

petreminder Application with Library
0.0.1

Generated by Doxygen 1.15.0

1 PetReminder Application	1
1.1 Overview	1
1.2 Features	1
1.3 Architecture	1
1.4 How to Run	2
1.5 Documentation	2
1.6 Team Members	2
1.7 License	2
2 Namespace Index	3
2.1 Package List	3
3 Hierarchical Index	5
3.1 Class Hierarchy	5
4 Class Index	7
4.1 Class List	7
5 File Index	11
5.1 File List	11
6 Namespace Documentation	15
6.1 Package com.mehdi.petreminder.config	15
6.1.1 Detailed Description	15
6.2 Package com.mehdi.petreminder.gui	15
6.2.1 Detailed Description	15
6.3 Package com.mehdi.petreminder.gui.dialogs	16
6.3.1 Detailed Description	16
6.4 Package com.mehdi.petreminder.gui.panels	16
6.4.1 Detailed Description	16
6.5 Package com.mehdi.petreminder.gui.util	16
6.5.1 Detailed Description	17
6.6 Package com.mehdi.petreminder.model	17
6.6.1 Detailed Description	17
6.7 Package com.mehdi.petreminder.observer	17
6.7.1 Detailed Description	18
6.8 Package com.mehdi.petreminder.repository	18
6.8.1 Detailed Description	18
6.9 Package com.mehdi.petreminder.service	19
6.9.1 Detailed Description	19
7 Class Documentation	21
7.1 com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable > Class Template Reference	21
7.1.1 Detailed Description	22

7.1.2 Constructor & Destructor Documentation	22
7.1.2.1 BinaryRepository()	22
7.1.3 Member Function Documentation	23
7.1.3.1 close()	23
7.1.3.2 count()	23
7.1.3.3 delete()	23
7.1.3.4 deleteAll()	24
7.1.3.5 findAll()	24
7.1.3.6 findById()	25
7.1.3.7 getFilePath()	26
7.1.3.8 save()	26
7.1.3.9 update()	27
7.2 com.mehdi.petreminder.model.Bird Class Reference	27
7.2.1 Detailed Description	29
7.2.2 Constructor & Destructor Documentation	29
7.2.2.1 Bird() [1/3]	29
7.2.2.2 Bird() [2/3]	30
7.2.2.3 Bird() [3/3]	30
7.2.3 Member Function Documentation	31
7.2.3.1 getBirdType()	31
7.2.3.2 getIconName()	31
7.2.3.3 isCanTalk()	31
7.2.3.4 makeSound()	32
7.2.3.5 setBirdType()	32
7.2.3.6 setCanTalk()	32
7.2.3.7 toString()	33
7.3 com.mehdi.petreminder.model.Cat Class Reference	33
7.3.1 Detailed Description	35
7.3.2 Constructor & Destructor Documentation	35
7.3.2.1 Cat() [1/3]	35
7.3.2.2 Cat() [2/3]	36
7.3.2.3 Cat() [3/3]	36
7.3.3 Member Function Documentation	37
7.3.3.1 getBreed()	37
7.3.3.2 getIconName()	37
7.3.3.3 isIndoor()	37
7.3.3.4 makeSound()	38
7.3.3.5 setBreed()	38
7.3.3.6 setIndoor()	38
7.3.3.7 toString()	39
7.4 com.mehdi.petreminder.ConsoleApp Class Reference	39
7.4.1 Detailed Description	40

7.4.2 Constructor & Destructor Documentation	40
7.4.2.1 ConsoleApp() [1/2]	40
7.4.2.2 ConsoleApp() [2/2]	40
7.4.3 Member Function Documentation	41
7.4.3.1 exitApp()	41
7.4.3.2 handleMainMenuChoice()	41
7.4.3.3 handleStorageSwitch()	41
7.4.3.4 isRunning()	42
7.4.3.5 printMainMenu()	42
7.4.3.6 readInput()	42
7.4.3.7 setRunning()	43
7.4.3.8 setScannerSource()	43
7.4.3.9 showMainMenu()	43
7.4.3.10 showMedicalMenu()	44
7.4.3.11 showPetsMenu()	44
7.4.3.12 showRemindersMenu()	44
7.4.3.13 showSettingsMenu()	45
7.4.3.14 showVetMenu()	45
7.4.3.15 start()	46
7.5 com.mehdi.petreminder.gui.panels.DashboardPanel Class Reference	46
7.5.1 Detailed Description	46
7.5.2 Constructor & Destructor Documentation	47
7.5.2.1 DashboardPanel()	47
7.5.3 Member Function Documentation	47
7.5.3.1 refresh()	47
7.6 com.mehdi.petreminder.model.Dog Class Reference	48
7.6.1 Detailed Description	49
7.6.2 Constructor & Destructor Documentation	50
7.6.2.1 Dog() [1/3]	50
7.6.2.2 Dog() [2/3]	50
7.6.2.3 Dog() [3/3]	50
7.6.3 Member Function Documentation	51
7.6.3.1 getBreed()	51
7.6.3.2 getIconName()	51
7.6.3.3 isTrained()	52
7.6.3.4 makeSound()	52
7.6.3.5 setBreed()	52
7.6.3.6 setTrained()	53
7.6.3.7 toString()	53
7.7 com.mehdi.petreminder.observer.EventManager Class Reference	53
7.7.1 Detailed Description	54
7.7.2 Member Function Documentation	54

7.7.2.1 clearAll()	54
7.7.2.2 getInstance()	55
7.7.2.3 getObserverCount()	55
7.7.2.4 notify()	56
7.7.2.5 resetInstance()	56
7.7.2.6 subscribe()	56
7.7.2.7 unsubscribe()	57
7.7.2.8 unsubscribeAll()	57
7.8 com.mehdi.petreminder.observer.EventType Enum Reference	58
7.8.1 Detailed Description	58
7.8.2 Constructor & Destructor Documentation	59
7.8.2.1 EventType()	59
7.8.3 Member Function Documentation	59
7.8.3.1 getDescription()	59
7.8.3.2 toString()	59
7.8.4 Member Data Documentation	60
7.8.4.1 MEDICAL_RECORD_ADDED	60
7.8.4.2 MEDICAL_RECORD_DELETED	60
7.8.4.3 PET_ADDED	60
7.8.4.4 PET_DELETED	60
7.8.4.5 PET_UPDATED	60
7.8.4.6 REMINDER_ADDED	61
7.8.4.7 REMINDER_COMPLETED	61
7.8.4.8 REMINDER_DELETED	61
7.8.4.9 STORAGE_CHANGED	61
7.9 com.mehdi.petreminder.model.ExerciseReminder Class Reference	61
7.9.1 Detailed Description	64
7.9.2 Constructor & Destructor Documentation	64
7.9.2.1 ExerciseReminder() [1/3]	64
7.9.2.2 ExerciseReminder() [2/3]	64
7.9.2.3 ExerciseReminder() [3/3]	65
7.9.3 Member Function Documentation	65
7.9.3.1 getDistanceKm()	65
7.9.3.2 getDurationMinutes()	66
7.9.3.3 getExerciseType()	66
7.9.3.4 getReminderType()	66
7.9.3.5 getSummary()	66
7.9.3.6 setDistanceKm()	67
7.9.3.7 setDurationMinutes()	67
7.9.3.8 setExerciseType()	67
7.9.3.9 toString()	68
7.9.3.10 validate()	68

7.10 com.mehdi.petreminder.model.FeedingReminder Class Reference	68
7.10.1 Detailed Description	71
7.10.2 Constructor & Destructor Documentation	71
7.10.2.1 FeedingReminder() [1/3]	71
7.10.2.2 FeedingReminder() [2/3]	71
7.10.2.3 FeedingReminder() [3/3]	72
7.10.3 Member Function Documentation	73
7.10.3.1 getFoodType()	73
7.10.3.2 getPortionGrams()	73
7.10.3.3 getReminderType()	73
7.10.3.4 getSummary()	74
7.10.3.5 isIncludesWaterChange()	74
7.10.3.6 setFoodType()	74
7.10.3.7 setIncludesWaterChange()	75
7.10.3.8 setPortionGrams()	75
7.10.3.9 toString()	75
7.10.3.10 validate()	76
7.11 com.mehdi.petreminder.model.GroomingReminder Class Reference	76
7.11.1 Detailed Description	78
7.11.2 Constructor & Destructor Documentation	79
7.11.2.1 GroomingReminder() [1/3]	79
7.11.2.2 GroomingReminder() [2/3]	79
7.11.2.3 GroomingReminder() [3/3]	79
7.11.3 Member Function Documentation	80
7.11.3.1 getGroomerName()	80
7.11.3.2 getGroomingType()	80
7.11.3.3 getReminderType()	81
7.11.3.4 getSummary()	81
7.11.3.5 isProfessional()	81
7.11.3.6 setGroomerName()	82
7.11.3.7 setGroomingType()	82
7.11.3.8 setProfessional()	82
7.11.3.9 toString()	83
7.11.3.10 validate()	83
7.12 com.mehdi.petreminder.gui.util.GuiConstants Class Reference	83
7.12.1 Detailed Description	84
7.12.2 Member Data Documentation	85
7.12.2.1 BG_COLOR	85
7.12.2.2 BODY_FONT	85
7.12.2.3 BORDER_COLOR	85
7.12.2.4 CARD_BG	85
7.12.2.5 DANGER	86

7.12.2.6 GAP	86
7.12.2.7 GAP_LARGE	86
7.12.2.8 NAV_ACTIVE	86
7.12.2.9 NAV_COLOR	86
7.12.2.10 NAV_FONT	87
7.12.2.11 NAV_HOVER	87
7.12.2.12 NAV_WIDTH	87
7.12.2.13 PRIMARY	87
7.12.2.14 SECTION_FONT	87
7.12.2.15 SMALL_FONT	88
7.12.2.16 SUCCESS	88
7.12.2.17 TEXT_PRIMARY	88
7.12.2.18 TEXT_SECONDARY	88
7.12.2.19 TITLE_FONT	88
7.12.2.20 WARNING	89
7.13 com.mehdi.petreminder.repository.IRepository< T > Interface Template Reference	89
7.13.1 Detailed Description	90
7.13.2 Member Function Documentation	90
7.13.2.1 close()	90
7.13.2.2 count()	90
7.13.2.3 delete()	91
7.13.2.4 deleteAll()	91
7.13.2.5 findAll()	91
7.13.2.6 findById()	92
7.13.2.7 save()	92
7.13.2.8 update()	93
7.14 com.mehdi.petreminder.gui.MainFrame Class Reference	93
7.14.1 Detailed Description	94
7.14.2 Constructor & Destructor Documentation	95
7.14.2.1 MainFrame()	95
7.14.3 Member Function Documentation	95
7.14.3.1 launch()	95
7.14.3.2 navigateTo()	95
7.14.3.3 onStorageChanged()	96
7.14.4 Member Data Documentation	96
7.14.4.1 CARD_DASHBOARD	96
7.14.4.2 CARD_MEDICAL	96
7.14.4.3 CARD_PETS	96
7.14.4.4 CARD_REMINDERS	97
7.14.4.5 CARD_SETTINGS	97
7.14.4.6 CARD_VET	97
7.15 com.mehdi.petreminder.gui.panels.MedicalPanel Class Reference	97

7.15.1 Detailed Description	98
7.15.2 Constructor & Destructor Documentation	98
7.15.2.1 MedicalPanel()	98
7.15.3 Member Function Documentation	99
7.15.3.1 refresh()	99
7.16 com.mehdi.petreminder.model.MedicalRecord Class Reference	99
7.16.1 Detailed Description	101
7.16.2 Constructor & Destructor Documentation	101
7.16.2.1 MedicalRecord() [1/2]	101
7.16.2.2 MedicalRecord() [2/2]	101
7.16.3 Member Function Documentation	102
7.16.3.1 equals()	102
7.16.3.2 getCost()	102
7.16.3.3 getDiagnosis()	103
7.16.3.4 getFormattedDate()	103
7.16.3.5 getId()	103
7.16.3.6 getNextCheckDate()	104
7.16.3.7 getNotes()	104
7.16.3.8 getPetId()	104
7.16.3.9 getPetName()	104
7.16.3.10 getRecordDate()	105
7.16.3.11 getRecordType()	105
7.16.3.12 getTreatment()	105
7.16.3.13 getVaccineName()	105
7.16.3.14 getVeterinarianName()	106
7.16.3.15 hashCode()	106
7.16.3.16 setCost()	106
7.16.3.17 setDiagnosis()	107
7.16.3.18 setId()	107
7.16.3.19 setNextCheckDate()	107
7.16.3.20 setNotes()	108
7.16.3.21 setPetId()	108
7.16.3.22 setPetName()	108
7.16.3.23 setRecordDate()	109
7.16.3.24 setRecordType()	109
7.16.3.25 setTreatment()	109
7.16.3.26 setVaccineName()	110
7.16.3.27 setVeterinarianName()	110
7.16.3.28 toString()	110
7.17 com.mehdi.petreminder.repository.MedicalRecordMySqlRepository Class Reference	111
7.17.1 Detailed Description	112
7.17.2 Constructor & Destructor Documentation	113

7.17.2.1 MedicalRecordMySqlRepository() [1/2]	113
7.17.2.2 MedicalRecordMySqlRepository() [2/2]	113
7.17.3 Member Function Documentation	113
7.17.3.1 getCreateTableSql()	113
7.17.3.2 getInsertSql()	114
7.17.3.3 getTableName()	114
7.17.3.4 getUpdateSql()	114
7.17.3.5 mapRow()	114
7.17.3.6 setInsertParams()	115
7.17.3.7 setUpdateParams()	115
7.18 com.mehdi.petreminder.service.MedicalRecordService Class Reference	115
7.18.1 Detailed Description	116
7.18.2 Constructor & Destructor Documentation	116
7.18.2.1 MedicalRecordService() [1/2]	116
7.18.2.2 MedicalRecordService() [2/2]	117
7.18.3 Member Function Documentation	117
7.18.3.1 addRecord()	117
7.18.3.2 close()	117
7.18.3.3 deleteAllRecords()	118
7.18.3.4 deleteRecord()	118
7.18.3.5 getAllRecords()	118
7.18.3.6 getRecordById()	119
7.18.3.7 getRecordCount()	119
7.18.3.8 getRecordsByPetId()	119
7.18.3.9 updateRecord()	120
7.19 com.mehdi.petreminder.repository.MedicalRecordSqliteRepository Class Reference	120
7.19.1 Detailed Description	122
7.19.2 Constructor & Destructor Documentation	122
7.19.2.1 MedicalRecordSqliteRepository() [1/2]	122
7.19.2.2 MedicalRecordSqliteRepository() [2/2]	123
7.19.3 Member Function Documentation	123
7.19.3.1 findByPetId()	123
7.19.3.2 getCreateTableSql()	124
7.19.3.3 getInsertSql()	124
7.19.3.4 getTableName()	124
7.19.3.5 getUpdateSql()	124
7.19.3.6 mapRow()	125
7.19.3.7 setInsertParams()	125
7.19.3.8 setUpdateParams()	126
7.20 com.mehdi.petreminder.model.MedicationReminder Class Reference	126
7.20.1 Detailed Description	128
7.20.2 Constructor & Destructor Documentation	129

7.20.2.1 MedicationReminder() [1/3]	129
7.20.2.2 MedicationReminder() [2/3]	129
7.20.2.3 MedicationReminder() [3/3]	130
7.20.3 Member Function Documentation	130
7.20.3.1 getDosage()	130
7.20.3.2 getDosageUnit()	131
7.20.3.3 getMedicationName()	131
7.20.3.4 getReminderType()	131
7.20.3.5 getSummary()	132
7.20.3.6 getVeterinarianName()	132
7.20.3.7 isPrescribed()	132
7.20.3.8 setDosage()	133
7.20.3.9 setDosageUnit()	133
7.20.3.10 setMedicationName()	133
7.20.3.11 setPrescribed()	134
7.20.3.12 setVeterinarianName()	134
7.20.3.13 toString()	134
7.20.3.14 validate()	135
7.21 com.mehdi.petreminder.repository.MySqlRepository< T > Class Template Reference	135
7.21.1 Detailed Description	136
7.21.2 Constructor & Destructor Documentation	137
7.21.2.1 MySqlRepository() [1/2]	137
7.21.2.2 MySqlRepository() [2/2]	137
7.21.3 Member Function Documentation	138
7.21.3.1 close()	138
7.21.3.2 count()	138
7.21.3.3 createTableIfNotExists()	138
7.21.3.4 delete()	139
7.21.3.5 deleteAll()	139
7.21.3.6 findAll()	140
7.21.3.7 findById()	140
7.21.3.8 getConnection()	141
7.21.3.9 getCreateTableSql()	141
7.21.3.10 getInsertSql()	141
7.21.3.11 getTableName()	141
7.21.3.12 getUpdateSql()	142
7.21.3.13 mapRow()	142
7.21.3.14 save()	142
7.21.3.15 setEntityId()	143
7.21.3.16 setInsertParams()	144
7.21.3.17 setUpdateParams()	144
7.21.3.18 update()	144

7.21.4 Member Data Documentation	145
7.21.4.1 connection	145
7.21.4.2 logger	145
7.22 com.mehdi.petreminder.gui.panels.NavPanel Class Reference	146
7.22.1 Detailed Description	146
7.22.2 Constructor & Destructor Documentation	147
7.22.2.1 NavPanel()	147
7.22.3 Member Function Documentation	147
7.22.3.1 setActiveButton()	147
7.23 com.mehdi.petreminder.model.Pet Class Reference	148
7.23.1 Detailed Description	149
7.23.2 Constructor & Destructor Documentation	149
7.23.2.1 Pet() [1/2]	149
7.23.2.2 Pet() [2/2]	150
7.23.3 Member Function Documentation	150
7.23.3.1 equals()	150
7.23.3.2 getAge()	151
7.23.3.3 getAgeString()	151
7.23.3.4 getBirthDate()	152
7.23.3.5 getGender()	152
7.23.3.6 getIconName()	152
7.23.3.7 getId()	152
7.23.3.8 getName()	153
7.23.3.9 getNotes()	153
7.23.3.10 getOwnerId()	153
7.23.3.11 getSpecies()	153
7.23.3.12 getWeight()	154
7.23.3.13 hashCode()	154
7.23.3.14 makeSound()	154
7.23.3.15 setBirthDate()	155
7.23.3.16 setGender()	155
7.23.3.17 setId()	155
7.23.3.18 setName()	156
7.23.3.19 setNotes()	156
7.23.3.20 setOwnerId()	156
7.23.3.21 setSpecies()	157
7.23.3.22 setWeight()	157
7.23.3.23 toString()	157
7.24 com.mehdi.petreminder.gui.dialogs.PetDialog Class Reference	158
7.24.1 Detailed Description	158
7.24.2 Constructor & Destructor Documentation	159
7.24.2.1 PetDialog()	159

7.24.3 Member Function Documentation	159
7.24.3.1 getPet()	159
7.24.3.2 isConfirmed()	159
7.25 com.mehdi.petreminder.repository.PetMySqlRepository Class Reference	160
7.25.1 Detailed Description	161
7.25.2 Constructor & Destructor Documentation	162
7.25.2.1 PetMySqlRepository() [1/2]	162
7.25.2.2 PetMySqlRepository() [2/2]	162
7.25.3 Member Function Documentation	162
7.25.3.1 findByOwnerId()	162
7.25.3.2 getCreateTableSql()	163
7.25.3.3 getInsertSql()	163
7.25.3.4 getTableName()	163
7.25.3.5 getUpdateSql()	163
7.25.3.6 mapRow()	164
7.25.3.7 setInsertParams()	164
7.25.3.8 setUpdateParams()	165
7.26 com.mehdi.petreminder.petreminder Class Reference	165
7.26.1 Detailed Description	166
7.26.2 Constructor & Destructor Documentation	166
7.26.2.1 petreminder()	166
7.26.3 Member Function Documentation	166
7.26.3.1 add()	166
7.26.3.2 divide()	167
7.26.3.3 getCurrentTimeFormatted()	168
7.26.3.4 isNullOrEmpty()	168
7.26.3.5 multiply()	168
7.26.3.6 printHeader()	169
7.26.3.7 printPets()	169
7.26.3.8 printReminders()	169
7.26.3.9 printSeparator()	170
7.26.3.10 subtract()	170
7.27 com.mehdi.petreminder.petreminderApp Class Reference	171
7.27.1 Detailed Description	172
7.27.2 Member Function Documentation	172
7.27.2.1 getAppName()	172
7.27.2.2 getVersion()	172
7.27.2.3 isGuiRequested()	173
7.27.2.4 main()	173
7.27.2.5 parseStorageArg()	174
7.27.2.6 startConsole()	174
7.27.2.7 startGui()	175

7.27.3 Member Data Documentation	175
7.27.3.1 APP_NAME	175
7.27.3.2 APP_VERSION	175
7.28 com.mehdi.petreminder.observer.PetReminderObserver Interface Reference	176
7.28.1 Detailed Description	176
7.28.2 Member Function Documentation	176
7.28.2.1 onEvent()	176
7.29 com.mehdi.petreminder.service.PetService Class Reference	177
7.29.1 Detailed Description	177
7.29.2 Constructor & Destructor Documentation	178
7.29.2.1 PetService() [1/2]	178
7.29.2.2 PetService() [2/2]	178
7.29.3 Member Function Documentation	178
7.29.3.1 addPet()	178
7.29.3.2 close()	179
7.29.3.3 deleteAllPets()	179
7.29.3.4 deletePet()	179
7.29.3.5 filterBySpecies()	180
7.29.3.6 getAllPets()	180
7.29.3.7 getPetById()	181
7.29.3.8 getPetCount()	181
7.29.3.9 searchByName()	181
7.29.3.10 updatePet()	182
7.30 com.mehdi.petreminder.gui.panels.PetsPanel Class Reference	183
7.30.1 Detailed Description	183
7.30.2 Constructor & Destructor Documentation	184
7.30.2.1 PetsPanel()	184
7.30.3 Member Function Documentation	184
7.30.3.1 refresh()	184
7.31 com.mehdi.petreminder.repository.PetSqliteRepository Class Reference	185
7.31.1 Detailed Description	186
7.31.2 Constructor & Destructor Documentation	187
7.31.2.1 PetSqliteRepository() [1/2]	187
7.31.2.2 PetSqliteRepository() [2/2]	187
7.31.3 Member Function Documentation	187
7.31.3.1 findByOwnerId()	187
7.31.3.2 getCreateTableSql()	188
7.31.3.3 getInsertSql()	188
7.31.3.4 getTableName()	188
7.31.3.5 getUpdateSql()	188
7.31.3.6 mapRow()	189
7.31.3.7 setInsertParams()	189

7.31.3.8 setUpdateParams()	190
7.32 com.mehdi.petreminder.model.Reminder Class Reference	190
7.32.1 Detailed Description	192
7.32.2 Constructor & Destructor Documentation	192
7.32.2.1 Reminder() [1/2]	192
7.32.2.2 Reminder() [2/2]	192
7.32.3 Member Function Documentation	193
7.32.3.1 equals()	193
7.32.3.2 getDescription()	193
7.32.3.3 getFormattedTime()	194
7.32.3.4 getId()	194
7.32.3.5 getPetId()	194
7.32.3.6 getPetName()	195
7.32.3.7 getPriority()	195
7.32.3.8 getRecurrencePattern()	195
7.32.3.9 getReminderType()	196
7.32.3.10 getScheduledTime()	196
7.32.3.11 getSummary()	196
7.32.3.12 hashCode()	197
7.32.3.13 isCompleted()	197
7.32.3.14 isOverdue()	197
7.32.3.15 isRecurring()	197
7.32.3.16 setCompleted()	198
7.32.3.17 setDescription()	198
7.32.3.18 setId()	198
7.32.3.19 setPetId()	199
7.32.3.20 setPetName()	199
7.32.3.21 setPriority()	199
7.32.3.22 setRecurrencePattern()	200
7.32.3.23 setRecurring()	200
7.32.3.24 setScheduledTime()	200
7.32.3.25 toString()	201
7.32.3.26 validate()	201
7.32.4 Member Data Documentation	201
7.32.4.1 FORMATTER	201
7.33 com.mehdi.petreminder.gui.dialogs.ReminderDialog Class Reference	202
7.33.1 Detailed Description	202
7.33.2 Constructor & Destructor Documentation	203
7.33.2.1 ReminderDialog()	203
7.33.3 Member Function Documentation	203
7.33.3.1 getReminder()	203
7.33.3.2 isConfirmed()	203

7.34 com.mehdi.petreminder.repository.ReminderMySqlRepository Class Reference	204
7.34.1 Detailed Description	205
7.34.2 Constructor & Destructor Documentation	206
7.34.2.1 ReminderMySqlRepository() [1/2]	206
7.34.2.2 ReminderMySqlRepository() [2/2]	206
7.34.3 Member Function Documentation	206
7.34.3.1 getCreateTableSql()	206
7.34.3.2 getInsertSql()	207
7.34.3.3 getTableName()	207
7.34.3.4 getUpdateSql()	207
7.34.3.5 mapRow()	208
7.34.3.6 setInsertParams()	208
7.34.3.7 setUpdateParams()	209
7.35 com.mehdi.petreminder.service.ReminderService Class Reference	209
7.35.1 Detailed Description	210
7.35.2 Constructor & Destructor Documentation	211
7.35.2.1 ReminderService() [1/2]	211
7.35.2.2 ReminderService() [2/2]	211
7.35.3 Member Function Documentation	211
7.35.3.1 addReminder()	211
7.35.3.2 close()	212
7.35.3.3 deleteAllReminders()	212
7.35.3.4 deleteReminder()	212
7.35.3.5 getAllReminders()	213
7.35.3.6 getOverdueReminders()	213
7.35.3.7 getPendingReminders()	213
7.35.3.8 getReminderById()	214
7.35.3.9 getReminderCount()	214
7.35.3.10 getRemindersByPetId()	214
7.35.3.11 markCompleted()	215
7.35.3.12 updateReminder()	215
7.36 com.mehdi.petreminder.gui.panels.RemindersPanel Class Reference	216
7.36.1 Detailed Description	216
7.36.2 Constructor & Destructor Documentation	216
7.36.2.1 RemindersPanel()	216
7.36.3 Member Function Documentation	217
7.36.3.1 refresh()	217
7.37 com.mehdi.petreminder.repository.ReminderSqliteRepository Class Reference	217
7.37.1 Detailed Description	219
7.37.2 Constructor & Destructor Documentation	219
7.37.2.1 ReminderSqliteRepository() [1/2]	219
7.37.2.2 ReminderSqliteRepository() [2/2]	220

7.37.3 Member Function Documentation	220
7.37.3.1 findByPetId()	220
7.37.3.2 getCreateTableSql()	221
7.37.3.3 getInsertSql()	221
7.37.3.4 getTableName()	221
7.37.3.5 getUpdateSql()	222
7.37.3.6 mapRow()	222
7.37.3.7 setInsertParams()	223
7.37.3.8 setUpdateParams()	224
7.38 com.mehdi.petreminder.repository.RepositoryException Class Reference	224
7.38.1 Detailed Description	224
7.38.2 Constructor & Destructor Documentation	225
7.38.2.1 RepositoryException() [1/2]	225
7.38.2.2 RepositoryException() [2/2]	225
7.39 com.mehdi.petreminder.repository.RepositoryFactory Class Reference	225
7.39.1 Detailed Description	226
7.39.2 Member Function Documentation	226
7.39.2.1 createMedicalRecordRepository()	226
7.39.2.2 createPetRepository()	227
7.39.2.3 createReminderRepository()	227
7.39.2.4 createUserRepository()	228
7.39.2.5 getActiveType()	228
7.40 com.mehdi.petreminder.service.ServiceException Class Reference	228
7.40.1 Detailed Description	229
7.40.2 Constructor & Destructor Documentation	229
7.40.2.1 ServiceException() [1/2]	229
7.40.2.2 ServiceException() [2/2]	230
7.41 com.mehdi.petreminder.gui.panels.SettingsPanel Class Reference	230
7.41.1 Detailed Description	230
7.41.2 Constructor & Destructor Documentation	231
7.41.2.1 SettingsPanel()	231
7.41.3 Member Function Documentation	231
7.41.3.1 refresh()	231
7.42 com.mehdi.petreminder.repository.SQLiteRepository< T > Class Template Reference	231
7.42.1 Detailed Description	233
7.42.2 Constructor & Destructor Documentation	233
7.42.2.1 SQLiteRepository() [1/2]	233
7.42.2.2 SQLiteRepository() [2/2]	234
7.42.3 Member Function Documentation	234
7.42.3.1 close()	234
7.42.3.2 count()	234
7.42.3.3 createTableIfNotExists()	235

7.42.3.4 delete()	235
7.42.3.5 deleteAll()	236
7.42.3.6 findAll()	236
7.42.3.7 findById()	237
7.42.3.8 getConnection()	237
7.42.3.9 getCreateTableSql()	238
7.42.3.10 getInsertSql()	238
7.42.3.11 getTableName()	238
7.42.3.12 getUpdateSql()	238
7.42.3.13 mapRow()	239
7.42.3.14 save()	239
7.42.3.15 setEntityId()	240
7.42.3.16 setInsertParams()	240
7.42.3.17 setUpdateParams()	241
7.42.3.18 update()	241
7.42.4 Member Data Documentation	242
7.42.4.1 connection	242
7.42.4.2 logger	242
7.43 com.mehdi.petreminder.config.StorageConfig Class Reference	242
7.43.1 Detailed Description	243
7.43.2 Member Function Documentation	244
7.43.2.1 getActiveBackend()	244
7.43.2.2 getBinaryDirectory()	244
7.43.2.3 getMysqlPassword()	244
7.43.2.4 getMysqlUrl()	245
7.43.2.5 getMysqlUsername()	245
7.43.2.6 getSqliteFilePath()	245
7.43.2.7 loadFromFile()	246
7.43.2.8 reset()	246
7.43.2.9 saveToFile()	246
7.43.2.10 setActiveBackend()	247
7.43.2.11 setMysqlUrl()	247
7.44 com.mehdi.petreminder.config.StorageType Enum Reference	248
7.44.1 Detailed Description	248
7.44.2 Constructor & Destructor Documentation	248
7.44.2.1 StorageType()	248
7.44.3 Member Function Documentation	249
7.44.3.1 getDisplayName()	249
7.44.3.2 toString()	249
7.44.4 Member Data Documentation	249
7.44.4.1 BINARY	249
7.44.4.2 MYSQL	250

7.44.4.3 SQLITE	250
7.45 com.mehdi.petreminder.model.User Class Reference	250
7.45.1 Detailed Description	251
7.45.2 Constructor & Destructor Documentation	252
7.45.2.1 User() [1/2]	252
7.45.2.2 User() [2/2]	252
7.45.3 Member Function Documentation	252
7.45.3.1 equals()	252
7.45.3.2 getCreatedAt()	253
7.45.3.3 getEmail()	253
7.45.3.4 getFullName()	253
7.45.3.5 getId()	254
7.45.3.6 getLastLogin()	254
7.45.3.7 getPasswordHash()	254
7.45.3.8 getUsername()	254
7.45.3.9 hashCode()	255
7.45.3.10 isActive()	255
7.45.3.11 setActive()	255
7.45.3.12 setCreatedAt()	256
7.45.3.13 setEmail()	256
7.45.3.14 setFullName()	256
7.45.3.15 setId()	257
7.45.3.16 setLastLogin()	257
7.45.3.17 setPasswordHash()	257
7.45.3.18 setUsername()	258
7.45.3.19 toString()	258
7.46 com.mehdi.petreminder.repository.UserMySqlRepository Class Reference	258
7.46.1 Detailed Description	260
7.46.2 Constructor & Destructor Documentation	260
7.46.2.1 UserMySqlRepository() [1/2]	260
7.46.2.2 UserMySqlRepository() [2/2]	261
7.46.3 Member Function Documentation	261
7.46.3.1 getCreateTableSql()	261
7.46.3.2 getInsertSql()	261
7.46.3.3 getTableName()	261
7.46.3.4 getUpdateSql()	262
7.46.3.5 mapRow()	262
7.46.3.6 setInsertParams()	262
7.46.3.7 setUpdateParams()	263
7.47 com.mehdi.petreminder.repository.UserSqliteRepository Class Reference	263
7.47.1 Detailed Description	265
7.47.2 Constructor & Destructor Documentation	265

7.47.2.1 UserSqliteRepository() [1/2]	265
7.47.2.2 UserSqliteRepository() [2/2]	265
7.47.3 Member Function Documentation	266
7.47.3.1 findByUsername()	266
7.47.3.2 getCreateTableSql()	266
7.47.3.3 getInsertSql()	266
7.47.3.4 getTableName()	267
7.47.3.5 getUpdateSql()	267
7.47.3.6 mapRow()	267
7.47.3.7 setInsertParams()	267
7.47.3.8 setUpdateParams()	268
7.48 com.mehdi.petreminder.model.VetAppointment Class Reference	268
7.48.1 Detailed Description	270
7.48.2 Constructor & Destructor Documentation	271
7.48.2.1 VetAppointment() [1/3]	271
7.48.2.2 VetAppointment() [2/3]	271
7.48.2.3 VetAppointment() [3/3]	272
7.48.3 Member Function Documentation	272
7.48.3.1 getClinicName()	272
7.48.3.2 getClinicPhone()	273
7.48.3.3 getEstimatedCost()	273
7.48.3.4 getReason()	273
7.48.3.5 getReminderType()	274
7.48.3.6 getSummary()	274
7.48.3.7 getVeterinarianName()	274
7.48.3.8 isConfirmed()	275
7.48.3.9 setClinicName()	275
7.48.3.10 setClinicPhone()	275
7.48.3.11 setConfirmed()	276
7.48.3.12 setEstimatedCost()	276
7.48.3.13 setReason()	276
7.48.3.14 setVeterinarianName()	277
7.48.3.15 toString()	277
7.48.3.16 validate()	277
7.49 com.mehdi.petreminder.gui.panels.VetPanel Class Reference	278
7.49.1 Detailed Description	278
7.49.2 Constructor & Destructor Documentation	278
7.49.2.1 VetPanel()	278
7.49.3 Member Function Documentation	279
7.49.3.1 refresh()	279

8.1 petreminder-app/src/main/java/com/mehdi/petreminder/config/StorageConfig.java File Reference . . .	281
8.1.1 Detailed Description	281
8.2 StorageConfig.java	282
8.3 petreminder-app/src/main/java/com/mehdi/petreminder/config/StorageType.java File Reference . . .	283
8.3.1 Detailed Description	283
8.4 StorageType.java	284
8.5 petreminder-app/src/main/java/com/mehdi/petreminder/ConsoleApp.java File Reference	284
8.5.1 Detailed Description	285
8.6 ConsoleApp.java	285
8.7 petreminder-app/src/main/java/com/mehdi/petreminder/gui/dialogs/PetDialog.java File Reference . .	293
8.7.1 Detailed Description	294
8.8 PetDialog.java	294
8.9 petreminder-app/src/main/java/com/mehdi/petreminder/gui/dialogs/ReminderDialog.java File Reference	296
8.9.1 Detailed Description	297
8.10 ReminderDialog.java	297
8.11 petreminder-app/src/main/java/com/mehdi/petreminder/gui/MainFrame.java File Reference	300
8.11.1 Detailed Description	300
8.12 MainFrame.java	301
8.13 petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/DashboardPanel.java File Refer- ence	303
8.13.1 Detailed Description	303
8.14 DashboardPanel.java	303
8.15 petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/MedicalPanel.java File Reference	305
8.15.1 Detailed Description	306
8.16 MedicalPanel.java	306
8.17 petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/NavPanel.java File Reference . .	308
8.17.1 Detailed Description	308
8.18 NavPanel.java	309
8.19 petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/PetsPanel.java File Reference . .	310
8.19.1 Detailed Description	310
8.20 PetsPanel.java	311
8.21 petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/RemindersPanel.java File Refer- ence	313
8.21.1 Detailed Description	314
8.22 RemindersPanel.java	314
8.23 petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/SettingsPanel.java File Reference	316
8.23.1 Detailed Description	317
8.24 SettingsPanel.java	317
8.25 petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/VetPanel.java File Reference . .	319
8.25.1 Detailed Description	319
8.26 VetPanel.java	320
8.27 petreminder-app/src/main/java/com/mehdi/petreminder/gui/util/GuiConstants.java File Reference . .	321
8.27.1 Detailed Description	322

8.28	GuiConstants.java	322
8.29	petreminder-app/src/main/java/com/mehdi/petreminder/model/Bird.java File Reference	323
8.29.1	Detailed Description	323
8.30	Bird.java	323
8.31	petreminder-app/src/main/java/com/mehdi/petreminder/model/Cat.java File Reference	324
8.31.1	Detailed Description	324
8.32	Cat.java	324
8.33	petreminder-app/src/main/java/com/mehdi/petreminder/model/Dog.java File Reference	325
8.33.1	Detailed Description	325
8.34	Dog.java	326
8.35	petreminder-app/src/main/java/com/mehdi/petreminder/model/ExerciseReminder.java File Reference	326
8.35.1	Detailed Description	327
8.36	ExerciseReminder.java	327
8.37	petreminder-app/src/main/java/com/mehdi/petreminder/model/FeedingReminder.java File Reference	328
8.37.1	Detailed Description	328
8.38	FeedingReminder.java	328
8.39	petreminder-app/src/main/java/com/mehdi/petreminder/model/GroomingReminder.java File Reference	329
8.39.1	Detailed Description	330
8.40	GroomingReminder.java	330
8.41	petreminder-app/src/main/java/com/mehdi/petreminder/model/MedicalRecord.java File Reference	331
8.41.1	Detailed Description	331
8.42	MedicalRecord.java	331
8.43	petreminder-app/src/main/java/com/mehdi/petreminder/model/MedicationReminder.java File Reference	333
8.43.1	Detailed Description	333
8.44	MedicationReminder.java	333
8.45	petreminder-app/src/main/java/com/mehdi/petreminder/model/Pet.java File Reference	335
8.45.1	Detailed Description	335
8.46	Pet.java	335
8.47	petreminder-app/src/main/java/com/mehdi/petreminder/model/Reminder.java File Reference	337
8.47.1	Detailed Description	337
8.48	Reminder.java	337
8.49	petreminder-app/src/main/java/com/mehdi/petreminder/model/User.java File Reference	339
8.49.1	Detailed Description	339
8.50	User.java	339
8.51	petreminder-app/src/main/java/com/mehdi/petreminder/model/VetAppointment.java File Reference	340
8.51.1	Detailed Description	341
8.52	VetAppointment.java	341
8.53	petreminder-app/src/main/java/com/mehdi/petreminder/observer/EventManager.java File Reference	342
8.53.1	Detailed Description	343
8.54	EventManager.java	343
8.55	petreminder-app/src/main/java/com/mehdi/petreminder/observer/EventType.java File Reference	344

8.55.1 Detailed Description	344
8.56 EventType.java	345
8.57 petreminder-app/src/main/java/com/mehdi/petreminder/observer/PetReminderObserver.java File Reference	345
8.57.1 Detailed Description	345
8.58 PetReminderObserver.java	346
8.59 petreminder-app/src/main/java/com/mehdi/petreminder/petreminder.java File Reference	346
8.59.1 Detailed Description	346
8.60 petreminder.java	346
8.61 petreminder-app/src/main/java/com/mehdi/petreminder/petreminderApp.java File Reference	348
8.61.1 Detailed Description	348
8.62 petreminderApp.java	348
8.63 petreminder-app/src/main/java/com/mehdi/petreminder/repository/BinaryRepository.java File Reference	350
8.63.1 Detailed Description	350
8.64 BinaryRepository.java	350
8.65 petreminder-app/src/main/java/com/mehdi/petreminder/repository/IRepository.java File Reference	352
8.65.1 Detailed Description	353
8.66 IRepository.java	353
8.67 petreminder-app/src/main/java/com/mehdi/petreminder/repository/MedicalRecordMySqlRepository.java File Reference	353
8.67.1 Detailed Description	354
8.68 MedicalRecordMySqlRepository.java	354
8.69 petreminder-app/src/main/java/com/mehdi/petreminder/repository/MedicalRecordSqliteRepository.java File Reference	355
8.69.1 Detailed Description	356
8.70 MedicalRecordSqliteRepository.java	356
8.71 petreminder-app/src/main/java/com/mehdi/petreminder/repository/MySqlRepository.java File Reference	357
8.71.1 Detailed Description	357
8.72 MySqlRepository.java	358
8.73 petreminder-app/src/main/java/com/mehdi/petreminder/repository/PetMySqlRepository.java File Reference	360
8.73.1 Detailed Description	360
8.74 PetMySqlRepository.java	360
8.75 petreminder-app/src/main/java/com/mehdi/petreminder/repository/PetSqliteRepository.java File Reference	362
8.75.1 Detailed Description	362
8.76 PetSqliteRepository.java	363
8.77 petreminder-app/src/main/java/com/mehdi/petreminder/repository/ReminderMySqlRepository.java File Reference	364
8.77.1 Detailed Description	365
8.78 ReminderMySqlRepository.java	365
8.79 petreminder-app/src/main/java/com/mehdi/petreminder/repository/ReminderSqliteRepository.java File Reference	367

8.79.1 Detailed Description	368
8.80 ReminderSqliteRepository.java	368
8.81 petreminder-app/src/main/java/com/mehdi/petreminder/repository/RepositoryException.java File Reference	370
8.81.1 Detailed Description	371
8.82 RepositoryException.java	371
8.83 petreminder-app/src/main/java/com/mehdi/petreminder/repository/RepositoryFactory.java File Reference	371
8.83.1 Detailed Description	371
8.84 RepositoryFactory.java	372
8.85 petreminder-app/src/main/java/com/mehdi/petreminder/repository/SqliteRepository.java File Reference	372
8.85.1 Detailed Description	373
8.86 SqliteRepository.java	373
8.87 petreminder-app/src/main/java/com/mehdi/petreminder/repository/UserMySqlRepository.java File Reference	375
8.87.1 Detailed Description	376
8.88 UserMySqlRepository.java	376
8.89 petreminder-app/src/main/java/com/mehdi/petreminder/repository/UserSqliteRepository.java File Reference	377
8.89.1 Detailed Description	377
8.90 UserSqliteRepository.java	378
8.91 petreminder-app/src/main/java/com/mehdi/petreminder/service/MedicalRecordService.java File Reference	379
8.91.1 Detailed Description	379
8.92 MedicalRecordService.java	379
8.93 petreminder-app/src/main/java/com/mehdi/petreminder/service/PetService.java File Reference	381
8.93.1 Detailed Description	381
8.94 PetService.java	381
8.95 petreminder-app/src/main/java/com/mehdi/petreminder/service/ReminderService.java File Reference	383
8.95.1 Detailed Description	383
8.96 ReminderService.java	383
8.97 petreminder-app/src/main/java/com/mehdi/petreminder/service/ServiceException.java File Reference	385
8.97.1 Detailed Description	385
8.98 ServiceException.java	385

Chapter 1

PetReminder Application

1.1 Overview

PetReminder is a comprehensive Java-based desktop application designed to help pet owners manage their pets' care routines, medical records, and veterinary appointments. Built using modern Java technologies including Swing (with FlatLaf), JDBC (SQLite/MySQL), and JUnit 5, it ensures robust performance, excellent test coverage, and a user-friendly interface.

1.2 Features

- **Pet Management:** Add, view, edit, and delete pet profiles (Cats, Dogs, Birds, etc.).
- **Reminders:** Schedule feeding, medication, exercise, and grooming routines.
- **Medical Records:** Keep track of vaccinations, surgeries, and treatments.
- **Vet Appointments:** Log and schedule upcoming visits to the veterinarian.
- **Data Persistence:** Supports SQLite for local storage and MySQL for distributed usage.

1.3 Architecture

- **Language:** Java 11+
- **UI Framework:** Java Swing with FlatLaf Light Theme and MigLayout
- **Database:** SQLite (default), MySQL, H2 (Testing)
- **Testing:** JUnit 5, Mockito
- **Build Tool:** Apache Maven

1.4 How to Run

Use the provided batch scripts in the repository root:

1. `4-install-required-apps.bat` - Install required local dependencies such as Doxygen, Lcov, ReportGenerator.
2. `7-build-app.bat` - Compiles the project, runs tests, and generates the fat JAR along with documentation (Doxygen) and coverage reports.
3. `8-run-app.bat` - Launches the PetReminder desktop application standalone.
4. `9-run-webpage.bat` - Starts a local web server (Maven Site) to view this documentation and Doxygen HTML outputs (port 9000).

1.5 Documentation

The source code is thoroughly documented using Javadoc. To view the Doxygen documentation, build the project using `7-build-app.bat` and run `9-run-webpage.bat`. Then navigate to <http://localhost:9000/doxygen/html/index.html>.

1.6 Team Members

Developed by students at Recep Tayyip Erdogan University (RTEU):

- Ibrahim Demirci
- Muhammed Mehdi Karagulle
- Zumre Uykun

1.7 License

This project is licensed under the **GNU Affero General Public License v3.0 (AGPL-3.0)**. See the [LICENSE](#) file in the root directory for more details.

Chapter 2

Namespace Index

2.1 Package List

Here are the packages with brief descriptions (if available):

com.mehdi.petreminder.config	15
com.mehdi.petreminder.gui	15
com.mehdi.petreminder.gui.dialogs	16
com.mehdi.petreminder.gui.panels	16
com.mehdi.petreminder.gui.util	16
com.mehdi.petreminder.model	17
com.mehdi.petreminder.observer	17
com.mehdi.petreminder.repository	18
com.mehdi.petreminder.service	19

Chapter 3

Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

com.mehdi.petreminder.ConsoleApp	39
com.mehdi.petreminder.observer.EventManager	53
com.mehdi.petreminder.observer.EventType	58
com.mehdi.petreminder.gui.util.GuiConstants	83
com.mehdi.petreminder.repository.IRepository< T >	89
com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >	21
com.mehdi.petreminder.repository.MySqlRepository< T >	135
com.mehdi.petreminder.repository.SQLiteRepository< T >	231
JDialog	
com.mehdi.petreminder.gui.dialogs.PetDialog	158
com.mehdi.petreminder.gui.dialogs.ReminderDialog	202
JFrame	
com.mehdi.petreminder.gui.MainFrame	93
JPanel	
com.mehdi.petreminder.gui.panels.DashboardPanel	46
com.mehdi.petreminder.gui.panels.MedicalPanel	97
com.mehdi.petreminder.gui.panels.NavPanel	146
com.mehdi.petreminder.gui.panels.PetsPanel	183
com.mehdi.petreminder.gui.panels.RemindersPanel	216
com.mehdi.petreminder.gui.panels.SettingsPanel	230
com.mehdi.petreminder.gui.panels.VetPanel	278
com.mehdi.petreminder.service.MedicalRecordService	115
com.mehdi.petreminder.repository.MySqlRepository< MedicalRecord >	135
com.mehdi.petreminder.repository.MedicalRecordMySqlRepository	111
com.mehdi.petreminder.repository.MySqlRepository< Pet >	135
com.mehdi.petreminder.repository.PetMySqlRepository	160
com.mehdi.petreminder.repository.MySqlRepository< Reminder >	135
com.mehdi.petreminder.repository.ReminderMySqlRepository	204
com.mehdi.petreminder.repository.MySqlRepository< User >	135
com.mehdi.petreminder.repository.UserMySqlRepository	258
com.mehdi.petreminder.petreminder	165
com.mehdi.petreminder.petreminderApp	171
com.mehdi.petreminder.observer.PetReminderObserver	176
com.mehdi.petreminder.service.PetService	177

com.mehdi.petreminder.service.ReminderService	209
com.mehdi.petreminder.repository.RepositoryFactory	225
RuntimeException	
com.mehdi.petreminder.repository.RepositoryException	224
com.mehdi.petreminder.service.ServiceException	228
Serializable	
com.mehdi.petreminder.model.MedicalRecord	99
com.mehdi.petreminder.model.Pet	148
com.mehdi.petreminder.model.Bird	27
com.mehdi.petreminder.model.Cat	33
com.mehdi.petreminder.model.Dog	48
com.mehdi.petreminder.model.Reminder	190
com.mehdi.petreminder.model.ExerciseReminder	61
com.mehdi.petreminder.model.FeedingReminder	68
com.mehdi.petreminder.model.GroomingReminder	76
com.mehdi.petreminder.model.MedicationReminder	126
com.mehdi.petreminder.model.VetAppointment	268
com.mehdi.petreminder.model.User	250
com.mehdi.petreminder.repository.SqliteRepository< MedicalRecord >	231
com.mehdi.petreminder.repository.MedicalRecordSqliteRepository	120
com.mehdi.petreminder.repository.SqliteRepository< Pet >	231
com.mehdi.petreminder.repository.PetSqliteRepository	185
com.mehdi.petreminder.repository.SqliteRepository< Reminder >	231
com.mehdi.petreminder.repository.ReminderSqliteRepository	217
com.mehdi.petreminder.repository.SqliteRepository< User >	231
com.mehdi.petreminder.repository.UserSqliteRepository	263
com.mehdi.petreminder.config.StorageConfig	242
com.mehdi.petreminder.config.StorageType	248

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >	
Generic binary (Object serialization) repository implementasyonu	21
com.mehdi.petreminder.model.Bird	
Kus evcil hayvan sinifi	27
com.mehdi.petreminder.model.Cat	
Kedi evcil hayvan sinifi	33
com.mehdi.petreminder.ConsoleApp	
Console application main loop and menu manager	39
com.mehdi.petreminder.gui.panels.DashboardPanel	
Ana kontrol paneli — özet kartlar ve bekleyen hatırlatıcılar	46
com.mehdi.petreminder.model.Dog	
Kopek evcil hayvan sinifi	48
com.mehdi.petreminder.observer.EventManager	
Observer Pattern Subject — olay yönetici	53
com.mehdi.petreminder.observer.EventType	58
com.mehdi.petreminder.model.ExerciseReminder	
Evcil hayvan egzersiz hatırlatıcısı	61
com.mehdi.petreminder.model.FeedingReminder	
Evcil hayvan besleme hatırlatıcısı	68
com.mehdi.petreminder.model.GroomingReminder	
Evcil hayvan tıraş ve bakım hatırlatıcısı	76
com.mehdi.petreminder.gui.util.Constants	
GUI bileşenlerinde kullanılan sabitler	83
com.mehdi.petreminder.repository.IRepository< T >	
Generic CRUD repository arayüzü	89
com.mehdi.petreminder.gui.MainFrame	
Uygulamanın ana Swing penceresi	93
com.mehdi.petreminder.gui.panels.MedicalPanel	
Sağlık kayıtları CRUD paneli	97
com.mehdi.petreminder.model.MedicalRecord	
Evcil hayvanın sağlık kaydı	99
com.mehdi.petreminder.repository.MedicalRecordMySQLRepository	
MySQL üzerinde MedicalRecord CRUD	111
com.mehdi.petreminder.service.MedicalRecordService	
MedicalRecord CRUD ve iş kuralları	115

com.mehdi.petreminder.repository.MedicalRecordSqliteRepository	
SQLite üzerinde MedicalRecord CRUD	120
com.mehdi.petreminder.model.MedicationReminder	
Evcil hayvan ilaç hatırlatıcısı	126
com.mehdi.petreminder.repository.MySqlRepository< T >	
MySQL tabanlı abstract repository temel sınıfı	135
com.mehdi.petreminder.gui.panels.NavPanel	
Sol navigasyon çubuğu — menü butonları	146
com.mehdi.petreminder.model.Pet	
Evcil hayvanın temel soyut sınıfı	148
com.mehdi.petreminder.gui.dialogs.PetDialog	
Pet ekleme ve düzenleme modal dialog'u	158
com.mehdi.petreminder.repository.PetMySqlRepository	
MySQL üzerinde Pet CRUD işlemleri	160
com.mehdi.petreminder.petreminder	
Pet Care Reminder System yardımcı/facade sınıfı	165
com.mehdi.petreminder.petreminderApp	
Pet Care Reminder System ana sınıfı	171
com.mehdi.petreminder.observer.PetReminderObserver	
Gözlemci arayuzu — durum değişikliklerini dinler	176
com.mehdi.petreminder.service.PetService	
Pet CRUD iş mantığı katmanı	177
com.mehdi.petreminder.gui.panels.PetsPanel	
Evcil hayvanlar listesi ve CRUD işlemleri	183
com.mehdi.petreminder.repository.PetSqliteRepository	
SQLite üzerinde Pet CRUD işlemleri	185
com.mehdi.petreminder.model.Reminder	
Evcil hayvan bakım hatırlatıcısının soyut temel sınıfı	190
com.mehdi.petreminder.gui.dialogs.ReminderDialog	
Reminder ekleme ve düzenleme modal dialog'u	202
com.mehdi.petreminder.repository.ReminderMySqlRepository	
MySQL üzerinde Reminder CRUD işlemleri	204
com.mehdi.petreminder.service.ReminderService	
Reminder CRUD ve iş kuralları	209
com.mehdi.petreminder.gui.panels.RemindersPanel	
Hatırlatıcılar CRUD paneli	216
com.mehdi.petreminder.repository.ReminderSqliteRepository	
SQLite üzerinde Reminder (tüm alt türler) CRUD işlemleri	217
com.mehdi.petreminder.repository.RepositoryException	
Repository işlemlerinde fırlatılan unchecked istisna	224
com.mehdi.petreminder.repository.RepositoryFactory	
Aktif storage backend'e göre repository üretir	225
com.mehdi.petreminder.service.ServiceException	
Servis katmanında fırlatılan unchecked istisna	228
com.mehdi.petreminder.gui.panels.SettingsPanel	
Storage backend ve uygulama ayarları	230
com.mehdi.petreminder.repository.SqliteRepository< T >	
SQLite tabanlı abstract repository temel sınıfı	231
com.mehdi.petreminder.config.StorageConfig	
Aktif storage backend'i yönetir	242
com.mehdi.petreminder.config.StorageType	248
com.mehdi.petreminder.model.User	
Uygulamayı kullanan evcil hayvan sahibi	250
com.mehdi.petreminder.repository.UserMySqlRepository	
MySQL üzerinde User CRUD işlemleri	258
com.mehdi.petreminder.repository.UserSqliteRepository	
SQLite üzerinde User CRUD işlemleri	263

com.mehdi.petreminder.model.VetAppointment	
Veteriner randevusu — Reminder 'dan türetilmiş somut sınıf	268
com.mehdi.petreminder.gui.panels.VetPanel	
Veteriner randevuları CRUD	278

Chapter 5

File Index

5.1 File List

Here is a list of all documented files with brief descriptions:

petreminder-app/src/main/java/com/mehdi/petreminder/ ConsoleApp.java	
Console application main loop	284
petreminder-app/src/main/java/com/mehdi/petreminder/ petreminder.java	
Pet Care Reminder System kütüphane yardımcı sınıfı	346
petreminder-app/src/main/java/com/mehdi/petreminder/ petreminderApp.java	
Pet Care Reminder System ana uygulama giriş noktası	348
petreminder-app/src/main/java/com/mehdi/petreminder/config/ StorageConfig.java	
Storage backend konfigürasyonu — runtime switch destekler	281
petreminder-app/src/main/java/com/mehdi/petreminder/config/ StorageType.java	
Storage backend türleri	283
petreminder-app/src/main/java/com/mehdi/petreminder/gui/ MainFrame.java	
Ana Swing penceresi — tüm panelleri barındırır	300
petreminder-app/src/main/java/com/mehdi/petreminder/gui/dialogs/ PetDialog.java	
Pet ekleme / düzenleme diyalogu	293
petreminder-app/src/main/java/com/mehdi/petreminder/gui/dialogs/ ReminderDialog.java	
Hatırlatıcı ekleme / düzenleme diyalogu	296
petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/ DashboardPanel.java	
Dashboard — özet istatistikler ve son hatırlatıcılar	303
petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/ MedicalPanel.java	
Sağlık kayıtları yönetim paneli	305
petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/ NavPanel.java	
Sol navigasyon paneli	308
petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/ PetsPanel.java	
Evcil hayvanlar yönetim paneli — tam CRUD	310
petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/ RemindersPanel.java	
Hatırlatıcılar yönetim paneli	313
petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/ SettingsPanel.java	
Ayarlar paneli — runtime storage backend değiştirme	316
petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/ VetPanel.java	
Veteriner randevuları paneli	319
petreminder-app/src/main/java/com/mehdi/petreminder/gui/util/ GuiConstants.java	
GUI için renk, font ve boyut sabitleri	321
petreminder-app/src/main/java/com/mehdi/petreminder/model/ Bird.java	
Kus turunu temsil eden somut sınıf	323
petreminder-app/src/main/java/com/mehdi/petreminder/model/ Cat.java	
Kedi turunu temsil eden somut sınıf	324

petreminder-app/src/main/java/com/mehdi/petreminder/model/ Dog.java	
Kopek turunu temsil eden somut sınıf	325
petreminder-app/src/main/java/com/mehdi/petreminder/model/ ExerciseReminder.java	
Egzersiz hatırlatıcısı sınıfı	326
petreminder-app/src/main/java/com/mehdi/petreminder/model/ FeedingReminder.java	
Besleme hatırlatıcısı sınıfı	328
petreminder-app/src/main/java/com/mehdi/petreminder/model/ GroomingReminder.java	
Tıraş/bakım hatırlatıcısı sınıfı	329
petreminder-app/src/main/java/com/mehdi/petreminder/model/ MedicalRecord.java	
Evcil hayvan sağlık kaydı modeli	331
petreminder-app/src/main/java/com/mehdi/petreminder/model/ MedicationReminder.java	
İlaç hatırlatıcısı sınıfı	333
petreminder-app/src/main/java/com/mehdi/petreminder/model/ Pet.java	
Tüm evcil hayvan türleri için soyut temel sınıf	335
petreminder-app/src/main/java/com/mehdi/petreminder/model/ Reminder.java	
Tüm hatırlatıcı türleri için soyut temel sınıf	337
petreminder-app/src/main/java/com/mehdi/petreminder/model/ User.java	
Kullanıcı modeli sınıfı	339
petreminder-app/src/main/java/com/mehdi/petreminder/model/ VetAppointment.java	
Veteriner randevu hatırlatıcısı sınıfı	340
petreminder-app/src/main/java/com/mehdi/petreminder/observer/ EventManager.java	
Observer Pattern — Subject (yayinci) sınıfı	342
petreminder-app/src/main/java/com/mehdi/petreminder/observer/ EventType.java	
Observer Pattern olay türleri	344
petreminder-app/src/main/java/com/mehdi/petreminder/observer/ PetReminderObserver.java	
Observer Design Pattern — gözlemci arayuzu	345
petreminder-app/src/main/java/com/mehdi/petreminder/repository/ BinaryRepository.java	
Binary file I/O repository — ObjectOutputStream / .bin dosyası	350
petreminder-app/src/main/java/com/mehdi/petreminder/repository/ IRepository.java	
Generic repository interface — PDF LO.6 zorunluluğu	352
petreminder-app/src/main/java/com/mehdi/petreminder/repository/ MedicalRecordMySqlRepository.java	
MedicalRecord için MySQL repository	353
petreminder-app/src/main/java/com/mehdi/petreminder/repository/ MedicalRecordSqliteRepository.java	
MedicalRecord için SQLite repository	355
petreminder-app/src/main/java/com/mehdi/petreminder/repository/ MySqlRepository.java	
MySQL JDBC repository — mysql-connector-j + Docker Compose	357
petreminder-app/src/main/java/com/mehdi/petreminder/repository/ PetMySqlRepository.java	
Pet varlığı için MySQL repository implementasyonu	360
petreminder-app/src/main/java/com/mehdi/petreminder/repository/ PetSqliteRepository.java	
Pet varlığı için SQLite repository implementasyonu	362
petreminder-app/src/main/java/com/mehdi/petreminder/repository/ ReminderMySqlRepository.java	
Reminder varlığı için MySQL repository	364
petreminder-app/src/main/java/com/mehdi/petreminder/repository/ ReminderSqliteRepository.java	
Reminder varlığı için SQLite repository	367
petreminder-app/src/main/java/com/mehdi/petreminder/repository/ RepositoryException.java	
Repository katmanı için özel istisna sınıfı	370
petreminder-app/src/main/java/com/mehdi/petreminder/repository/ RepositoryFactory.java	
Repository nesnesi üretici — Factory Pattern	371
petreminder-app/src/main/java/com/mehdi/petreminder/repository/ SqliteRepository.java	
SQLite JDBC repository — org.xerial:sqlite-jdbc	372
petreminder-app/src/main/java/com/mehdi/petreminder/repository/ UserMySqlRepository.java	
User varlığı için MySQL repository	375
petreminder-app/src/main/java/com/mehdi/petreminder/repository/ UserSqliteRepository.java	
User varlığı için SQLite repository	377
petreminder-app/src/main/java/com/mehdi/petreminder/service/ MedicalRecordService.java	
Sağlık kayıtları servis katmanı	379
petreminder-app/src/main/java/com/mehdi/petreminder/service/ PetService.java	
Pet iş mantığı servis katmanı	381

petreminder-app/src/main/java/com/mehdi/petreminder/service/ ReminderService.java	
Reminder iş mantığı servis katmanı	383
petreminder-app/src/main/java/com/mehdi/petreminder/service/ ServiceException.java	
Servis katmanı özel istisna sınıfı	385

Chapter 6

Namespace Documentation

6.1 Package com.mehdi.petreminder.config

Classes

- class [StorageConfig](#)
Aktif storage backend'i yönetir.
- enum [StorageType](#)

6.1.1 Detailed Description

Member documentation.

6.2 Package com.mehdi.petreminder.gui

Packages

- package [dialogs](#)
- package [panels](#)
- package [util](#)

Classes

- class [MainFrame](#)
Uygulamanın ana Swing penceresi.

6.2.1 Detailed Description

Member documentation.

6.3 Package com.mehdi.petreminder.gui.dialogs

Classes

- class [PetDialog](#)
Pet ekleme ve düzenleme modal dialog'u.
- class [ReminderDialog](#)
Reminder ekleme ve düzenleme modal dialog'u.

6.3.1 Detailed Description

Member documentation.

6.4 Package com.mehdi.petreminder.gui.panels

Classes

- class [DashboardPanel](#)
Ana kontrol paneli — özet kartlar ve bekleyen hatırlatıcılar.
- class [MedicalPanel](#)
Sağlık kayıtları CRUD paneli.
- class [NavPanel](#)
Sol navigasyon çubuğu — menü butonları.
- class [PetsPanel](#)
Evcil hayvanlar listesi ve CRUD işlemleri.
- class [RemindersPanel](#)
Hatırlatıcılar CRUD paneli.
- class [SettingsPanel](#)
Storage backend ve uygulama ayarları.
- class [VetPanel](#)
Veteriner randevuları CRUD.

6.4.1 Detailed Description

Member documentation.

6.5 Package com.mehdi.petreminder.gui.util

Classes

- class [GuiConstants](#)
GUI bileşenlerinde kullanılan sabitler.

6.5.1 Detailed Description

Member documentation.

6.6 Package com.mehdi.petreminder.model

Classes

- class [Bird](#)
Kus evcil hayvan sinifi.
- class [Cat](#)
Kedi evcil hayvan sinifi.
- class [Dog](#)
Kopek evcil hayvan sinifi.
- class [ExerciseReminder](#)
Evcil hayvan egzersiz hatırlatıcısı.
- class [FeedingReminder](#)
Evcil hayvan besleme hatırlatıcısı.
- class [GroomingReminder](#)
Evcil hayvan tıraş ve bakım hatırlatıcısı.
- class [MedicalRecord](#)
Evcil hayvanın sağlık kaydı.
- class [MedicationReminder](#)
Evcil hayvan ilaç hatırlatıcısı.
- class [Pet](#)
Evcil hayvanın temel soyut sinifi.
- class [Reminder](#)
Evcil hayvan bakım hatırlatıcısının soyut temel sınıfı.
- class [User](#)
Uygulamayı kullanan evcil hayvan sahibi.
- class [VetAppointment](#)
Veteriner randevusu — [Reminder](#)'dan türetilmiş somut sınıf.

6.6.1 Detailed Description

Member documentation.

6.7 Package com.mehdi.petreminder.observer

Classes

- class [EventManager](#)
Observer Pattern Subject — olay yönetici.
- enum [EventType](#)
- interface [PetReminderObserver](#)
Gözlemci arayuzu — durum değişikliklerini dinler.

6.7.1 Detailed Description

Member documentation.

6.8 Package com.mehdi.petreminder.repository

Classes

- class [BinaryRepository](#)
Generic binary (Object serialization) repository implementasyonu.
- interface [IRepository](#)
Generic CRUD repository arayüzü.
- class [MedicalRecordMySqlRepository](#)
MySQL üzerinde [MedicalRecord](#) CRUD.
- class [MedicalRecordSqliteRepository](#)
SQLite üzerinde [MedicalRecord](#) CRUD.
- class [MySqlRepository](#)
MySQL tabanlı abstract repository temel sınıfı.
- class [PetMySqlRepository](#)
MySQL üzerinde [Pet](#) CRUD işlemleri.
- class [PetSqliteRepository](#)
SQLite üzerinde [Pet](#) CRUD işlemleri.
- class [ReminderMySqlRepository](#)
MySQL üzerinde [Reminder](#) CRUD işlemleri.
- class [ReminderSqliteRepository](#)
SQLite üzerinde [Reminder](#) (tüm alt türler) CRUD işlemleri.
- class [RepositoryException](#)
Repository işlemlerinde fırlatılan unchecked istisna.
- class [RepositoryFactory](#)
Aktif storage backend'e göre repository üretir.
- class [SqliteRepository](#)
SQLite tabanlı abstract repository temel sınıfı.
- class [UserMySqlRepository](#)
MySQL üzerinde [User](#) CRUD işlemleri.
- class [UserSqliteRepository](#)
SQLite üzerinde [User](#) CRUD işlemleri.

6.8.1 Detailed Description

Member documentation.

6.9 Package com.mehdi.petreminder.service

Classes

- class [MedicalRecordService](#)
MedicalRecord CRUD ve iş kuralları.
- class [PetService](#)
Pet CRUD iş mantığı katmanı.
- class [ReminderService](#)
Reminder CRUD ve iş kuralları.
- class [ServiceException](#)
Servis katmanında fırlatılan unchecked istisna.

6.9.1 Detailed Description

Member documentation.

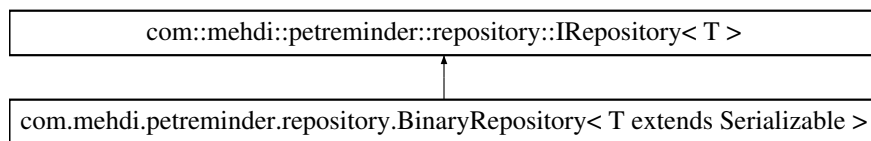
Chapter 7

Class Documentation

7.1 com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable > Class Template Reference

Generic binary (Object serialization) repository implementasyonu.

Inheritance diagram for com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >:



Public Member Functions

- [BinaryRepository](#) (String directoryPath, String entityName)

BinaryRepository yapıcısı.

- int [save](#) (T entity)
- Optional< T > [findById](#) (int id)
- List< T > [findAll](#) ()
- boolean [update](#) (T entity)
- boolean [delete](#) (int id)
- void [deleteAll](#) ()
- int [count](#) ()
- void [close](#) ()
- String [getFilePath](#) ()

Dosya yolunu döndürür (test için).

7.1.1 Detailed Description

Generic binary (Object serialization) repository implementasyonu.

OOP Generics: T extends Serializable. Her varlık türü için ayrı .bin dosyası kullanılır. Thread-safe değil — tek kullanıcı varsayılır.

Parameters

<T>	Serializable varlık türü
-----	--------------------------

Author

Muhammed Mehdi Karagülle
Ibrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 29 of file [BinaryRepository.java](#).

7.1.2 Constructor & Destructor Documentation

7.1.2.1 BinaryRepository()

```
com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >.BinaryRepository (
    String directoryPath,
    String entityName)
```

[BinaryRepository](#) yapıcısı.

Parameters

<i>directoryPath</i>	Dizin yolu (örn. data/binary)
<i>entityName</i>	Varlık adı (örn. pets, reminders)

Definition at line 46 of file [BinaryRepository.java](#).

```
00046                                     {
00047         File dir = new File(directoryPath);
00048         if (!dir.exists()) {
00049             dir.mkdirs();
00050         }
00051         this.filePath = directoryPath + File.separator + entityName + ".bin";
00052         initNextId();
00053     }
```

7.1.3 Member Function Documentation

7.1.3.1 close()

```
void com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >.close ()
```

Repository bağlantısını/kaynaklarını kapatır.

Implements [com.mehdi.petreminder.repository.IRepository< T >](#).

Definition at line 224 of file [BinaryRepository.java](#).

```
00224         {
00225             // Binary repository için kaynak kapatma gerekmez
00226             logger.debug("BinaryRepository kapatıldı.");
00227         }
```

7.1.3.2 count()

```
int com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >.count ()
```

Varlık sayısını döndürür.

Returns

Toplam kayıt sayısı

Implements [com.mehdi.petreminder.repository.IRepository< T >](#).

Definition at line 216 of file [BinaryRepository.java](#).

```
00216         {
00217             return findAll().size();
00218         }
```

References [findAll\(\)](#).

7.1.3.3 delete()

```
boolean com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >.delete (
    int id)
```

ID'ye göre varlık siler.

Parameters

<i>id</i>	Silinecek varlığın ID'si
-----------	--------------------------

Returns

Silme başarılıysa true

Exceptions

RepositoryException	Silme başarısız olursa
-------------------------------------	------------------------

Implements [com.mehdi.petreminder.repository.IRepository< T >](#).

Definition at line 192 of file [BinaryRepository.java](#).

```

00192             {
00193         List<T> all = findAll();
00194         boolean removed = all.removeIf(e -> getEntityId(e) == id);
00195         if (removed) {
00196             writeAll(all);
00197             logger.debug("Silindi: id={}", id);
00198         }
00199         return removed;
00200     }

```

References [findAll\(\)](#).

7.1.3.4 deleteAll()

void [com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >](#).deleteAll ()

Tüm varlıkları siler.

Exceptions

RepositoryException	Silme başarısız olursa
-------------------------------------	------------------------

Implements [com.mehdi.petreminder.repository.IRepository< T >](#).

Definition at line 206 of file [BinaryRepository.java](#).

```

00206             {
00207         writeAll(new ArrayList<>());
00208         nextId = 1;
00209         logger.debug("Tümü silindi: {}", filePath);
00210     }

```

7.1.3.5 findAll()

List< T > [com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >](#).findAll
()

Tüm varlıkları listeler.

Returns

Varlık listesi (boş olabilir)

Exceptions

RepositoryException	Okuma başarısız olursa
-------------------------------------	------------------------

Implements [com.mehdi.petreminder.repository.IRepository< T >](#).

Definition at line 145 of file [BinaryRepository.java](#).

```

00145         {
00146             File file = new File(filePath);
00147             if (!file.exists() || file.length() == 0) {
00148                 return new ArrayList<>();
00149             }
00150             try (FileInputStream fis = new FileInputStream(file);
00151                  ObjectInputStream ois = new ObjectInputStream(fis)) {
00152                 Object obj = ois.readObject();
00153                 if (obj instanceof List) {
00154                     return (List<T>) obj;
00155                 }
00156             } catch (EOFException e) {
00157                 return new ArrayList<>();
00158             } catch (Exception e) {
00159                 logger.error("Binary okuma hatası: {}", e.getMessage());
00160                 throw new RepositoryException("Binary okuma hatası", e);
00161             }
00162             return new ArrayList<>();
00163         }

```

References [findAll\(\)](#).

Referenced by [count\(\)](#), [delete\(\)](#), [findAll\(\)](#), [findById\(\)](#), [save\(\)](#), and [update\(\)](#).

7.1.3.6 findById()

Optional< T > [com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >.findById](#)(
 ById (

int id)

ID'ye göre varlık getirir.

Parameters

<i>id</i>	Varlık ID'si
-----------	--------------

Returns

Varlık (bulunamazsa empty Optional)

Exceptions

RepositoryException	Okuma başarısız olursa
-------------------------------------	------------------------

Implements [com.mehdi.petreminder.repository.IRepository< T >](#).

Definition at line 134 of file [BinaryRepository.java](#).

```

00134         {
00135             return findAll().stream()
00136                 .filter(e -> getEntityId(e) == id)
00137                 .findFirst();
00138         }

```

References [findAll\(\)](#).

7.1.3.7 getFilePath()

String `com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >.getFilePath()`

Dosya yolunu döndürür (test için).

Returns

Dosya yolu

Definition at line 247 of file [BinaryRepository.java](#).

```
00247         {
00248             return filePath;
00249         }
```

7.1.3.8 save()

int `com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >.save (T entity)`

Yeni bir varlık kaydeder.

Parameters

<i>entity</i>	Kaydedilecek varlık
---------------	---------------------

Returns

Oluşturulan varlığın ID'si

Exceptions

<i>RepositoryException</i>	Kayıt başarısız olursa
--	------------------------

Implements `com.mehdi.petreminder.repository.IRepository< T >`.

Definition at line 117 of file [BinaryRepository.java](#).

```
00117         {
00118             if (entity == null) {
00119                 throw new RepositoryException("Entity null olamaz");
00120             }
00121             List<T> all = findAll();
00122             int id = nextId++;
00123             setEntityId(entity, id);
00124             all.add(entity);
00125             writeAll(all);
00126             logger.debug("Kaydedildi: id={}, tip={}", id, entity.getClass().getSimpleName());
00127             return id;
00128         }
```

References [findAll\(\)](#).

7.1.3.9 update()

```
boolean com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >.update (
    T entity)
```

Mevcut varlığı günceller.

Parameters

<i>entity</i>	Güncellenecek varlık
---------------	----------------------

Returns

Güncelleme başarılıysa true

Exceptions

RepositoryException	Güncelleme başarısız olursa
-------------------------------------	-----------------------------

Implements [com.mehdi.petreminder.repository.IRepository< T >](#).

Definition at line 169 of file [BinaryRepository.java](#).

```
00169         {
00170             if (entity == null) return false;
00171             int id = getEntityId(entity);
00172             List<T> all = findAll();
00173             boolean found = false;
00174             for (int i = 0; i < all.size(); i++) {
00175                 if (getEntityId(all.get(i)) == id) {
00176                     all.set(i, entity);
00177                     found = true;
00178                     break;
00179                 }
00180             }
00181             if (found) {
00182                 writeAll(all);
00183                 logger.debug("Güncellendi: id={}", id);
00184             }
00185             return found;
00186         }
```

References [findAll\(\)](#).

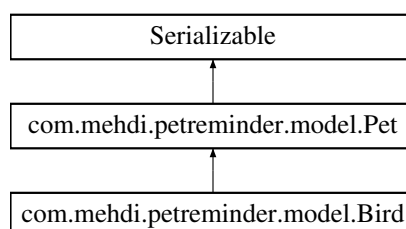
The documentation for this class was generated from the following file:

- petreminder-app/src/main/java/com/mehdi/petreminder/repository/[BinaryRepository.java](#)

7.2 com.mehdi.petreminder.model.Bird Class Reference

Kus evcil hayvan sınıfı.

Inheritance diagram for com.mehdi.petreminder.model.Bird:



Public Member Functions

- [Bird](#) ()
Varsayilan yapici.
- [Bird](#) (int id, String name, LocalDate birthDate, int ownerId)
Temel bilgilerle yapici.
- [Bird](#) (int id, String name, LocalDate birthDate, int ownerId, String gender, double weight, String notes, String birdType, boolean canTalk)
Tam parametreli yapici.
- String [getBirdType](#) ()
Kus turu getter.
- void [setBirdType](#) (String birdType)
Kus turu setter.
- boolean [isCanTalk](#) ()
Konusma getter.
- void [setCanTalk](#) (boolean canTalk)
Konusma setter.
- String [makeSound](#) ()
Kusa ozgu ses.
- String [getIconName](#) ()
GUI ikon adi.
- String [toString](#) ()
String temsili.

Public Member Functions inherited from [com.mehdi.petreminder.model.Pet](#)

- int [getId](#) ()
ID getter.
- void [setId](#) (int id)
ID setter.
- String [getName](#) ()
Isim getter.
- void [setName](#) (String name)
Isim setter.
- String [getSpecies](#) ()
Tur getter.
- void [setSpecies](#) (String species)
Tur setter.
- LocalDate [getBirthDate](#) ()
Dogum tarihi getter.
- void [setBirthDate](#) (LocalDate birthDate)
Dogum tarihi setter.
- int [getOwnerId](#) ()
Sahip ID getter.
- void [setOwnerId](#) (int ownerId)
Sahip ID setter.
- String [getGender](#) ()
Cinsiyet getter.
- void [setGender](#) (String gender)
Cinsiyet setter.

- double [getWeight](#) ()
Agirlik getter.
- void [setWeight](#) (double weight)
Agirlik setter.
- String [getNotes](#) ()
Notlar getter.
- void [setNotes](#) (String notes)
Notlar setter.
- int [getAge](#) ()
Yas hesaplar.
- String [getAgeString](#) ()
Yas bilgisini formatlı string olarak dondurur.
- boolean [equals](#) (Object obj)
İki [Pet](#) nesnesinin eşitliğini karşılaştırır.
- int [hashCode](#) ()
Hash kodu hesaplar.

Additional Inherited Members

Protected Member Functions inherited from [com.mehdi.petreminder.model.Pet](#)

- [Pet](#) ()
Varsayılan yapıcı metod.
- [Pet](#) (int id, String name, String species, LocalDate birthDate, int ownerId, String gender, double weight, String notes)
Tam parametrelili yapıcı metod.

7.2.1 Detailed Description

Kus evcil hayvan sınıfı.

OOP Inheritance: [Pet](#) sınıfından miras alır. OOP Polymorphism: [makeSound\(\)](#) ve [getIconName\(\)](#) override edilir.

Author

Muhammed Mehdi Karagulle, Ibrahim Demirci, Zumre Uykun

Version

1.0

Definition at line 20 of file [Bird.java](#).

7.2.2 Constructor & Destructor Documentation

7.2.2.1 Bird() [1/3]

```
com.mehdi.petreminder.model.Bird.Bird ()
```

Varsayılan yapıcı.

Definition at line 32 of file [Bird.java](#).

```
00032 { super(); }
```

7.2.2.2 Bird() [2/3]

```
com.mehdi.petreminder.model.Bird.Bird (  
    int id,  
    String name,  
    LocalDate birthDate,  
    int ownerId)
```

Temel bilgilerle yapici.

Parameters

<i>id</i>	Kimlik
<i>name</i>	Isim
<i>birthDate</i>	Dogum tarihi
<i>ownerId</i>	Sahip ID

Definition at line 41 of file [Bird.java](#).

```
00041  
00042         super(id, name, "Kus", birthDate, ownerId, "Erkek", 0.0, "");  
00043         this.birdType = "Muhabbet Kusu";  
00044         this.canTalk = false;  
00045     }
```

7.2.2.3 Bird() [3/3]

```
com.mehdi.petreminder.model.Bird.Bird (  
    int id,  
    String name,  
    LocalDate birthDate,  
    int ownerId,  
    String gender,  
    double weight,  
    String notes,  
    String birdType,  
    boolean canTalk)
```

Tam parametrelili yapici.

Parameters

<i>id</i>	Kimlik
<i>name</i>	Isim
<i>birthDate</i>	Dogum tarihi
<i>ownerId</i>	Sahip ID
<i>gender</i>	Cinsiyet
<i>weight</i>	Agirlik
<i>notes</i>	Notlar

<i>birdType</i>	Kus turu
<i>canTalk</i>	Konusabiliyor mu

Definition at line 59 of file [Bird.java](#).

```
00061         {
00062             super(id, name, "Kus", birthDate, ownerId, gender, weight, notes);
00063             this.birdType = birdType;
00064             this.canTalk = canTalk;
00065         }
```

7.2.3 Member Function Documentation

7.2.3.1 getBirdType()

`String com.mehdi.petreminder.model.Bird.getBirdType ()`

Kus turu getter.

Returns

Kus turu

Definition at line 68 of file [Bird.java](#).

```
00068 { return birdType; }
```

7.2.3.2 getIconName()

`String com.mehdi.petreminder.model.Bird.getIconName ()`

GUI ikon adi.

Returns

"bird"

Reimplemented from [com.mehdi.petreminder.model.Pet](#).

Definition at line 93 of file [Bird.java](#).

```
00093 { return "bird"; }
```

7.2.3.3 isCanTalk()

`boolean com.mehdi.petreminder.model.Bird.isCanTalk ()`

Konusma getter.

Returns

Konusabiliyor mu

Definition at line 74 of file [Bird.java](#).

```
00074 { return canTalk; }
```

7.2.3.4 makeSound()

```
String com.mehdi.petreminder.model.Bird.makeSound ()
```

Kusa ozgu ses.

Returns

"Cik Cik!" veya "Merhaba!" (konusabiliyorsa)

Reimplemented from [com.mehdi.petreminder.model.Pet](#).

Definition at line 84 of file [Bird.java](#).

```
00084         {  
00085         return canTalk ? "Merhaba!" : "Cik Cik!";  
00086     }
```

7.2.3.5 setBirdType()

```
void com.mehdi.petreminder.model.Bird.setBirdType (  
    String birdType)
```

Kus turu setter.

Parameters

<i>birdType</i>	Kus turu
-----------------	----------

Definition at line 71 of file [Bird.java](#).

```
00071 { this.birdType = birdType; }
```

Referenced by [com.mehdi.petreminder.repository.PetMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository.PetSqliteRe](#)

7.2.3.6 setCanTalk()

```
void com.mehdi.petreminder.model.Bird.setCanTalk (  
    boolean canTalk)
```

Konusma setter.

Parameters

<i>canTalk</i>	Konusabiliyor mu
----------------	------------------

Definition at line 77 of file [Bird.java](#).

```
00077 { this.canTalk = canTalk; }
```

Referenced by [com.mehdi.petreminder.repository.PetMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository.PetSqliteRe](#)

7.2.3.7 toString()

```
String com.mehdi.petreminder.model.Bird.toString ()
```

String temsili.

Returns

Kus bilgilerini iceren string

Reimplemented from [com.mehdi.petreminder.model.Pet](#).

Definition at line 100 of file [Bird.java](#).

```
00100      {
00101      return String.format("Bird{id=%d, name='%s', type='%s', age=%s, canTalk=%b}",
00102      getId(), getName(), birdType, getAgeString(), canTalk);
00103      }
```

References [com.mehdi.petreminder.model.Pet.getAgeString\(\)](#), [com.mehdi.petreminder.model.Pet.getId\(\)](#), and [com.mehdi.petreminder.model.Pet.getName\(\)](#).

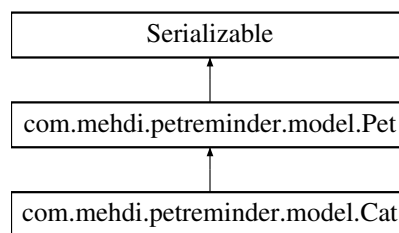
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/model/Bird.java](#)

7.3 com.mehdi.petreminder.model.Cat Class Reference

Kedi evcil hayvan sinifi.

Inheritance diagram for com.mehdi.petreminder.model.Cat:



Public Member Functions

- [Cat \(\)](#)
Varsayilan yapici.
- [Cat \(int id, String name, LocalDate birthDate, int ownerId\)](#)
Temel bilgilerle yapici.
- [Cat \(int id, String name, LocalDate birthDate, int ownerId, String gender, double weight, String notes, String breed, boolean indoor\)](#)
Tam parametreli yapici.
- String [getBreed \(\)](#)
lrk getter.
- void [setBreed \(String breed\)](#)

- lrc setter.*
- boolean `isIndoor` ()
- lc mekan getter.*
- void `setIndoor` (boolean indoor)
- lc mekan setter.*
- String `makeSound` ()
- Kediye ozgu ses.*
- String `getIconName` ()
- GUI ikon adi.*
- String `toString` ()
- String temsili.*

Public Member Functions inherited from `com.mehdi.petreminder.model.Pet`

- int `getId` ()
- ID getter.*
- void `setId` (int id)
- ID setter.*
- String `getName` ()
- Isim getter.*
- void `setName` (String name)
- Isim setter.*
- String `getSpecies` ()
- Tur getter.*
- void `setSpecies` (String species)
- Tur setter.*
- LocalDate `getBirthDate` ()
- Dogum tarihi getter.*
- void `setBirthDate` (LocalDate birthDate)
- Dogum tarihi setter.*
- int `getOwnerId` ()
- Sahip ID getter.*
- void `setOwnerId` (int ownerId)
- Sahip ID setter.*
- String `getGender` ()
- Cinsiyet getter.*
- void `setGender` (String gender)
- Cinsiyet setter.*
- double `getWeight` ()
- Agirlik getter.*
- void `setWeight` (double weight)
- Agirlik setter.*
- String `getNotes` ()
- Notlar getter.*
- void `setNotes` (String notes)
- Notlar setter.*
- int `getAge` ()
- Yas hesaplar.*
- String `getAgeString` ()
- Yas bilgisini formatlı string olarak dondurur.*
- boolean `equals` (Object obj)
- İki Pet nesnesinin eşitliğini karşılaştırır.*
- int `hashCode` ()
- Hash kodu hesaplar.*

Additional Inherited Members

Protected Member Functions inherited from [com.mehdi.petreminder.model.Pet](#)

- [Pet](#) ()
Varsayilan yapici metod.
- [Pet](#) (int id, String name, String species, LocalDate birthDate, int ownerId, String gender, double weight, String notes)
Tam parametrelili yapici metod.

7.3.1 Detailed Description

Kedi evcil hayvan sinifi.

OOP Inheritance: [Pet](#) sinifindan miras alir. OOP Polymorphism: [makeSound\(\)](#) ve [getIconName\(\)](#) override edilir.

Author

Muhammed Mehdi Karagulle, Ibrahim Demirci, Zumre Uykun

Version

1.0

Definition at line 17 of file [Cat.java](#).

7.3.2 Constructor & Destructor Documentation

7.3.2.1 [Cat\(\)](#) [1/3]

```
com.mehdi.petreminder.model.Cat.Cat ()
```

Varsayilan yapici.

Definition at line 29 of file [Cat.java](#).

```
00029 { super(); }
```

7.3.2.2 Cat() [2/3]

```
com.mehdi.petreminder.model.Cat.Cat (
    int id,
    String name,
    LocalDate birthDate,
    int ownerId)
```

Temel bilgilerle yapici.

Parameters

<i>id</i>	Kimlik
<i>name</i>	Isim
<i>birthDate</i>	Dogum tarihi
<i>ownerId</i>	Sahip ID

Definition at line 38 of file [Cat.java](#).

```
00038
00039         super(id, name, "Kedi", birthDate, ownerId, "Disi", 0.0, "");
00040         this.breed = "Karisik";
00041         this.indoor = true;
00042     }
```

7.3.2.3 Cat() [3/3]

```
com.mehdi.petreminder.model.Cat.Cat (
    int id,
    String name,
    LocalDate birthDate,
    int ownerId,
    String gender,
    double weight,
    String notes,
    String breed,
    boolean indoor)
```

Tam parametrelili yapici.

Parameters

<i>id</i>	Kimlik
<i>name</i>	Isim
<i>birthDate</i>	Dogum tarihi
<i>ownerId</i>	Sahip ID
<i>gender</i>	Cinsiyet
<i>weight</i>	Agirlik
<i>notes</i>	Notlar
<i>breed</i>	Irak

<i>indoor</i>	lc mekan mi
---------------	-------------

Definition at line 56 of file [Cat.java](#).

```
00058      {
00059          super(id, name, "Kedi", birthDate, ownerId, gender, weight, notes);
00060          this.breed = breed;
00061          this.indoor = indoor;
00062      }
```

7.3.3 Member Function Documentation

7.3.3.1 getBreed()

String com.mehdi.petreminder.model.Cat.getBreed ()

Irk getter.

Returns

Kedi irki

Definition at line 65 of file [Cat.java](#).

```
00065 { return breed; }
```

7.3.3.2 getIconName()

String com.mehdi.petreminder.model.Cat.getIconName ()

GUI ikon adi.

Returns

"cat"

Reimplemented from [com.mehdi.petreminder.model.Pet](#).

Definition at line 88 of file [Cat.java](#).

```
00088 { return "cat"; }
```

7.3.3.3 isIndoor()

boolean com.mehdi.petreminder.model.Cat.isIndoor ()

Ic mekan getter.

Returns

Ic mekan mi

Definition at line 71 of file [Cat.java](#).

```
00071 { return indoor; }
```

7.3.3.4 makeSound()

```
String com.mehdi.petreminder.model.Cat.makeSound ()
```

Kediye ozgu ses.

Returns

"Miyav!"

Reimplemented from [com.mehdi.petreminder.model.Pet](#).

Definition at line 81 of file [Cat.java](#).

```
00081 { return "Miyav!"; }
```

7.3.3.5 setBreed()

```
void com.mehdi.petreminder.model.Cat.setBreed (  
    String breed)
```

Irk setter.

Parameters

<i>breed</i>	Kedi irki
--------------	-----------

Definition at line 68 of file [Cat.java](#).

```
00068 { this.breed = breed; }
```

Referenced by [com.mehdi.petreminder.repository.PetMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository.PetSqliteRe](#)

7.3.3.6 setIndoor()

```
void com.mehdi.petreminder.model.Cat.setIndoor (  
    boolean indoor)
```

Ic mekan setter.

Parameters

<i>indoor</i>	Ic mekan mi
---------------	-------------

Definition at line 74 of file [Cat.java](#).

```
00074 { this.indoor = indoor; }
```

Referenced by [com.mehdi.petreminder.repository.PetMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository.PetSqliteRe](#)

7.3.3.7 toString()

```
String com.mehdi.petreminder.model.Cat.toString ()
```

String temsili.

Returns

Kedi bilgilerini iceren string

Reimplemented from [com.mehdi.petreminder.model.Pet](#).

Definition at line 95 of file [Cat.java](#).

```
00095     {
00096         return String.format("Cat{id=%d, name='%s', breed='%s', age=%s, indoor=%b}",
00097                               getId(), getName(), breed, getAgeString(), indoor);
00098     }
```

References [com.mehdi.petreminder.model.Pet.getAgeString\(\)](#), [com.mehdi.petreminder.model.Pet.getId\(\)](#), and [com.mehdi.petreminder.model.Pet.getName\(\)](#).

The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/model/Cat.java](#)

7.4 com.mehdi.petreminder.ConsoleApp Class Reference

Console application main loop and menu manager.

Public Member Functions

- [ConsoleApp](#) ()
ConsoleApp method.
- [ConsoleApp](#) (Scanner scanner)
ConsoleApp method.
- void [start](#) ()
start method.
- void [showMainMenu](#) ()
showMainMenu method.
- void [printMainMenu](#) ()
printMainMenu method.
- void [handleMainMenuChoice](#) (String choice)
handleMainMenuChoice method.
- void [showPetsMenu](#) ()
showPetsMenu method.
- void [showRemindersMenu](#) ()
showRemindersMenu method.
- void [showVetMenu](#) ()
showVetMenu method.
- void [showMedicalMenu](#) ()
showMedicalMenu method.

- void [showSettingsMenu](#) ()
showSettingsMenu method.
- void [handleStorageSwitch](#) (String choice)
handleStorageSwitch method.
- void [exitApp](#) ()
exitApp method.
- String [readInput](#) ()
readInput method.
- boolean [isRunning](#) ()
isRunning getter.
- void [setRunning](#) (boolean running)
setRunning setter.

Protected Member Functions

- void [setScannerSource](#) (String input)
setScannerSource method.

7.4.1 Detailed Description

Console application main loop and menu manager.

Fully functional CRUD CLI mode connecting to Service layer.

Definition at line 33 of file [ConsoleApp.java](#).

7.4.2 Constructor & Destructor Documentation

7.4.2.1 ConsoleApp() [1/2]

```
com.mehdi.petreminder.ConsoleApp.ConsoleApp ()
```

[ConsoleApp](#) method.

Definition at line 59 of file [ConsoleApp.java](#).

```
00059         {
00060             this.scanner = new Scanner(System.in);
00061             this.running = false;
00062             initTerminal();
00063             initServices();
00064         }
```

7.4.2.2 ConsoleApp() [2/2]

```
com.mehdi.petreminder.ConsoleApp.ConsoleApp (
    Scanner scanner)
```

[ConsoleApp](#) method.

Definition at line 69 of file [ConsoleApp.java](#).

```
00069         {
00070             this.scanner = scanner;
00071             this.running = false;
00072             initServices();
00073         }
```


7.4.3 Member Function Documentation

7.4.3.1 exitApp()

```
void com.mehdi.petreminder.ConsoleApp.exitApp ()
```

exitApp method.

Definition at line 748 of file [ConsoleApp.java](#).

```
00748         {
00749             running = false;
00750             System.out.println("\nClosing Pet Care Reminder System...");
00751             System.out.println("Goodbye!");
00752         }
```

Referenced by [handleMainMenuChoice\(\)](#).

7.4.3.2 handleMainMenuChoice()

```
void com.mehdi.petreminder.ConsoleApp.handleMainMenuChoice (
    String choice)
```

handleMainMenuChoice method.

Definition at line 141 of file [ConsoleApp.java](#).

```
00141         {
00142             if (choice == null) return;
00143             switch (choice.trim()) {
00144                 case "1": showPetsMenu(); break;
00145                 case "2": showRemindersMenu(); break;
00146                 case "3": showVetMenu(); break;
00147                 case "4": showMedicalMenu(); break;
00148                 case "5": showSettingsMenu(); initServices(); break; // re-init on storage change
00149                 case "0": exitApp(); break;
00150                 default:
00151                     System.out.println("Invalid choice. Please try again.");
00152             }
00153         }
```

References [exitApp\(\)](#), [showMedicalMenu\(\)](#), [showPetsMenu\(\)](#), [showRemindersMenu\(\)](#), [showSettingsMenu\(\)](#), and [showVetMenu\(\)](#).

Referenced by [showMainMenu\(\)](#).

7.4.3.3 handleStorageSwitch()

```
void com.mehdi.petreminder.ConsoleApp.handleStorageSwitch (
    String choice)
```

handleStorageSwitch method.

Definition at line 719 of file [ConsoleApp.java](#).

```
00719         {
00720             if (choice == null) return;
00721             switch (choice.trim()) {
00722                 case "1":
00723                     StorageConfig.setActiveBackend(StorageType.BINARY);
00724                     System.out.println("Storage switched to: Binary File I/O");
00725                     break;
00726                 case "2":
00727                     StorageConfig.setActiveBackend(StorageType.SQLITE);
00728                     System.out.println("Storage switched to: SQLite");
00729                     break;
```

```

00730         case "3":
00731             StorageConfig.setActiveBackend(StorageType.MYSQL);
00732             System.out.println("Storage switched to: MySQL");
00733             System.out.println("NOTE: MySQL requires Docker Compose to be running.");
00734             break;
00735         case "0":
00736             break;
00737         default:
00738             System.out.println("Invalid choice.");
00739     }
00740 }

```

References [com.mehdi.petreminder.config.StorageType.BINARY](#), [com.mehdi.petreminder.config.StorageType.MYSQL](#), [com.mehdi.petreminder.config.StorageConfig.setActiveBackend\(\)](#), and [com.mehdi.petreminder.config.StorageType.SQLite](#).

Referenced by [showSettingsMenu\(\)](#).

7.4.3.4 isRunning()

`boolean com.mehdi.petreminder.ConsoleApp.isRunning ()`

`isRunning` getter.

Returns

boolean running state

Definition at line 833 of file [ConsoleApp.java](#).

```
00833 { return running; }
```

7.4.3.5 printMainMenu()

`void com.mehdi.petreminder.ConsoleApp.printMainMenu ()`

`printMainMenu` method.

Definition at line 124 of file [ConsoleApp.java](#).

```

00124     {
00125         System.out.println("\n=====");
00126         System.out.println("  PET CARE REMINDER SYSTEM - MAIN MENU  ");
00127         System.out.println("=====");
00128         System.out.println("  Pets");
00129         System.out.println("  Reminders");
00130         System.out.println("  Vet Appointments");
00131         System.out.println("  Medical Records");
00132         System.out.println("  Settings");
00133         System.out.println("  Exit");
00134         System.out.println("=====");
00135         System.out.print("Your choice: ");
00136     }

```

7.4.3.6 readInput()

`String com.mehdi.petreminder.ConsoleApp.readInput ()`

`readInput` method.

Definition at line 757 of file [ConsoleApp.java](#).

```

00757     {
00758         try {
00759             if (scanner.hasNextLine()) {
00760                 return scanner.nextLine().trim();
00761             }
00762         } catch (Exception e) {
00763             logger.error("Input reading error: {}", e.getMessage());
00764         }
00765         return "";
00766     }

```

7.4.3.7 setRunning()

```
void com.mehdi.petreminder.ConsoleApp.setRunning (
    boolean running)
```

setRunning setter.

Parameters

<i>running</i>	running state
----------------	---------------

Definition at line 839 of file [ConsoleApp.java](#).

```
00839 { this.running = running; }
```

7.4.3.8 setScannerSource()

```
void com.mehdi.petreminder.ConsoleApp.setScannerSource (
    String input) [protected]
```

setScannerSource method.

Definition at line 772 of file [ConsoleApp.java](#).

```
00772                                     {
00773         // Only used in tests via reflection
00774     }
```

7.4.3.9 showMainMenu()

```
void com.mehdi.petreminder.ConsoleApp.showMainMenu ()
```

showMainMenu method.

Definition at line 110 of file [ConsoleApp.java](#).

```
00110                                     {
00111         while (running) {
00112             String choice = selectMenuOption(
00113                 "\n=====
PET CARE REMINDER SYSTEM - MAIN MENU
\n=====
",
00114                 new String[]{"Pets", "Reminders", "Vet Appointments", "Medical Records", "Settings
(Storage: " + StorageConfig.getActiveBackend().getDisplayName() + ") ", "Exit"},
00115                 new String[]{"1", "2", "3", "4", "5", "0"}
00116             );
00117             handleMainMenuChoice(choice);
00118         }
00119     }
```

References [com.mehdi.petreminder.config.StorageConfig.getActiveBackend\(\)](#), [com.mehdi.petreminder.config.StorageType.getDisplay](#) and [handleMainMenuChoice\(\)](#).

Referenced by [start\(\)](#).

7.4.3.10 showMedicalMenu()

```
void com.mehdi.petreminder.ConsoleApp.showMedicalMenu ()
```

showMedicalMenu method.

Definition at line 630 of file [ConsoleApp.java](#).

```
00630                                     {
00631         while (running) {
00632             String choice = selectMenuOption(
00633                 "\n--- MEDICAL RECORDS ---",
00634                 new String[]{"List Records", "Add New Record", "Back to Main Menu"},
00635                 new String[]{"1", "2", "0"}
00636             );
00637
00638             switch (choice) {
00639                 case "1": listMedicalRecords(); break;
00640                 case "2": addMedicalRecord(); break;
00641                 case "0": return;
00642                 default: System.out.println("Invalid choice.");
00643             }
00644         }
00645     }
```

Referenced by [handleMainMenuChoice\(\)](#).

7.4.3.11 showPetsMenu()

```
void com.mehdi.petreminder.ConsoleApp.showPetsMenu ()
```

showPetsMenu method.

Definition at line 229 of file [ConsoleApp.java](#).

```
00229                                     {
00230         while (running) {
00231             String choice = selectMenuOption(
00232                 "\n--- PETS ---",
00233                 new String[]{"List All Pets", "Add New Pet", "Edit Pet", "Delete Pet", "Back to Main
00234 Menu"},
00235                 new String[]{"1", "2", "3", "4", "0"}
00236             );
00237
00238             switch (choice) {
00239                 case "1": listAllPets(); break;
00240                 case "2": addNewPet(); break;
00241                 case "3": editPet(); break;
00242                 case "4": deletePet(); break;
00243                 case "0": return;
00244                 default: System.out.println("Invalid choice.");
00245             }
00246         }
```

Referenced by [handleMainMenuChoice\(\)](#).

7.4.3.12 showRemindersMenu()

```
void com.mehdi.petreminder.ConsoleApp.showRemindersMenu ()
```

showRemindersMenu method.

Definition at line 395 of file [ConsoleApp.java](#).

```
00395                                     {
00396         while (running) {
00397             String choice = selectMenuOption(
00398                 "\n--- REMINDERS ---",
```

```

00399         new String[]{"List All Pending Reminders", "Add New Reminder", "Mark Reminder as
Completed", "Delete Reminder", "Back to Main Menu"},
00400         new String[]{"1", "2", "3", "4", "0"}
00401     );
00402
00403     switch (choice) {
00404         case "1": listPendingReminders(); break;
00405         case "2": addNewReminder(); break;
00406         case "3": markReminderCompleted(); break;
00407         case "4": deleteReminder(); break;
00408         case "0": return;
00409         default: System.out.println("Invalid choice.");
00410     }
00411 }
00412 }

```

Referenced by [handleMainMenuChoice\(\)](#).

7.4.3.13 showSettingsMenu()

```
void com.mehdi.petreminder.ConsoleApp.showSettingsMenu ()
```

showSettingsMenu method.

Definition at line 707 of file [ConsoleApp.java](#).

```

00707     {
00708         String choice = selectMenuOption(
00709             "\n--- SETTINGS ---\nCurrent Storage: " +
StorageConfig.getActiveBackend().getDisplayName(),
00710         new String[]{"Binary File I/O (.bin)", "SQLite (.db)", "MySQL (Docker required)", "Back"},
00711         new String[]{"1", "2", "3", "0"}
00712     );
00713     handleStorageSwitch(choice);
00714 }

```

References [com.mehdi.petreminder.config.StorageConfig.getActiveBackend\(\)](#), [com.mehdi.petreminder.config.StorageType.getDisplayName\(\)](#) and [handleStorageSwitch\(\)](#).

Referenced by [handleMainMenuChoice\(\)](#).

7.4.3.14 showVetMenu()

```
void com.mehdi.petreminder.ConsoleApp.showVetMenu ()
```

showVetMenu method.

Definition at line 552 of file [ConsoleApp.java](#).

```

00552     {
00553         while (running) {
00554             String choice = selectMenuOption(
00555                 "\n--- VET APPOINTMENTS ---",
00556                 new String[]{"List Vet Appointments", "Add New Appointment", "Back to Main Menu"},
00557                 new String[]{"1", "2", "0"}
00558             );
00559
00560             switch (choice) {
00561                 case "1": listVetAppointments(); break;
00562                 case "2": addVetAppointment(); break;
00563                 case "0": return;
00564                 default: System.out.println("Invalid choice.");
00565             }
00566         }
00567     }

```

Referenced by [handleMainMenuChoice\(\)](#).

7.4.3.15 start()

```
void com.mehdi.petreminder.ConsoleApp.start ()
```

start method.

Definition at line 101 of file [ConsoleApp.java](#).

```
00101      {
00102          this.running = true;
00103          logger.info("ConsoleApp started.");
00104          showMainMenu();
00105      }
```

References [showMainMenu\(\)](#).

Referenced by [com.mehdi.petreminder.petreminderApp.startConsole\(\)](#).

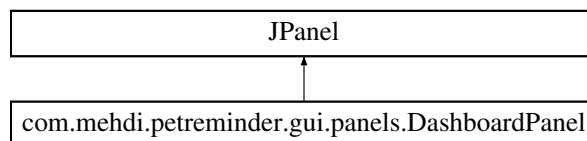
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/ConsoleApp.java](#)

7.5 com.mehdi.petreminder.gui.panels.DashboardPanel Class Reference

Ana kontrol paneli — özet kartlar ve bekleyen hatırlatıcılar.

Inheritance diagram for `com.mehdi.petreminder.gui.panels.DashboardPanel`:



Public Member Functions

- [DashboardPanel](#) ([MainFrame](#) mainFrame)
DashboardPanel yapıcısı.
- void [refresh](#) ()
Verileri yeniler.

7.5.1 Detailed Description

Ana kontrol paneli — özet kartlar ve bekleyen hatırlatıcılar.

Author

Muhammed Mehdi Karagülle
İbrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 30 of file [DashboardPanel.java](#).

7.5.2 Constructor & Destructor Documentation

7.5.2.1 DashboardPanel()

com.mehdi.petreminder.gui.panels.DashboardPanel.DashboardPanel (
[MainFrame](#) *mainFrame*)

[DashboardPanel](#) yapıcısı.

Parameters

<i>mainFrame</i>	Ana pencere
------------------	-------------

Definition at line 61 of file [DashboardPanel.java](#).

```
00061                                     {
00062         this.mainFrame = mainFrame;
00063         petService      = new PetService();
00064         reminderService = new ReminderService();
00065         medicalService  = new MedicalRecordService();
00066         setBackground(GuiConstants.BG_COLOR);
00067         setLayout(new BorderLayout(GuiConstants.GAP, GuiConstants.GAP));
00068         setBorder(new EmptyBorder(GuiConstants.GAP_LARGE, GuiConstants.GAP_LARGE,
00069         GuiConstants.GAP_LARGE, GuiConstants.GAP_LARGE));
00070         buildUI();
00071     }
```

References [com.mehdi.petreminder.gui.util.GuiConstants.BG_COLOR](#), [com.mehdi.petreminder.gui.util.GuiConstants.GAP](#), and [com.mehdi.petreminder.gui.util.GuiConstants.GAP_LARGE](#).

7.5.3 Member Function Documentation

7.5.3.1 refresh()

void com.mehdi.petreminder.gui.panels.DashboardPanel.refresh ()

Verileri yeniler.

Definition at line 177 of file [DashboardPanel.java](#).

```
00177                                     {
00178         try {
00179             lblPetCount.setText(String.valueOf(petService.getPetCount()));
00180             lblReminderCount.setText(String.valueOf(reminderService.getReminderCount()));
00181             List<Reminder> overdue = reminderService.getOverdueReminders();
00182             lblOverdueCount.setText(String.valueOf(overdue.size()));
00183             lblMedicalCount.setText(String.valueOf(medicalService.getRecordCount()));
00184
00185             pendingModel.clear();
00186             List<Reminder> pending = reminderService.getPendingReminders();
00187             for (Reminder r : pending) {
00188                 String item = String.format("[%s] %s -- %s %s",
00189                     r.getPriority(),
00190                     r.getPetName(),
00191                     r.getReminderType(),
00192                     r.getFormattedTime());
00193                 if (r.isOverdue()) item = " " + item;
00194                 pendingModel.addElement(item);
00195             }
00196         } catch (Exception e) {
00197             // Veri yoksa sessizce devam et
00198         }
00199     }
```

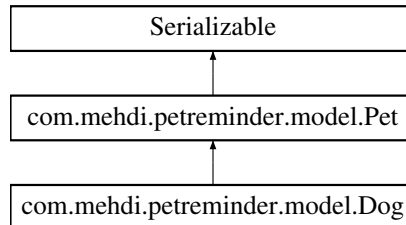
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/DashboardPanel.java](#)

7.6 com.mehdi.petreminder.model.Dog Class Reference

Kopek evcil hayvan sinifi.

Inheritance diagram for com.mehdi.petreminder.model.Dog:



Public Member Functions

- [Dog](#) ()
Varsayilan yapici.
- [Dog](#) (int id, String name, LocalDate birthDate, int ownerId)
Temel bilgilerle yapici.
- [Dog](#) (int id, String name, LocalDate birthDate, int ownerId, String gender, double weight, String notes, String breed, boolean trained)
Tam parametreli yapici.
- String [getBreed](#) ()
Irk getter.
- void [setBreed](#) (String breed)
Irk setter.
- boolean [isTrained](#) ()
Egitim durumu getter.
- void [setTrained](#) (boolean trained)
Egitim durumu setter.
- String [makeSound](#) ()
Kopege ozgu ses.
- String [getIconName](#) ()
GUI ikon adi.
- String [toString](#) ()
String temsili.

Public Member Functions inherited from [com.mehdi.petreminder.model.Pet](#)

- int [getId](#) ()
ID getter.
- void [setId](#) (int id)
ID setter.
- String [getName](#) ()
Isim getter.
- void [setName](#) (String name)
Isim setter.
- String [getSpecies](#) ()

- Tur getter.*
- void [setSpecies](#) (String species)
- Tur setter.*
- LocalDate [getBirthDate](#) ()
- Dogum tarihi getter.*
- void [setBirthDate](#) (LocalDate birthDate)
- Dogum tarihi setter.*
- int [getOwnerId](#) ()
- Sahip ID getter.*
- void [setOwnerId](#) (int ownerId)
- Sahip ID setter.*
- String [getGender](#) ()
- Cinsiyet getter.*
- void [setGender](#) (String gender)
- Cinsiyet setter.*
- double [getWeight](#) ()
- Agirlik getter.*
- void [setWeight](#) (double weight)
- Agirlik setter.*
- String [getNotes](#) ()
- Notlar getter.*
- void [setNotes](#) (String notes)
- Notlar setter.*
- int [getAge](#) ()
- Yas hesaplar.*
- String [getAgeString](#) ()
- Yas bilgisini formatlı string olarak dondurur.*
- boolean [equals](#) (Object obj)
- İki [Pet](#) nesnesinin eşitliğini karşılaştırır.*
- int [hashCode](#) ()
- Hash kodu hesaplar.*

Additional Inherited Members

Protected Member Functions inherited from [com.mehdi.petreminder.model.Pet](#)

- [Pet](#) ()
- Varsayılan yapıcı metod.*
- [Pet](#) (int id, String name, String species, LocalDate birthDate, int ownerId, String gender, double weight, String notes)
- Tam parametrelili yapıcı metod.*

7.6.1 Detailed Description

Kopek evcil hayvan sınıfı.

OOP Inheritance: [Pet](#) sınıfından miras alır. OOP Polymorphism: [makeSound\(\)](#) ve [getIconName\(\)](#) override edilir.

Author

Muhammed Mehdi Karagulle, Ibrahim Demirci, Zumre Uykun

Version

1.0

Definition at line 17 of file [Dog.java](#).

7.6.2 Constructor & Destructor Documentation

7.6.2.1 Dog() [1/3]

```
com.mehdi.petreminder.model.Dog.Dog ()
```

Varsayılan yapıcı.

Definition at line 29 of file [Dog.java](#).

```
00029 { super(); }
```

7.6.2.2 Dog() [2/3]

```
com.mehdi.petreminder.model.Dog.Dog (  
    int id,  
    String name,  
    LocalDate birthDate,  
    int ownerId)
```

Temel bilgilerle yapıcı.

Parameters

<i>id</i>	Kimlik
<i>name</i>	İsim
<i>birthDate</i>	Dogum tarihi
<i>ownerId</i>	Sahip ID

Definition at line 38 of file [Dog.java](#).

```
00038  
00039     super(id, name, "Kopek", birthDate, ownerId, "Erkek", 0.0, "");  
00040     this.breed = "Karisik";  
00041     this.trained = false;  
00042 }
```

7.6.2.3 Dog() [3/3]

```
com.mehdi.petreminder.model.Dog.Dog (  
    int id,  
    String name,  
    LocalDate birthDate,  
    int ownerId,  
    String gender,  
    double weight,  
    String notes,  
    String breed,  
    boolean trained)
```

Tam parametrelili yapıcı.

Parameters

<i>id</i>	Kimlik
<i>name</i>	Isim
<i>birthDate</i>	Dogum tarihi
<i>ownerId</i>	Sahip ID
<i>gender</i>	Cinsiyet
<i>weight</i>	Agirlik
<i>notes</i>	Notlar
<i>breed</i>	Irk
<i>trained</i>	Egitimli mi

Definition at line 56 of file [Dog.java](#).

```
00058      {
00059          super(id, name, "Kopek", birthDate, ownerId, gender, weight, notes);
00060          this.breed = breed;
00061          this.trained = trained;
00062      }
```

7.6.3 Member Function Documentation

7.6.3.1 getBreed()

```
String com.mehdi.petreminder.model.Dog.getBreed ()
```

Irk getter.

Returns

Kopek irki

Definition at line 65 of file [Dog.java](#).

```
00065 { return breed; }
```

7.6.3.2 getIconName()

```
String com.mehdi.petreminder.model.Dog.getIconName ()
```

GUI ikon adi.

Returns

"dog"

Reimplemented from [com.mehdi.petreminder.model.Pet](#).

Definition at line 88 of file [Dog.java](#).

```
00088 { return "dog"; }
```

7.6.3.3 isTrained()

```
boolean com.mehdi.petreminder.model.Dog.isTrained ()
```

Egitim durumu getter.

Returns

Egitimli mi

Definition at line 71 of file [Dog.java](#).

```
00071 { return trained; }
```

7.6.3.4 makeSound()

```
String com.mehdi.petreminder.model.Dog.makeSound ()
```

Kopege ozgu ses.

Returns

"Hav! Hav!"

Reimplemented from [com.mehdi.petreminder.model.Pet](#).

Definition at line 81 of file [Dog.java](#).

```
00081 { return "Hav! Hav!"; }
```

7.6.3.5 setBreed()

```
void com.mehdi.petreminder.model.Dog.setBreed (  
    String breed)
```

Irk setter.

Parameters

<i>breed</i>	Kopek irki
--------------	------------

Definition at line 68 of file [Dog.java](#).

```
00068 { this.breed = breed; }
```

Referenced by [com.mehdi.petreminder.repository.PetMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository.PetSqliteRe](#)

7.6.3.6 setTrained()

```
void com.mehdi.petreminder.model.Dog.setTrained (
    boolean trained)
```

Egitim durumu setter.

Parameters

<i>trained</i>	Egitimli mi
----------------	-------------

Definition at line 74 of file [Dog.java](#).

```
00074 { this.trained = trained; }
```

Referenced by [com.mehdi.petreminder.repository.PetMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository.PetSqliteRepository.mapRow\(\)](#).

7.6.3.7 toString()

```
String com.mehdi.petreminder.model.Dog.toString ()
```

String temsili.

Returns

Kopek bilgilerini iceren string

Reimplemented from [com.mehdi.petreminder.model.Pet](#).

Definition at line 95 of file [Dog.java](#).

```
00095 {
00096     return String.format("Dog{id=%d, name='%s', breed='%s', age=%s, trained=%b}",
00097         getId(), getName(), breed, getAgeString(), trained);
00098 }
```

References [com.mehdi.petreminder.model.Pet.getAgeString\(\)](#), [com.mehdi.petreminder.model.Pet.getId\(\)](#), and [com.mehdi.petreminder.model.Pet.getName\(\)](#).

The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/model/Dog.java](#)

7.7 com.mehdi.petreminder.observer.EventManager Class Reference

Observer Pattern Subject — olay yoneticisi.

Public Member Functions

- void [subscribe](#) ([EventType](#) eventType, [PetReminderObserver](#) observer)
Belirli bir olay turune observer kaydeder.
- void [unsubscribe](#) ([EventType](#) eventType, [PetReminderObserver](#) observer)
Observer'i belirli bir olay turundan cikarir.
- void [unsubscribeAll](#) ([PetReminderObserver](#) observer)
Tum olay turlerinden observer'i cikarir.
- void [notify](#) ([EventType](#) eventType, Object data)
Belirli bir olay turu icin tum observer'lara bildirim gonderir.
- int [getObserverCount](#) ([EventType](#) eventType)
Belirli olay turune kayitli observer sayisini dondurur.
- void [clearAll](#) ()
Tum observer'lari temizler (test icin).

Static Public Member Functions

- static synchronized [EventManager](#) [getInstance](#) ()
Singleton instance dondurur.
- static synchronized void [resetInstance](#) ()
Singleton instance'i sifirlar (test icin).

7.7.1 Detailed Description

Observer Pattern Subject — olay yoneticisi.

PDF zorunlulugu: Observer Design Pattern. Bu sinif "Subject" rolundedir. Observer'lari kayit altina alir ve bir olay gerceklestiginde tum kayitli observer'lara bildirim gonderir. Thread-safe: CopyOnWriteArrayList kullanilir (GUI thread guvenli). Singleton: Uygulama genelinde tek instance.

Author

Muhammed Mehdi Karagulle
Ibrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 37 of file [EventManager.java](#).

7.7.2 Member Function Documentation

7.7.2.1 clearAll()

```
void com.mehdi.petreminder.observer.EventManager.clearAll ()
```

Tum observer'lari temizler (test icin).

Definition at line 153 of file [EventManager.java](#).

```
00153         {
00154             for (EventType type : EventType.values()) {
00155                 listeners.get(type).clear();
00156             }
00157             logger.debug("Tum observer'lar temizlendi.");
00158         }
```

7.7.2.2 getInstance()

```
synchronized EntityManager com.mehdi.petreminder.observer.EventManager.getInstance () [static]
```

Singleton instance dondurur.

Returns

[EventManager](#) tek instance

Definition at line 69 of file [EventManager.java](#).

```
00069                                     {
00070         if (instance == null) {
00071             instance = new EntityManager();
00072         }
00073         return instance;
00074     }
```

Referenced by [com.mehdi.petreminder.service.PetService.addPet\(\)](#), [com.mehdi.petreminder.service.MedicalRecordService.addRecord\(\)](#), [com.mehdi.petreminder.service.ReminderService.addReminder\(\)](#), [com.mehdi.petreminder.service.PetService.deletePet\(\)](#), [com.mehdi.petreminder.service.MedicalRecordService.deleteRecord\(\)](#), [com.mehdi.petreminder.service.ReminderService.deleteReminder\(\)](#), [com.mehdi.petreminder.service.ReminderService.markCompleted\(\)](#), and [com.mehdi.petreminder.service.PetService.updatePet\(\)](#).

7.7.2.3 getObserverCount()

```
int com.mehdi.petreminder.observer.EventManager.getObserverCount (
    EventType eventType)
```

Belirli olay turune kayitli observer sayisini dondurur.

Parameters

<i>eventType</i>	Olay turu
------------------	-----------

Returns

Observer sayisi

Definition at line 145 of file [EventManager.java](#).

```
00145                                     {
00146         if (eventType == null) return 0;
00147         return listeners.get(eventType).size();
00148     }
```

7.7.2.4 notify()

```
void com.mehdi.petreminder.observer.EventManager.notify (
    EventType eventType,
    Object data)
```

Belirli bir olay turu icin tum observer'lara bildirim gonderir.

Parameters

<i>eventType</i>	Gerceklesen olay turu
<i>data</i>	Olayla ilgili veri (null olabilir)

Definition at line 125 of file [EventManager.java](#).

```
00125                                     {
00126     if (eventType == null) return;
00127     List<PetReminderObserver> list = listeners.get(eventType);
00128     logger.info("Olay tetiklendi: {} ({} observer)",
00129         eventType, list.size());
00130     for (PetReminderObserver observer : list) {
00131         try {
00132             observer.onEvent(eventType, data);
00133         } catch (Exception e) {
00134             logger.error("Observer bildirim hatasi: {} - {}",
00135                 observer.getClass().getSimpleName(), e.getMessage());
00136         }
00137     }
00138 }
```

Referenced by [com.mehdi.petreminder.service.PetService.addPet\(\)](#), [com.mehdi.petreminder.service.MedicalRecordService.addRecord\(\)](#), [com.mehdi.petreminder.service.ReminderService.addReminder\(\)](#), [com.mehdi.petreminder.service.PetService.deletePet\(\)](#), [com.mehdi.petreminder.service.MedicalRecordService.deleteRecord\(\)](#), [com.mehdi.petreminder.service.ReminderService.deleteReminder\(\)](#), [com.mehdi.petreminder.service.ReminderService.markCompleted\(\)](#), and [com.mehdi.petreminder.service.PetService.updatePet\(\)](#).

7.7.2.5 resetInstance()

```
synchronized void com.mehdi.petreminder.observer.EventManager.resetInstance () [static]
```

Singleton instance'i sıfırlar (test için).

Definition at line 163 of file [EventManager.java](#).

```
00163                                     {
00164     instance = null;
00165 }
```

7.7.2.6 subscribe()

```
void com.mehdi.petreminder.observer.EventManager.subscribe (
    EventType eventType,
    PetReminderObserver observer)
```

Belirli bir olay turune observer kaydeder.

Parameters

<i>eventType</i>	Dinlenecek olay turu
------------------	----------------------

<i>observer</i>	Kayit edilecek observer
-----------------	-------------------------

Definition at line 81 of file [EventManager.java](#).

```

00081                                     {
00082         if (eventType == null || observer == null) {
00083             logger.warn("subscribe: null parametre, atlaniyor.");
00084             return;
00085         }
00086         List<PetReminderObserver> list = listeners.get(eventType);
00087         if (!list.contains(observer)) {
00088             list.add(observer);
00089             logger.debug("Observer kaydedildi: {} -> {}",
00090                 eventType, observer.getClass().getSimpleName());
00091         }
00092     }

```

7.7.2.7 unsubscribe()

```

void com.mehdi.petreminder.observer.EventManager.unsubscribe (
    EventType eventType,
    PetReminderObserver observer)

```

Observer'i belirli bir olay turundan cikarir.

Parameters

<i>eventType</i>	Olay turu
<i>observer</i>	Cikarilacak observer

Definition at line 99 of file [EventManager.java](#).

```

00099                                     {
00100         if (eventType == null || observer == null) return;
00101         List<PetReminderObserver> list = listeners.get(eventType);
00102         list.remove(observer);
00103         logger.debug("Observer cikarildi: {} -> {}",
00104             eventType, observer.getClass().getSimpleName());
00105     }

```

7.7.2.8 unsubscribeAll()

```

void com.mehdi.petreminder.observer.EventManager.unsubscribeAll (
    PetReminderObserver observer)

```

Tum olay turlerinden observer'i cikarir.

Parameters

<i>observer</i>	Cikarilacak observer
-----------------	----------------------

Definition at line 111 of file [EventManager.java](#).

```

00111                                     {
00112         if (observer == null) return;
00113         for (EventType type : EventType.values()) {
00114             listeners.get(type).remove(observer);
00115         }
00116         logger.debug("Observer tum olaylardan cikarildi: {}",
00117             observer.getClass().getSimpleName());
00118     }

```

The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/observer/EventManager.java](#)

7.8 com.mehdi.petreminder.observer.EventType Enum Reference

Public Member Functions

- [EventType](#) (String description)
Yapici.
- String [getDescription](#) ()
Aciklamayi dondurur.
- String [toString](#) ()
toString override.

Public Attributes

- [STORAGE_CHANGED](#) =("Storage Backend Degisti")
Storage backend degisti (BINARY, SQLite, MySQL).
- [PET_ADDED](#) =("Yeni [Pet](#) Eklendi")
Yeni pet eklendi.
- [PET_DELETED](#) =("Pet Silindi")
[Pet](#) silindi.
- [PET_UPDATED](#) =("Pet Guncellendi")
[Pet](#) guncellendi.
- [REMINDER_ADDED](#) =("Yeni Hatirlatici Eklendi")
Yeni hatirlatici eklendi.
- [REMINDER_COMPLETED](#) =("Hatirlatici Tamamlandi")
Hatirlatici tamamlandi.
- [REMINDER_DELETED](#) =("Hatirlatici Silindi")
Hatirlatici silindi.
- [MEDICAL_RECORD_ADDED](#) =("Saglik Kaydi Eklendi")
Saglik kaydi eklendi.
- [MEDICAL_RECORD_DELETED](#) =("Saglik Kaydi Silindi")
Saglik kaydi silindi.

7.8.1 Detailed Description

Enum documentation.

Definition at line 21 of file [EventType.java](#).

7.8.2 Constructor & Destructor Documentation

7.8.2.1 EventType()

```
com.mehdi.petreminder.observer.EventType.EventType (
    String description)
```

Yapici.

Parameters

<i>description</i>	Olay aciklamasi
--------------------	-----------------

Definition at line 57 of file [EventType.java](#).

```
00057         {
00058             this.description = description;
00059         }
```

7.8.3 Member Function Documentation

7.8.3.1 getDescription()

```
String com.mehdi.petreminder.observer.EventType.getDescription ()
```

Aciklamayi dondurur.

Returns

Olay aciklamasi

Definition at line 65 of file [EventType.java](#).

```
00065         {
00066             return description;
00067         }
```

7.8.3.2 toString()

```
String com.mehdi.petreminder.observer.EventType.toString ()
```

toString override.

Returns

Olay aciklamasi

Definition at line 74 of file [EventType.java](#).

```
00074         {
00075             return description;
00076         }
```

7.8.4 Member Data Documentation

7.8.4.1 MEDICAL_RECORD_ADDED

```
com.mehdi.petreminder.observer.EventType.MEDICAL_RECORD_ADDED = ("Sağlık Kaydı Eklendi")
```

Sağlık kaydı eklendi.

Definition at line 45 of file [EventType.java](#).

Referenced by [com.mehdi.petreminder.service.MedicalRecordService.addRecord\(\)](#).

7.8.4.2 MEDICAL_RECORD_DELETED

```
com.mehdi.petreminder.observer.EventType.MEDICAL_RECORD_DELETED = ("Sağlık Kaydı Silindi")
```

Sağlık kaydı silindi.

Definition at line 48 of file [EventType.java](#).

Referenced by [com.mehdi.petreminder.service.MedicalRecordService.deleteRecord\(\)](#).

7.8.4.3 PET_ADDED

```
com.mehdi.petreminder.observer.EventType.PET_ADDED = ("Yeni Pet Eklendi")
```

Yeni pet eklendi.

Definition at line 27 of file [EventType.java](#).

Referenced by [com.mehdi.petreminder.service.PetService.addPet\(\)](#).

7.8.4.4 PET_DELETED

```
com.mehdi.petreminder.observer.EventType.PET_DELETED = ("Pet Silindi")
```

[Pet](#) silindi.

Definition at line 30 of file [EventType.java](#).

Referenced by [com.mehdi.petreminder.service.PetService.deletePet\(\)](#).

7.8.4.5 PET_UPDATED

```
com.mehdi.petreminder.observer.EventType.PET_UPDATED = ("Pet Güncellendi")
```

[Pet](#) güncellendi.

Definition at line 33 of file [EventType.java](#).

Referenced by [com.mehdi.petreminder.service.PetService.updatePet\(\)](#).

7.8.4.6 REMINDER_ADDED

```
com.mehdi.petreminder.observer.EventType.REMINDER_ADDED = ("Yeni Hatirlatici Eklendi")
```

Yeni hatirlatici eklendi.

Definition at line 36 of file [EventType.java](#).

Referenced by [com.mehdi.petreminder.service.ReminderService.addReminder\(\)](#).

7.8.4.7 REMINDER_COMPLETED

```
com.mehdi.petreminder.observer.EventType.REMINDER_COMPLETED = ("Hatirlatici Tamamlandi")
```

Hatirlatici tamamlandi.

Definition at line 39 of file [EventType.java](#).

Referenced by [com.mehdi.petreminder.service.ReminderService.markCompleted\(\)](#).

7.8.4.8 REMINDER_DELETED

```
com.mehdi.petreminder.observer.EventType.REMINDER_DELETED = ("Hatirlatici Silindi")
```

Hatirlatici silindi.

Definition at line 42 of file [EventType.java](#).

Referenced by [com.mehdi.petreminder.service.ReminderService.deleteReminder\(\)](#).

7.8.4.9 STORAGE_CHANGED

```
com.mehdi.petreminder.observer.EventType.STORAGE_CHANGED = ("Storage Backend Degisti")
```

Storage backend degisti (BINARY, SQLite, MySQL).

Definition at line 24 of file [EventType.java](#).

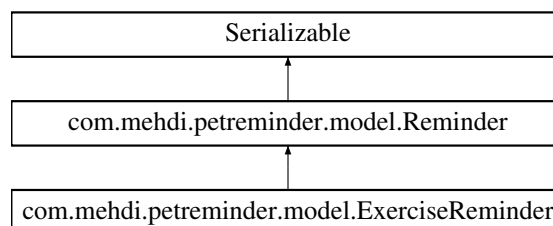
The documentation for this enum was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/observer/EventType.java](#)

7.9 com.mehdi.petreminder.model.ExerciseReminder Class Reference

Evcil hayvan egzersiz hatirlatici.

Inheritance diagram for com.mehdi.petreminder.model.ExerciseReminder:



Public Member Functions

- [ExerciseReminder](#) ()
Varsayılan yapıcı.
- [ExerciseReminder](#) (int id, int petId, String petName, LocalDateTime scheduledTime, String exerciseType, int durationMinutes)
Temel parametrelili yapıcı.
- [ExerciseReminder](#) (int id, int petId, String petName, LocalDateTime scheduledTime, String description, String priority, boolean recurring, String recurrencePattern, String exerciseType, int durationMinutes, double distanceKm)
Tam parametrelili yapıcı.
- String [getExerciseType](#) ()
Egzersiz türü getter.
- void [setExerciseType](#) (String exerciseType)
Egzersiz türü setter.
- int [getDurationMinutes](#) ()
Süre getter.
- void [setDurationMinutes](#) (int durationMinutes)
Süre setter.
- double [getDistanceKm](#) ()
Mesafe getter.
- void [setDistanceKm](#) (double distanceKm)
Mesafe setter.
- String [getReminderType](#) ()
Hatırlatıcı türü.
- boolean [validate](#) ()
Egzersiz hatırlatıcısını doğrular.
- String [getSummary](#) ()
Özet bilgi.
- String [toString](#) ()
String temsili.

Public Member Functions inherited from [com.mehdi.petreminder.model.Reminder](#)

- int [getId](#) ()
ID getter.
- void [setId](#) (int id)
ID setter.
- int [getPetId](#) ()
Pet ID getter.
- void [setPetId](#) (int petId)
Pet ID setter.
- String [getPetName](#) ()
Pet adı getter.
- void [setPetName](#) (String petName)
Pet adı setter.
- LocalDateTime [getScheduledTime](#) ()
Planlanan zaman getter.
- void [setScheduledTime](#) (LocalDateTime scheduledTime)
Planlanan zaman setter.
- boolean [isCompleted](#) ()

- Tamamlandı mı getter.*
- void [setCompleted](#) (boolean completed)
- Tamamlandı setter.*
- String [getDescription](#) ()
- Açıklama getter.*
- void [setDescription](#) (String description)
- Açıklama setter.*
- String [getPriority](#) ()
- Öncelik getter.*
- void [setPriority](#) (String priority)
- Öncelik setter.*
- boolean [isRecurring](#) ()
- Tekrarlıyor mu getter.*
- void [setRecurring](#) (boolean recurring)
- Tekrarlama setter.*
- String [getRecurrencePattern](#) ()
- Tekrar deseni getter.*
- void [setRecurrencePattern](#) (String recurrencePattern)
- Tekrar deseni setter.*
- String [getFormattedTime](#) ()
- Planlanan zamanı biçimli string olarak döndürür.*
- boolean [isOverdue](#) ()
- Hatırlatıcının geçmiş mi olduğunu kontrol eder.*
- boolean [equals](#) (Object obj)
- Eşitlik kontrolü — ID bazlı.*
- int [hashCode](#) ()
- Hash kodu.*

Additional Inherited Members

Protected Member Functions inherited from [com.mehdi.petreminder.model.Reminder](#)

- [Reminder](#) ()
- Varsayılan yapıcı — Serialization için.*
- [Reminder](#) (int id, int petId, String petName, LocalDateTime scheduledTime, String description, String priority, boolean recurring, String recurrencePattern)
- Tam parametrelili yapıcı.*

Static Protected Attributes inherited from [com.mehdi.petreminder.model.Reminder](#)

- static final DateFormatter [FORMATTER](#)
- Tarih saat formatı.*

7.9.1 Detailed Description

Evcil hayvan egzersiz hatırlatıcısı.

OOP Inheritance: [Reminder](#)'dan miras alır.

Author

Muhammed Mehdi Karagülle, İbrahim Demirci, Zümre Uykun

Version

1.0

Definition at line 16 of file [ExerciseReminder.java](#).

7.9.2 Constructor & Destructor Documentation

7.9.2.1 ExerciseReminder() [1/3]

```
com.mehdi.petreminder.model.ExerciseReminder.ExerciseReminder ()
```

Varsayılan yapıcı.

Definition at line 31 of file [ExerciseReminder.java](#).

```
00031 { super(); }
```

7.9.2.2 ExerciseReminder() [2/3]

```
com.mehdi.petreminder.model.ExerciseReminder.ExerciseReminder (
    int id,
    int petId,
    String petName,
    LocalDateTime scheduledTime,
    String exerciseType,
    int durationMinutes)
```

Temel parametrelili yapıcı.

Parameters

<i>id</i>	ID
<i>petId</i>	Pet ID
<i>petName</i>	Pet adı
<i>scheduledTime</i>	Zaman
<i>exerciseType</i>	Egzersiz türü
<i>durationMinutes</i>	Süre (dk)

Definition at line 42 of file [ExerciseReminder.java](#).

```
00044
00045     super(id, petId, petName, scheduledTime,
00046           "Egzersiz: " + exerciseType, "ORTA", false, "");
00047     this.exerciseType = exerciseType;
00048     this.durationMinutes = durationMinutes;
00049     this.distanceKm = 0.0;
00050 }
```


7.9.2.3 ExerciseReminder() [3/3]

```
com.mehdi.petreminder.model.ExerciseReminder.ExerciseReminder (
    int id,
    int petId,
    String petName,
    LocalDateTime scheduledTime,
    String description,
    String priority,
    boolean recurring,
    String recurrencePattern,
    String exerciseType,
    int durationMinutes,
    double distanceKm)
```

Tam parametrelili yapıcı.

Parameters

<i>id</i>	ID
<i>petId</i>	Pet ID
<i>petName</i>	Pet adı
<i>scheduledTime</i>	Zaman
<i>description</i>	Açıklama
<i>priority</i>	Öncelik
<i>recurring</i>	Tekrarlıyor mu
<i>recurrencePattern</i>	Tekrar deseni
<i>exerciseType</i>	Egzersiz türü
<i>durationMinutes</i>	Süre
<i>distanceKm</i>	Mesafe (km)

Definition at line 66 of file [ExerciseReminder.java](#).

```
00070
00071         super(id, petId, petName, scheduledTime, description,
00072               priority, recurring, recurrencePattern);
00073         this.exerciseType = exerciseType;
00074         this.durationMinutes = durationMinutes;
00075         this.distanceKm = distanceKm;
00076     }
```

7.9.3 Member Function Documentation

7.9.3.1 getDistanceKm()

```
double com.mehdi.petreminder.model.ExerciseReminder.getDistanceKm ()
```

Mesafe getter.

Returns

Mesafe (km)

Definition at line 93 of file [ExerciseReminder.java](#).

```
00093 { return distanceKm; }
```

7.9.3.2 getDurationMinutes()

```
int com.mehdi.petreminder.model.ExerciseReminder.getDurationMinutes ()
```

Süre getter.

Returns

Süre (dk)

Definition at line 85 of file [ExerciseReminder.java](#).

```
00085 { return durationMinutes; }
```

7.9.3.3 getExerciseType()

```
String com.mehdi.petreminder.model.ExerciseReminder.getExerciseType ()
```

Egzersiz türü getter.

Returns

Egzersiz türü

Definition at line 79 of file [ExerciseReminder.java](#).

```
00079 { return exerciseType; }
```

7.9.3.4 getReminderType()

```
String com.mehdi.petreminder.model.ExerciseReminder.getReminderType ()
```

Hatırlatıcı türü.

Returns

"Egzersiz"

Reimplemented from [com.mehdi.petreminder.model.Reminder](#).

Definition at line 100 of file [ExerciseReminder.java](#).

```
00100 { return "Egzersiz"; }
```

7.9.3.5 getSummary()

```
String com.mehdi.petreminder.model.ExerciseReminder.getSummary ()
```

Özet bilgi.

Returns

"Egzersiz: Yürüyüş 30dk"

Reimplemented from [com.mehdi.petreminder.model.Reminder](#).

Definition at line 117 of file [ExerciseReminder.java](#).

```
00117 {
00118     String s = String.format("Egzersiz: %s %d dk", exerciseType, durationMinutes);
00119     if (distanceKm > 0) s += String.format(" (%.1f km)", distanceKm);
00120     return s;
00121 }
```

7.9.3.6 setDistanceKm()

```
void com.mehdi.petreminder.model.ExerciseReminder.setDistanceKm (
    double distanceKm)
```

Mesafe setter.

Parameters

<i>distanceKm</i>	Mesafe (km)
-------------------	-------------

Definition at line 96 of file [ExerciseReminder.java](#).

```
00096 { this.distanceKm = distanceKm; }
```

7.9.3.7 setDurationMinutes()

```
void com.mehdi.petreminder.model.ExerciseReminder.setDurationMinutes (
    int durationMinutes)
```

Süre setter.

Parameters

<i>durationMinutes</i>	Süre (dk)
------------------------	-----------

Definition at line 88 of file [ExerciseReminder.java](#).

```
00088                                     {
00089     this.durationMinutes = durationMinutes;
00090 }
```

7.9.3.8 setExerciseType()

```
void com.mehdi.petreminder.model.ExerciseReminder.setExerciseType (
    String exerciseType)
```

Egzersiz türü setter.

Parameters

<i>exerciseType</i>	Egzersiz türü
---------------------	---------------

Definition at line 82 of file [ExerciseReminder.java](#).

```
00082 { this.exerciseType = exerciseType; }
```

7.9.3.9 toString()

String com.mehdi.petreminder.model.ExerciseReminder.toString ()

String temsili.

Returns

Egzersiz hatırlatıcısı bilgisi

Reimplemented from [com.mehdi.petreminder.model.Reminder](#).

Definition at line 125 of file [ExerciseReminder.java](#).

```
00125         {
00126             return String.format("ExerciseReminder{id=%d, pet='%s', type='%s', %ddk}",
00127                                 getId(), getPetName(), exerciseType, durationMinutes);
00128         }
```

References [com.mehdi.petreminder.model.Reminder.getId\(\)](#), and [com.mehdi.petreminder.model.Reminder.getPetName\(\)](#).

7.9.3.10 validate()

boolean com.mehdi.petreminder.model.ExerciseReminder.validate ()

Egzersiz hatırlatıcısını doğrular.

Returns

Egzersiz türü ve süre geçerliyse true

Reimplemented from [com.mehdi.petreminder.model.Reminder](#).

Definition at line 107 of file [ExerciseReminder.java](#).

```
00107         {
00108             return exerciseType != null && !exerciseType.trim().isEmpty()
00109                    && durationMinutes > 0;
00110         }
```

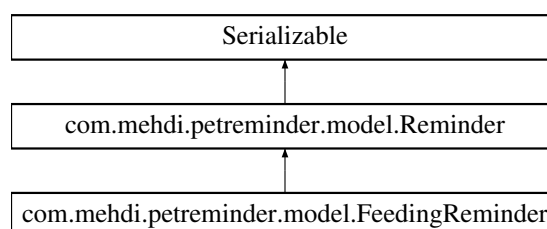
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/model/ExerciseReminder.java](#)

7.10 com.mehdi.petreminder.model.FeedingReminder Class Reference

Evcil hayvan besleme hatırlatıcısı.

Inheritance diagram for com.mehdi.petreminder.model.FeedingReminder:



Public Member Functions

- [FeedingReminder](#) ()
Varsayılan yapıcı.
- [FeedingReminder](#) (int id, int petId, String petName, LocalDateTime scheduledTime, String foodType, double portionGrams)
Temel parametrelili yapıcı.
- [FeedingReminder](#) (int id, int petId, String petName, LocalDateTime scheduledTime, String description, String priority, boolean recurring, String recurrencePattern, String foodType, double portionGrams, boolean includesWaterChange)
Tam parametrelili yapıcı.
- String [getFoodType](#) ()
Besin türü getter.
- void [setFoodType](#) (String foodType)
Besin türü setter.
- double [getPortionGrams](#) ()
Porsiyon getter.
- void [setPortionGrams](#) (double portionGrams)
Porsiyon setter.
- boolean [isIncludesWaterChange](#) ()
Su değişimi getter.
- void [setIncludesWaterChange](#) (boolean includesWaterChange)
Su değişimi setter.
- String [getReminderType](#) ()
Hatırlatıcı türü.
- boolean [validate](#) ()
Besleme hatırlatıcısını doğrular.
- String [getSummary](#) ()
Özet bilgi.
- String [toString](#) ()
String temsili.

Public Member Functions inherited from [com.mehdi.petreminder.model.Reminder](#)

- int [getId](#) ()
ID getter.
- void [setId](#) (int id)
ID setter.
- int [getPetId](#) ()
Pet ID getter.
- void [setPetId](#) (int petId)
Pet ID setter.
- String [getPetName](#) ()
Pet adı getter.
- void [setPetName](#) (String petName)
Pet adı setter.
- LocalDateTime [getScheduledTime](#) ()
Planlanan zaman getter.
- void [setScheduledTime](#) (LocalDateTime scheduledTime)
Planlanan zaman setter.
- boolean [isCompleted](#) ()

- *Tamamlandı mı getter.*
- void [setCompleted](#) (boolean completed)
- *Tamamlandı setter.*
- String [getDescription](#) ()
- *Açıklama getter.*
- void [setDescription](#) (String description)
- *Açıklama setter.*
- String [getPriority](#) ()
- *Öncelik getter.*
- void [setPriority](#) (String priority)
- *Öncelik setter.*
- boolean [isRecurring](#) ()
- *Tekrarlıyor mu getter.*
- void [setRecurring](#) (boolean recurring)
- *Tekrarlama setter.*
- String [getRecurrencePattern](#) ()
- *Tekrar deseni getter.*
- void [setRecurrencePattern](#) (String recurrencePattern)
- *Tekrar deseni setter.*
- String [getFormattedTime](#) ()
- *Planlanan zamanı biçimli string olarak döndürür.*
- boolean [isOverdue](#) ()
- *Hatırlatıcının geçmiş mi olduğunu kontrol eder.*
- boolean [equals](#) (Object obj)
- *Eşitlik kontrolü — ID bazlı.*
- int [hashCode](#) ()
- *Hash kodu.*

Additional Inherited Members

Protected Member Functions inherited from [com.mehdi.petreminder.model.Reminder](#)

- [Reminder](#) ()
- *Varsayılan yapıcı — Serialization için.*
- [Reminder](#) (int id, int petId, String petName, LocalDateTime scheduledTime, String description, String priority, boolean recurring, String recurrencePattern)
- *Tam parametrelili yapıcı.*

Static Protected Attributes inherited from [com.mehdi.petreminder.model.Reminder](#)

- static final DateFormatter [FORMATTER](#)
- *Tarih saat formatı.*

7.10.1 Detailed Description

Evcil hayvan besleme hatırlatıcısı.

OOP Inheritance: [Reminder](#)'dan miras alır. OOP Polymorphism: [getReminderType\(\)](#), [validate\(\)](#), [getSummary\(\)](#) override.

Author

Muhammed Mehdi Karagülle, İbrahim Demirci, Zümre Uykun

Version

1.0

Definition at line 17 of file [FeedingReminder.java](#).

7.10.2 Constructor & Destructor Documentation

7.10.2.1 FeedingReminder() [1/3]

```
com.mehdi.petreminder.model.FeedingReminder.FeedingReminder ()
```

Varsayılan yapıcı.

Definition at line 32 of file [FeedingReminder.java](#).

```
00032 { super(); }
```

7.10.2.2 FeedingReminder() [2/3]

```
com.mehdi.petreminder.model.FeedingReminder.FeedingReminder (  
    int id,  
    int petId,  
    String petName,  
    LocalDateTime scheduledTime,  
    String foodType,  
    double portionGrams)
```

Temel parametrelili yapıcı.

Parameters

<i>id</i>	ID
<i>petId</i>	Pet ID
<i>petName</i>	Pet adı
<i>scheduledTime</i>	Zaman
<i>foodType</i>	Besin türü

<i>portionGrams</i>	Porsiyon (gram)
---------------------	-----------------

Definition at line 43 of file [FeedingReminder.java](#).

```

00045
00046         super(id, petId, petName, scheduledTime,
00047             "Besleme: " + foodType, "ORTA", false, "");
00048         this.foodType = foodType;
00049         this.portionGrams = portionGrams;
00050         this.includesWaterChange = false;
00051     }

```

7.10.2.3 FeedingReminder() [3/3]

```

com.mehdi.petreminder.model.FeedingReminder.FeedingReminder (
    int id,
    int petId,
    String petName,
    LocalDateTime scheduledTime,
    String description,
    String priority,
    boolean recurring,
    String recurrencePattern,
    String foodType,
    double portionGrams,
    boolean includesWaterChange)

```

Tam parametrelili yapıcı.

Parameters

<i>id</i>	ID
<i>petId</i>	Pet ID
<i>petName</i>	Pet adı
<i>scheduledTime</i>	Zaman
<i>description</i>	Açıklama
<i>priority</i>	Öncelik
<i>recurring</i>	Tekrarlıyor mu
<i>recurrencePattern</i>	Tekrar deseni
<i>foodType</i>	Besin türü
<i>portionGrams</i>	Porsiyon (gram)
<i>includesWaterChange</i>	Su değişimi dahil mi

Definition at line 67 of file [FeedingReminder.java](#).

```

00071
00072         super(id, petId, petName, scheduledTime, description,
00073             priority, recurring, recurrencePattern);
00074         this.foodType = foodType;
00075         this.portionGrams = portionGrams;
00076         this.includesWaterChange = includesWaterChange;
00077     }

```


7.10.3 Member Function Documentation

7.10.3.1 getFoodType()

```
String com.mehdi.petreminder.model.FeedingReminder.getFoodType ()
```

Besin türü getter.

Returns

Besin türü

Definition at line 80 of file [FeedingReminder.java](#).

```
00080 { return foodType; }
```

7.10.3.2 getPortionGrams()

```
double com.mehdi.petreminder.model.FeedingReminder.getPortionGrams ()
```

Porsiyon getter.

Returns

Porsiyon (gram)

Definition at line 86 of file [FeedingReminder.java](#).

```
00086 { return portionGrams; }
```

7.10.3.3 getReminderType()

```
String com.mehdi.petreminder.model.FeedingReminder.getReminderType ()
```

Hatırlatıcı türü.

Returns

"Besleme"

Reimplemented from [com.mehdi.petreminder.model.Reminder](#).

Definition at line 104 of file [FeedingReminder.java](#).

```
00104 { return "Besleme"; }
```

7.10.3.4 getSummary()

```
String com.mehdi.petreminder.model.FeedingReminder.getSummary ()
```

Özet bilgi.

Returns

"Besleme: Kuru Mama (150g)"

Reimplemented from [com.mehdi.petreminder.model.Reminder](#).

Definition at line 120 of file [FeedingReminder.java](#).

```
00120         {
00121             return String.format("Besleme: %s (%.0fg)%s",
00122                 foodType, portionGrams,
00123                 includesWaterChange ? " + Su değişimi" : "");
00124         }
```

7.10.3.5 isIncludesWaterChange()

```
boolean com.mehdi.petreminder.model.FeedingReminder.isIncludesWaterChange ()
```

Su değişimi getter.

Returns

Su değişimi dahil mi

Definition at line 92 of file [FeedingReminder.java](#).

```
00092 { return includesWaterChange; }
```

7.10.3.6 setFoodType()

```
void com.mehdi.petreminder.model.FeedingReminder.setFoodType (
    String foodType)
```

Besin türü setter.

Parameters

<i>foodType</i>	Besin türü
-----------------	------------

Definition at line 83 of file [FeedingReminder.java](#).

```
00083 { this.foodType = foodType; }
```

7.10.3.7 setIncludesWaterChange()

```
void com.mehdi.petreminder.model.FeedingReminder.setIncludesWaterChange (
    boolean includesWaterChange)
```

Su değişimi setter.

Parameters

<i>includesWaterChange</i>	Su değişimi
----------------------------	-------------

Definition at line 95 of file [FeedingReminder.java](#).

```
00095                                     {
00096         this.includesWaterChange = includesWaterChange;
00097     }
```

7.10.3.8 setPortionGrams()

```
void com.mehdi.petreminder.model.FeedingReminder.setPortionGrams (
    double portionGrams)
```

Porsiyon setter.

Parameters

<i>portionGrams</i>	Porsiyon (gram)
---------------------	-----------------

Definition at line 89 of file [FeedingReminder.java](#).

```
00089 { this.portionGrams = portionGrams; }
```

7.10.3.9 toString()

```
String com.mehdi.petreminder.model.FeedingReminder.toString ()
```

String temsili.

Returns

Besleme hatırlatıcısı bilgisi

Reimplemented from [com.mehdi.petreminder.model.Reminder](#).

Definition at line 131 of file [FeedingReminder.java](#).

```
00131     {
00132         return String.format("FeedingReminder{id=%d, pet='%s', food='%s', portion=%.0fg}",
00133             getId(), getPetName(), foodType, portionGrams);
00134     }
```

References [com.mehdi.petreminder.model.Reminder.getId\(\)](#), and [com.mehdi.petreminder.model.Reminder.getPetName\(\)](#).

7.10.3.10 validate()

```
boolean com.mehdi.petreminder.model.FeedingReminder.validate ()
```

Besleme hatırlatıcısını doğrular.

Returns

foodType boş değilse ve porsiyon > 0 ise true

Reimplemented from [com.mehdi.petreminder.model.Reminder](#).

Definition at line 111 of file [FeedingReminder.java](#).

```
00111         {
00112         return foodType != null && !foodType.trim().isEmpty() && portionGrams > 0;
00113     }
```

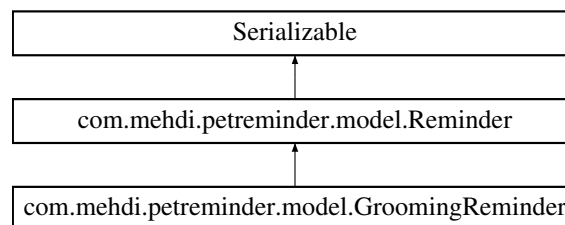
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/model/FeedingReminder.java](#)

7.11 com.mehdi.petreminder.model.GroomingReminder Class Reference

Evcil hayvan tıraş ve bakım hatırlatıcısı.

Inheritance diagram for com.mehdi.petreminder.model.GroomingReminder:



Public Member Functions

- [GroomingReminder \(\)](#)
Varsayılan yapıcı.
- [GroomingReminder \(int id, int petId, String petName, LocalDateTime scheduledTime, String groomingType, boolean professional\)](#)
Temel parametrelili yapıcı.
- [GroomingReminder \(int id, int petId, String petName, LocalDateTime scheduledTime, String description, String priority, boolean recurring, String recurrencePattern, String groomingType, boolean professional, String groomerName\)](#)
Tam parametrelili yapıcı.
- String [getGroomingType \(\)](#)
Bakım türü getter.
- void [setGroomingType \(String groomingType\)](#)
Bakım türü setter.
- boolean [isProfessional \(\)](#)

- Profesyonel mi getter.*
- void [setProfessional](#) (boolean professional)
- Profesyonel setter.*
- String [getGroomerName](#) ()
- Groomer adı getter.*
- void [setGroomerName](#) (String groomerName)
- Groomer adı setter.*
- String [getReminderType](#) ()
- Hatırlatıcı türü.*
- boolean [validate](#) ()
- Bakım hatırlatıcısını doğrular.*
- String [getSummary](#) ()
- Özet bilgi.*
- String [toString](#) ()
- String temsili.*

Public Member Functions inherited from [com.mehdi.petreminder.model.Reminder](#)

- int [getId](#) ()
- ID getter.*
- void [setId](#) (int id)
- ID setter.*
- int [getPetId](#) ()
- Pet ID getter.*
- void [setPetId](#) (int petId)
- Pet ID setter.*
- String [getPetName](#) ()
- Pet adı getter.*
- void [setPetName](#) (String petName)
- Pet adı setter.*
- LocalDateTime [getScheduledTime](#) ()
- Planlanan zaman getter.*
- void [setScheduledTime](#) (LocalDateTime scheduledTime)
- Planlanan zaman setter.*
- boolean [isCompleted](#) ()
- Tamamlandı mı getter.*
- void [setCompleted](#) (boolean completed)
- Tamamlandı setter.*
- String [getDescription](#) ()
- Açıklama getter.*
- void [setDescription](#) (String description)
- Açıklama setter.*
- String [getPriority](#) ()
- Öncelik getter.*
- void [setPriority](#) (String priority)
- Öncelik setter.*
- boolean [isRecurring](#) ()
- Tekrarlıyor mu getter.*
- void [setRecurring](#) (boolean recurring)
- Tekrarlama setter.*

- String [getRecurrencePattern](#) ()
Tekrar deseni getter.
- void [setRecurrencePattern](#) (String recurrencePattern)
Tekrar deseni setter.
- String [getFormattedTime](#) ()
Planlanan zamanı biçimli string olarak döndürür.
- boolean [isOverdue](#) ()
Hatırlatıcının geçmiş mi olduğunu kontrol eder.
- boolean [equals](#) (Object obj)
Eşitlik kontrolü — ID bazlı.
- int [hashCode](#) ()
Hash kodu.

Additional Inherited Members

Protected Member Functions inherited from [com.mehdi.petreminder.model.Reminder](#)

- [Reminder](#) ()
Varsayılan yapıcı — Serialization için.
- [Reminder](#) (int id, int petId, String petName, LocalDateTime scheduledTime, String description, String priority, boolean recurring, String recurrencePattern)
Tam parametrelili yapıcı.

Static Protected Attributes inherited from [com.mehdi.petreminder.model.Reminder](#)

- static final DateTimeFormatter [FORMATTER](#)
Tarih saat formatı.

7.11.1 Detailed Description

Evcil hayvan tıraş ve bakım hatırlatıcısı.

OOP Inheritance: [Reminder](#)'dan miras alır.

Author

Muhammed Mehdi Karagülle, İbrahim Demirci, Zümre Uykun

Version

1.0

Definition at line 16 of file [GroomingReminder.java](#).

7.11.2 Constructor & Destructor Documentation

7.11.2.1 GroomingReminder() [1/3]

com.mehdi.petreminder.model.GroomingReminder.GroomingReminder ()

Varsayılan yapıcı.

Definition at line 31 of file [GroomingReminder.java](#).

```
00031 { super(); }
```

7.11.2.2 GroomingReminder() [2/3]

```
com.mehdi.petreminder.model.GroomingReminder.GroomingReminder (
    int id,
    int petId,
    String petName,
    LocalDateTime scheduledTime,
    String groomingType,
    boolean professional)
```

Temel parametrelili yapıcı.

Parameters

<i>id</i>	ID
<i>petId</i>	Pet ID
<i>petName</i>	Pet adı
<i>scheduledTime</i>	Zaman
<i>groomingType</i>	Bakım türü
<i>professional</i>	Profesyonel mi

Definition at line 42 of file [GroomingReminder.java](#).

```
00044
00045         super(id, petId, petName, scheduledTime,
00046             "Bakim: " + groomingType, "DUSUK", false, "");
00047         this.groomingType = groomingType;
00048         this.professional = professional;
00049         this.groomerName = "";
00050     }
```

7.11.2.3 GroomingReminder() [3/3]

```
com.mehdi.petreminder.model.GroomingReminder.GroomingReminder (
    int id,
    int petId,
    String petName,
    LocalDateTime scheduledTime,
    String description,
    String priority,
    boolean recurring,
```

```
String recurrencePattern,
String groomingType,
boolean professional,
String groomerName)
```

Tam parametrelili yapıcı.

Parameters

<i>id</i>	ID
<i>petId</i>	Pet ID
<i>petName</i>	Pet adı
<i>scheduledTime</i>	Zaman
<i>description</i>	Açıklama
<i>priority</i>	Öncelik
<i>recurring</i>	Tekrarlıyor mu
<i>recurrencePattern</i>	Tekrar deseni
<i>groomingType</i>	Bakım türü
<i>professional</i>	Profesyonel mi
<i>groomerName</i>	Groomer adı

Definition at line 66 of file [GroomingReminder.java](#).

```
00070
00071         super(id, petId, petName, scheduledTime, description,
00072             priority, recurring, recurrencePattern);
00073         this.groomingType = groomingType;
00074         this.professional = professional;
00075         this.groomerName = groomerName;
00076     }
```

7.11.3 Member Function Documentation

7.11.3.1 getGroomerName()

```
String com.mehdi.petreminder.model.GroomingReminder.getGroomerName ()
```

Groomer adı getter.

Returns

Groomer adı

Definition at line 91 of file [GroomingReminder.java](#).

```
00091 { return groomerName; }
```

7.11.3.2 getGroomingType()

```
String com.mehdi.petreminder.model.GroomingReminder.getGroomingType ()
```

Bakım türü getter.

Returns

Bakım türü

Definition at line 79 of file [GroomingReminder.java](#).

```
00079 { return groomingType; }
```


7.11.3.3 getReminderType()

```
String com.mehdi.petreminder.model.GroomingReminder.getReminderType ()
```

Hatırlatıcı türü.

Returns

"Bakim"

Reimplemented from [com.mehdi.petreminder.model.Reminder](#).

Definition at line 98 of file [GroomingReminder.java](#).

```
00098 { return "Bakim"; }
```

7.11.3.4 getSummary()

```
String com.mehdi.petreminder.model.GroomingReminder.getSummary ()
```

Özet bilgi.

Returns

"Bakım: Tıraş (Profesyonel)"

Reimplemented from [com.mehdi.petreminder.model.Reminder](#).

Definition at line 114 of file [GroomingReminder.java](#).

```
00114         {
00115             return String.format("Bakım: %s%s%s",
00116                 groomingType,
00117                 professional ? " (Profesyonel)" : "",
00118                 groomerName != null && !groomerName.isEmpty() ? " - " + groomerName : "");
00119         }
```

7.11.3.5 isProfessional()

```
boolean com.mehdi.petreminder.model.GroomingReminder.isProfessional ()
```

Profesyonel mi getter.

Returns

Profesyonel mi

Definition at line 85 of file [GroomingReminder.java](#).

```
00085 { return professional; }
```

7.11.3.6 setGroomerName()

```
void com.mehdi.petreminder.model.GroomingReminder.setGroomerName (
    String groomerName)
```

Groomer adı setter.

Parameters

<i>groomerName</i>	Groomer adı
--------------------	-------------

Definition at line 94 of file [GroomingReminder.java](#).

```
00094 { this.groomerName = groomerName; }
```

7.11.3.7 setGroomingType()

```
void com.mehdi.petreminder.model.GroomingReminder.setGroomingType (
    String groomingType)
```

Bakım türü setter.

Parameters

<i>groomingType</i>	Bakım türü
---------------------	------------

Definition at line 82 of file [GroomingReminder.java](#).

```
00082 { this.groomingType = groomingType; }
```

7.11.3.8 setProfessional()

```
void com.mehdi.petreminder.model.GroomingReminder.setProfessional (
    boolean professional)
```

Profesyonel setter.

Parameters

<i>professional</i>	Profesyonel mi
---------------------	----------------

Definition at line 88 of file [GroomingReminder.java](#).

```
00088 { this.professional = professional; }
```

7.11.3.9 toString()

```
String com.mehdi.petreminder.model.GroomingReminder.toString ()
```

String temsili.

Returns

Bakım hatırlatıcısı bilgisi

Reimplemented from [com.mehdi.petreminder.model.Reminder](#).

Definition at line 123 of file [GroomingReminder.java](#).

```
00123         {
00124         return String.format("GroomingReminder{id=%d, pet='%s', type='%s', prof=%b}",
00125             getId(), getPetName(), groomingType, professional);
00126     }
```

References [com.mehdi.petreminder.model.Reminder.getId\(\)](#), and [com.mehdi.petreminder.model.Reminder.getPetName\(\)](#).

7.11.3.10 validate()

```
boolean com.mehdi.petreminder.model.GroomingReminder.validate ()
```

Bakım hatırlatıcısını doğrular.

Returns

Bakım türü boş değilse true

Reimplemented from [com.mehdi.petreminder.model.Reminder](#).

Definition at line 105 of file [GroomingReminder.java](#).

```
00105         {
00106         return groomingType != null && !groomingType.trim().isEmpty();
00107     }
```

The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/model/GroomingReminder.java](#)

7.12 com.mehdi.petreminder.gui.util.GuiConstants Class Reference

GUI bileşenlerinde kullanılan sabitler.

Static Public Attributes

- static final Color **NAV_COLOR** = new Color(45, 55, 72)
Sol nav paneli arka plan rengi.
- static final Color **NAV_HOVER** = new Color(74, 85, 104)
Nav buton hover rengi.
- static final Color **NAV_ACTIVE** = new Color(99, 102, 241)
Nav aktif buton rengi.
- static final Color **BG_COLOR** = new Color(248, 250, 252)
İçerik alanı arka plan.
- static final Color **CARD_BG** = Color.WHITE
Kart arka plan rengi.
- static final Color **PRIMARY** = new Color(99, 102, 241)
Birincil vurgu rengi (indigo).
- static final Color **DANGER** = new Color(239, 68, 68)
Tehlike rengi (kırmızı).
- static final Color **SUCCESS** = new Color(34, 197, 94)
Başarı rengi (yeşil).
- static final Color **WARNING** = new Color(245, 158, 11)
Uyarı rengi (turuncu).
- static final Color **TEXT_PRIMARY** = new Color(17, 24, 39)
Başlık metni rengi.
- static final Color **TEXT_SECONDARY** = new Color(107, 114, 128)
İkincil metin rengi.
- static final Color **BORDER_COLOR** = new Color(229, 231, 235)
Kenarlık rengi.
- static final Font **TITLE_FONT** = new Font("Segoe UI", Font.BOLD, 22)
Büyük başlık fontu.
- static final Font **SECTION_FONT** = new Font("Segoe UI", Font.BOLD, 16)
Bölüm başlık fontu.
- static final Font **BODY_FONT** = new Font("Segoe UI", Font.PLAIN, 13)
Normal metin fontu.
- static final Font **SMALL_FONT** = new Font("Segoe UI", Font.PLAIN, 11)
Küçük metin fontu.
- static final Font **NAV_FONT** = new Font("Segoe UI", Font.PLAIN, 13)
Nav buton fontu.
- static final int **NAV_WIDTH** = 200
Sol nav genişliği.
- static final int **GAP** = 12
Standart boşluk.
- static final int **GAP_LARGE** = 24
Büyük boşluk.

7.12.1 Detailed Description

GUI bileşenlerinde kullanılan sabitler.

Tüm renk, font ve boyut değerleri merkezi olarak burada tanımlanır.

Author

Muhammed Mehdi Karagülle
Ibrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 21 of file [GuiConstants.java](#).

7.12.2 Member Data Documentation

7.12.2.1 BG_COLOR

```
final Color com.mehdi.petreminder.gui.util.GuiConstants.BG_COLOR = new Color(248, 250, 252)
[static]
```

İçerik alanı arka plan.

Definition at line 38 of file [GuiConstants.java](#).

Referenced by [com.mehdi.petreminder.gui.panels.DashboardPanel.DashboardPanel\(\)](#), [com.mehdi.petreminder.gui.panels.MedicalPa](#), [com.mehdi.petreminder.gui.panels.PetsPanel.PetsPanel\(\)](#), [com.mehdi.petreminder.gui.panels.RemindersPanel.RemindersPanel\(\)](#), [com.mehdi.petreminder.gui.panels.SettingsPanel.SettingsPanel\(\)](#), and [com.mehdi.petreminder.gui.panels.VetPanel.VetPanel\(\)](#).

7.12.2.2 BODY_FONT

```
final Font com.mehdi.petreminder.gui.util.GuiConstants.BODY_FONT = new Font("Segoe UI", Font.↔
PLAIN, 13) [static]
```

Normal metin fontu.

Definition at line 73 of file [GuiConstants.java](#).

7.12.2.3 BORDER_COLOR

```
final Color com.mehdi.petreminder.gui.util.GuiConstants.BORDER_COLOR = new Color(229, 231,
235) [static]
```

Kenarlık rengi.

Definition at line 62 of file [GuiConstants.java](#).

7.12.2.4 CARD_BG

```
final Color com.mehdi.petreminder.gui.util.GuiConstants.CARD_BG = Color.WHITE [static]
```

Kart arka plan rengi.

Definition at line 41 of file [GuiConstants.java](#).

7.12.2.5 DANGER

```
final Color com.mehdi.petreminder.gui.util.GuiConstants.DANGER = new Color(239, 68, 68) [static]
```

Tehlike rengi (kırmızı).

Definition at line 47 of file [GuiConstants.java](#).

7.12.2.6 GAP

```
final int com.mehdi.petreminder.gui.util.GuiConstants.GAP = 12 [static]
```

Standart boşluk.

Definition at line 87 of file [GuiConstants.java](#).

Referenced by [com.mehdi.petreminder.gui.panels.DashboardPanel.DashboardPanel\(\)](#), [com.mehdi.petreminder.gui.panels.MedicalPanel.MedicalPanel\(\)](#), [com.mehdi.petreminder.gui.panels.PetsPanel.PetsPanel\(\)](#), [com.mehdi.petreminder.gui.panels.RemindersPanel.RemindersPanel\(\)](#), and [com.mehdi.petreminder.gui.panels.VetPanel.VetPanel\(\)](#).

7.12.2.7 GAP_LARGE

```
final int com.mehdi.petreminder.gui.util.GuiConstants.GAP_LARGE = 24 [static]
```

Büyük boşluk.

Definition at line 90 of file [GuiConstants.java](#).

Referenced by [com.mehdi.petreminder.gui.panels.DashboardPanel.DashboardPanel\(\)](#), [com.mehdi.petreminder.gui.panels.MedicalPanel.MedicalPanel\(\)](#), [com.mehdi.petreminder.gui.panels.PetsPanel.PetsPanel\(\)](#), [com.mehdi.petreminder.gui.panels.RemindersPanel.RemindersPanel\(\)](#), [com.mehdi.petreminder.gui.panels.SettingsPanel.SettingsPanel\(\)](#), and [com.mehdi.petreminder.gui.panels.VetPanel.VetPanel\(\)](#).

7.12.2.8 NAV_ACTIVE

```
final Color com.mehdi.petreminder.gui.util.GuiConstants.NAV_ACTIVE = new Color(99, 102, 241) [static]
```

Nav aktif buton rengi.

Definition at line 35 of file [GuiConstants.java](#).

Referenced by [com.mehdi.petreminder.gui.panels.NavPanel.setActiveButton\(\)](#).

7.12.2.9 NAV_COLOR

```
final Color com.mehdi.petreminder.gui.util.GuiConstants.NAV_COLOR = new Color(45, 55, 72) [static]
```

Sol nav paneli arka plan rengi.

Definition at line 29 of file [GuiConstants.java](#).

Referenced by [com.mehdi.petreminder.gui.panels.NavPanel.NavPanel\(\)](#), and [com.mehdi.petreminder.gui.panels.NavPanel.setActiveButton\(\)](#).

7.12.2.10 NAV_FONT

```
final Font com.mehdi.petreminder.gui.util.GuiConstants.NAV_FONT = new Font("Segoe UI", Font.↵  
PLAIN, 13) [static]
```

Nav buton fontu.

Definition at line 79 of file [GuiConstants.java](#).

7.12.2.11 NAV_HOVER

```
final Color com.mehdi.petreminder.gui.util.GuiConstants.NAV_HOVER = new Color(74, 85, 104)  
[static]
```

Nav buton hover rengi.

Definition at line 32 of file [GuiConstants.java](#).

7.12.2.12 NAV_WIDTH

```
final int com.mehdi.petreminder.gui.util.GuiConstants.NAV_WIDTH = 200 [static]
```

Sol nav genişliği.

Definition at line 84 of file [GuiConstants.java](#).

Referenced by [com.mehdi.petreminder.gui.panels.NavPanel.NavPanel\(\)](#).

7.12.2.13 PRIMARY

```
final Color com.mehdi.petreminder.gui.util.GuiConstants.PRIMARY = new Color(99, 102, 241)  
[static]
```

Birincil vurgu rengi (indigo).

Definition at line 44 of file [GuiConstants.java](#).

7.12.2.14 SECTION_FONT

```
final Font com.mehdi.petreminder.gui.util.GuiConstants.SECTION_FONT = new Font("Segoe UI",  
Font.BOLD, 16) [static]
```

Bölüm başlık fontu.

Definition at line 70 of file [GuiConstants.java](#).

7.12.2.15 SMALL_FONT

```
final Font com.mehdi.petreminder.gui.util.GuiConstants.SMALL_FONT = new Font("Segoe UI", Font.↵  
PLAIN, 11) [static]
```

Küçük metin fontu.

Definition at line 76 of file [GuiConstants.java](#).

7.12.2.16 SUCCESS

```
final Color com.mehdi.petreminder.gui.util.GuiConstants.SUCCESS = new Color(34, 197, 94) [static]
```

Başarı rengi (yeşil).

Definition at line 50 of file [GuiConstants.java](#).

7.12.2.17 TEXT_PRIMARY

```
final Color com.mehdi.petreminder.gui.util.GuiConstants.TEXT_PRIMARY = new Color(17, 24, 39)  
[static]
```

Başlık metni rengi.

Definition at line 56 of file [GuiConstants.java](#).

7.12.2.18 TEXT_SECONDARY

```
final Color com.mehdi.petreminder.gui.util.GuiConstants.TEXT_SECONDARY = new Color(107, 114,  
128) [static]
```

İkincil metin rengi.

Definition at line 59 of file [GuiConstants.java](#).

7.12.2.19 TITLE_FONT

```
final Font com.mehdi.petreminder.gui.util.GuiConstants.TITLE_FONT = new Font("Segoe UI", Font.↵  
BOLD, 22) [static]
```

Büyük başlık fontu.

Definition at line 67 of file [GuiConstants.java](#).

7.12.2.20 WARNING

```
final Color com.mehdi.petreminder.gui.util.GuiConstants.WARNING = new Color(245, 158, 11)
[static]
```

Uyarı rengi (turuncu).

Definition at line 53 of file [GuiConstants.java](#).

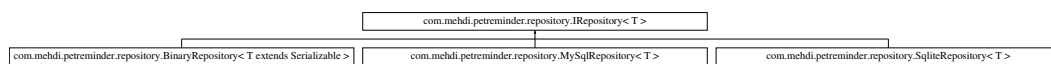
The documentation for this class was generated from the following file:

- petreminder-app/src/main/java/com/mehdi/petreminder/gui/util/[GuiConstants.java](#)

7.13 com.mehdi.petreminder.repository.IRepository< T > Interface Template Reference

Generic CRUD repository arayüzü.

Inheritance diagram for com.mehdi.petreminder.repository.IRepository< T >:



Public Member Functions

- int [save](#) (T entity)
Yeni bir varlık kaydeder.
- Optional< T > [findById](#) (int id)
ID'ye göre varlık getirir.
- List< T > [findAll](#) ()
Tüm varlıkları listeler.
- boolean [update](#) (T entity)
Mevcut varlığı günceller.
- boolean [delete](#) (int id)
ID'ye göre varlık siler.
- void [deleteAll](#) ()
Tüm varlıkları siler.
- int [count](#) ()
Varlık sayısını döndürür.
- void [close](#) ()
Repository bağlantısını/kaynaklarını kapatır.

7.13.1 Detailed Description

Generic CRUD repository arayüzü.

PDF zorunluluğu: [IRepository](#)<T> interface with [RepositoryFactory](#). OOP Abstraction: Binary / SQLite / MySQL aynı interface üzerinden kullanılır.

Parameters

<T>	Varlık türü (Pet , Reminder , User , MedicalRecord)
-----	---

Author

Muhammed Mehdi Karagülle
Ibrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 23 of file [IRepository.java](#).

7.13.2 Member Function Documentation

7.13.2.1 close()

```
void com.mehdi.petreminder.repository.IRepository< T >.close ()
```

Repository bağlantısını/kaynaklarını kapatır.

Implemented in [com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >](#), [com.mehdi.petreminder.repository.M](#) and [com.mehdi.petreminder.repository.SQLiteRepository< T >](#).

7.13.2.2 count()

```
int com.mehdi.petreminder.repository.IRepository< T >.count ()
```

Varlık sayısını döndürür.

Returns

Toplam kayıt sayısı

Implemented in [com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >](#), [com.mehdi.petreminder.repository.M](#) and [com.mehdi.petreminder.repository.SQLiteRepository< T >](#).

7.13.2.3 delete()

```
boolean com.mehdi.petreminder.repository.IRepository< T >.delete (
    int id)
```

ID'ye göre varlık siler.

Parameters

<i>id</i>	Silinecek varlığın ID'si
-----------	--------------------------

Returns

Silme başarılıysa true

Exceptions

<i>RepositoryException</i>	Silme başarısız olursa
--	------------------------

Implemented in [com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >](#), [com.mehdi.petreminder.repository.M](#) and [com.mehdi.petreminder.repository.SqliteRepository< T >](#).

7.13.2.4 deleteAll()

```
void com.mehdi.petreminder.repository.IRepository< T >.deleteAll ()
```

Tüm varlıkları siler.

Exceptions

<i>RepositoryException</i>	Silme başarısız olursa
--	------------------------

Implemented in [com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >](#), [com.mehdi.petreminder.repository.M](#) and [com.mehdi.petreminder.repository.SqliteRepository< T >](#).

7.13.2.5 findAll()

```
List< T > com.mehdi.petreminder.repository.IRepository< T >.findAll ()
```

Tüm varlıkları listeler.

Returns

Varlık listesi (boş olabilir)

Exceptions

RepositoryException	Okuma başarısız olursa
-------------------------------------	------------------------

Implemented in [com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >](#), [com.mehdi.petreminder.repository.M](#) and [com.mehdi.petreminder.repository.SqliteRepository< T >](#).

7.13.2.6 findById()

```
Optional< T > com.mehdi.petreminder.repository.IRepository< T >.findById (  
    int id)
```

ID'ye göre varlık getirir.

Parameters

<i>id</i>	Varlık ID'si
-----------	--------------

Returns

Varlık (bulunamazsa empty Optional)

Exceptions

RepositoryException	Okuma başarısız olursa
-------------------------------------	------------------------

Implemented in [com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >](#), [com.mehdi.petreminder.repository.M](#) and [com.mehdi.petreminder.repository.SqliteRepository< T >](#).

7.13.2.7 save()

```
int com.mehdi.petreminder.repository.IRepository< T >.save (  
    T entity)
```

Yeni bir varlık kaydeder.

Parameters

<i>entity</i>	Kaydedilecek varlık
---------------	---------------------

Returns

Oluşturulan varlığın ID'si

Exceptions

RepositoryException	Kayıt başarısız olursa
-------------------------------------	------------------------

Implemented in [com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >](#), [com.mehdi.petreminder.repository.M](#) and [com.mehdi.petreminder.repository.SqliteRepository< T >](#).

7.13.2.8 update()

```
boolean com.mehdi.petreminder.repository.IRepository< T >.update (
    T entity)
```

Mevcut varlığı günceller.

Parameters

<i>entity</i>	Güncellenecek varlık
---------------	----------------------

Returns

Güncelleme başarılıysa true

Exceptions

RepositoryException	Güncelleme başarısız olursa
-------------------------------------	-----------------------------

Implemented in [com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >](#), [com.mehdi.petreminder.repository.M](#) and [com.mehdi.petreminder.repository.SqliteRepository< T >](#).

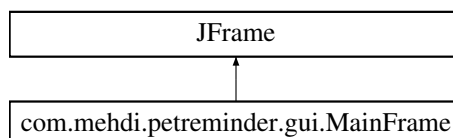
The documentation for this interface was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/repository/IRepository.java](#)

7.14 com.mehdi.petreminder.gui.MainFrame Class Reference

Uygulamanın ana Swing penceresi.

Inheritance diagram for com.mehdi.petreminder.gui.MainFrame:



Public Member Functions

- [MainFrame](#) ()
MainFrame yapıcısı — GUI bileşenlerini başlatır.
- void [navigateTo](#) (String cardName)
Belirtilen kartı gösterir.
- void [onStorageChanged](#) ([StorageType](#) type)
Storage backend değişince tüm panelleri günceller.

Static Public Member Functions

- static void [launch](#) ()
Uygulamayı başlatır (EDT'de çalışır).

Static Public Attributes

- static final String [CARD_DASHBOARD](#) = "DASHBOARD"
Kart isimleri.
- static final String [CARD_PETS](#) = "PETS"
Pets kart adı.
- static final String [CARD_REMINDERS](#) = "REMINDERS"
Reminders kart adı.
- static final String [CARD_VET](#) = "VET"
Vet kart adı.
- static final String [CARD_MEDICAL](#) = "MEDICAL"
Medical kart adı.
- static final String [CARD_SETTINGS](#) = "SETTINGS"
Settings kart adı.

7.14.1 Detailed Description

Uygulamanın ana Swing penceresi.

PDF zorunluluğu: Swing GUI — ayrı GUI katmanı. FlatLaf light tema kullanılır (modern görünüm). Sol [NavPanel](#) + sağ CardLayout içerik alanı.

Author

Muhammed Mehdi Karagülle
İbrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 33 of file [MainFrame.java](#).

7.14.2 Constructor & Destructor Documentation

7.14.2.1 MainFrame()

`com.mehdi.petreminder.gui.MainFrame.MainFrame ()`

[MainFrame](#) yapıcısı — GUI bileşenlerini başlatır.

Definition at line 88 of file [MainFrame.java](#).

```
00088         {
00089             super(TITLE);
00090             initLookAndFeel();
00091             initComponents();
00092             setupLayout();
00093             setupFrame();
00094             navigateTo(CARD_DASHBOARD);
00095             logger.info("MainFrame başlatıldı.");
00096         }
```

References [CARD_DASHBOARD](#), and [navigateTo\(\)](#).

Referenced by [launch\(\)](#).

7.14.3 Member Function Documentation

7.14.3.1 launch()

`void com.mehdi.petreminder.gui.MainFrame.launch () [static]`

Uygulamayı başlatır (EDT'de çalışır).

Definition at line 241 of file [MainFrame.java](#).

```
00241         {
00242             SwingUtilities.invokeLater(() -> {
00243                 MainFrame frame = new MainFrame();
00244                 frame.setVisible(true);
00245             });
00246     }
```

References [MainFrame\(\)](#).

7.14.3.2 navigateTo()

`void com.mehdi.petreminder.gui.MainFrame.navigateTo (String cardName)`

Belirtilen kartı gösterir.

Parameters

<i>cardName</i>	Kart adı (CARD_* sabitlerinden biri)
-----------------	--------------------------------------

Definition at line 196 of file [MainFrame.java](#).

```
00196         {
00197             cardLayout.show(contentPanel, cardName);
00198             navPanel.setActiveButton(cardName);
00199             refreshCurrentPanel(cardName);
00200             logger.debug("Navigasyon: {}", cardName);
00201     }
```

Referenced by [MainFrame\(\)](#).

7.14.3.3 onStorageChanged()

```
void com.mehdi.petreminder.gui.MainFrame.onStorageChanged (
    StorageType type)
```

Storage backend değişince tüm panelleri günceller.

Parameters

<i>type</i>	Yeni StorageType
-------------	----------------------------------

Definition at line 223 of file [MainFrame.java](#).

```
00223                                     {
00224         StorageConfig.setActiveBackend(type);
00225         // Status bar'ı güncelle
00226         Component[] comps = ((JPanel) getContentPane().getComponent(2)).getComponents();
00227         for (Component c : comps) {
00228             if (c instanceof JLabel && ((JLabel) c).getText().startsWith("Storage:")) {
00229                 ((JLabel) c).setText("Storage: " + type.getDisplayName());
00230             }
00231         }
00232         JOptionPane.showMessageDialog(this,
00233             "Storage backend değiştirildi:\n" + type.getDisplayName(),
00234             "Ayarlar", JOptionPane.INFORMATION_MESSAGE);
00235         logger.info("Storage değişti: {}", type);
00236     }
```

References [com.mehdi.petreminder.config.StorageType.getDisplayName\(\)](#), and [com.mehdi.petreminder.config.StorageConfig.setActiveBackend\(\)](#).

7.14.4 Member Data Documentation

7.14.4.1 CARD_DASHBOARD

```
final String com.mehdi.petreminder.gui.MainFrame.CARD_DASHBOARD = "DASHBOARD" [static]
```

Kart isimleri.

Definition at line 46 of file [MainFrame.java](#).

Referenced by [MainFrame\(\)](#).

7.14.4.2 CARD_MEDICAL

```
final String com.mehdi.petreminder.gui.MainFrame.CARD_MEDICAL = "MEDICAL" [static]
```

Medical kart adı.

Definition at line 54 of file [MainFrame.java](#).

7.14.4.3 CARD_PETS

```
final String com.mehdi.petreminder.gui.MainFrame.CARD_PETS = "PETS" [static]
```

Pets kart adı.

Definition at line 48 of file [MainFrame.java](#).

7.14.4.4 CARD_REMINDERS

```
final String com.mehdi.petreminder.gui.MainFrame.CARD_REMINDERS = "REMINDERS" [static]
```

Reminders kart adı.

Definition at line 50 of file [MainFrame.java](#).

7.14.4.5 CARD_SETTINGS

```
final String com.mehdi.petreminder.gui.MainFrame.CARD_SETTINGS = "SETTINGS" [static]
```

Settings kart adı.

Definition at line 56 of file [MainFrame.java](#).

7.14.4.6 CARD_VET

```
final String com.mehdi.petreminder.gui.MainFrame.CARD_VET = "VET" [static]
```

Vet kart adı.

Definition at line 52 of file [MainFrame.java](#).

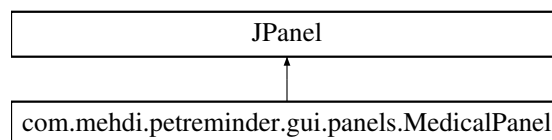
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/gui/MainFrame.java](#)

7.15 com.mehdi.petreminder.gui.panels.MedicalPanel Class Reference

Sağlık kayıtları CRUD paneli.

Inheritance diagram for com.mehdi.petreminder.gui.panels.MedicalPanel:



Public Member Functions

- [MedicalPanel](#) ([MainFrame](#) mainFrame)
MedicalPanel yapıcısı.
- void [refresh](#) ()
Tabloyu yeniler.

7.15.1 Detailed Description

Sağlık kayıtları CRUD paneli.

Author

Muhammed Mehdi Karagülle
İbrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 28 of file [MedicalPanel.java](#).

7.15.2 Constructor & Destructor Documentation

7.15.2.1 MedicalPanel()

```
com.mehdi.petreminder.gui.panels.MedicalPanel.MedicalPanel (  
    MainFrame mainFrame)
```

[MedicalPanel](#) yapıcısı.

Parameters

<i>mainFrame</i>	Ana pencere
------------------	-------------

Definition at line 49 of file [MedicalPanel.java](#).

```
00049      {  
00050          this.mainFrame      = mainFrame;  
00051          this.medicalService = new MedicalRecordService();  
00052          setBackground(GuiConstants.BG_COLOR);  
00053          setLayout(new BorderLayout(GuiConstants.GAP, GuiConstants.GAP));  
00054          setBorder(new EmptyBorder(GuiConstants.GAP_LARGE, GuiConstants.GAP_LARGE,  
00055                                   GuiConstants.GAP_LARGE, GuiConstants.GAP_LARGE));  
00056          buildUI();  
00057      }
```

References [com.mehdi.petreminder.gui.util.GuiConstants.BG_COLOR](#), [com.mehdi.petreminder.gui.util.GuiConstants.GAP](#), and [com.mehdi.petreminder.gui.util.GuiConstants.GAP_LARGE](#).

7.15.3 Member Function Documentation

7.15.3.1 refresh()

```
void com.mehdi.petreminder.gui.panels.MedicalPanel.refresh ()
```

Tabloyu yeniler.

Definition at line 157 of file [MedicalPanel.java](#).

```
00157         {
00158             tableModel.setRowCount(0);
00159             try {
00160                 List<MedicalRecord> list = medicalService.getAllRecords();
00161                 for (MedicalRecord r : list) {
00162                     tableModel.addRow(new Object[]{
00163                         r.getId(), r.getPetName(), r.getRecordType(),
00164                         r.getDiagnosis(), r.getVeterinarianName(),
00165                         r.getFormattedDate(),
00166                         String.format("%.2f", r.getCost())
00167                     });
00168                 }
00169             } catch (Exception e) { /* sessiz */ }
00170         }
```

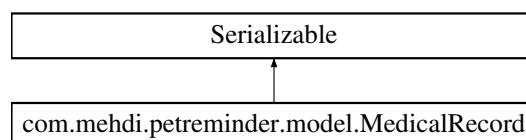
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/MedicalPanel.java](#)

7.16 com.mehdi.petreminder.model.MedicalRecord Class Reference

Evcil hayvanın sağlık kaydı.

Inheritance diagram for com.mehdi.petreminder.model.MedicalRecord:



Public Member Functions

- [MedicalRecord](#) ()
Varsayılan yapıcı.
- [MedicalRecord](#) (int id, int petId, String petName, LocalDate recordDate, String recordType, String diagnosis, String veterinarianName)
Temel parametrelili yapıcı.
- int [getId](#) ()
ID getter.
- void [setId](#) (int id)
ID setter.
- int [getPetId](#) ()
Pet ID getter.
- void [setPetId](#) (int petId)
Pet ID setter.

- String [getPetName](#) ()
Pet adı getter.
- void [setPetName](#) (String petName)
Pet adı setter.
- LocalDate [getRecordDate](#) ()
Kayıt tarihi getter.
- void [setRecordDate](#) (LocalDate recordDate)
Kayıt tarihi setter.
- String [getRecordType](#) ()
Kayıt türü getter.
- void [setRecordType](#) (String recordType)
Kayıt türü setter.
- String [getDiagnosis](#) ()
Teşhis getter.
- void [setDiagnosis](#) (String diagnosis)
Teşhis setter.
- String [getTreatment](#) ()
Tedavi getter.
- void [setTreatment](#) (String treatment)
Tedavi setter.
- String [getVeterinarianName](#) ()
Veteriner getter.
- void [setVeterinarianName](#) (String veterinarianName)
Veteriner setter.
- double [getCost](#) ()
Maliyet getter.
- void [setCost](#) (double cost)
Maliyet setter.
- LocalDate [getNextCheckDate](#) ()
Sonraki kontrol getter.
- void [setNextCheckDate](#) (LocalDate nextCheckDate)
Sonraki kontrol setter.
- String [getNotes](#) ()
Notlar getter.
- void [setNotes](#) (String notes)
Notlar setter.
- String [getVaccineName](#) ()
Aşı adı getter.
- void [setVaccineName](#) (String vaccineName)
Aşı adı setter.
- String [getFormattedDate](#) ()
Biçimli tarih döndürür.
- String [toString](#) ()
String temsili.
- boolean [equals](#) (Object obj)
Eşitlik kontrolü.
- int [hashCode](#) ()
Hash kodu.

7.16.1 Detailed Description

Evcil hayvanın sağlık kaydı.

OOP Composition: [Pet](#) nesnesiyle composition ilişkisi. OOP Encapsulation: tüm alanlar private. Serializable: BinaryRepository için.

Author

Muhammed Mehdi Karagülle, İbrahim Demirci, Zümre Uykun

Version

1.0

Definition at line 21 of file [MedicalRecord.java](#).

7.16.2 Constructor & Destructor Documentation

7.16.2.1 MedicalRecord() [1/2]

```
com.mehdi.petreminder.model.MedicalRecord.MedicalRecord ()
```

Varsayılan yapıcı.

Definition at line 70 of file [MedicalRecord.java](#).

```
00070 {}
```

Referenced by [equals\(\)](#).

7.16.2.2 MedicalRecord() [2/2]

```
com.mehdi.petreminder.model.MedicalRecord.MedicalRecord (
    int id,
    int petId,
    String petName,
    LocalDate recordDate,
    String recordType,
    String diagnosis,
    String veterinarianName)
```

Temel parametrelili yapıcı.

Parameters

<i>id</i>	Kayıt ID
<i>petId</i>	Pet ID
<i>petName</i>	Pet adı

<i>recordDate</i>	Tarih
<i>recordType</i>	Tür
<i>diagnosis</i>	Teşhis
<i>veterinarianName</i>	Veteriner adı

Definition at line 82 of file [MedicalRecord.java](#).

```

00084
00085         this.id = id;
00086         this.petId = petId;
00087         this.petName = petName;
00088         this.recordDate = recordDate;
00089         this.recordType = recordType;
00090         this.diagnosis = diagnosis;
00091         this.veterinarianName = veterinarianName;
00092         this.treatment = "";
00093         this.cost = 0.0;
00094         this.notes = "";
00095         this.vaccineName = "";
00096     }

```

7.16.3 Member Function Documentation

7.16.3.1 equals()

```
boolean com.mehdi.petreminder.model.MedicalRecord.equals (
    Object obj)
```

Eşitlik kontrolü.

Parameters

<i>obj</i>	Karşılaştırılan nesne
------------	-----------------------

Returns

ID'ler eşitse true

Definition at line 199 of file [MedicalRecord.java](#).

```

00199
00200         if (this == obj) return true;
00201         if (obj == null || !(obj instanceof MedicalRecord)) return false;
00202         MedicalRecord other = (MedicalRecord) obj;
00203         return this.id == other.id;
00204     }

```

References [MedicalRecord\(\)](#).

7.16.3.2 getCost()

```
double com.mehdi.petreminder.model.MedicalRecord.getCost ()
```

Maliyet getter.

Returns

Maliyet (TL)

Definition at line 149 of file [MedicalRecord.java](#).

```
00149 { return cost; }
```

7.16.3.3 getDiagnosis()

```
String com.mehdi.petreminder.model.MedicalRecord.getDiagnosis ()
```

Teşhis getter.

Returns

Teşhis

Definition at line 129 of file [MedicalRecord.java](#).

```
00129 { return diagnosis; }
```

7.16.3.4 getFormattedDate()

```
String com.mehdi.petreminder.model.MedicalRecord.getFormattedDate ()
```

Biçimli tarih döndürür.

Returns

dd/MM/yyyy formatında tarih

Definition at line 178 of file [MedicalRecord.java](#).

```
00178                                     {  
00179         if (recordDate == null) return "Belirsiz";  
00180         return recordDate.format (DATE_FMT);  
00181     }
```

Referenced by [toString\(\)](#).

7.16.3.5 getId()

```
int com.mehdi.petreminder.model.MedicalRecord.getId ()
```

ID getter.

Returns

Kayıt ID

Definition at line 99 of file [MedicalRecord.java](#).

```
00099 { return id; }
```

Referenced by [com.mehdi.petreminder.service.MedicalRecordService.updateRecord\(\)](#).

7.16.3.6 getNextCheckDate()

`LocalDate com.mehdi.petreminder.model.MedicalRecord.getNextCheckDate ()`

Sonraki kontrol getter.

Returns

Sonraki kontrol tarihi

Definition at line 155 of file [MedicalRecord.java](#).

```
00155 { return nextCheckDate; }
```

7.16.3.7 getNotes()

`String com.mehdi.petreminder.model.MedicalRecord.getNotes ()`

Notlar getter.

Returns

Notlar

Definition at line 163 of file [MedicalRecord.java](#).

```
00163 { return notes; }
```

7.16.3.8 getPetId()

`int com.mehdi.petreminder.model.MedicalRecord.getPetId ()`

[Pet](#) ID getter.

Returns

[Pet](#) ID

Definition at line 105 of file [MedicalRecord.java](#).

```
00105 { return petId; }
```

Referenced by [com.mehdi.petreminder.service.MedicalRecordService.addRecord\(\)](#).

7.16.3.9 getPetName()

`String com.mehdi.petreminder.model.MedicalRecord.getPetName ()`

[Pet](#) adı getter.

Returns

[Pet](#) adı

Definition at line 111 of file [MedicalRecord.java](#).

```
00111 { return petName; }
```


7.16.3.10 getRecordDate()

`LocalDate com.mehdi.petreminder.model.MedicalRecord.getRecordDate ()`

Kayıt tarihi getter.

Returns

Kayıt tarihi

Definition at line 117 of file [MedicalRecord.java](#).

```
00117 { return recordDate; }
```

Referenced by [com.mehdi.petreminder.service.MedicalRecordService.getRecordsByPetId\(\)](#).

7.16.3.11 getRecordType()

`String com.mehdi.petreminder.model.MedicalRecord.getRecordType ()`

Kayıt türü getter.

Returns

Kayıt türü

Definition at line 123 of file [MedicalRecord.java](#).

```
00123 { return recordType; }
```

7.16.3.12 getTreatment()

`String com.mehdi.petreminder.model.MedicalRecord.getTreatment ()`

Tedavi getter.

Returns

Tedavi

Definition at line 135 of file [MedicalRecord.java](#).

```
00135 { return treatment; }
```

7.16.3.13 getVaccineName()

`String com.mehdi.petreminder.model.MedicalRecord.getVaccineName ()`

Aşı adı getter.

Returns

Aşı adı

Definition at line 169 of file [MedicalRecord.java](#).

```
00169 { return vaccineName; }
```

7.16.3.14 `getVeterinarianName()`

```
String com.mehdi.petreminder.model.MedicalRecord.getVeterinarianName ()
```

Veteriner getter.

Returns

Veteriner

Definition at line 141 of file [MedicalRecord.java](#).

```
00141 { return veterinarianName; }
```

7.16.3.15 `hashCode()`

```
int com.mehdi.petreminder.model.MedicalRecord.hashCode ()
```

Hash kodu.

Returns

ID'ye göre hash

Definition at line 211 of file [MedicalRecord.java](#).

```
00211 {  
00212     return Integer.hashCode(id);  
00213 }
```

7.16.3.16 `setCost()`

```
void com.mehdi.petreminder.model.MedicalRecord.setCost (  
    double cost)
```

Maliyet setter.

Parameters

<i>cost</i>	Maliyet
-------------	---------

Definition at line 152 of file [MedicalRecord.java](#).

```
00152 { this.cost = cost; }
```

Referenced by [com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository](#)

7.16.3.17 setDiagnosis()

```
void com.mehdi.petreminder.model.MedicalRecord.setDiagnosis (
    String diagnosis)
```

Teşhis setter.

Parameters

<i>diagnosis</i>	Teşhis
------------------	--------

Definition at line 132 of file [MedicalRecord.java](#).

```
00132 { this.diagnosis = diagnosis; }
```

Referenced by [com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository](#)

7.16.3.18 setId()

```
void com.mehdi.petreminder.model.MedicalRecord.setId (
    int id)
```

ID setter.

Parameters

<i>id</i>	Kayıt ID
-----------	----------

Definition at line 102 of file [MedicalRecord.java](#).

```
00102 { this.id = id; }
```

Referenced by [com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository](#)

7.16.3.19 setNextCheckDate()

```
void com.mehdi.petreminder.model.MedicalRecord.setNextCheckDate (
    LocalDate nextCheckDate)
```

Sonraki kontrol setter.

Parameters

<i>nextCheckDate</i>	Tarih
----------------------	-------

Definition at line 158 of file [MedicalRecord.java](#).

```
00158
00159         this.nextCheckDate = nextCheckDate;
00160     }
```

Referenced by [com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository](#)

7.16.3.20 setNotes()

```
void com.mehdi.petreminder.model.MedicalRecord.setNotes (  
    String notes)
```

Notlar setter.

Parameters

<i>notes</i>	Notlar
--------------	--------

Definition at line 166 of file [MedicalRecord.java](#).

```
00166 { this.notes = notes; }
```

Referenced by [com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository](#)

7.16.3.21 setPetId()

```
void com.mehdi.petreminder.model.MedicalRecord.setPetId (  
    int petId)
```

[Pet](#) ID setter.

Parameters

<i>petId</i>	Pet ID
--------------	------------------------

Definition at line 108 of file [MedicalRecord.java](#).

```
00108 { this.petId = petId; }
```

Referenced by [com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository](#)

7.16.3.22 setPetName()

```
void com.mehdi.petreminder.model.MedicalRecord.setPetName (  
    String petName)
```

[Pet](#) adı setter.

Parameters

<i>petName</i>	Pet adı
----------------	-------------------------

Definition at line 114 of file [MedicalRecord.java](#).

```
00114 { this.petName = petName; }
```

Referenced by [com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository](#)

7.16.3.23 setRecordDate()

```
void com.mehdi.petreminder.model.MedicalRecord.setRecordDate (
    LocalDate recordDate)
```

Kayıt tarihi setter.

Parameters

<i>recordDate</i>	Kayıt tarihi
-------------------	--------------

Definition at line 120 of file [MedicalRecord.java](#).

```
00120 { this.recordDate = recordDate; }
```

Referenced by [com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository](#)

7.16.3.24 setRecordType()

```
void com.mehdi.petreminder.model.MedicalRecord.setRecordType (
    String recordType)
```

Kayıt türü setter.

Parameters

<i>recordType</i>	Kayıt türü
-------------------	------------

Definition at line 126 of file [MedicalRecord.java](#).

```
00126 { this.recordType = recordType; }
```

Referenced by [com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository](#)

7.16.3.25 setTreatment()

```
void com.mehdi.petreminder.model.MedicalRecord.setTreatment (
    String treatment)
```

Tedavi setter.

Parameters

<i>treatment</i>	Tedavi
------------------	--------

Definition at line 138 of file [MedicalRecord.java](#).

```
00138 { this.treatment = treatment; }
```

Referenced by [com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository](#)

7.16.3.26 setVaccineName()

```
void com.mehdi.petreminder.model.MedicalRecord.setVaccineName (  
    String vaccineName)
```

Aşı adı setter.

Parameters

<i>vaccineName</i>	Aşı adı
--------------------	---------

Definition at line 172 of file [MedicalRecord.java](#).

```
00172 { this.vaccineName = vaccineName; }
```

Referenced by [com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository](#)

7.16.3.27 setVeterinarianName()

```
void com.mehdi.petreminder.model.MedicalRecord.setVeterinarianName (  
    String veterinarianName)
```

Veteriner setter.

Parameters

<i>veterinarianName</i>	Veteriner
-------------------------	-----------

Definition at line 144 of file [MedicalRecord.java](#).

```
00144  
00145         this.veterinarianName = veterinarianName; {  
00146     }
```

Referenced by [com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository](#)

7.16.3.28 toString()

```
String com.mehdi.petreminder.model.MedicalRecord.toString ()
```

String temsili.

Returns

Sağlık kaydı özeti

Definition at line 188 of file [MedicalRecord.java](#).

```
00188         {  
00189             return String.format("MedicalRecord{id=%d, pet='%s', type='%s', date='%s'}",  
00190                 id, petName, recordType, getFormattedDate());  
00191         }
```

References [getFormattedDate\(\)](#).

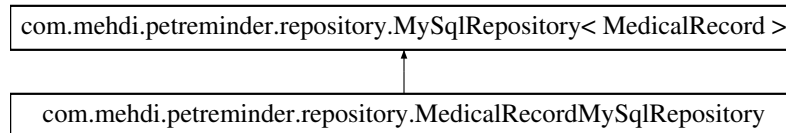
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/model/MedicalRecord.java](#)

7.17 com.mehdi.petreminder.repository.MedicalRecordMySqlRepository Class Reference

MySQL üzerinde [MedicalRecord](#) CRUD.

Inheritance diagram for com.mehdi.petreminder.repository.MedicalRecordMySqlRepository:



Public Member Functions

- [MedicalRecordMySqlRepository](#) ()
Varsayılan yapıcı.
- [MedicalRecordMySqlRepository](#) (Connection [connection](#))
Test yapıcısı.

Public Member Functions inherited from

[com.mehdi.petreminder.repository.MySqlRepository](#) < [MedicalRecord](#) >

- [MySqlRepository](#) ()
MySQL bağlantısını açar.
- int [save](#) (T entity)
- Optional< T > [findById](#) (int id)
- List< T > [findAll](#) ()
- boolean [update](#) (T entity)
- boolean [delete](#) (int id)
- void [deleteAll](#) ()
- int [count](#) ()
- void [close](#) ()
- Connection [getConnection](#) ()
Bağlantıyı döndürür.

Protected Member Functions

- String [getTableName](#) ()
- String [getCreateTableSql](#) ()
- String [getInsertSql](#) ()
- String [getUpdateSql](#) ()
- void [setInsertParams](#) (PreparedStatement ps, [MedicalRecord](#) m) throws SQLException
- void [setUpdateParams](#) (PreparedStatement ps, [MedicalRecord](#) m) throws SQLException
- [MedicalRecord](#) [mapRow](#) (ResultSet rs) throws SQLException

Protected Member Functions inherited from**[com.mehdi.petreminder.repository.MySqlRepository](#) < [MedicalRecord](#) >**

- abstract String [getTableName](#) ()
Tablo adı.
- abstract String [getCreateTableSql](#) ()
Tablo oluşturma SQL.
- abstract String [getInsertSql](#) ()
INSERT SQL.
- abstract String [getUpdateSql](#) ()
UPDATE SQL.
- abstract void [setInsertParams](#) (PreparedStatement ps, T entity) throws SQLException
INSERT parametrelerini set eder.
- abstract void [setUpdateParams](#) (PreparedStatement ps, T entity) throws SQLException
UPDATE parametrelerini set eder.
- abstract T [mapRow](#) (ResultSet rs) throws SQLException
ResultSet'ten varlık oluşturur.
- void [createTableIfNotExists](#) ()
Tabloyu oluşturur (yoksa).
- void [setEntityId](#) (T entity, int id)
Reflection ile entity'ye id set eder.

Additional Inherited Members**Protected Attributes inherited from****[com.mehdi.petreminder.repository.MySqlRepository](#) < [MedicalRecord](#) >**

- Connection [connection](#)
MySQL JDBC bağlantısı.

Static Protected Attributes inherited from**[com.mehdi.petreminder.repository.MySqlRepository](#) < [MedicalRecord](#) >**

- static final Logger [logger](#)
Logger.

7.17.1 Detailed Description

MySQL üzerinde [MedicalRecord](#) CRUD.

Author

Muhammed Mehdi Karagülle
İbrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 26 of file [MedicalRecordMySqlRepository.java](#).

7.17.2 Constructor & Destructor Documentation

7.17.2.1 MedicalRecordMySqlRepository() [1/2]

com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.MedicalRecordMySqlRepository ()

Varsayılan yapıcı.

Definition at line 29 of file [MedicalRecordMySqlRepository.java](#).

```
00029 { super(); }
```

7.17.2.2 MedicalRecordMySqlRepository() [2/2]

com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.MedicalRecordMySqlRepository (
Connection connection)

Test yapıcısı.

Parameters

<i>connection</i>	Test bağlantısı
-------------------	-----------------

Definition at line 35 of file [MedicalRecordMySqlRepository.java](#).

```
00035 { super(connection); }
```

References [com.mehdi.petreminder.repository.MySqlRepository< MedicalRecord >.connection](#).

7.17.3 Member Function Documentation

7.17.3.1 getCreateTableSql()

String com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.getCreateTableSql ()
[protected]

Member documentation.

Definition at line 49 of file [MedicalRecordMySqlRepository.java](#).

```
00049                                     {
00050     return "CREATE TABLE IF NOT EXISTS medical_records ("
00051         + "id INT AUTO_INCREMENT PRIMARY KEY,"
00052         + "pet_id INT NOT NULL,"
00053         + "pet_name VARCHAR(50),"
00054         + "record_date DATE NOT NULL,"
00055         + "record_type VARCHAR(30) NOT NULL,"
00056         + "diagnosis TEXT,"
00057         + "treatment TEXT,"
00058         + "veterinarian_name VARCHAR(100),"
00059         + "cost DOUBLE DEFAULT 0,"
00060         + "next_check_date DATE,"
00061         + "notes TEXT,"
00062         + "vaccine_name VARCHAR(100)"
00063         + ")";
00064 }
```

7.17.3.2 getInsertSql()

String com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.getInsertSql () [protected]

Member documentation.

Definition at line 71 of file [MedicalRecordMySqlRepository.java](#).

```
00071         {
00072             return "INSERT INTO medical_records (pet_id,pet_name,record_date,record_type,"
00073                 + "diagnosis,treatment,veterinarian_name,cost,next_check_date,notes,vaccine_name) "
00074                 + " VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?) ";
00075         }
```

7.17.3.3 getTableName()

String com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.getTableName () [protected]

Member documentation.

Definition at line 42 of file [MedicalRecordMySqlRepository.java](#).

```
00042 { return "medical_records"; }
```

7.17.3.4 getUpdateSql()

String com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.getUpdateSql () [protected]

Member documentation.

Definition at line 82 of file [MedicalRecordMySqlRepository.java](#).

```
00082         {
00083             return "UPDATE medical_records SET pet_id=?,pet_name=?,record_date=?,record_type=?, "
00084                 + "diagnosis=?,treatment=?,veterinarian_name=?,cost=?,next_check_date=?, "
00085                 + "notes=?,vaccine_name=? WHERE id=?";
00086         }
```

7.17.3.5 mapRow()

[MedicalRecord](#) com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.mapRow (ResultSet rs) throws SQLException [protected]

Member documentation.

Definition at line 124 of file [MedicalRecordMySqlRepository.java](#).

```
00124         {
00125             MedicalRecord m = new MedicalRecord();
00126             m.setId(rs.getInt("id"));
00127             m.setPetId(rs.getInt("pet_id"));
00128             m.setPetName(rs.getString("pet_name"));
00129             java.sql.Date rd = rs.getDate("record_date");
00130             if (rd != null) m.setRecordDate(rd.toLocalDate());
00131             m.setRecordType(rs.getString("record_type"));
00132             m.setDiagnosis(rs.getString("diagnosis"));
00133             m.setTreatment(rs.getString("treatment"));
00134             m.setVeterinarianName(rs.getString("veterinarian_name"));
00135             m.setCost(rs.getDouble("cost"));
00136             java.sql.Date ncd = rs.getDate("next_check_date");
00137             if (ncd != null) m.setNextCheckDate(ncd.toLocalDate());
00138             m.setNotes(rs.getString("notes"));
00139             m.setVaccineName(rs.getString("vaccine_name"));
00140             return m;
00141         }
```

References [com.mehdi.petreminder.model.MedicalRecord.setCost\(\)](#), [com.mehdi.petreminder.model.MedicalRecord.setDiagnosis\(\)](#), [com.mehdi.petreminder.model.MedicalRecord.setId\(\)](#), [com.mehdi.petreminder.model.MedicalRecord.setNextCheckDate\(\)](#), [com.mehdi.petreminder.model.MedicalRecord.setNotes\(\)](#), [com.mehdi.petreminder.model.MedicalRecord.setPetId\(\)](#), [com.mehdi.petreminder.model.MedicalRecord.setPetName\(\)](#), [com.mehdi.petreminder.model.MedicalRecord.setRecordDate\(\)](#), [com.mehdi.petreminder.model.MedicalRecord.setRecordType\(\)](#), [com.mehdi.petreminder.model.MedicalRecord.setTreatment\(\)](#), [com.mehdi.petreminder.model.MedicalRecord.setVaccineName\(\)](#), and [com.mehdi.petreminder.model.MedicalRecord.setVeterinarianName\(\)](#).

7.17.3.6 setInsertParams()

```
void com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.setInsertParams (
    PreparedStatement ps,
    MedicalRecord m) throws SQLException [protected]
```

Member documentation.

Definition at line 93 of file [MedicalRecordMySqlRepository.java](#).

```
00093                                     {
00094         ps.setInt(1, m.getPetId());
00095         ps.setString(2, m.getPetName());
00096         ps.setObject(3, m.getRecordDate() != null
00097             ? java.sql.Date.valueOf(m.getRecordDate()) : null);
00098         ps.setString(4, m.getRecordType());
00099         ps.setString(5, m.getDiagnosis());
00100         ps.setString(6, m.getTreatment());
00101         ps.setString(7, m.getVeterinarianName());
00102         ps.setDouble(8, m.getCost());
00103         ps.setObject(9, m.getNextCheckDate() != null
00104             ? java.sql.Date.valueOf(m.getNextCheckDate()) : null);
00105         ps.setString(10, m.getNotes());
00106         ps.setString(11, m.getVaccineName());
00107     }
```

Referenced by [setUpdateParams\(\)](#).

7.17.3.7 setUpdateParams()

```
void com.mehdi.petreminder.repository.MedicalRecordMySqlRepository.setUpdateParams (
    PreparedStatement ps,
    MedicalRecord m) throws SQLException [protected]
```

Member documentation.

Definition at line 114 of file [MedicalRecordMySqlRepository.java](#).

```
00114                                     {
00115         setInsertParams(ps, m);
00116         ps.setInt(12, m.getId());
00117     }
```

References [setInsertParams\(\)](#).

The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/repository/MedicalRecordMySqlRepository.java](#)

7.18 com.mehdi.petreminder.service.MedicalRecordService Class Reference

[MedicalRecord](#) CRUD ve iş kuralları.

Public Member Functions

- [MedicalRecordService](#) ()
Varsayılan yapıcı.
- [MedicalRecordService](#) ([IRepository](#)< [MedicalRecord](#) > repository)
Test yapıcısı.
- [MedicalRecord](#) [addRecord](#) ([MedicalRecord](#) record)
Kayıt ekler.
- [MedicalRecord](#) [getRecordById](#) (int id)
ID ile kayıt getirir.
- List< [MedicalRecord](#) > [getAllRecords](#) ()
Tüm kayıtları listeler.
- List< [MedicalRecord](#) > [getRecordsByPetId](#) (int petId)
Pet'e ait kayıtları getirir.
- void [updateRecord](#) ([MedicalRecord](#) record)
Kayıt günceller.
- void [deleteRecord](#) (int id)
Kayıt siler.
- void [deleteAllRecords](#) ()
Tüm kayıtları siler.
- int [getRecordCount](#) ()
Toplam kayıt sayısı.
- void [close](#) ()
Repository kapatır.

7.18.1 Detailed Description

[MedicalRecord](#) CRUD ve iş kuralları.

Observer Pattern: Kayıt eklendiğinde veya silindiğinde [EventManager](#) üzerinden bildirim gönderilir.

Author

Muhammed Mehdi Karagülle
Ibrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 32 of file [MedicalRecordService.java](#).

7.18.2 Constructor & Destructor Documentation

7.18.2.1 [MedicalRecordService](#)() [1/2]

```
com.mehdi.petreminder.service.MedicalRecordService.MedicalRecordService ()
```

Varsayılan yapıcı.

Definition at line 44 of file [MedicalRecordService.java](#).

```
00044                                     {
00045         this.repository = RepositoryFactory.createMedicalRecordRepository();
00046     }
```

References [com.mehdi.petreminder.repository.RepositoryFactory.createMedicalRecordRepository\(\)](#).

7.18.2.2 MedicalRecordService() [2/2]

```
com.mehdi.petreminder.service.MedicalRecordService.MedicalRecordService (
    IRepository< MedicalRecord > repository)
```

Test yapıcısı.

Parameters

<i>repository</i>	Test repository
-------------------	-----------------

Definition at line 52 of file [MedicalRecordService.java](#).

```
00052                                     {
00053         this.repository = repository;
00054     }
```

7.18.3 Member Function Documentation

7.18.3.1 addRecord()

```
MedicalRecord com.mehdi.petreminder.service.MedicalRecordService.addRecord (
    MedicalRecord record)
```

Kayıt ekler.

Parameters

<i>record</i>	Eklenecek MedicalRecord
---------------	---

Returns

Eklenen kayıt

Definition at line 61 of file [MedicalRecordService.java](#).

```
00061                                     {
00062         validateRecord(record);
00063         repository.save(record);
00064         logger.info("MedicalRecord eklendi: petId={}", record.getPetId());
00065         // Observer Pattern: Saglik kaydi eklendi bildirimi
00066         EventManager.getInstance().notify(EventType.MEDICAL_RECORD_ADDED, record);
00067         return record;
00068     }
```

References [com.mehdi.petreminder.observer.EventManager.getInstance\(\)](#), [com.mehdi.petreminder.model.MedicalRecord.getPetId\(\)](#), [com.mehdi.petreminder.observer.EventType.MEDICAL_RECORD_ADDED](#), and [com.mehdi.petreminder.observer.EventManager.not](#)

7.18.3.2 close()

```
void com.mehdi.petreminder.service.MedicalRecordService.close ()
```

Repository kapatır.

Definition at line 142 of file [MedicalRecordService.java](#).

```
00142                                     {
00143         repository.close();
00144     }
```

7.18.3.3 deleteAllRecords()

```
void com.mehdi.petreminder.service.MedicalRecordService.deleteAllRecords ()
```

Tüm kayıtları siler.

Definition at line 127 of file [MedicalRecordService.java](#).

```
00127         {
00128             repository.deleteAll();
00129         }
```

7.18.3.4 deleteRecord()

```
void com.mehdi.petreminder.service.MedicalRecordService.deleteRecord (
    int id)
```

Kayıt siler.

Parameters

<i>id</i>	Kayıt ID
-----------	----------

Definition at line 116 of file [MedicalRecordService.java](#).

```
00116         {
00117             boolean deleted = repository.delete(id);
00118             if (!deleted) throw new ServiceException("Kayıt silinemedi: id=" + id);
00119             logger.info("MedicalRecord silindi: id={}", id);
00120             // Observer Pattern: Sağlık kaydı silindi bildirimi
00121             EventManager.getInstance().notify(EventType.MEDICAL_RECORD_DELETED, id);
00122         }
```

References [com.mehdi.petreminder.observer.EventManager.getInstance\(\)](#), [com.mehdi.petreminder.observer.EventType.MEDICAL_RECORD_DELETED](#) and [com.mehdi.petreminder.observer.EventManager.notify\(\)](#).

7.18.3.5 getAllRecords()

```
List< MedicalRecord > com.mehdi.petreminder.service.MedicalRecordService.getAllRecords ()
```

Tüm kayıtları listeler.

Returns

Liste

Definition at line 84 of file [MedicalRecordService.java](#).

```
00084         {
00085             return repository.findAll();
00086         }
```

7.18.3.6 getRecordById()

`MedicalRecord` com.mehdi.petreminder.service.MedicalRecordService.getRecordById (
 int id)

ID ile kayıt getirir.

Parameters

<i>id</i>	Kayıt ID
-----------	----------

Returns

`MedicalRecord`

Definition at line 75 of file `MedicalRecordService.java`.

```
00075                                     {
00076         return repository.findById(id)
00077         .orElseThrow(() -> new ServiceException("Kayıt bulunamadı: id=" + id));
00078     }
```

7.18.3.7 getRecordCount()

int com.mehdi.petreminder.service.MedicalRecordService.getRecordCount ()

Toplam kayıt sayısı.

Returns

Sayı

Definition at line 135 of file `MedicalRecordService.java`.

```
00135                                     {
00136         return repository.count();
00137     }
```

7.18.3.8 getRecordsByPetId()

List< `MedicalRecord` > com.mehdi.petreminder.service.MedicalRecordService.getRecordsByPetId (
 int petId)

`Pet`'e ait kayıtları getirir.

Parameters

<i>petId</i>	<code>Pet</code> ID
--------------	---------------------

Returns

`MedicalRecord` listesi

Definition at line 93 of file `MedicalRecordService.java`.

```
00093                                     {
00094         return repository.findAll().stream()
00095         .filter(r -> r.getPetId() == petId)
00096         .sorted(Comparator.comparing(
00097             MedicalRecord::getRecordDate,
00098             Comparator.nullsLast(Comparator.reverseOrder())))
00099         .collect(Collectors.toList());
00100     }
```

References `com.mehdi.petreminder.model.MedicalRecord.getRecordDate()`.

7.18.3.9 updateRecord()

```
void com.mehdi.petreminder.service.MedicalRecordService.updateRecord (
    MedicalRecord record)
```

Kayıt günceller.

Parameters

<i>record</i>	Güncellenecek kayıt
---------------	---------------------

Definition at line 106 of file [MedicalRecordService.java](#).

```
00106                                     {
00107         validateRecord(record);
00108         boolean updated = repository.update(record);
00109         if (!updated) throw new ServiceException("Kayıt güncellenemedi: id=" + record.getId());
00110     }
```

References [com.mehdi.petreminder.model.MedicalRecord.getId\(\)](#).

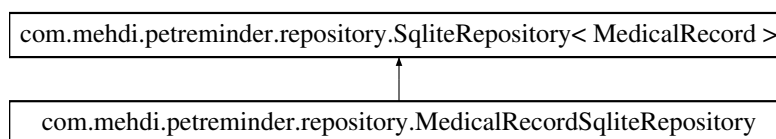
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/service/MedicalRecordService.java](#)

7.19 com.mehdi.petreminder.repository.MedicalRecordSqliteRepository Class Reference

SQLite üzerinde [MedicalRecord](#) CRUD.

Inheritance diagram for `com.mehdi.petreminder.repository.MedicalRecordSqliteRepository`:



Public Member Functions

- [MedicalRecordSqliteRepository \(\)](#)
Varsayılan yapıcı.
- [MedicalRecordSqliteRepository \(Connection connection\)](#)
Test yapıcısı.
- `List< MedicalRecord > findByPetId (int petId)`
PetId'ye göre sağlık kayıtlarını getirir.

Public Member Functions inherited from**com.mehdi.petreminder.repository.SqliteRepository< MedicalRecord >**

- [SqliteRepository](#) ()
SQLite bağlantısını açar ve tabloyu oluşturur.
- int [save](#) (T entity)
- Optional< T > [findById](#) (int id)
- List< T > [findAll](#) ()
- boolean [update](#) (T entity)
- boolean [delete](#) (int id)
- void [deleteAll](#) ()
- int [count](#) ()
- void [close](#) ()
- Connection [getConnection](#) ()
Bağlantıyı döndürür (test için).

Protected Member Functions

- String [getTableName](#) ()
- String [getCreateTableSql](#) ()
- String [getInsertSql](#) ()
- String [getUpdateSql](#) ()
- void [setInsertParams](#) (PreparedStatement ps, [MedicalRecord](#) m) throws SQLException
- void [setUpdateParams](#) (PreparedStatement ps, [MedicalRecord](#) m) throws SQLException
- [MedicalRecord](#) [mapRow](#) (ResultSet rs) throws SQLException

Protected Member Functions inherited from**com.mehdi.petreminder.repository.SqliteRepository< MedicalRecord >**

- abstract String [getTableName](#) ()
Tablo adını döndürür.
- abstract String [getCreateTableSql](#) ()
Tablo oluşturma SQL'ini döndürür.
- abstract String [getInsertSql](#) ()
INSERT SQL'ini döndürür (ID hariç).
- abstract String [getUpdateSql](#) ()
UPDATE SQL'ini döndürür.
- abstract void [setInsertParams](#) (PreparedStatement ps, T entity) throws SQLException
PreparedStatement'a INSERT parametrelerini set eder.
- abstract void [setUpdateParams](#) (PreparedStatement ps, T entity) throws SQLException
PreparedStatement'a UPDATE parametrelerini set eder.
- abstract T [mapRow](#) (ResultSet rs) throws SQLException
ResultSet'ten varlık oluşturur.
- void [createTableIfNotExists](#) ()
Tabloyu oluşturur (yoksa).
- void [setEntityId](#) (T entity, int id)
Reflection ile entity'ye id set eder.

Additional Inherited Members

Protected Attributes inherited from

[com.mehdi.petreminder.repository.SqliteRepository](#)< [MedicalRecord](#) >

- Connection [connection](#)
SQLite JDBC bağlantısı.

Static Protected Attributes inherited from

[com.mehdi.petreminder.repository.SqliteRepository](#)< [MedicalRecord](#) >

- static final Logger [logger](#)
Logger.

7.19.1 Detailed Description

SQLite üzerinde [MedicalRecord](#) CRUD.

Author

Muhammed Mehdi Karagülle
Ibrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 21 of file [MedicalRecordSqliteRepository.java](#).

7.19.2 Constructor & Destructor Documentation

7.19.2.1 [MedicalRecordSqliteRepository\(\)](#) [1/2]

```
com.mehdi.petreminder.repository.MedicalRecordSqliteRepository.MedicalRecordSqliteRepository  
( )
```

Varsayılan yapıcı.

Definition at line 24 of file [MedicalRecordSqliteRepository.java](#).

```
00024 { super(); }
```

7.19.2.2 MedicalRecordSqliteRepository() [2/2]

```
com.mehdi.petreminder.repository.MedicalRecordSqliteRepository.MedicalRecordSqliteRepository (
    Connection connection)
```

Test yapıcısı.

Parameters

<i>connection</i>	Test bağlantısı
-------------------	-----------------

Definition at line 30 of file [MedicalRecordSqliteRepository.java](#).

```
00030 { super(connection); }
```

References [com.mehdi.petreminder.repository.SqliteRepository< MedicalRecord >.connection](#).

7.19.3 Member Function Documentation

7.19.3.1 findByPetId()

```
List< MedicalRecord > com.mehdi.petreminder.repository.MedicalRecordSqliteRepository.findBy↵
PetId (
    int petId)
```

PetId'ye göre sağlık kayıtlarını getirir.

Parameters

<i>pet↵ Id</i>	Pet ID
--------------------	--------

Returns

[MedicalRecord](#) listesi

Definition at line 141 of file [MedicalRecordSqliteRepository.java](#).

```
00141
00142     List<MedicalRecord> list = new ArrayList<>();
00143     try (PreparedStatement ps = connection.prepareStatement(
00144         "SELECT * FROM medical_records WHERE pet_id = ?")) {
00145         ps.setInt(1, petId);
00146         ResultSet rs = ps.executeQuery();
00147         while (rs.next()) list.add(mapRow(rs));
00148     } catch (SQLException e) {
00149         throw new RepositoryException("findByPetId hatası", e);
00150     }
00151     return list;
00152 }
```

References [com.mehdi.petreminder.repository.SqliteRepository< MedicalRecord >.connection](#), and [mapRow\(\)](#).

7.19.3.2 getCreateTableSql()

String com.mehdi.petreminder.repository.MedicalRecordSqliteRepository.getCreateTableSql ()
[protected]

Member documentation.

Definition at line 44 of file [MedicalRecordSqliteRepository.java](#).

```
00044      {
00045          return "CREATE TABLE IF NOT EXISTS medical_records ("
00046              + "id INTEGER PRIMARY KEY AUTOINCREMENT,"
00047              + "pet_id INTEGER NOT NULL,"
00048              + "pet_name TEXT,"
00049              + "record_date TEXT NOT NULL,"
00050              + "record_type TEXT NOT NULL,"
00051              + "diagnosis TEXT,"
00052              + "treatment TEXT,"
00053              + "veterinarian_name TEXT,"
00054              + "cost REAL DEFAULT 0,"
00055              + "next_check_date TEXT,"
00056              + "notes TEXT,"
00057              + "vaccine_name TEXT"
00058              + ")";
00059      }
```

7.19.3.3 getInsertSql()

String com.mehdi.petreminder.repository.MedicalRecordSqliteRepository.getInsertSql () [protected]

Member documentation.

Definition at line 66 of file [MedicalRecordSqliteRepository.java](#).

```
00066      {
00067          return "INSERT INTO medical_records (pet_id,pet_name,record_date,record_type,"
00068              + "diagnosis,treatment,veterinarian_name,cost,next_check_date,notes,vaccine_name) "
00069              + " VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?) ";
00070      }
```

7.19.3.4 getTableName()

String com.mehdi.petreminder.repository.MedicalRecordSqliteRepository.getTableName () [protected]

Member documentation.

Definition at line 37 of file [MedicalRecordSqliteRepository.java](#).

```
00037 { return "medical_records"; }
```

7.19.3.5 getUpdateSql()

String com.mehdi.petreminder.repository.MedicalRecordSqliteRepository.getUpdateSql () [protected]

Member documentation.

Definition at line 77 of file [MedicalRecordSqliteRepository.java](#).

```
00077      {
00078          return "UPDATE medical_records SET pet_id=?,pet_name=?,record_date=?,record_type=?, "
00079              + "diagnosis=?,treatment=?,veterinarian_name=?,cost=?,next_check_date=?, "
00080              + "notes=?,vaccine_name=? WHERE id=?";
00081      }
```

7.19.3.6 mapRow()

`MedicalRecord` com.mehdi.petreminder.repository.MedicalRecordSqliteRepository.mapRow (
 `ResultSet rs`) throws `SQLException` [protected]

Member documentation.

Definition at line 117 of file `MedicalRecordSqliteRepository.java`.

```
00117                                     {
00118         MedicalRecord m = new MedicalRecord();
00119         m.setId(rs.getInt("id"));
00120         m.setPetId(rs.getInt("pet_id"));
00121         m.setPetName(rs.getString("pet_name"));
00122         String rd = rs.getString("record_date");
00123         if (rd != null) m.setRecordDate(LocalDate.parse(rd));
00124         m.setRecordType(rs.getString("record_type"));
00125         m.setDiagnosis(rs.getString("diagnosis"));
00126         m.setTreatment(rs.getString("treatment"));
00127         m.setVeterinarianName(rs.getString("veterinarian_name"));
00128         m.setCost(rs.getDouble("cost"));
00129         String ncd = rs.getString("next_check_date");
00130         if (ncd != null) m.setNextCheckDate(LocalDate.parse(ncd));
00131         m.setNotes(rs.getString("notes"));
00132         m.setVaccineName(rs.getString("vaccine_name"));
00133         return m;
00134     }
```

References `com.mehdi.petreminder.model.MedicalRecord.setCost()`, `com.mehdi.petreminder.model.MedicalRecord.setDiagnosis()`, `com.mehdi.petreminder.model.MedicalRecord.setId()`, `com.mehdi.petreminder.model.MedicalRecord.setNextCheckDate()`, `com.mehdi.petreminder.model.MedicalRecord.setNotes()`, `com.mehdi.petreminder.model.MedicalRecord.setPetId()`, `com.mehdi.petreminder.model.MedicalRecord.setPetName()`, `com.mehdi.petreminder.model.MedicalRecord.setRecordDate()`, `com.mehdi.petreminder.model.MedicalRecord.setRecordType()`, `com.mehdi.petreminder.model.MedicalRecord.setTreatment()`, `com.mehdi.petreminder.model.MedicalRecord.setVaccineName()`, and `com.mehdi.petreminder.model.MedicalRecord.setVeterinarianName()`.

Referenced by `findByPetId()`.

7.19.3.7 setInsertParams()

`void` com.mehdi.petreminder.repository.MedicalRecordSqliteRepository.setInsertParams (
 `PreparedStatement ps`,
 `MedicalRecord m`) throws `SQLException` [protected]

Member documentation.

Definition at line 88 of file `MedicalRecordSqliteRepository.java`.

```
00088                                     {
00089         ps.setInt(1, m.getPetId());
00090         ps.setString(2, m.getPetName());
00091         ps.setString(3, m.getRecordDate() != null ? m.getRecordDate().toString() : null);
00092         ps.setString(4, m.getRecordType());
00093         ps.setString(5, m.getDiagnosis());
00094         ps.setString(6, m.getTreatment());
00095         ps.setString(7, m.getVeterinarianName());
00096         ps.setDouble(8, m.getCost());
00097         ps.setString(9, m.getNextCheckDate() != null ? m.getNextCheckDate().toString() : null);
00098         ps.setString(10, m.getNotes());
00099         ps.setString(11, m.getVaccineName());
00100     }
```

Referenced by `setUpdateParams()`.

7.19.3.8 setUpdateParams()

```
void com.mehdi.petreminder.repository.MedicalRecordSqliteRepository.setUpdateParams (
    PreparedStatement ps,
    MedicalRecord m) throws SQLException [protected]
```

Member documentation.

Definition at line 107 of file [MedicalRecordSqliteRepository.java](#).

```
00107
00108         setInsertParams(ps, m);
00109         ps.setInt(12, m.getId());
00110     }
```

References [setInsertParams\(\)](#).

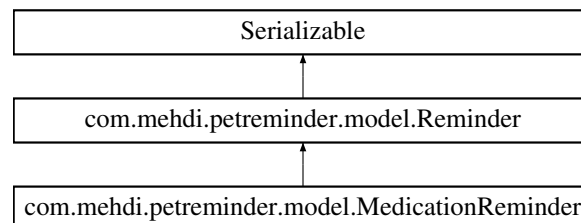
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/repository/MedicalRecordSqliteRepository.java](#)

7.20 com.mehdi.petreminder.model.MedicationReminder Class Reference

Evcil hayvan ilaç hatırlatıcısı.

Inheritance diagram for com.mehdi.petreminder.model.MedicationReminder:



Public Member Functions

- [MedicationReminder \(\)](#)
Varsayılan yapıcı.
- [MedicationReminder \(int id, int petId, String petName, LocalDateTime scheduledTime, String medicationName, double dosage, String dosageUnit\)](#)
Temel parametrelili yapıcı.
- [MedicationReminder \(int id, int petId, String petName, LocalDateTime scheduledTime, String description, String priority, boolean recurring, String recurrencePattern, String medicationName, double dosage, String dosageUnit, boolean prescribed, String veterinarianName\)](#)
Tam parametrelili yapıcı.
- String [getMedicationName \(\)](#)
İlaç adı getter.
- void [setMedicationName \(String medicationName\)](#)
İlaç adı setter.
- double [getDosage \(\)](#)

- Doz getter.*
- void [setDosage](#) (double dosage)
- Doz setter.*
- String [getDosageUnit](#) ()
- Birim getter.*
- void [setDosageUnit](#) (String dosageUnit)
- Birim setter.*
- boolean [isPrescribed](#) ()
- Reçeteli mi getter.*
- void [setPrescribed](#) (boolean prescribed)
- Reçeteli setter.*
- String [getVeterinarianName](#) ()
- Veteriner adı getter.*
- void [setVeterinarianName](#) (String veterinarianName)
- Veteriner adı setter.*
- String [getReminderType](#) ()
- Hatırlatıcı türü.*
- boolean [validate](#) ()
- İlaç hatırlatıcısını doğrular.*
- String [getSummary](#) ()
- Özet bilgi.*
- String [toString](#) ()
- String temsili.*

Public Member Functions inherited from [com.mehdi.petreminder.model.Reminder](#)

- int [getId](#) ()
- ID getter.*
- void [setId](#) (int id)
- ID setter.*
- int [getPetId](#) ()
- Pet ID getter.*
- void [setPetId](#) (int petId)
- Pet ID setter.*
- String [getPetName](#) ()
- Pet adı getter.*
- void [setPetName](#) (String petName)
- Pet adı setter.*
- LocalDateTime [getScheduledTime](#) ()
- Planlanan zaman getter.*
- void [setScheduledTime](#) (LocalDateTime scheduledTime)
- Planlanan zaman setter.*
- boolean [isCompleted](#) ()
- Tamamlandı mı getter.*
- void [setCompleted](#) (boolean completed)
- Tamamlandı setter.*
- String [getDescription](#) ()
- Açıklama getter.*
- void [setDescription](#) (String description)
- Açıklama setter.*

- String [getPriority](#) ()
Öncelik getter.
- void [setPriority](#) (String priority)
Öncelik setter.
- boolean [isRecurring](#) ()
Tekrarlıyor mu getter.
- void [setRecurring](#) (boolean recurring)
Tekrarlama setter.
- String [getRecurrencePattern](#) ()
Tekrar deseni getter.
- void [setRecurrencePattern](#) (String recurrencePattern)
Tekrar deseni setter.
- String [getFormattedTime](#) ()
Planlanan zamanı biçimli string olarak döndürür.
- boolean [isOverdue](#) ()
Hatırlatıcının geçmiş mi olduğunu kontrol eder.
- boolean [equals](#) (Object obj)
Eşitlik kontrolü — ID bazlı.
- int [hashCode](#) ()
Hash kodu.

Additional Inherited Members

Protected Member Functions inherited from [com.mehdi.petreminder.model.Reminder](#)

- [Reminder](#) ()
Varsayılan yapıcı — Serialization için.
- [Reminder](#) (int id, int petId, String petName, LocalDateTime scheduledTime, String description, String priority, boolean recurring, String recurrencePattern)
Tam parametrelili yapıcı.

Static Protected Attributes inherited from [com.mehdi.petreminder.model.Reminder](#)

- static final DateFormatter [FORMATTER](#)
Tarih saat formatı.

7.20.1 Detailed Description

Evcil hayvan ilaç hatırlatıcısı.

OOP Inheritance: [Reminder](#)'dan miras alır.

Author

Muhammed Mehdi Karagülle, İbrahim Demirci, Zümre Uykun

Version

1.0

Definition at line 16 of file [MedicationReminder.java](#).

7.20.2 Constructor & Destructor Documentation

7.20.2.1 MedicationReminder() [1/3]

com.mehdi.petreminder.model.MedicationReminder.MedicationReminder ()

Varsayılan yapıcı.

Definition at line 37 of file [MedicationReminder.java](#).

```
00037 { super(); }
```

7.20.2.2 MedicationReminder() [2/3]

```
com.mehdi.petreminder.model.MedicationReminder.MedicationReminder (
    int id,
    int petId,
    String petName,
    LocalDateTime scheduledTime,
    String medicationName,
    double dosage,
    String dosageUnit)
```

Temel parametrelili yapıcı.

Parameters

<i>id</i>	ID
<i>petId</i>	Pet ID
<i>petName</i>	Pet adı
<i>scheduledTime</i>	Zaman
<i>medicationName</i>	İlaç adı
<i>dosage</i>	Doz
<i>dosageUnit</i>	Birim

Definition at line 49 of file [MedicationReminder.java](#).

```
00052
00053         super(id, petId, petName, scheduledTime,
00054             "İlaç: " + medicationName, "YUKSEK", false, "");
00055         this.medicationName = medicationName;
00056         this.dosage = dosage;
00057         this.dosageUnit = dosageUnit;
00058         this.prescribed = false;
00059         this.veterinarianName = "";
00060     }
```

7.20.2.3 MedicationReminder() [3/3]

```
com.mehdi.petreminder.model.MedicationReminder.MedicationReminder (
    int id,
    int petId,
    String petName,
    LocalDateTime scheduledTime,
    String description,
    String priority,
    boolean recurring,
    String recurrencePattern,
    String medicationName,
    double dosage,
    String dosageUnit,
    boolean prescribed,
    String veterinarianName)
```

Tam parametrelili yapıcı.

Parameters

<i>id</i>	ID
<i>petId</i>	Pet ID
<i>petName</i>	Pet adı
<i>scheduledTime</i>	Zaman
<i>description</i>	Açıklama
<i>priority</i>	Öncelik
<i>recurring</i>	Tekrarlıyor mu
<i>recurrencePattern</i>	Tekrar deseni
<i>medicationName</i>	İlaç adı
<i>dosage</i>	Doz
<i>dosageUnit</i>	Birim
<i>prescribed</i>	Reçeteli mi
<i>veterinarianName</i>	Veteriner adı

Definition at line 78 of file [MedicationReminder.java](#).

```
00083
00084         super(id, petId, petName, scheduledTime, description,
00085             priority, recurring, recurrencePattern);
00086         this.medicationName = medicationName;
00087         this.dosage = dosage;
00088         this.dosageUnit = dosageUnit;
00089         this.prescribed = prescribed;
00090         this.veterinarianName = veterinarianName;
00091     }
```

7.20.3 Member Function Documentation

7.20.3.1 getDosage()

```
double com.mehdi.petreminder.model.MedicationReminder.getDosage ()
```

Doz getter.

Returns

Doz

Definition at line 102 of file [MedicationReminder.java](#).

```
00102 { return dosage; }
```

7.20.3.2 getDosageUnit()

```
String com.mehdi.petreminder.model.MedicationReminder.getDosageUnit ()
```

Birim getter.

Returns

Birim

Definition at line 108 of file [MedicationReminder.java](#).

```
00108 { return dosageUnit; }
```

7.20.3.3 getMedicationName()

```
String com.mehdi.petreminder.model.MedicationReminder.getMedicationName ()
```

İlaç adı getter.

Returns

İlaç adı

Definition at line 94 of file [MedicationReminder.java](#).

```
00094 { return medicationName; }
```

7.20.3.4 getReminderType()

```
String com.mehdi.petreminder.model.MedicationReminder.getReminderType ()
```

Hatırlatıcı türü.

Returns

"Ilac"

Reimplemented from [com.mehdi.petreminder.model.Reminder](#).

Definition at line 129 of file [MedicationReminder.java](#).

```
00129 { return "Ilac"; }
```

7.20.3.5 getSummary()

```
String com.mehdi.petreminder.model.MedicationReminder.getSummary ()
```

Özet bilgi.

Returns

"İlaç: Antibiyotik 250mg"

Reimplemented from [com.mehdi.petreminder.model.Reminder](#).

Definition at line 147 of file [MedicationReminder.java](#).

```
00147         {  
00148             return String.format("İlaç: %s %.1f%s%s",  
00149                 medicationName, dosage, dosageUnit,  
00150                 prescribed ? " (Receteli)" : "");  
00151         }
```

7.20.3.6 getVeterinarianName()

```
String com.mehdi.petreminder.model.MedicationReminder.getVeterinarianName ()
```

Veteriner adı getter.

Returns

Veteriner adı

Definition at line 120 of file [MedicationReminder.java](#).

```
00120 { return veterinarianName; }
```

7.20.3.7 isPrescribed()

```
boolean com.mehdi.petreminder.model.MedicationReminder.isPrescribed ()
```

Reçeteli mi getter.

Returns

Reçeteli mi

Definition at line 114 of file [MedicationReminder.java](#).

```
00114 { return prescribed; }
```

7.20.3.8 setDosage()

```
void com.mehdi.petreminder.model.MedicationReminder.setDosage (
    double dosage)
```

Doz setter.

Parameters

<i>dosage</i>	Doz
---------------	-----

Definition at line 105 of file [MedicationReminder.java](#).

```
00105 { this.dosage = dosage; }
```

7.20.3.9 setDosageUnit()

```
void com.mehdi.petreminder.model.MedicationReminder.setDosageUnit (
    String dosageUnit)
```

Birim setter.

Parameters

<i>dosageUnit</i>	Birim
-------------------	-------

Definition at line 111 of file [MedicationReminder.java](#).

```
00111 { this.dosageUnit = dosageUnit; }
```

7.20.3.10 setMedicationName()

```
void com.mehdi.petreminder.model.MedicationReminder.setMedicationName (
    String medicationName)
```

İlaç adı setter.

Parameters

<i>medicationName</i>	İlaç adı
-----------------------	----------

Definition at line 97 of file [MedicationReminder.java](#).

```
00097                                     {
00098     this.medicationName = medicationName;
00099 }
```

7.20.3.11 setPrescribed()

```
void com.mehdi.petreminder.model.MedicationReminder.setPrescribed (
    boolean prescribed)
```

Reçeteli setter.

Parameters

<i>prescribed</i>	Reçeteli mi
-------------------	-------------

Definition at line 117 of file [MedicationReminder.java](#).

```
00117 { this.prescribed = prescribed; }
```

7.20.3.12 setVeterinarianName()

```
void com.mehdi.petreminder.model.MedicationReminder.setVeterinarianName (
    String veterinarianName)
```

Veteriner adı setter.

Parameters

<i>veterinarianName</i>	Veteriner adı
-------------------------	---------------

Definition at line 123 of file [MedicationReminder.java](#).

```
00123                                     {
00124         this.veterinarianName = veterinarianName;
00125     }
```

7.20.3.13 toString()

```
String com.mehdi.petreminder.model.MedicationReminder.toString ()
```

String temsili.

Returns

İlaç hatırlatıcısı bilgisi

Reimplemented from [com.mehdi.petreminder.model.Reminder](#).

Definition at line 158 of file [MedicationReminder.java](#).

```
00158         {
00159         return String.format("MedicationReminder{id=%d, pet='%s', med='%s', dose=%.1f%s}",
00160             getId(), getPetName(), medicationName, dosage, dosageUnit);
00161     }
```

References [com.mehdi.petreminder.model.Reminder.getId\(\)](#), and [com.mehdi.petreminder.model.Reminder.getPetName\(\)](#).

7.20.3.14 validate()

```
boolean com.mehdi.petreminder.model.MedicationReminder.validate ()
```

İlaç hatırlatıcısını doğrular.

Returns

İlaç adı ve doz geçerliyse true

Reimplemented from [com.mehdi.petreminder.model.Reminder](#).

Definition at line 136 of file [MedicationReminder.java](#).

```
00136         {
00137             return medicationName != null && !medicationName.trim().isEmpty()
00138                && dosage > 0
00139                && dosageUnit != null && !dosageUnit.trim().isEmpty();
00140     }
```

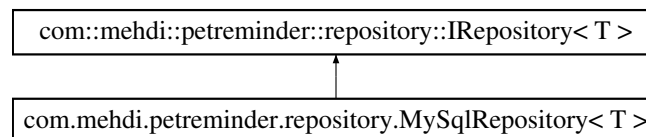
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/model/MedicationReminder.java](#)

7.21 com.mehdi.petreminder.repository.MySqlRepository< T > Class Template Reference

MySQL tabanlı abstract repository temel sınıfı.

Inheritance diagram for com.mehdi.petreminder.repository.MySqlRepository< T >:



Public Member Functions

- [MySqlRepository](#) ()
MySQL bağlantısını açar.
- [MySqlRepository](#) (Connection [connection](#))
Test amaçlı yapıcı.
- int [save](#) (T entity)
- Optional< T > [findById](#) (int id)
- List< T > [findAll](#) ()
- boolean [update](#) (T entity)
- boolean [delete](#) (int id)
- void [deleteAll](#) ()
- int [count](#) ()
- void [close](#) ()
- Connection [getConnection](#) ()
Bağlantıyı döndürür.

Protected Member Functions

- abstract String [getTableName](#) ()
Tablo adı.
- abstract String [getCreateTableSql](#) ()
Tablo oluşturma SQL.
- abstract String [getInsertSql](#) ()
INSERT SQL.
- abstract String [getUpdateSql](#) ()
UPDATE SQL.
- abstract void [setInsertParams](#) (PreparedStatement ps, T entity) throws SQLException
INSERT parametrelerini set eder.
- abstract void [setUpdateParams](#) (PreparedStatement ps, T entity) throws SQLException
UPDATE parametrelerini set eder.
- abstract T [mapRow](#) (ResultSet rs) throws SQLException
ResultSet'ten varlık oluşturur.
- void [createTableIfNotExists](#) ()
Tabloyu oluşturur (yoksa).
- void [setEntityId](#) (T entity, int id)
Reflection ile entity'ye id set eder.

Protected Attributes

- Connection [connection](#)
MySQL JDBC bağlantısı.

Static Protected Attributes

- static final Logger [logger](#)
Logger.

7.21.1 Detailed Description

MySQL tabanlı abstract repository temel sınıfı.

[SqliteRepository](#) ile aynı Template Method Pattern kullanır. Docker Compose ile ayağa kaldırılan MySQL 8.0'a bağlanır.

Parameters

< T >	Varlık türü
-------	-------------

Author

Muhammed Mehdi Karagülle
İbrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 26 of file [MySqlRepository.java](#).

7.21.2 Constructor & Destructor Documentation

7.21.2.1 MySqlRepository() [1/2]

`com.mehdi.petreminder.repository.MySqlRepository< T >.MySqlRepository ()`

MySQL bağlantısını açar.

Exceptions

<i>RepositoryException</i>	Bağlantı başarısız olursa
--	---------------------------

Definition at line 39 of file [MySqlRepository.java](#).

```

00039                                     {
00040         try {
00041             Class.forName("com.mysql.cj.jdbc.Driver");
00042             connection = DriverManager.getConnection(
00043                 StorageConfig.getMysqlUrl(),
00044                 StorageConfig.getMysqlUsername(),
00045                 StorageConfig.getMysqlPassword());
00046             connection.setAutoCommit(true);
00047             createTableIfNotExists();
00048             logger.info("MySQL bağlantısı açıldı: {}", StorageConfig.getMysqlUrl());
00049         } catch (Exception e) {
00050             throw new RepositoryException("MySQL bağlantı hatası -- Docker çalışıyor mu?", e);
00051         }
00052     }

```

References [connection](#), [createTableIfNotExists\(\)](#), [com.mehdi.petreminder.config.StorageConfig.getMysqlPassword\(\)](#), [com.mehdi.petreminder.config.StorageConfig.getMysqlUrl\(\)](#), [com.mehdi.petreminder.config.StorageConfig.getMysqlUsername\(\)](#), and [logger](#).

7.21.2.2 MySqlRepository() [2/2]

`com.mehdi.petreminder.repository.MySqlRepository< T >.MySqlRepository (Connection connection)`

Test amaçlı yapıcı.

Parameters

<i>connection</i>	Test bağlantısı (H2 MySQL-mode)
-------------------	---------------------------------

Definition at line 58 of file [MySqlRepository.java](#).

```

00058                                     {
00059         this.connection = connection;
00060         try {
00061             createTableIfNotExists();
00062         } catch (Exception e) {
00063             throw new RepositoryException("Tablo oluşturma hatası", e);
00064         }
00065     }

```

References [connection](#), and [createTableIfNotExists\(\)](#).

7.21.3 Member Function Documentation

7.21.3.1 close()

```
void com.mehdi.petreminder.repository.MySqlRepository< T >.close ()
```

Repository bağlantısını/kaynaklarını kapatır.

Implements [com.mehdi.petreminder.repository.IRepository< T >](#).

Definition at line 211 of file [MySqlRepository.java](#).

```
00211         {
00212             try {
00213                 if (connection != null && !connection.isClosed()) {
00214                     connection.close();
00215                     logger.info("MySQL bağlantısı kapatıldı.");
00216                 }
00217             } catch (SQLException e) {
00218                 logger.warn("MySQL kapatma hatası: {}", e.getMessage());
00219             }
00220         }
```

References [connection](#), and [logger](#).

7.21.3.2 count()

```
int com.mehdi.petreminder.repository.MySqlRepository< T >.count ()
```

Varlık sayısını döndürür.

Returns

Toplam kayıt sayısı

Implements [com.mehdi.petreminder.repository.IRepository< T >](#).

Definition at line 199 of file [MySqlRepository.java](#).

```
00199         {
00200             try (Statement st = connection.createStatement();
00201                 ResultSet rs = st.executeQuery("SELECT COUNT(*) FROM " + getTableName())) {
00202                 if (rs.next()) return rs.getInt(1);
00203             } catch (SQLException e) {
00204                 throw new RepositoryException("MySQL count hatası", e);
00205             }
00206             return 0;
00207         }
```

References [connection](#), and [getTableName\(\)](#).

7.21.3.3 createTableIfNotExists()

```
void com.mehdi.petreminder.repository.MySqlRepository< T >.createTableIfNotExists () [protected]
```

Tabloyu oluşturur (yoksa).

Definition at line 108 of file [MySqlRepository.java](#).

```
00108         {
00109             try (Statement st = connection.createStatement()) {
00110                 st.execute(getCreateTableSql());
00111             } catch (SQLException e) {
00112                 throw new RepositoryException("Tablo oluşturma hatası", e);
00113             }
00114         }
```

References [connection](#), and [getCreateTableSql\(\)](#).

Referenced by [MySqlRepository\(\)](#), and [MySqlRepository\(\)](#).

7.21.3.4 delete()

```
boolean com.mehdi.petreminder.repository.MySqlRepository< T >.delete (
    int id)
```

ID'ye göre varlık siler.

Parameters

<i>id</i>	Silinecek varlığın ID'si
-----------	--------------------------

Returns

Silme başarılıysa true

Exceptions

RepositoryException	Silme başarısız olursa
-------------------------------------	------------------------

Implements [com.mehdi.petreminder.repository.IRepository< T >](#).

Definition at line 177 of file [MySqlRepository.java](#).

```
00177         {
00178             try (PreparedStatement ps = connection.prepareStatement(
00179                 "DELETE FROM " + getTableName() + " WHERE id = ?")) {
00180                 ps.setInt(1, id);
00181                 return ps.executeUpdate() > 0;
00182             } catch (SQLException e) {
00183                 throw new RepositoryException("MySQL delete hatası", e);
00184             }
00185         }
```

References [connection](#), and [getTableName\(\)](#).

7.21.3.5 deleteAll()

```
void com.mehdi.petreminder.repository.MySqlRepository< T >.deleteAll ()
```

Tüm varlıkları siler.

Exceptions

RepositoryException	Silme başarısız olursa
-------------------------------------	------------------------

Implements [com.mehdi.petreminder.repository.IRepository< T >](#).

Definition at line 189 of file [MySqlRepository.java](#).

```
00189         {
00190             try (Statement st = connection.createStatement()) {
00191                 st.execute("DELETE FROM " + getTableName());
00192             } catch (SQLException e) {
00193                 throw new RepositoryException("MySQL deleteAll hatası", e);
00194             }
00195         }
```

References [connection](#), and [getTableName\(\)](#).

7.21.3.6 findAll()

List< T > [com.mehdi.petreminder.repository.MySqlRepository](#)< T >.findAll ()

Tüm varlıkları listeler.

Returns

Varlık listesi (boş olabilir)

Exceptions

RepositoryException	Okuma başarısız olursa
-------------------------------------	------------------------

Implements [com.mehdi.petreminder.repository.IRepository](#)< T >.

Definition at line 152 of file [MySqlRepository.java](#).

```

00152         {
00153             List<T> list = new ArrayList<>();
00154             try (Statement st = connection.createStatement();
00155                 ResultSet rs = st.executeQuery("SELECT * FROM " + getTableName())) {
00156                 while (rs.next()) list.add(mapRow(rs));
00157             } catch (SQLException e) {
00158                 throw new RepositoryException("MySQL findAll hatası", e);
00159             }
00160             return list;
00161         }

```

References [connection](#), [getTableName\(\)](#), and [mapRow\(\)](#).

7.21.3.7 findById()

Optional< T > [com.mehdi.petreminder.repository.MySqlRepository](#)< T >.findById (int id)

ID'ye göre varlık getirir.

Parameters

<i>id</i>	Varlık ID'si
-----------	--------------

Returns

Varlık (bulunamazsa empty Optional)

Exceptions

RepositoryException	Okuma başarısız olursa
-------------------------------------	------------------------

Implements [com.mehdi.petreminder.repository.IRepository](#)< T >.

Definition at line 138 of file [MySqlRepository.java](#).

```

00138         {
00139             String sql = "SELECT * FROM " + getTableName() + " WHERE id = ?";
00140             try (PreparedStatement ps = connection.prepareStatement(sql)) {
00141                 ps.setInt(1, id);
00142                 ResultSet rs = ps.executeQuery();
00143                 if (rs.next()) return Optional.of(mapRow(rs));
00144             } catch (SQLException e) {
00145                 throw new RepositoryException("MySQL findById hatası", e);
00146             }
00147             return Optional.empty();
00148         }

```

References [connection](#), [getTableName\(\)](#), and [mapRow\(\)](#).

7.21.3.8 getConnection()

Connection [com.mehdi.petreminder.repository.MySqlRepository< T >.getConnection \(\)](#)

Bağlantıyı döndürür.

Returns

JDBC Connection

Definition at line 226 of file [MySqlRepository.java](#).

```
00226                                     {
00227         return connection;
00228     }
```

References [connection](#).

7.21.3.9 getCreateTableSql()

abstract String [com.mehdi.petreminder.repository.MySqlRepository< T >.getCreateTableSql \(\)](#) [abstract], [protected]

Tablo oluşturma SQL.

Returns

CREATE TABLE sql

Referenced by [createTableIfNotExists\(\)](#).

7.21.3.10 getInsertSql()

abstract String [com.mehdi.petreminder.repository.MySqlRepository< T >.getInsertSql \(\)](#) [abstract], [protected]

INSERT SQL.

Returns

INSERT sql

Referenced by [save\(\)](#).

7.21.3.11 getTableName()

abstract String [com.mehdi.petreminder.repository.MySqlRepository< T >.getTableName \(\)](#) [abstract], [protected]

Tablo adı.

Returns

SQL tablo adı

Referenced by [count\(\)](#), [delete\(\)](#), [deleteAll\(\)](#), [findAll\(\)](#), and [findById\(\)](#).

7.21.3.12 getUpdateSql()

```
abstract String com.mehdi.petreminder.repository.MySqlRepository< T >.getUpdateSql () [abstract],  
[protected]
```

UPDATE SQL.

Returns

UPDATE sql

Referenced by [update\(\)](#).

7.21.3.13 mapRow()

```
abstract T com.mehdi.petreminder.repository.MySqlRepository< T >.mapRow (  
    ResultSet rs) throws SQLException [abstract], [protected]
```

ResultSet'ten varlık oluşturur.

Parameters

<i>rs</i>	ResultSet
-----------	-----------

Returns

Varlık nesnesi

Exceptions

<i>SQLException</i>	SQL hatası
---------------------	------------

Referenced by [findAll\(\)](#), and [findById\(\)](#).

7.21.3.14 save()

```
int com.mehdi.petreminder.repository.MySqlRepository< T >.save (  
    T entity)
```

Yeni bir varlık kaydeder.

Parameters

<i>entity</i>	Kaydedilecek varlık
---------------	---------------------

Returns

Oluşturulan varlığın ID'si

Exceptions

RepositoryException	Kayıt başarısız olursa
-------------------------------------	------------------------

Implements [com.mehdi.petreminder.repository.IRepository< T >](#).

Definition at line 118 of file [MySqlRepository.java](#).

```

00118         {
00119             if (entity == null) throw new RepositoryException("Entity null olamaz");
00120             try (PreparedStatement ps = connection.prepareStatement(
00121                 getInsertSql(), Statement.RETURN_GENERATED_KEYS)) {
00122                 setInsertParams(ps, entity);
00123                 ps.executeUpdate();
00124                 ResultSet keys = ps.getGeneratedKeys();
00125                 if (keys.next()) {
00126                     int id = keys.getInt(1);
00127                     setEntityId(entity, id);
00128                     return id;
00129                 }
00130                 throw new RepositoryException("MySQL ID üretilemedi");
00131             } catch (SQLException e) {
00132                 throw new RepositoryException("MySQL save hatası", e);
00133             }
00134         }

```

References [connection](#), [getInsertSql\(\)](#), [setEntityId\(\)](#), and [setInsertParams\(\)](#).

7.21.3.15 setEntityId()

```

void com.mehdi.petreminder.repository.MySqlRepository< T >.setEntityId (
    T entity,
    int id) [protected]

```

Reflection ile entity'ye id set eder.

Parameters

<i>entity</i>	Varlık
<i>id</i>	ID

Definition at line 235 of file [MySqlRepository.java](#).

```

00235         {
00236             try {
00237                 var method = entity.getClass().getMethod("setId", int.class);
00238                 method.invoke(entity, id);
00239             } catch (Exception e) {
00240                 logger.warn("setId çağrılmadı: {}", e.getMessage());
00241             }
00242         }

```

References [logger](#).

Referenced by [save\(\)](#).

7.21.3.16 setInsertParams()

```
abstract void com.mehdi.petreminder.repository.MySqlRepository< T >.setInsertParams (
    PreparedStatement ps,
    T entity) throws SQLException [abstract], [protected]
```

INSERT parametrelerini set eder.

Parameters

<i>ps</i>	PreparedStatement
<i>entity</i>	Varlık

Exceptions

<i>SQLException</i>	SQL hatası
---------------------	------------

Referenced by [save\(\)](#).

7.21.3.17 setUpdateParams()

```
abstract void com.mehdi.petreminder.repository.MySqlRepository< T >.setUpdateParams (
    PreparedStatement ps,
    T entity) throws SQLException [abstract], [protected]
```

UPDATE parametrelerini set eder.

Parameters

<i>ps</i>	PreparedStatement
<i>entity</i>	Varlık

Exceptions

<i>SQLException</i>	SQL hatası
---------------------	------------

Referenced by [update\(\)](#).

7.21.3.18 update()

```
boolean com.mehdi.petreminder.repository.MySqlRepository< T >.update (
    T entity)
```

Mevcut varlığı günceller.

Parameters

<i>entity</i>	Güncellenecek varlık
---------------	----------------------

Returns

Güncelleme başarılıysa true

Exceptions

RepositoryException	Güncelleme başarısız olursa
-------------------------------------	-----------------------------

Implements [com.mehdi.petreminder.repository.IRepository< T >](#).

Definition at line 165 of file [MySqlRepository.java](#).

```

00165         {
00166             if (entity == null) return false;
00167             try (PreparedStatement ps = connection.prepareStatement(getUpdateSql())) {
00168                 setUpdateParams(ps, entity);
00169                 return ps.executeUpdate() > 0;
00170             } catch (SQLException e) {
00171                 throw new RepositoryException("MySQL update hatası", e);
00172             }
00173         }

```

References [connection](#), [getUpdateSql\(\)](#), and [setUpdateParams\(\)](#).

7.21.4 Member Data Documentation

7.21.4.1 connection

Connection [com.mehdi.petreminder.repository.MySqlRepository< T >.connection](#) [protected]

MySQL JDBC bağlantısı.

Definition at line 33 of file [MySqlRepository.java](#).

Referenced by [close\(\)](#), [count\(\)](#), [createTableIfNotExists\(\)](#), [delete\(\)](#), [deleteAll\(\)](#), [findAll\(\)](#), [findById\(\)](#), [getConnection\(\)](#), [MySqlRepository\(\)](#), [MySqlRepository\(\)](#), [save\(\)](#), and [update\(\)](#).

7.21.4.2 logger

final Logger [com.mehdi.petreminder.repository.MySqlRepository< T >.logger](#) [static], [protected]

Initial value:

```

=
    (Logger) LoggerFactory.getLogger(MySqlRepository.class)

```

Logger.

Definition at line 29 of file [MySqlRepository.java](#).

Referenced by [close\(\)](#), [MySqlRepository\(\)](#), and [setEntityId\(\)](#).

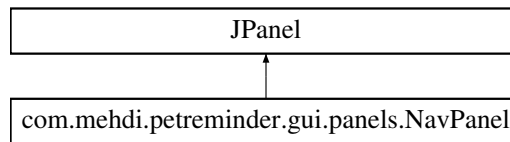
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/repository/MySqlRepository.java](#)

7.22 com.mehdi.petreminder.gui.panels.NavPanel Class Reference

Sol navigasyon çubuğu — menü butonları.

Inheritance diagram for com.mehdi.petreminder.gui.panels.NavPanel:



Public Member Functions

- [NavPanel](#) ([MainFrame](#) mainFrame)
NavPanel yapıcısı.
- void [setActiveButton](#) (String cardName)
Aktif butonu vurgular.

7.22.1 Detailed Description

Sol navigasyon çubuğu — menü butonları.

Her buton ilgili karta navigasyon yapar. Aktif buton vurgulanır.

Author

Muhammed Mehdi Karagülle
Ibrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 33 of file [NavPanel.java](#).

7.22.2 Constructor & Destructor Documentation

7.22.2.1 NavPanel()

`com.mehdi.petreminder.gui.panels.NavPanel.NavPanel (`
 [MainFrame](#) *mainFrame*)

[NavPanel](#) yapıcısı.

Parameters

<i>mainFrame</i>	Ana pencere
------------------	-------------

Definition at line 51 of file [NavPanel.java](#).

```
00051         {
00052             this.mainFrame = mainFrame;
00053             setBackground(GuiConstants.NAV_COLOR);
00054             setPreferredSize(new Dimension(GuiConstants.NAV_WIDTH, 0));
00055             setLayout(new BoxLayout(this, BoxLayout.Y_AXIS));
00056             buildNav();
00057         }
```

References [com.mehdi.petreminder.gui.util.GuiConstants.NAV_COLOR](#), and [com.mehdi.petreminder.gui.util.GuiConstants.NAV_WID](#)

7.22.3 Member Function Documentation

7.22.3.1 setActiveButton()

`void com.mehdi.petreminder.gui.panels.NavPanel.setActiveButton (`
 String *cardName*)

Aktif butonu vurgular.

Parameters

<i>cardName</i>	Aktif kart adı
-----------------	----------------

Definition at line 139 of file [NavPanel.java](#).

```
00139         {
00140             if (activeCard != null && buttons.containsKey(activeCard)) {
00141                 JButton prev = buttons.get(activeCard);
00142                 prev.setBackground(GuiConstants.NAV_COLOR);
00143                 prev.setForeground(new Color(200, 210, 230));
00144             }
00145             activeCard = cardName;
00146             if (buttons.containsKey(cardName)) {
00147                 JButton active = buttons.get(cardName);
00148                 active.setBackground(GuiConstants.NAV_ACTIVE);
00149                 active.setForeground(Color.WHITE);
00150             }
00151         }
```

References [com.mehdi.petreminder.gui.util.GuiConstants.NAV_ACTIVE](#), and [com.mehdi.petreminder.gui.util.GuiConstants.NAV_CO](#)

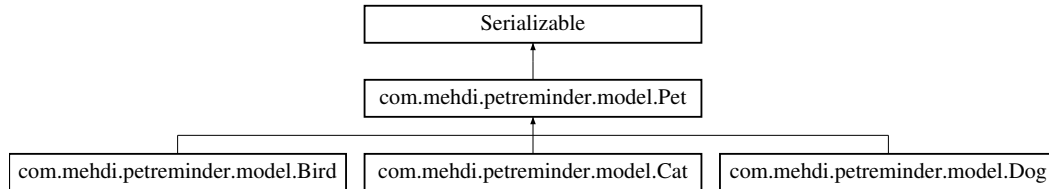
The documentation for this class was generated from the following file:

- `petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/NavPanel.java`

7.23 com.mehdi.petreminder.model.Pet Class Reference

Evcil hayvanın temel soyut sınıfı.

Inheritance diagram for com.mehdi.petreminder.model.Pet:



Public Member Functions

- `int getId ()`
ID getter.
- `void setId (int id)`
ID setter.
- `String getName ()`
Isim getter.
- `void setName (String name)`
Isim setter.
- `String getSpecies ()`
Tur getter.
- `void setSpecies (String species)`
Tur setter.
- `LocalDate getBirthDate ()`
Dogum tarihi getter.
- `void setBirthDate (LocalDate birthDate)`
Dogum tarihi setter.
- `int getOwnerId ()`
Sahip ID getter.
- `void setOwnerId (int ownerId)`
Sahip ID setter.
- `String getGender ()`
Cinsiyet getter.
- `void setGender (String gender)`
Cinsiyet setter.
- `double getWeight ()`
Agirlik getter.
- `void setWeight (double weight)`
Agirlik setter.
- `String getNotes ()`
Notlar getter.
- `void setNotes (String notes)`
Notlar setter.
- `abstract String makeSound ()`
Hayvana ozgu ses dondurur.
- `abstract String getIconName ()`

- `int getAge ()`
Yas hesaplar.
- `String getAgeString ()`
Yas bilgisini formatlı string olarak dondurur.
- `String toString ()`
Nesneyi string olarak temsil eder.
- `boolean equals (Object obj)`
İki `Pet` nesnesinin esitliğini karsilastirir.
- `int hashCode ()`
Hash kodu hesaplar.

Protected Member Functions

- `Pet ()`
Varsayilan yapici metod.
- `Pet (int id, String name, String species, LocalDate birthDate, int ownerId, String gender, double weight, String notes)`
Tam parametrelili yapici metod.

7.23.1 Detailed Description

Evcil hayvanin temel soyut sinifi.

OOP Abstraction: abstract class ile somutlastirilamaz. OOP Encapsulation: tum alanlar private, getter/setter ile erisim. Serializable: BinaryRepository icin zorunlu.

Author

Muhammed Mehdi Karagulle, Ibrahim Demirci, Zumre Uykun

Version

1.0

Definition at line 22 of file [Pet.java](#).

7.23.2 Constructor & Destructor Documentation

7.23.2.1 `Pet()` [1/2]

```
com.mehdi.petreminder.model.Pet.Pet () [protected]
```

Varsayilan yapici metod.

Serialization icin gereklidir.

Definition at line 55 of file [Pet.java](#).

```
00055 {}
```

Referenced by [equals\(\)](#).

7.23.2.2 Pet() [2/2]

```
com.mehdi.petreminder.model.Pet.Pet (
    int id,
    String name,
    String species,
    LocalDate birthDate,
    int ownerId,
    String gender,
    double weight,
    String notes) [protected]
```

Tam parametrelili yapıcı metod.

Parameters

<i>id</i>	Benzersiz kimlik numarası
<i>name</i>	Evciil hayvanın ismi
<i>species</i>	Hayvan türü
<i>birthDate</i>	Dogum tarihi
<i>ownerId</i>	Sahip kullanıcı ID'si
<i>gender</i>	Cinsiyet (Erkek/Disi)
<i>weight</i>	Ağırlık (kg)
<i>notes</i>	Ek notlar

Definition at line 68 of file [Pet.java](#).

```
00069                                     {
00070         this.id = id;
00071         this.name = name;
00072         this.species = species;
00073         this.birthDate = birthDate;
00074         this.ownerId = ownerId;
00075         this.gender = gender;
00076         this.weight = weight;
00077         this.notes = notes;
00078     }
```

7.23.3 Member Function Documentation

7.23.3.1 equals()

```
boolean com.mehdi.petreminder.model.Pet.equals (
    Object obj)
```

İki [Pet](#) nesnesinin eşitliğini karşılaştırır.

Parameters

<i>obj</i>	Karşılaştırılacak nesne
------------	-------------------------

Returns

ID'ler esitse true

Definition at line 181 of file [Pet.java](#).

```
00181         {
00182             if (this == obj) return true;
00183             if (obj == null || !(obj instanceof Pet)) return false;
00184             Pet other = (Pet) obj;
00185             return this.id == other.id;
00186         }
```

References [Pet\(\)](#).

7.23.3.2 getAge()

```
int com.mehdi.petreminder.model.Pet.getAge ()
```

Yas hesaplar.

Bugun ile dogum tarihi arasindaki farki yil cinsinden dondurur.

Returns

Yas (yil), dogum tarihi null ise -1

Definition at line 150 of file [Pet.java](#).

```
00150         {
00151             if (birthDate == null) return -1;
00152             return Period.between(birthDate, LocalDate.now()).getYears();
00153         }
```

7.23.3.3 getAgeString()

```
String com.mehdi.petreminder.model.Pet.getAgeString ()
```

Yas bilgisini formatlı string olarak dondurur.

Returns

"2 yil 3 ay" formatında yas

Definition at line 159 of file [Pet.java](#).

```
00159         {
00160             if (birthDate == null) return "Bilinmiyor";
00161             Period p = Period.between(birthDate, LocalDate.now());
00162             return p.getYears() + " yil " + p.getMonths() + " ay";
00163         }
```

Referenced by [com.mehdi.petreminder.model.Bird.toString\(\)](#), [com.mehdi.petreminder.model.Cat.toString\(\)](#), [com.mehdi.petreminder.model.Dog.toString\(\)](#), and [toString\(\)](#).

7.23.3.4 getBirthDate()

```
LocalDate com.mehdi.petreminder.model.Pet.getBirthDate ()
```

Dogum tarihi getter.

Returns

Dogum tarihi

Definition at line 101 of file [Pet.java](#).

```
00101 { return birthDate; }
```

Referenced by [com.mehdi.petreminder.repository.PetMySqlRepository.setInsertParams\(\)](#).

7.23.3.5 getGender()

```
String com.mehdi.petreminder.model.Pet.getGender ()
```

Cinsiyet getter.

Returns

Cinsiyet

Definition at line 113 of file [Pet.java](#).

```
00113 { return gender; }
```

Referenced by [com.mehdi.petreminder.repository.PetMySqlRepository.setInsertParams\(\)](#).

7.23.3.6 getIconName()

```
abstract String com.mehdi.petreminder.model.Pet.getIconName () [abstract]
```

Hayvana ozgu ikon adi dondurur (GUI icin).

Returns

Ikon adi (ornk. "dog.png")

Reimplemented in [com.mehdi.petreminder.model.Bird](#), [com.mehdi.petreminder.model.Cat](#), and [com.mehdi.petreminder.model.Dog](#).

7.23.3.7 getId()

```
int com.mehdi.petreminder.model.Pet.getId ()
```

ID getter.

Returns

Evcil hayvan ID'si

Definition at line 83 of file [Pet.java](#).

```
00083 { return id; }
```

Referenced by [com.mehdi.petreminder.model.Bird.toString\(\)](#), [com.mehdi.petreminder.model.Cat.toString\(\)](#), [com.mehdi.petreminder.model.Dog.toString\(\)](#), and [com.mehdi.petreminder.service.PetService.updatePet\(\)](#).

7.23.3.8 getName()

```
String com.mehdi.petreminder.model.Pet.getName ()
```

Isim getter.

Returns

Evcil hayvan ismi

Definition at line 89 of file [Pet.java](#).

```
00089 { return name; }
```

Referenced by [com.mehdi.petreminder.service.PetService.addPet\(\)](#), [com.mehdi.petreminder.model.Bird.toString\(\)](#), [com.mehdi.petreminder.model.Cat.toString\(\)](#), and [com.mehdi.petreminder.model.Dog.toString\(\)](#).

7.23.3.9 getNotes()

```
String com.mehdi.petreminder.model.Pet.getNotes ()
```

Notlar getter.

Returns

Notlar

Definition at line 125 of file [Pet.java](#).

```
00125 { return notes; }
```

Referenced by [com.mehdi.petreminder.repository.PetMySqlRepository.setInsertParams\(\)](#).

7.23.3.10 getOwnerId()

```
int com.mehdi.petreminder.model.Pet.getOwnerId ()
```

Sahip ID getter.

Returns

Sahip kullanici ID'si

Definition at line 107 of file [Pet.java](#).

```
00107 { return ownerId; }
```

7.23.3.11 getSpecies()

```
String com.mehdi.petreminder.model.Pet.getSpecies ()
```

Tur getter.

Returns

Hayvan turu

Definition at line 95 of file [Pet.java](#).

```
00095 { return species; }
```

7.23.3.12 `getWeight()`

```
double com.mehdi.petreminder.model.Pet.getWeight ()
```

Agirlik getter.

Returns

Agirlik (kg)

Definition at line 119 of file [Pet.java](#).

```
00119 { return weight; }
```

Referenced by [com.mehdi.petreminder.repository.PetMySqlRepository.setInsertParams\(\)](#).

7.23.3.13 `hashCode()`

```
int com.mehdi.petreminder.model.Pet.hashCode ()
```

Hash kodu hesaplar.

Returns

ID'ye gore hash kodu

Definition at line 193 of file [Pet.java](#).

```
00193         {  
00194             return Integer.hashCode(id);  
00195         }
```

7.23.3.14 `makeSound()`

```
abstract String com.mehdi.petreminder.model.Pet.makeSound () [abstract]
```

Hayvana ozgu ses dondurur.

OOP Polymorphism: her alt sinif farkli implementasyon saglar.

Returns

Hayvan sesi string'i

Reimplemented in [com.mehdi.petreminder.model.Bird](#), [com.mehdi.petreminder.model.Cat](#), and [com.mehdi.petreminder.model.Dog](#).

7.23.3.15 setBirthDate()

```
void com.mehdi.petreminder.model.Pet.setBirthDate (
    LocalDate birthDate)
```

Dogum tarihi setter.

Parameters

<i>birthDate</i>	Dogum tarihi
------------------	--------------

Definition at line 104 of file [Pet.java](#).

```
00104 { this.birthDate = birthDate; }
```

7.23.3.16 setGender()

```
void com.mehdi.petreminder.model.Pet.setGender (
    String gender)
```

Cinsiyet setter.

Parameters

<i>gender</i>	Cinsiyet
---------------	----------

Definition at line 116 of file [Pet.java](#).

```
00116 { this.gender = gender; }
```

Referenced by [com.mehdi.petreminder.repository.PetMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository.PetSqliteRepository.mapRow\(\)](#).

7.23.3.17 setId()

```
void com.mehdi.petreminder.model.Pet.setId (
    int id)
```

ID setter.

Parameters

<i>id</i>	Yeni ID degeri
-----------	----------------

Definition at line 86 of file [Pet.java](#).

```
00086 { this.id = id; }
```

7.23.3.18 setName()

```
void com.mehdi.petreminder.model.Pet.setName (
    String name)
```

Isim setter.

Parameters

<i>name</i>	Yeni isim
-------------	-----------

Definition at line 92 of file [Pet.java](#).

```
00092 { this.name = name; }
```

7.23.3.19 setNotes()

```
void com.mehdi.petreminder.model.Pet.setNotes (
    String notes)
```

Notlar setter.

Parameters

<i>notes</i>	Notlar
--------------	--------

Definition at line 128 of file [Pet.java](#).

```
00128 { this.notes = notes; }
```

Referenced by [com.mehdi.petreminder.repository.PetMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository.PetSqliteRe](#)

7.23.3.20 setOwnerId()

```
void com.mehdi.petreminder.model.Pet.setOwnerId (
    int ownerId)
```

Sahip ID setter.

Parameters

<i>owner↔ Id</i>	Sahip kullanici ID'si
----------------------	-----------------------

Definition at line 110 of file [Pet.java](#).

```
00110 { this.ownerId = ownerId; }
```

7.23.3.21 setSpecies()

```
void com.mehdi.petreminder.model.Pet.setSpecies (
    String species)
```

Tur setter.

Parameters

<i>species</i>	Hayvan turu
----------------	-------------

Definition at line 98 of file [Pet.java](#).

```
00098 { this.species = species; }
```

7.23.3.22 setWeight()

```
void com.mehdi.petreminder.model.Pet.setWeight (
    double weight)
```

Agirlik setter.

Parameters

<i>weight</i>	Agirlik (kg)
---------------	--------------

Definition at line 122 of file [Pet.java](#).

```
00122 { this.weight = weight; }
```

Referenced by [com.mehdi.petreminder.repository.PetMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository.PetSqliteRepository.mapRow\(\)](#).

7.23.3.23 toString()

```
String com.mehdi.petreminder.model.Pet.toString ()
```

Nesneyi string olarak temsil eder.

Returns

Formatli pet bilgisi

Reimplemented in [com.mehdi.petreminder.model.Bird](#), [com.mehdi.petreminder.model.Cat](#), and [com.mehdi.petreminder.model.Dog](#).

Definition at line 170 of file [Pet.java](#).

```
00170 {
00171     return String.format("Pet{id=%d, name='%s', species='%s', age=%s}",
00172         id, name, species, getAgeString());
00173 }
```

References [getAgeString\(\)](#).

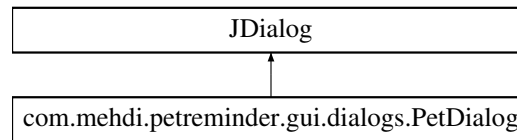
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/model/Pet.java](#)

7.24 com.mehdi.petreminder.gui.dialogs.PetDialog Class Reference

[Pet](#) ekleme ve düzenleme modal dialog'u.

Inheritance diagram for com.mehdi.petreminder.gui.dialogs.PetDialog:



Public Member Functions

- [PetDialog](#) (Frame parent, [Pet](#) editPet)
[PetDialog](#) yapıcısı.
- boolean [isConfirmed](#) ()
Onaylandı mı kontrolü.
- [Pet](#) [getPet](#) ()
Oluşturulan / düzenlenen pet'i döndürür.

7.24.1 Detailed Description

[Pet](#) ekleme ve düzenleme modal dialog'u.

Yeni pet için null geçerli; düzenleme için mevcut [Pet](#) nesnesi geçerli.

Author

Muhammed Mehdi Karagülle
İbrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 34 of file [PetDialog.java](#).

7.24.2 Constructor & Destructor Documentation

7.24.2.1 PetDialog()

```
com.mehdi.petreminder.gui.dialogs.PetDialog.PetDialog (
    Frame parent,
    Pet editPet)
```

[PetDialog](#) yapıcısı.

Parameters

<i>parent</i>	Ana pencere
<i>editPet</i>	Düzenlenecek pet (yeni için null)

Definition at line 70 of file [PetDialog.java](#).

```
00070
00071         {
00072             super(parent, editPet == null ? "Yeni Hayvan Ekle" : "Hayvanı Düzenle", true);
00073             this.editPet = editPet;
00074             buildUI();
00075             if (editPet != null) fillForm(editPet);
00076             pack();
00077             setLocationRelativeTo(parent);
00078             setMinimumSize(new Dimension(420, 500));
00079         }
```

7.24.3 Member Function Documentation

7.24.3.1 getPet()

```
Pet com.mehdi.petreminder.gui.dialogs.PetDialog.getPet ()
```

Oluşturulan / düzenlenen pet'i döndürür.

Returns

[Pet](#) nesnesi

Definition at line 296 of file [PetDialog.java](#).

```
00296 { return resultPet; }
```

7.24.3.2 isConfirmed()

```
boolean com.mehdi.petreminder.gui.dialogs.PetDialog.isConfirmed ()
```

Onaylandı mı kontrolü.

Returns

true ise kullanıcı "Kaydet" tıkladı

Definition at line 290 of file [PetDialog.java](#).

```
00290 { return confirmed; }
```

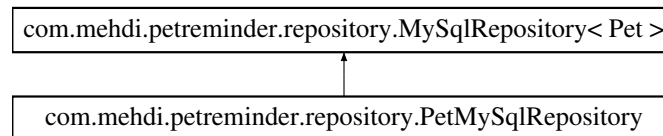
The documentation for this class was generated from the following file:

- petreminder-app/src/main/java/com/mehdi/petreminder/gui/dialogs/[PetDialog.java](#)

7.25 com.mehdi.petreminder.repository.PetMySqlRepository Class Reference

MySQL üzerinde [Pet](#) CRUD işlemleri.

Inheritance diagram for com.mehdi.petreminder.repository.PetMySqlRepository:



Public Member Functions

- [PetMySqlRepository](#) ()
Varsayılan yapıcı.
- [PetMySqlRepository](#) (Connection [connection](#))
Test yapıcısı.
- List< [Pet](#) > [findByOwnerId](#) (int ownerId)
OwnerId'ye göre petleri listeler.

Public Member Functions inherited from [com.mehdi.petreminder.repository.MySqlRepository< Pet >](#)

- [MySqlRepository](#) ()
MySQL bağlantısını açar.
- int [save](#) (T entity)
- Optional< T > [findById](#) (int id)
- List< T > [findAll](#) ()
- boolean [update](#) (T entity)
- boolean [delete](#) (int id)
- void [deleteAll](#) ()
- int [count](#) ()
- void [close](#) ()
- Connection [getConnection](#) ()
Bağlantıyı döndürür.

Protected Member Functions

- String [getTableName](#) ()
- String [getCreateTableSql](#) ()
- String [getInsertSql](#) ()
- String [getUpdateSql](#) ()
- void [setInsertParams](#) (PreparedStatement ps, [Pet](#) pet) throws SQLException
- void [setUpdateParams](#) (PreparedStatement ps, [Pet](#) pet) throws SQLException
- [Pet](#) [mapRow](#) (ResultSet rs) throws SQLException

Protected Member Functions inherited from [com.mehdi.petreminder.repository.MySqlRepository](#) < [Pet](#) >

- abstract String [getTableName](#) ()
Tablo adı.
- abstract String [getCreateTableSql](#) ()
Tablo oluşturma SQL.
- abstract String [getInsertSql](#) ()
INSERT SQL.
- abstract String [getUpdateSql](#) ()
UPDATE SQL.
- abstract void [setInsertParams](#) (PreparedStatement ps, T entity) throws SQLException
INSERT parametrelerini set eder.
- abstract void [setUpdateParams](#) (PreparedStatement ps, T entity) throws SQLException
UPDATE parametrelerini set eder.
- abstract T [mapRow](#) (ResultSet rs) throws SQLException
ResultSet'ten varlık oluşturur.
- void [createTableIfNotExists](#) ()
Tabloyu oluşturur (yoksa).
- void [setEntityId](#) (T entity, int id)
Reflection ile entity'ye id set eder.

Additional Inherited Members

Protected Attributes inherited from [com.mehdi.petreminder.repository.MySqlRepository](#) < [Pet](#) >

- Connection [connection](#)
MySQL JDBC bağlantısı.

Static Protected Attributes inherited from [com.mehdi.petreminder.repository.MySqlRepository](#) < [Pet](#) >

- static final Logger [logger](#)
Logger.

7.25.1 Detailed Description

MySQL üzerinde [Pet](#) CRUD işlemleri.

[MySqlRepository](#)'yi extend eder; [PetSqliteRepository](#) ile aynı mantık.

Author

Muhammed Mehdi Karagülle
Ibrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 23 of file [PetMySqlRepository.java](#).

7.25.2 Constructor & Destructor Documentation

7.25.2.1 PetMySqlRepository() [1/2]

`com.mehdi.petreminder.repository.PetMySqlRepository.PetMySqlRepository ()`

Varsayılan yapıcı.

Definition at line 28 of file [PetMySqlRepository.java](#).

```
00028                                     {
00029         super();
00030     }
```

7.25.2.2 PetMySqlRepository() [2/2]

`com.mehdi.petreminder.repository.PetMySqlRepository.PetMySqlRepository (Connection connection)`

Test yapıcısı.

Parameters

<i>connection</i>	Test bağlantısı
-------------------	-----------------

Definition at line 36 of file [PetMySqlRepository.java](#).

```
00036                                     {
00037         super(connection);
00038     }
```

References [com.mehdi.petreminder.repository.MySqlRepository< Pet >.connection](#).

7.25.3 Member Function Documentation

7.25.3.1 findByIdOwnerId()

`List< Pet > com.mehdi.petreminder.repository.PetMySqlRepository.findByIdOwnerId (int ownerId)`

OwnerId'ye göre petleri listeler.

Parameters

<i>ownerId</i>	Kullanıcı ID
----------------	--------------

Returns

Kullanıcıya ait pet listesi

Definition at line 167 of file [PetMySqlRepository.java](#).

```
00167                                     {
00168         List<Pet> list = new ArrayList<>();
00169         String sql = "SELECT * FROM pets WHERE owner_id = ?";
00170         try (PreparedStatement ps = connection.prepareStatement(sql)) {
00171             ps.setInt(1, ownerId);
00172             ResultSet rs = ps.executeQuery();
00173             while (rs.next()) list.add(mapRow(rs));
00174         } catch (SQLException e) {
00175             throw new RepositoryException("findByIdOwnerId hatası", e);
00176         }
00177         return list;
00178     }
```

References [com.mehdi.petreminder.repository.MySqlRepository< Pet >.connection](#), and [mapRow\(\)](#).

7.25.3.2 getCreateTableSql()

String com.mehdi.petreminder.repository.PetMySqlRepository.getCreateTableSql () [protected]

Member documentation.

Definition at line 54 of file [PetMySqlRepository.java](#).

```

00054     {
00055         return "CREATE TABLE IF NOT EXISTS pets ("
00056             + "id INT AUTO_INCREMENT PRIMARY KEY,"
00057             + "owner_id INT NOT NULL DEFAULT 1,"
00058             + "name VARCHAR(50) NOT NULL,"
00059             + "species VARCHAR(30) NOT NULL,"
00060             + "breed VARCHAR(50),"
00061             + "birth_date DATE,"
00062             + "gender VARCHAR(10),"
00063             + "weight DOUBLE DEFAULT 0,"
00064             + "notes TEXT,"
00065             + "can_talk BOOLEAN DEFAULT FALSE,"
00066             + "bird_type VARCHAR(50),"
00067             + "is_trained BOOLEAN DEFAULT FALSE,"
00068             + "is_indoor BOOLEAN DEFAULT TRUE"
00069             + ");";
00070     }

```

7.25.3.3 getInsertSql()

String com.mehdi.petreminder.repository.PetMySqlRepository.getInsertSql () [protected]

Member documentation.

Definition at line 77 of file [PetMySqlRepository.java](#).

```

00077     {
00078         return "INSERT INTO pets (owner_id,name,species,breed,birth_date,gender,"
00079             + "weight,notes,can_talk,bird_type,is_trained,is_indoor) "
00080             + "VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";
00081     }

```

7.25.3.4 getTableName()

String com.mehdi.petreminder.repository.PetMySqlRepository.getTableName () [protected]

Member documentation.

Definition at line 45 of file [PetMySqlRepository.java](#).

```

00045     {
00046         return "pets";
00047     }

```

7.25.3.5 getUpdateSql()

String com.mehdi.petreminder.repository.PetMySqlRepository.getUpdateSql () [protected]

Member documentation.

Definition at line 88 of file [PetMySqlRepository.java](#).

```

00088     {
00089         return "UPDATE pets SET owner_id=?,name=?,species=?,breed=?,birth_date=,"
00090             + "gender=?,weight=?,notes=?,can_talk=?,bird_type=?,is_trained=?,is_indoor=? "
00091             + "WHERE id=?";
00092     }

```

7.25.3.6 mapRow()

`Pet` `com.mehdi.petreminder.repository.PetMySQLRepository.mapRow (`
`ResultSet rs)` throws `SQLException` [protected]

Member documentation.

Definition at line 131 of file `PetMySQLRepository.java`.

```

00131                                     {
00132         String species = rs.getString("species");
00133         int id = rs.getInt("id");
00134         String name = rs.getString("name");
00135         Date sqlDate = rs.getDate("birth_date");
00136         LocalDate birthDate = sqlDate != null ? sqlDate.toLocalDate() : null;
00137         int ownerId = rs.getInt("owner_id");
00138
00139         Pet pet;
00140         if ("Dog".equalsIgnoreCase(species)) {
00141             Dog dog = new Dog(id, name, birthDate, ownerId);
00142             dog.setBreed(rs.getString("breed"));
00143             dog.setTrained(rs.getBoolean("is_trained"));
00144             pet = dog;
00145         } else if ("Cat".equalsIgnoreCase(species)) {
00146             Cat cat = new Cat(id, name, birthDate, ownerId);
00147             cat.setBreed(rs.getString("breed"));
00148             cat.setIndoor(rs.getBoolean("is_indoor"));
00149             pet = cat;
00150         } else {
00151             Bird bird = new Bird(id, name, birthDate, ownerId);
00152             bird.setBirdType(rs.getString("bird_type"));
00153             bird.setCanTalk(rs.getBoolean("can_talk"));
00154             pet = bird;
00155         }
00156         pet.setGender(rs.getString("gender"));
00157         pet.setWeight(rs.getDouble("weight"));
00158         pet.setNotes(rs.getString("notes"));
00159         return pet;
00160     }

```

References `com.mehdi.petreminder.model.Bird.setBirdType()`, `com.mehdi.petreminder.model.Cat.setBreed()`, `com.mehdi.petreminder.model.Dog.setBreed()`, `com.mehdi.petreminder.model.Bird.setCanTalk()`, `com.mehdi.petreminder.model.Pet.setNotes()`, `com.mehdi.petreminder.model.Cat.setIndoor()`, `com.mehdi.petreminder.model.Pet.setNotes()`, `com.mehdi.petreminder.model.Dog.setTrained()` and `com.mehdi.petreminder.model.Pet.setWeight()`.

Referenced by `findByOwnerId()`.

7.25.3.7 setInsertParams()

`void` `com.mehdi.petreminder.repository.PetMySQLRepository.setInsertParams (`
`PreparedStatement ps,`
`Pet pet)` throws `SQLException` [protected]

Member documentation.

Definition at line 99 of file `PetMySQLRepository.java`.

```

00099                                     {
00100         ps.setInt(1, pet.getOwnerId());
00101         ps.setString(2, pet.getName());
00102         ps.setString(3, pet.getSpecies());
00103         ps.setString(4, pet instanceof Dog ? ((Dog) pet).getBreed()
00104             : pet instanceof Cat ? ((Cat) pet).getBreed() : null);
00105         ps.setObject(5, pet.getBirthDate() != null
00106             ? java.sql.Date.valueOf(pet.getBirthDate()) : null);
00107         ps.setString(6, pet.getGender());
00108         ps.setDouble(7, pet.getWeight());
00109         ps.setString(8, pet.getNotes());
00110         ps.setBoolean(9, pet instanceof Bird && ((Bird) pet).isCanTalk());
00111         ps.setString(10, pet instanceof Bird ? ((Bird) pet).getBirdType() : null);
00112         ps.setBoolean(11, pet instanceof Dog && ((Dog) pet).isTrained());
00113         ps.setBoolean(12, !(pet instanceof Cat) || ((Cat) pet).isIndoor());
00114     }

```

References `com.mehdi.petreminder.model.Pet.getBirthDate()`, `com.mehdi.petreminder.model.Pet.getGender()`, `com.mehdi.petreminder.model.Pet.getNotes()`, and `com.mehdi.petreminder.model.Pet.getWeight()`.

Referenced by `setUpdateParams()`.

7.25.3.8 setUpdateParams()

```
void com.mehdi.petreminder.repository.PetMySQLRepository.setUpdateParams (
    PreparedStatement ps,
    Pet pet) throws SQLException [protected]
```

Member documentation.

Definition at line 121 of file [PetMySQLRepository.java](#).

```
00121
00122         setInsertParams(ps, pet);
00123         ps.setInt(13, pet.getId());
00124     }
```

References [setInsertParams\(\)](#).

The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/repository/PetMySQLRepository.java](#)

7.26 com.mehdi.petreminder.petreminder Class Reference

[Pet](#) Care [Reminder](#) System yardımcı/facade sınıfı.

Public Member Functions

- [petreminder](#) ()
Varsayılan yapıcı.
- int [add](#) (int a, int b)
İki tamsayıyı toplar.
- int [subtract](#) (int a, int b)
İki tamsayıyı çıkarır.
- int [multiply](#) (int a, int b)
İki tamsayıyı çarpar.
- double [divide](#) (int a, int b)
İki tamsayıyı böler.
- void [printPets](#) (List< [Pet](#) > pets)
Pet listesini biçimli şekilde konsola yazdırır.
- void [printReminders](#) (List< [Reminder](#) > reminders)
Hatırlatıcı listesini biçimli şekilde konsola yazdırır.
- String [getCurrentTimeFormatted](#) ()
Mevcut tarih-saati biçimli string olarak döndürür.
- boolean [isNullOrEmpty](#) (String value)
Verilen string'in boş veya null olup olmadığını kontrol eder.
- void [printSeparator](#) (int width)
Konsol ayırıcı çizgi yazdırır.
- void [printHeader](#) ()
Uygulama başlığını konsola yazdırır.

7.26.1 Detailed Description

[Pet](#) Care [Reminder](#) System yardımcı/facade sınıfı.

PDF gereksinimi: Separate library (lib) and application (app) layers. Bu sınıf lib katmanının facade'ı olarak görev yapar. OOP Polymorphism: [Reminder](#) listesi üzerinden polimorfik işlemler. OOP Encapsulation: tüm alanlar private.

Author

Muhammed Mehdi Karagülle

Ibrahim Demirci

Zumre Uykun

Version

1.0

Date

2026-03-27

Definition at line [33](#) of file [petreminder.java](#).

7.26.2 Constructor & Destructor Documentation

7.26.2.1 `petreminder()`

```
com.mehdi.petreminder.petreminder.petreminder ()
```

Varsayılan yapıcı.

Definition at line [50](#) of file [petreminder.java](#).

```
00050         {
00051     logger.debug("petreminder yardimci sinifi olusturuldu.");
00052     }
```

7.26.3 Member Function Documentation

7.26.3.1 `add()`

```
int com.mehdi.petreminder.petreminder.add (
    int a,
    int b)
```

İki tamsayıyı toplar.

Template'deki add metodu — test coverage için korunur.

Parameters

<i>a</i>	Birinci sayı
<i>b</i>	İkinci sayı

Returns

Toplam

Definition at line 63 of file [petreminder.java](#).

```
00063         {
00064             logger.debug("add({}, {})", a, b);
00065             return a + b;
00066         }
```

7.26.3.2 divide()

```
double com.mehdi.petreminder.petreminder.divide (
    int a,
    int b)
```

İki tamsayıyı böler.

Sıfıra bölme durumunda `IllegalArgumentException` fırlatır. PDF gereksinimi: Exception handling — try-catch.

Parameters

<i>a</i>	Bölünen
<i>b</i>	Bölen (0 olamaz)

Returns

Bölüm (double)

Exceptions

<i>IllegalArgumentException</i>	bölen sıfır ise
---------------------------------	-----------------

Definition at line 99 of file [petreminder.java](#).

```
00099         {
00100             if (b == 0) {
00101                 logger.error("Sifira bolme denemesi: {} / {}", a, b);
00102                 throw new IllegalArgumentException("Sifira bolme yapilamaz.");
00103             }
00104             logger.debug("divide({}, {})", a, b);
00105             return (double) a / b;
00106         }
```

7.26.3.3 getCurrentTimeFormatted()

```
String com.mehdi.petreminder.petreminder.getCurrentTimeFormatted ()
```

Mevcut tarih-saati biçimli string olarak döndürür.

Returns

"dd/MM/yyyy HH:mm" formatında şimdiki zaman

Definition at line 156 of file [petreminder.java](#).

```
00156                                     {
00157         return LocalDateTime.now().format(DISPLAY_FORMAT);
00158     }
```

7.26.3.4 isNullOrEmpty()

```
boolean com.mehdi.petreminder.petreminder.isNullOrEmpty (
    String value)
```

Verilen string'in boş veya null olup olmadığını kontrol eder.

Parameters

<i>value</i>	Kontrol edilecek string
--------------	-------------------------

Returns

null veya boş ise true

Definition at line 165 of file [petreminder.java](#).

```
00165                                     {
00166         return value == null || value.trim().isEmpty();
00167     }
```

7.26.3.5 multiply()

```
int com.mehdi.petreminder.petreminder.multiply (
    int a,
    int b)
```

İki tamsayıyı çarpar.

Parameters

<i>a</i>	Birinci sayı
<i>b</i>	İkinci sayı

Returns

Çarpım

Definition at line 85 of file [petreminder.java](#).

```
00085                                     {
00086         logger.debug("multiply({}, {})", a, b);
00087         return a * b;
00088     }
```


7.26.3.6 printHeader()

```
void com.mehdi.petreminder.petreminder.printHeader ()
```

Uygulama başlığını konsola yazdırır.

Definition at line 180 of file [petreminder.java](#).

```
00180      {
00181          printSeparator(50);
00182          System.out.println("  " + petreminderApp.APP_NAME);
00183          System.out.println("  v" + petreminderApp.APP_VERSION);
00184          printSeparator(50);
00185      }
```

References [com.mehdi.petreminder.petreminderApp.APP_NAME](#), [com.mehdi.petreminder.petreminderApp.APP_VERSION](#), and [printSeparator\(\)](#).

7.26.3.7 printPets()

```
void com.mehdi.petreminder.petreminder.printPets (
    List< Pet > pets)
```

[Pet](#) listesini biçimli şekilde konsola yazdırır.

OOP Polymorphism: List<Pet> üzerinden polimorfik toString.

Parameters

<i>pets</i>	Pet listesi
-------------	-----------------------------

Definition at line 115 of file [petreminder.java](#).

```
00115      {
00116          if (pets == null || pets.isEmpty()) {
00117              System.out.println("Hic evcil hayvan bulunamadi.");
00118              return;
00119          }
00120          System.out.println(String.format("%-5s %-15s %-12s %-10s",
00121              "ID", "Ad", "Tur", "Yas"));
00122          System.out.println("-".repeat(45));
00123          for (Pet pet : pets) {
00124              System.out.println(String.format("%-5d %-15s %-12s %-10s",
00125                  pet.getId(), pet.getName(), pet.getSpecies(),
00126                  pet.getAgeString()));
00127          }
00128      }
```

7.26.3.8 printReminders()

```
void com.mehdi.petreminder.petreminder.printReminders (
    List< Reminder > reminders)
```

Hatırlatıcı listesini biçimli şekilde konsola yazdırır.

OOP Polymorphism: Her [Reminder](#) alt sınıfının getSummary() metodu çağrılır.

Parameters

<i>reminders</i>	Hatırlatıcı listesi
------------------	---------------------

Definition at line 135 of file [petreminder.java](#).

```

00135
00136         if (reminders == null || reminders.isEmpty()) {
00137             System.out.println("Hic hatirlatici bulunamadi.");
00138             return;
00139         }
00140         System.out.println(String.format("%-5s %-12s %-15s %-20s %-8s",
00141             "ID", "Tur", "Pet", "Zaman", "Durum"));
00142         System.out.println("-".repeat(65));
00143         for (Reminder r : reminders) {
00144             String durum = r.isCompleted() ? "[TAMAM]"
00145                 : r.isOverdue() ? "[GECMIS]" : "[BEKLIYOR]";
00146             System.out.println(String.format("%-5d %-12s %-15s %-20s %-8s",
00147                 r.getId(), r.getReminderType(), r.getPetName(),
00148                 r.getFormattedTime(), durum));
00149         }
00150     }

```

7.26.3.9 printSeparator()

```

void com.mehdi.petreminder.petreminder.printSeparator (
    int width)

```

Konsol ayırıcı çizgi yazdırır.

Parameters

<i>width</i>	Çizgi genişliği
--------------	-----------------

Definition at line 173 of file [petreminder.java](#).

```

00173
00174         System.out.println("=".repeat(width));
00175     }

```

Referenced by [printHeader\(\)](#).

7.26.3.10 subtract()

```

int com.mehdi.petreminder.petreminder.subtract (
    int a,
    int b)

```

İki tamsayıyı çıkarır.

Parameters

<i>a</i>	Birinci sayı
<i>b</i>	İkinci sayı

Returns

Fark

Definition at line 74 of file [petreminder.java](#).

```
00074         {
00075             logger.debug("subtract({}, {})", a, b);
00076             return a - b;
00077         }
```

The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/petreminder.java](#)

7.27 com.mehdi.petreminder.petreminderApp Class Reference

[Pet Care Reminder](#) System ana sınıfı.

Static Public Member Functions

- static void [main](#) (String[] args)
Ana giriş noktası.
- static void [startGui](#) ()
Swing GUI'yi başlatır (EDT'de).
- static void [startConsole](#) (String[] args)
Console modunu başlatır.
- static boolean [isGuiRequested](#) (String[] args)
–gui argümanı var mı kontrol eder.
- static [StorageType](#) [parseStorageArg](#) (String[] args)
–storage argümanını ayrıştırır.
- static String [getVersion](#) ()
Uygulama versiyonunu döndürür.
- static String [getAppName](#) ()
Uygulama adını döndürür.

Static Public Attributes

- static final String [APP_NAME](#) = "Pet Care [Reminder](#) System"
Uygulama adı.
- static final String [APP_VERSION](#) = "1.0.0"
Versiyon.

7.27.1 Detailed Description

[Pet Care Reminder](#) System ana sınıfı.

PDF zorunluluğu: Modular Architecture. `-gui` → Swing GUI başlatır. Swing yoksa console moduna düşer.

Author

Muhammed Mehdi Karagülle
İbrahim Demirci
Zumre Uykun

Version

1.0

Date

2026-03-27

Definition at line 30 of file [petreminderApp.java](#).

7.27.2 Member Function Documentation

7.27.2.1 `getAppName()`

```
String com.mehdi.petreminder.petreminderApp.getAppname () [static]
```

Uygulama adını döndürür.

Returns

Uygulama adı

Definition at line 155 of file [petreminderApp.java](#).

```
00155 { return APP_NAME; }
```

References [APP_NAME](#).

7.27.2.2 `getVersion()`

```
String com.mehdi.petreminder.petreminderApp.getVersion () [static]
```

Uygulama versiyonunu döndürür.

Returns

Versiyon string'i

Definition at line 149 of file [petreminderApp.java](#).

```
00149 { return APP_VERSION; }
```

References [APP_VERSION](#).

7.27.2.3 isGuiRequested()

```
boolean com.mehdi.petreminder.petreminderApp.isGuiRequested (
    String[] args) [static]
```

–gui argümanı var mı kontrol eder.

Parameters

<i>args</i>	Argümanlar
-------------	------------

Returns

true ise GUI isteniyor

Definition at line 115 of file [petreminderApp.java](#).

```
00115                                     {
00116         if (args == null) return false;
00117         for (String arg : args) {
00118             if ("--gui".equalsIgnoreCase(arg)) return true;
00119         }
00120         return false;
00121     }
```

Referenced by [main\(\)](#).

7.27.2.4 main()

```
void com.mehdi.petreminder.petreminderApp.main (
    String[] args) [static]
```

Ana giriş noktası.

Parameters

<i>args</i>	Komut satırı argümanları. –gui → Swing GUI başlatır –storage=binary → Binary backend –storage=sqlite → SQLite backend –storage=mysql → MySQL backend
-------------	--

Definition at line 50 of file [petreminderApp.java](#).

```
00050                                     {
00051         logger.info("{} v{} baslatiliyor...", APP_NAME, APP_VERSION);
00052
00053         // Storage backend
00054         StorageType storageType = parseStorageArg(args);
00055         StorageConfig.setActiveBackend(storageType);
00056         logger.info("Storage backend: {}", storageType);
00057
00058         boolean guiMode = isGuiRequested(args);
00059
00060         if (guiMode) {
00061             startGui();
00062         } else {
00063             startConsole(args);
00064         }
00065
00066         logger.info("Uygulama kapatildi.");
00067     }
```

References [APP_NAME](#), [APP_VERSION](#), [isGuiRequested\(\)](#), [parseStorageArg\(\)](#), [com.mehdi.petreminder.config.StorageConfig.setActiveBackend\(\)](#), [startConsole\(\)](#), and [startGui\(\)](#).

7.27.2.5 parseStorageArg()

`StorageType` `com.mehdi.petreminder.petreminderApp.parseStorageArg (String[] args) [static]`

–storage argümanını ayrıştırır.

Parameters

<code>args</code>	Argümanlar
-------------------	------------

Returns

`StorageType` (varsayılan BINARY)

Definition at line 128 of file `petreminderApp.java`.

```

00128                                     {
00129         if (args == null || args.length == 0) return StorageType.BINARY;
00130         for (String arg : args) {
00131             if (arg != null && arg.startsWith("--storage=")) {
00132                 String value = arg.substring("--storage=".length())
00133                     .trim().toUpperCase(java.util.Locale.ROOT);
00134                 try {
00135                     return StorageType.valueOf(value);
00136                 } catch (IllegalArgumentException e) {
00137                     logger.warn("Gecersiz storage: {}. BINARY kullaniliyor.", value);
00138                     return StorageType.BINARY;
00139                 }
00140             }
00141         }
00142         return StorageType.BINARY;
00143     }

```

References `com.mehdi.petreminder.config.StorageType.BINARY`.

Referenced by `main()`.

7.27.2.6 startConsole()

`void` `com.mehdi.petreminder.petreminderApp.startConsole (String[] args) [static]`

Console modunu başlatır.

Parameters

<code>args</code>	Argümanlar
-------------------	------------

Definition at line 90 of file `petreminderApp.java`.

```

00090                                     {
00091         System.out.println("=====");
00092         System.out.println(" " + APP_NAME + " v" + APP_VERSION);
00093         System.out.println(" CEN206 - OOP Term Project - Spring 2026");
00094         System.out.println("=====");
00095         System.out.println("Storage: " + StorageConfig.getActiveBackend().getDisplayName());
00096         System.out.println();
00097         System.out.println("Hint: GUI modda başlatmak için --gui argümanını kullanın.");
00098         System.out.println();
00099
00100         try {
00101             ConsoleApp consoleApp = new ConsoleApp();

```

```

00102         consoleApp.start();
00103     } catch (Exception e) {
00104         logger.error("Uygulama hatasi: {}", e.getMessage(), e);
00105         System.err.println("Kritik hata: " + e.getMessage());
00106         System.exit(1);
00107     }
00108 }

```

References [APP_NAME](#), [APP_VERSION](#), [com.mehdi.petreminder.config.StorageConfig.getActiveBackend\(\)](#), [com.mehdi.petreminder.config.StorageType.getDisplayName\(\)](#), and [com.mehdi.petreminder.ConsoleApp.start\(\)](#).

Referenced by [main\(\)](#).

7.27.2.7 startGui()

```
void com.mehdi.petreminder.petreminderApp.startGui () [static]
```

Swing GUI'yi başlatır (EDT'de).

Definition at line 72 of file [petreminderApp.java](#).

```

00072         {
00073             SwingUtilities.invokeLater(() -> {
00074                 try {
00075                     MainFrame frame = new MainFrame();
00076                     frame.setVisible(true);
00077                     logger.info("GUI baslatildi.");
00078                 } catch (Exception e) {
00079                     logger.error("GUI baslatilamadi: {}", e.getMessage(), e);
00080                     System.err.println("GUI baslatilamadi: " + e.getMessage());
00081                     System.exit(1);
00082                 }
00083             });
00084         }

```

Referenced by [main\(\)](#).

7.27.3 Member Data Documentation

7.27.3.1 APP_NAME

```
final String com.mehdi.petreminder.petreminderApp.APP_NAME = "Pet Care Reminder System" [static]
```

Uygulama adı.

Definition at line 37 of file [petreminderApp.java](#).

Referenced by [getAppName\(\)](#), [main\(\)](#), [com.mehdi.petreminder.petreminder.printHeader\(\)](#), and [startConsole\(\)](#).

7.27.3.2 APP_VERSION

```
final String com.mehdi.petreminder.petreminderApp.APP_VERSION = "1.0.0" [static]
```

Versiyon.

Definition at line 40 of file [petreminderApp.java](#).

Referenced by [getVersion\(\)](#), [main\(\)](#), [com.mehdi.petreminder.petreminder.printHeader\(\)](#), and [startConsole\(\)](#).

The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/petreminderApp.java](#)

7.28 com.mehdi.petreminder.observer.PetReminderObserver Interface Reference

Gozlemci arayuzu — durum degisikliklerini dinler.

Public Member Functions

- void [onEvent](#) ([EventType](#) event, Object data)
Olay gerceklestiginde cagrilan metod.

7.28.1 Detailed Description

Gozlemci arayuzu — durum degisikliklerini dinler.

Observer Design Pattern: Bu interface'i implement eden siniflar, Subject'teki degisikliklerden haberdar olur. Ornek: Storage degistiginde servisler guncellenir, [Reminder](#) eklendiginde Dashboard yenilenir.

Author

Muhammed Mehdi Karagulle
Ibrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 22 of file [PetReminderObserver.java](#).

7.28.2 Member Function Documentation

7.28.2.1 onEvent()

```
void com.mehdi.petreminder.observer.PetReminderObserver.onEvent (
    EventType event,
    Object data)
```

Olay gerceklestiginde cagrilan metod.

Subject durumu degistiginde tum kayitli observer'lara bu metod uzerinden bildirim gonderilir.

Parameters

<i>event</i>	Olay turu (ornegin STORAGE_CHANGED, REMINDER_ADDED)
<i>data</i>	Olayla ilgili veri (ornegin yeni StorageType , eklenen Reminder)

The documentation for this interface was generated from the following file:

- petreminder-app/src/main/java/com/mehdi/petreminder/observer/[PetReminderObserver.java](#)

7.29 com.mehdi.petreminder.service.PetService Class Reference

[Pet](#) CRUD iş mantığı katmanı.

Public Member Functions

- [PetService](#) ()
Varsayılan yapıcı — [RepositoryFactory](#) kullanır.
- [PetService](#) ([IRepository](#)< [Pet](#) > repository)
Test yapıcısı — dependency injection.
- [Pet](#) [addPet](#) ([Pet](#) pet)
Yeni pet ekler.
- [Pet](#) [getPetById](#) (int id)
ID ile pet getirir.
- List< [Pet](#) > [getAllPets](#) ()
Tüm petleri listeler.
- void [updatePet](#) ([Pet](#) pet)
[Pet](#) günceller.
- void [deletePet](#) (int id)
[Pet](#) siler.
- void [deleteAllPets](#) ()
Tüm petleri siler.
- int [getPetCount](#) ()
Toplam pet sayısını döndürür.
- List< [Pet](#) > [searchByName](#) (String name)
Ada göre pet arar (büyük/küçük harf duyarlı).
- List< [Pet](#) > [filterBySpecies](#) (String species)
Türe göre filtreler ([Dog](#), [Cat](#), [Bird](#)).
- void [close](#) ()
[Repository](#) kapatır.

7.29.1 Detailed Description

[Pet](#) CRUD iş mantığı katmanı.

OOP Composition: [IRepository](#) bağımlılığı constructor injection ile. PDF zorunluluğu: Service layer — business logic ayrılır. Observer Pattern: Her CRUD işleminde [EventManager](#) üzerinden kayıtlı observer'lara bildirim gönderilir.

Author

Muhammed Mehdi Karagülle
İbrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 33 of file [PetService.java](#).

7.29.2 Constructor & Destructor Documentation

7.29.2.1 PetService() [1/2]

`com.mehdi.petreminder.service.PetService.PetService ()`

Varsayılan yapıcı — [RepositoryFactory](#) kullanır.

Definition at line 45 of file [PetService.java](#).

```
00045         {
00046             this.repository = RepositoryFactory.createPetRepository();
00047         }
```

References [com.mehdi.petreminder.repository.RepositoryFactory.createPetRepository\(\)](#).

7.29.2.2 PetService() [2/2]

`com.mehdi.petreminder.service.PetService.PetService (
 IRepository< Pet > repository)`

Test yapıcısı — dependency injection.

Parameters

<i>repository</i>	Mock veya test repository'si
-------------------	------------------------------

Definition at line 53 of file [PetService.java](#).

```
00053         {
00054             this.repository = repository;
00055         }
```

7.29.3 Member Function Documentation

7.29.3.1 addPet()

`Pet com.mehdi.petreminder.service.PetService.addPet (
 Pet pet)`

Yeni pet ekler.

Parameters

<i>pet</i>	Eklenecek Pet nesnesi
------------	---------------------------------------

Returns

Oluşturulan [Pet](#) (ID set edilmiş)

Exceptions

ServiceException	Doğrulama başarısız olursa
----------------------------------	----------------------------

Definition at line 63 of file [PetService.java](#).

```

00063         {
00064             validatePet(pet);
00065             int id = repository.save(pet);
00066             logger.info("Pet eklendi: id={}, name={}", id, pet.getName());
00067             // Observer Pattern: Pet eklendi bildirimi
00068             EventManager.getInstance().notify(EventType.PET_ADDED, pet);
00069             return pet;
00070         }

```

References [com.mehdi.petreminder.observer.EventManager.getInstance\(\)](#), [com.mehdi.petreminder.model.Pet.getName\(\)](#), [com.mehdi.petreminder.observer.EventManager.notify\(\)](#), and [com.mehdi.petreminder.observer.EventType.PET_ADDED](#).

7.29.3.2 close()

```
void com.mehdi.petreminder.service.PetService.close ()
```

Repository kapatır.

Definition at line 166 of file [PetService.java](#).

```

00166         {
00167             repository.close();
00168         }

```

7.29.3.3 deleteAllPets()

```
void com.mehdi.petreminder.service.PetService.deleteAllPets ()
```

Tüm petleri siler.

Definition at line 125 of file [PetService.java](#).

```

00125         {
00126             repository.deleteAll();
00127         }

```

7.29.3.4 deletePet()

```
void com.mehdi.petreminder.service.PetService.deletePet (
    int id)
```

[Pet](#) siler.

Parameters

<i>id</i>	Silinecek Pet ID
-----------	----------------------------------

Exceptions

ServiceException	Bulunamazsa
----------------------------------	-------------

Definition at line 112 of file [PetService.java](#).

```

00112         {
00113             boolean deleted = repository.delete(id);
00114             if (!deleted) {
00115                 throw new ServiceException("Pet silinemedi: id=" + id);
00116             }
00117             logger.info("Pet silindi: id={}", id);
00118             // Observer Pattern: Pet silindi bildirimi
00119             EventManager.getInstance().notify(EventType.PET_DELETED, id);
00120         }

```

References [com.mehdi.petreminder.observer.EventManager.getInstance\(\)](#), [com.mehdi.petreminder.observer.EventManager.notify\(\)](#), and [com.mehdi.petreminder.observer.EventType.PET_DELETED](#).

7.29.3.5 filterBySpecies()

```
List< Pet > com.mehdi.petreminder.service.PetService.filterBySpecies (
    String species)
```

Türe göre filtreler ([Dog](#), [Cat](#), [Bird](#)).

Parameters

<i>species</i>	Tür adı
----------------	---------

Returns

Filtrelenmiş liste

Definition at line 155 of file [PetService.java](#).

```

00155         {
00156             if (species == null || species.trim().isEmpty()) return getAllPets();
00157             return repository.findAll().stream()
00158                 .filter(p -> p.getSpecies() != null
00159                     && p.getSpecies().equalsIgnoreCase(species.trim()))
00160                 .collect(Collectors.toList());
00161         }

```

References [getAllPets\(\)](#).

7.29.3.6 getAllPets()

```
List< Pet > com.mehdi.petreminder.service.PetService.getAllPets ()
```

Tüm petleri listeler.

Returns

[Pet](#) listesi

Definition at line 87 of file [PetService.java](#).

```

00087         {
00088             return repository.findAll();
00089         }

```

Referenced by [filterBySpecies\(\)](#), and [searchByName\(\)](#).

7.29.3.7 getPetById()

```
Pet com.mehdi.petreminder.service.PetService.getPetById (
    int id)
```

ID ile pet getirir.

Parameters

<i>id</i>	Pet ID
-----------	--------

Returns

Pet nesnesi

Exceptions

<i>ServiceException</i>	Bulunamazsa
-------------------------	-------------

Definition at line 78 of file [PetService.java](#).

```
00078      {
00079          return repository.findById(id)
00080              .orElseThrow(() -> new ServiceException("Pet bulunamadi: id=" + id));
00081      }
```

7.29.3.8 getPetCount()

```
int com.mehdi.petreminder.service.PetService.getPetCount ()
```

Toplam pet sayısını döndürür.

Returns

Sayı

Definition at line 133 of file [PetService.java](#).

```
00133      {
00134          return repository.count();
00135      }
```

7.29.3.9 searchByName()

```
List< Pet > com.mehdi.petreminder.service.PetService.searchByName (
    String name)
```

Ada göre pet arar (büyük/küçük harf duyarsız).

Parameters

<i>name</i>	Aranacak ad
-------------	-------------

Returns

Eşleşen [Pet](#) listesi

Definition at line 142 of file [PetService.java](#).

```

00142         {
00143             if (name == null || name.trim().isEmpty()) return getAllPets();
00144             String lower = name.toLowerCase().trim();
00145             return repository.findAll().stream()
00146                 .filter(p -> p.getName() != null && p.getName().toLowerCase().contains(lower))
00147                 .collect(Collectors.toList());
00148         }

```

References [getAllPets\(\)](#).

7.29.3.10 updatePet()

```

void com.mehdi.petreminder.service.PetService.updatePet (
    Pet pet)

```

[Pet](#) günceller.

Parameters

<i>pet</i>	Güncellenecek Pet
------------	-----------------------------------

Exceptions

ServiceException	Doğrulama başarısız veya bulunamazsa
----------------------------------	--------------------------------------

Definition at line 96 of file [PetService.java](#).

```

00096         {
00097             validatePet(pet);
00098             boolean updated = repository.update(pet);
00099             if (!updated) {
00100                 throw new ServiceException("Pet güncellenemedi: id=" + pet.getId());
00101             }
00102             logger.info("Pet güncellendi: id={}", pet.getId());
00103             // Observer Pattern: Pet güncellendi bildirimi
00104             EventManager.getInstance().notify(EventType.PET_UPDATED, pet);
00105         }

```

References [com.mehdi.petreminder.model.Pet.getId\(\)](#), [com.mehdi.petreminder.observer.EventManager.getInstance\(\)](#), [com.mehdi.petreminder.observer.EventManager.notify\(\)](#), and [com.mehdi.petreminder.observer.EventType.PET_UPDATED](#).

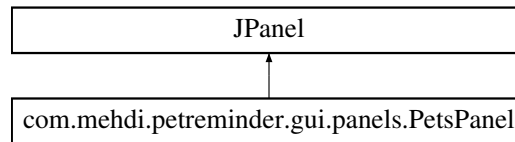
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/service/PetService.java](#)

7.30 com.mehdi.petreminder.gui.panels.PetsPanel Class Reference

Evcil hayvanlar listesi ve CRUD işlemleri.

Inheritance diagram for com.mehdi.petreminder.gui.panels.PetsPanel:



Public Member Functions

- [PetsPanel](#) ([MainFrame](#) mainFrame)
[PetsPanel](#) yapıcısı.
- void [refresh](#) ()
Tabloyu yeniler.

7.30.1 Detailed Description

Evcil hayvanlar listesi ve CRUD işlemleri.

JTable + toolbar + dialog pattern.

Author

Muhammed Mehdi Karagülle
Ibrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 35 of file [PetsPanel.java](#).

7.30.2 Constructor & Destructor Documentation

7.30.2.1 PetsPanel()

`com.mehdi.petreminder.gui.panels.PetsPanel.PetsPanel (`
 `MainFrame mainFrame)`

[PetsPanel](#) yapıcısı.

Parameters

<i>mainFrame</i>	Ana pencere
------------------	-------------

Definition at line 62 of file [PetsPanel.java](#).

```
00062         {
00063             this.mainFrame = mainFrame;
00064             this.petService = new PetService();
00065             setBackground(GuiConstants.BG_COLOR);
00066             setLayout(new BorderLayout(GuiConstants.GAP, GuiConstants.GAP));
00067             setBorder(new EmptyBorder(GuiConstants.GAP_LARGE, GuiConstants.GAP_LARGE,
00068                                     GuiConstants.GAP_LARGE, GuiConstants.GAP_LARGE));
00069             buildUI();
00070         }
```

References [com.mehdi.petreminder.gui.util.GuiConstants.BG_COLOR](#), [com.mehdi.petreminder.gui.util.GuiConstants.GAP](#), and [com.mehdi.petreminder.gui.util.GuiConstants.GAP_LARGE](#).

7.30.3 Member Function Documentation

7.30.3.1 refresh()

`void com.mehdi.petreminder.gui.panels.PetsPanel.refresh ()`

Tabloyu yeniler.

Definition at line 220 of file [PetsPanel.java](#).

```
00220         {
00221             tableModel.setRowCount(0);
00222             try {
00223                 String search = searchField != null ? searchField.getText().trim() : "";
00224                 String species = speciesFilter != null
00225                     ? (String) speciesFilter.getSelectedItem() : "Tümü";
00226
00227                 List<Pet> pets = search.isEmpty() ? petService.getAllPets()
00228                     : petService.searchByName(search);
00229                 if (!"Tümü".equals(species) && species != null) {
00230                     pets = petService.filterBySpecies(species);
00231                 }
00232
00233                 for (Pet p : pets) {
00234                     tableModel.addRow(new Object[]{
00235                         p.getId(),
00236                         p.getName(),
00237                         p.getSpecies(),
00238                         getBreed(p),
00239                         p.getBirthDate() != null ? p.getBirthDate().toString() : "",
00240                         p.getGender() != null ? p.getGender() : "",
00241                         String.format("%.1f", p.getWeight())
00242                     });
00243                 }
00244             } catch (Exception e) {
00245                 // Veri yoksa sessiz
00246             }
00247         }
```

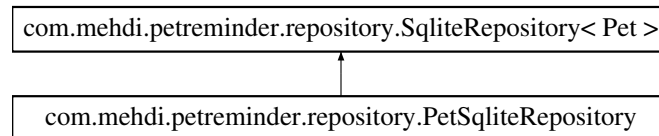
The documentation for this class was generated from the following file:

- `petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/PetsPanel.java`

7.31 com.mehdi.petreminder.repository.PetSqliteRepository Class Reference

SQLite üzerinde [Pet](#) CRUD işlemleri.

Inheritance diagram for com.mehdi.petreminder.repository.PetSqliteRepository:



Public Member Functions

- [PetSqliteRepository](#) ()
Varsayılan yapıcı — gerçek SQLite.
- [PetSqliteRepository](#) (Connection [connection](#))
Test yapıcısı.
- List< [Pet](#) > [findByOwnerId](#) (int ownerId)
OwnerId'ye göre petleri listeler.

Public Member Functions inherited from [com.mehdi.petreminder.repository.SqliteRepository< Pet >](#)

- [SqliteRepository](#) ()
SQLite bağlantısını açar ve tabloyu oluşturur.
- int [save](#) (T entity)
- Optional< T > [findById](#) (int id)
- List< T > [findAll](#) ()
- boolean [update](#) (T entity)
- boolean [delete](#) (int id)
- void [deleteAll](#) ()
- int [count](#) ()
- void [close](#) ()
- Connection [getConnection](#) ()
Bağlantıyı döndürür (test için).

Protected Member Functions

- String [getTableName](#) ()
- String [getCreateTableSql](#) ()
- String [getInsertSql](#) ()
- String [getUpdateSql](#) ()
- void [setInsertParams](#) (PreparedStatement ps, [Pet](#) pet) throws SQLException
- void [setUpdateParams](#) (PreparedStatement ps, [Pet](#) pet) throws SQLException
- [Pet](#) [mapRow](#) (ResultSet rs) throws SQLException

Protected Member Functions inherited from [com.mehdi.petreminder.repository.SqliteRepository](#)< [Pet](#) >

- abstract String [getTableName](#) ()
Tablo adını döndürür.
- abstract String [getCreateTableSql](#) ()
Tablo oluşturma SQL'ini döndürür.
- abstract String [getInsertSql](#) ()
INSERT SQL'ini döndürür (ID hariç).
- abstract String [getUpdateSql](#) ()
UPDATE SQL'ini döndürür.
- abstract void [setInsertParams](#) (PreparedStatement ps, T entity) throws SQLException
PreparedStatement'a INSERT parametrelerini set eder.
- abstract void [setUpdateParams](#) (PreparedStatement ps, T entity) throws SQLException
PreparedStatement'a UPDATE parametrelerini set eder.
- abstract T [mapRow](#) (ResultSet rs) throws SQLException
ResultSet'ten varlık oluşturur.
- void [createTableIfNotExists](#) ()
Tabloyu oluşturur (yoksa).
- void [setEntityId](#) (T entity, int id)
Reflection ile entity'ye id set eder.

Additional Inherited Members

Protected Attributes inherited from [com.mehdi.petreminder.repository.SqliteRepository](#)< [Pet](#) >

- Connection [connection](#)
SQLite JDBC bağlantısı.

Static Protected Attributes inherited from [com.mehdi.petreminder.repository.SqliteRepository](#)< [Pet](#) >

- static final Logger [logger](#)
Logger.

7.31.1 Detailed Description

SQLite üzerinde [Pet](#) CRUD işlemleri.

[SqliteRepository](#)'yi extend eder; Template Method Pattern. [Dog](#) / [Cat](#) / [Bird](#) tür ayrımı species sütununa göre yapılır.

Author

Muhammed Mehdi Karagülle
Ibrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 25 of file [PetSqliteRepository.java](#).

7.31.2 Constructor & Destructor Documentation

7.31.2.1 PetSqliteRepository() [1/2]

com.mehdi.petreminder.repository.PetSqliteRepository.PetSqliteRepository ()

Varsayılan yapıcı — gerçek SQLite.

Definition at line 34 of file [PetSqliteRepository.java](#).

```
00034         {
00035             super();
00036         }
```

7.31.2.2 PetSqliteRepository() [2/2]

com.mehdi.petreminder.repository.PetSqliteRepository.PetSqliteRepository (
Connection connection)

Test yapıcısı.

Parameters

<i>connection</i>	Test bağlantısı
-------------------	-----------------

Definition at line 42 of file [PetSqliteRepository.java](#).

```
00042         {
00043             super(connection);
00044         }
```

References [com.mehdi.petreminder.repository.SqliteRepository< Pet >.connection](#).

7.31.3 Member Function Documentation

7.31.3.1 findByOwnerId()

List< [Pet](#) > com.mehdi.petreminder.repository.PetSqliteRepository.findByOwnerId (
int ownerId)

OwnerId'ye göre petleri listeler.

Parameters

<i>ownerId</i>	Kullanıcı ID
----------------	--------------

Returns

Kullanıcıya ait pet listesi

Definition at line 182 of file [PetSqliteRepository.java](#).

```
00182         {
00183             List<Pet> list = new ArrayList<>();
00184             String sql = "SELECT * FROM pets WHERE owner_id = ?";
00185             try (PreparedStatement ps = connection.prepareStatement(sql)) {
00186                 ps.setInt(1, ownerId);
00187                 ResultSet rs = ps.executeQuery();
00188                 while (rs.next()) list.add(mapRow(rs));
00189             } catch (SQLException e) {
00190                 throw new RepositoryException("findByOwnerId hatası", e);
00191             }
00192             return list;
00193         }
```

References [com.mehdi.petreminder.repository.SqliteRepository< Pet >.connection](#), and [mapRow\(\)](#).

7.31.3.2 getCreateTableSql()

String com.mehdi.petreminder.repository.PetSqliteRepository.getCreateTableSql () [protected]

Member documentation.

Definition at line 60 of file [PetSqliteRepository.java](#).

```
00060      {
00061      return "CREATE TABLE IF NOT EXISTS pets ("
00062      + "id INTEGER PRIMARY KEY AUTOINCREMENT,"
00063      + "owner_id INTEGER NOT NULL DEFAULT 1,"
00064      + "name TEXT NOT NULL,"
00065      + "species TEXT NOT NULL,"
00066      + "breed TEXT,"
00067      + "birth_date TEXT,"
00068      + "gender TEXT,"
00069      + "weight REAL DEFAULT 0,"
00070      + "notes TEXT,"
00071      + "can_talk INTEGER DEFAULT 0,"
00072      + "bird_type TEXT,"
00073      + "is_trained INTEGER DEFAULT 0,"
00074      + "is_indoor INTEGER DEFAULT 1"
00075      + ");";
00076      }
```

7.31.3.3 getInsertSql()

String com.mehdi.petreminder.repository.PetSqliteRepository.getInsertSql () [protected]

Member documentation.

Definition at line 83 of file [PetSqliteRepository.java](#).

```
00083      {
00084      return "INSERT INTO pets (owner_id,name,species,breed,birth_date,gender,"
00085      + "weight,notes,can_talk,bird_type,is_trained,is_indoor) "
00086      + "VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";
00087      }
```

7.31.3.4 getTableName()

String com.mehdi.petreminder.repository.PetSqliteRepository.getTableName () [protected]

Member documentation.

Definition at line 51 of file [PetSqliteRepository.java](#).

```
00051      {
00052      return "pets";
00053      }
```

7.31.3.5 getUpdateSql()

String com.mehdi.petreminder.repository.PetSqliteRepository.getUpdateSql () [protected]

Member documentation.

Definition at line 94 of file [PetSqliteRepository.java](#).

```
00094      {
00095      return "UPDATE pets SET owner_id=?,name=?,species=?,breed=?,birth_date=?,
00096      + "gender=?,weight=?,notes=?,can_talk=?,bird_type=?,is_trained=?,is_indoor=? "
00097      + "WHERE id=?";
00098      }
```

7.31.3.6 mapRow()

`Pet` com.mehdi.petreminder.repository.PetSqliteRepository.mapRow (
 ResultSet rs) throws SQLException [protected]

Member documentation.

Definition at line 135 of file [PetSqliteRepository.java](#).

```
00135                                     {
00136         String species = rs.getString("species");
00137         int id = rs.getInt("id");
00138         String name = rs.getString("name");
00139         String birthDateStr = rs.getString("birth_date");
00140         LocalDate birthDate = birthDateStr != null ? LocalDate.parse(birthDateStr) : null;
00141         int ownerId = rs.getInt("owner_id");
00142
00143         Pet pet;
00144         if ("Dog".equalsIgnoreCase(species) || "Kopek".equalsIgnoreCase(species)) {
00145             Dog dog = new Dog(id, name, birthDate, ownerId);
00146             dog.setBreed(rs.getString("breed"));
00147             dog.setTrained(rs.getInt("is_trained") == 1);
00148             pet = dog;
00149         } else if ("Cat".equalsIgnoreCase(species) || "Kedi".equalsIgnoreCase(species)) {
00150             Cat cat = new Cat(id, name, birthDate, ownerId);
00151             cat.setBreed(rs.getString("breed"));
00152             cat.setIndoor(rs.getInt("is_indoor") == 1);
00153             pet = cat;
00154         } else {
00155             Bird bird = new Bird(id, name, birthDate, ownerId);
00156             bird.setBirdType(rs.getString("bird_type"));
00157             bird.setCanTalk(rs.getInt("can_talk") == 1);
00158             pet = bird;
00159         }
00160         pet.setGender(rs.getString("gender"));
00161         pet.setWeight(rs.getDouble("weight"));
00162         pet.setNotes(rs.getString("notes"));
00163         return pet;
00164     }
```

References [com.mehdi.petreminder.model.Bird.setBirdType\(\)](#), [com.mehdi.petreminder.model.Cat.setBreed\(\)](#), [com.mehdi.petreminder.model.Dog.setBreed\(\)](#), [com.mehdi.petreminder.model.Bird.setCanTalk\(\)](#), [com.mehdi.petreminder.model.Pet.setNotes\(\)](#), [com.mehdi.petreminder.model.Cat.setIndoor\(\)](#), [com.mehdi.petreminder.model.Pet.setNotes\(\)](#), [com.mehdi.petreminder.model.Dog.setTrained\(\)](#) and [com.mehdi.petreminder.model.Pet.setWeight\(\)](#).

Referenced by [findByOwnerId\(\)](#).

7.31.3.7 setInsertParams()

void com.mehdi.petreminder.repository.PetSqliteRepository.setInsertParams (
 PreparedStatement ps,
 Pet pet) throws SQLException [protected]

Member documentation.

Definition at line 105 of file [PetSqliteRepository.java](#).

```
00105                                     {
00106         ps.setInt(1, pet.getOwnerId());
00107         ps.setString(2, pet.getName());
00108         ps.setString(3, pet.getSpecies());
00109         ps.setString(4, pet.getBreed());
00110         ps.setString(5, pet.getBirthDate() != null ? pet.getBirthDate().toString() : null);
00111         ps.setString(6, pet.getGender());
00112         ps.setDouble(7, pet.getWeight());
00113         ps.setString(8, pet.getNotes());
00114         ps.setInt(9, (pet instanceof Bird && ((Bird) pet).isCanTalk()) ? 1 : 0);
00115         ps.setString(10, pet instanceof Bird ? ((Bird) pet).getBirdType() : null);
00116         ps.setInt(11, (pet instanceof Dog && ((Dog) pet).isTrained()) ? 1 : 0);
00117         ps.setInt(12, (pet instanceof Cat && ((Cat) pet).isIndoor()) ? 1 : 0);
00118     }
```

Referenced by [setUpdateParams\(\)](#).

7.31.3.8 setUpdateParams()

```
void com.mehdi.petreminder.repository.PetSqliteRepository.setUpdateParams (
    PreparedStatement ps,
    Pet pet) throws SQLException [protected]
```

Member documentation.

Definition at line 125 of file [PetSqliteRepository.java](#).

```
00125                                     {
00126         setInsertParams(ps, pet);
00127         ps.setInt(13, pet.getId());
00128     }
```

References [setInsertParams\(\)](#).

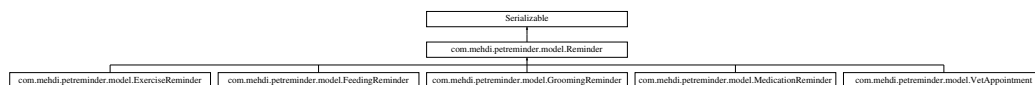
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/repository/PetSqliteRepository.java](#)

7.32 com.mehdi.petreminder.model.Reminder Class Reference

Evcil hayvan bakım hatırlatıcısının soyut temel sınıfı.

Inheritance diagram for com.mehdi.petreminder.model.Reminder:



Public Member Functions

- int [getId](#) ()
ID getter.
- void [setId](#) (int id)
ID setter.
- int [getPetId](#) ()
Pet ID getter.
- void [setPetId](#) (int petId)
Pet ID setter.
- String [getPetName](#) ()
Pet adı getter.
- void [setPetName](#) (String petName)
Pet adı setter.
- LocalDateTime [getScheduledTime](#) ()
Planlanan zaman getter.
- void [setScheduledTime](#) (LocalDateTime scheduledTime)
Planlanan zaman setter.
- boolean [isCompleted](#) ()
Tamamlandı mı getter.
- void [setCompleted](#) (boolean completed)

- Tamamlandı setter.*
- String [getDescription](#) ()
 - Açıklama getter.*
- void [setDescription](#) (String description)
 - Açıklama setter.*
- String [getPriority](#) ()
 - Öncelik getter.*
- void [setPriority](#) (String priority)
 - Öncelik setter.*
- boolean [isRecurring](#) ()
 - Tekrarlıyor mu getter.*
- void [setRecurring](#) (boolean recurring)
 - Tekrarlama setter.*
- String [getRecurrencePattern](#) ()
 - Tekrar deseni getter.*
- void [setRecurrencePattern](#) (String recurrencePattern)
 - Tekrar deseni setter.*
- abstract String [getReminderType](#) ()
 - Hatırlatıcının türünü döndürür.*
- abstract boolean [validate](#) ()
 - Hatırlatıcının geçerliliğini doğrular.*
- abstract String [getSummary](#) ()
 - Hatırlatıcının özet bilgisini döndürür (GUI'de gösterilir).*
- String [getFormattedTime](#) ()
 - Planlanan zamanı biçimli string olarak döndürür.*
- boolean [isOverdue](#) ()
 - Hatırlatıcının geçmiş mi olduğunu kontrol eder.*
- String [toString](#) ()
 - String temsili.*
- boolean [equals](#) (Object obj)
 - Eşitlik kontrolü — ID bazlı.*
- int [hashCode](#) ()
 - Hash kodu.*

Protected Member Functions

- [Reminder](#) ()
 - Varsayılan yapıcı — Serialization için.*
- [Reminder](#) (int id, int petId, String petName, LocalDateTime scheduledTime, String description, String priority, boolean recurring, String recurrencePattern)
 - Tam parametrelili yapıcı.*

Static Protected Attributes

- static final DateFormatter [FORMATTER](#)
 - Tarih saat formatı.*

7.32.1 Detailed Description

Evcil hayvan bakım hatırlatıcısının soyut temel sınıfı.

OOP Abstraction: abstract class ile somutlaştırılmaz. OOP Polymorphism: [validate\(\)](#) her alt sınıfta farklı davranır.
OOP Encapsulation: tüm alanlar private.

Author

Muhammed Mehdi Karagülle, İbrahim Demirci, Zümre Uykun

Version

1.0

Definition at line 22 of file [Reminder.java](#).

7.32.2 Constructor & Destructor Documentation

7.32.2.1 [Reminder\(\)](#) [1/2]

```
com.mehdi.petreminder.model.Reminder.Reminder () [protected]
```

Varsayılan yapıcı — Serialization için.

Definition at line 59 of file [Reminder.java](#).

```
00059 {}
```

Referenced by [equals\(\)](#).

7.32.2.2 [Reminder\(\)](#) [2/2]

```
com.mehdi.petreminder.model.Reminder.Reminder (  
    int id,  
    int petId,  
    String petName,  
    LocalDateTime scheduledTime,  
    String description,  
    String priority,  
    boolean recurring,  
    String recurrencePattern) [protected]
```

Tam parametrelili yapıcı.

Parameters

<i>id</i>	Benzersiz ID
<i>petId</i>	Pet ID

<i>petName</i>	Pet adı
<i>scheduledTime</i>	Planlanan zaman
<i>description</i>	Açıklama
<i>priority</i>	Öncelik (DUSUK/ORTA/YUKSEK)
<i>recurring</i>	Tekrarlıyor mu
<i>recurrencePattern</i>	Tekrar deseni

Definition at line 72 of file [Reminder.java](#).

```

00074                                     {
00075         this.id = id;
00076         this.petId = petId;
00077         this.petName = petName;
00078         this.scheduledTime = scheduledTime;
00079         this.completed = false;
00080         this.description = description;
00081         this.priority = priority;
00082         this.recurring = recurring;
00083         this.recurrencePattern = recurrencePattern;
00084     }

```

7.32.3 Member Function Documentation

7.32.3.1 equals()

```
boolean com.mehdi.petreminder.model.Reminder.equals (
    Object obj)
```

Eşitlik kontrolü — ID bazlı.

Parameters

<i>obj</i>	Karşılaştırılan nesne
------------	-----------------------

Returns

ID'ler eşitse true

Definition at line 204 of file [Reminder.java](#).

```

00204                                     {
00205         if (this == obj) return true;
00206         if (obj == null || !(obj instanceof Reminder)) return false;
00207         Reminder other = (Reminder) obj;
00208         return this.id == other.id;
00209     }

```

References [Reminder\(\)](#).

7.32.3.2 getDescription()

```
String com.mehdi.petreminder.model.Reminder.getDescription ()
```

Açıklama getter.

Returns

Açıklama

Definition at line 121 of file [Reminder.java](#).

```
00121 { return description; }
```

7.32.3.3 getFormattedTime()

```
String com.mehdi.petreminder.model.Reminder.getFormattedTime ()
```

Planlanan zamanı biçimli string olarak döndürür.

Returns

"dd/MM/yyyy HH:mm" formatında zaman

Definition at line 174 of file [Reminder.java](#).

```
00174                                     {
00175         if (scheduledTime == null) return "Belirsiz";
00176         return scheduledTime.format (FORMATTER);
00177     }
```

References [FORMATTER](#).

Referenced by [com.mehdi.petreminder.gui.panels.VetPanel.refresh\(\)](#), and [toString\(\)](#).

7.32.3.4 getId()

```
int com.mehdi.petreminder.model.Reminder.getId ()
```

ID getter.

Returns

Hatırlatıcı ID

Definition at line 89 of file [Reminder.java](#).

```
00089 { return id; }
```

Referenced by [com.mehdi.petreminder.gui.panels.VetPanel.refresh\(\)](#), [com.mehdi.petreminder.model.ExerciseReminder.toString\(\)](#), [com.mehdi.petreminder.model.FeedingReminder.toString\(\)](#), [com.mehdi.petreminder.model.GroomingReminder.toString\(\)](#), [com.mehdi.petreminder.model.MedicationReminder.toString\(\)](#), [com.mehdi.petreminder.model.VetAppointment.toString\(\)](#), and [com.mehdi.petreminder.service.ReminderService.updateReminder\(\)](#).

7.32.3.5 getPetId()

```
int com.mehdi.petreminder.model.Reminder.getPetId ()
```

[Pet](#) ID getter.

Returns

[Pet](#) ID

Definition at line 95 of file [Reminder.java](#).

```
00095 { return petId; }
```

7.32.3.6 getPetName()

```
String com.mehdi.petreminder.model.Reminder.getPetName ()
```

Pet adı getter.

Returns

Pet adı

Definition at line 101 of file [Reminder.java](#).

```
00101 { return petName; }
```

Referenced by [com.mehdi.petreminder.gui.panels.VetPanel.refresh\(\)](#), [com.mehdi.petreminder.model.ExerciseReminder.toString\(\)](#), [com.mehdi.petreminder.model.FeedingReminder.toString\(\)](#), [com.mehdi.petreminder.model.GroomingReminder.toString\(\)](#), [com.mehdi.petreminder.model.MedicationReminder.toString\(\)](#), and [com.mehdi.petreminder.model.VetAppointment.toString\(\)](#).

7.32.3.7 getPriority()

```
String com.mehdi.petreminder.model.Reminder.getPriority ()
```

Öncelik getter.

Returns

Öncelik

Definition at line 127 of file [Reminder.java](#).

```
00127 { return priority; }
```

7.32.3.8 getRecurrencePattern()

```
String com.mehdi.petreminder.model.Reminder.getRecurrencePattern ()
```

Tekrar deseni getter.

Returns

Tekrar deseni

Definition at line 139 of file [Reminder.java](#).

```
00139 { return recurrencePattern; }
```

7.32.3.9 getReminderType()

```
abstract String com.mehdi.petreminder.model.Reminder.getReminderType () [abstract]
```

Hatırlatıcının türünü döndürür.

OOP Polymorphism: her alt sınıf farklı tür döndürür.

Returns

Hatırlatıcı türü string'i

Reimplemented in [com.mehdi.petreminder.model.ExerciseReminder](#), [com.mehdi.petreminder.model.FeedingReminder](#), [com.mehdi.petreminder.model.GroomingReminder](#), [com.mehdi.petreminder.model.MedicationReminder](#), and [com.mehdi.petreminder.model.VetAppointment](#).

Referenced by [com.mehdi.petreminder.service.ReminderService.addReminder\(\)](#), and [toString\(\)](#).

7.32.3.10 getScheduledTime()

```
LocalDateTime com.mehdi.petreminder.model.Reminder.getScheduledTime ()
```

Planlanan zaman getter.

Returns

Planlanan zaman

Definition at line 107 of file [Reminder.java](#).

```
00107 { return scheduledTime; }
```

7.32.3.11 getSummary()

```
abstract String com.mehdi.petreminder.model.Reminder.getSummary () [abstract]
```

Hatırlatıcının özet bilgisini döndürür (GUI'de gösterilir).

Returns

Özet string

Reimplemented in [com.mehdi.petreminder.model.ExerciseReminder](#), [com.mehdi.petreminder.model.FeedingReminder](#), [com.mehdi.petreminder.model.GroomingReminder](#), [com.mehdi.petreminder.model.MedicationReminder](#), and [com.mehdi.petreminder.model.VetAppointment](#).

7.32.3.12 hashCode()

```
int com.mehdi.petreminder.model.Reminder.hashCode ()
```

Hash kodu.

Returns

ID'ye göre hash

Definition at line 216 of file [Reminder.java](#).

```
00216         {  
00217             return Integer.hashCode(id);  
00218         }
```

7.32.3.13 isCompleted()

```
boolean com.mehdi.petreminder.model.Reminder.isCompleted ()
```

Tamamlandı mı getter.

Returns

true ise tamamlandı

Definition at line 115 of file [Reminder.java](#).

```
00115 { return completed; }
```

7.32.3.14 isOverdue()

```
boolean com.mehdi.petreminder.model.Reminder.isOverdue ()
```

Hatırlatıcının geçmiş mi olduğunu kontrol eder.

Returns

Planlan zaman geçmişse ve tamamlanmamışsa true

Definition at line 183 of file [Reminder.java](#).

```
00183         {  
00184             if (scheduledTime == null || completed) return false;  
00185             return LocalDateTime.now().isAfter(scheduledTime);  
00186         }
```

Referenced by [com.mehdi.petreminder.service.ReminderService.getOverdueReminders\(\)](#).

7.32.3.15 isRecurring()

```
boolean com.mehdi.petreminder.model.Reminder.isRecurring ()
```

Tekrarlıyor mu getter.

Returns

Tekrarlıyor mu

Definition at line 133 of file [Reminder.java](#).

```
00133 { return recurring; }
```

7.32.3.16 setCompleted()

```
void com.mehdi.petreminder.model.Reminder.setCompleted (
    boolean completed)
```

Tamamlandı setter.

Parameters

<i>completed</i>	Durum
------------------	-------

Definition at line 118 of file [Reminder.java](#).

```
00118 { this.completed = completed; }
```

Referenced by [com.mehdi.petreminder.repository.ReminderMySqlRepository.mapRow\(\)](#), [com.mehdi.petreminder.repository.ReminderRepository.findById\(\)](#) and [com.mehdi.petreminder.service.ReminderService.markCompleted\(\)](#).

7.32.3.17 setDescription()

```
void com.mehdi.petreminder.model.Reminder.setDescription (
    String description)
```

Açıklama setter.

Parameters

<i>description</i>	Açıklama
--------------------	----------

Definition at line 124 of file [Reminder.java](#).

```
00124 { this.description = description; }
```

7.32.3.18 setId()

```
void com.mehdi.petreminder.model.Reminder.setId (
    int id)
```

ID setter.

Parameters

<i>id</i>	Yeni ID
-----------	---------

Definition at line 92 of file [Reminder.java](#).

```
00092 { this.id = id; }
```

7.32.3.19 setPetId()

```
void com.mehdi.petreminder.model.Reminder.setPetId (
    int petId)
```

Pet ID setter.

Parameters

<i>petId</i>	Pet ID
--------------	--------

Definition at line 98 of file [Reminder.java](#).

```
00098 { this.petId = petId; }
```

7.32.3.20 setPetName()

```
void com.mehdi.petreminder.model.Reminder.setPetName (
    String petName)
```

Pet adı setter.

Parameters

<i>petName</i>	Pet adı
----------------	---------

Definition at line 104 of file [Reminder.java](#).

```
00104 { this.petName = petName; }
```

7.32.3.21 setPriority()

```
void com.mehdi.petreminder.model.Reminder.setPriority (
    String priority)
```

Öncelik setter.

Parameters

<i>priority</i>	Öncelik
-----------------	---------

Definition at line 130 of file [Reminder.java](#).

```
00130 { this.priority = priority; }
```

7.32.3.22 setRecurrencePattern()

```
void com.mehdi.petreminder.model.Reminder.setRecurrencePattern (
    String recurrencePattern)
```

Tekrar deseni setter.

Parameters

<i>recurrencePattern</i>	Tekrar deseni
--------------------------	---------------

Definition at line 142 of file [Reminder.java](#).

```
00142
00143         this.recurrencePattern = recurrencePattern;
00144     }
```

7.32.3.23 setRecurring()

```
void com.mehdi.petreminder.model.Reminder.setRecurring (
    boolean recurring)
```

Tekrarlama setter.

Parameters

<i>recurring</i>	Tekrarlıyor mu
------------------	----------------

Definition at line 136 of file [Reminder.java](#).

```
00136 { this.recurring = recurring; }
```

7.32.3.24 setScheduledTime()

```
void com.mehdi.petreminder.model.Reminder.setScheduledTime (
    LocalDateTime scheduledTime)
```

Planlanan zaman setter.

Parameters

<i>scheduledTime</i>	Planlanan zaman
----------------------	-----------------

Definition at line 110 of file [Reminder.java](#).

```
00110
00111         this.scheduledTime = scheduledTime;
00112     }
```


7.32.3.25 toString()

```
String com.mehdi.petreminder.model.Reminder.toString ()
```

String temsili.

Returns

Hatırlatıcı bilgilerini içeren string

Reimplemented in [com.mehdi.petreminder.model.ExerciseReminder](#), [com.mehdi.petreminder.model.FeedingReminder](#), [com.mehdi.petreminder.model.GroomingReminder](#), [com.mehdi.petreminder.model.MedicationReminder](#), and [com.mehdi.petreminder.model.VetAppointment](#).

Definition at line 193 of file [Reminder.java](#).

```
00193         {
00194         return String.format("Reminder{id=%d, type='%s', pet='%s', time='%s', done=%b}",
00195         id, getReminderType(), petName, getFormattedTime(), completed);
00196     }
```

References [getFormattedTime\(\)](#), and [getReminderType\(\)](#).

7.32.3.26 validate()

```
abstract boolean com.mehdi.petreminder.model.Reminder.validate () [abstract]
```

Hatırlatıcının geçerliliğini doğrular.

OOP Polymorphism: her alt sınıf kendi kuralını uygular.

Returns

Geçerliyse true

Reimplemented in [com.mehdi.petreminder.model.ExerciseReminder](#), [com.mehdi.petreminder.model.FeedingReminder](#), [com.mehdi.petreminder.model.GroomingReminder](#), [com.mehdi.petreminder.model.MedicationReminder](#), and [com.mehdi.petreminder.model.VetAppointment](#).

7.32.4 Member Data Documentation

7.32.4.1 FORMATTER

```
final DateTimeFormatter com.mehdi.petreminder.model.Reminder.FORMATTER [static], [protected]
```

Initial value:

```
=
    DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm")
```

Tarih saat formatı.

Definition at line 28 of file [Reminder.java](#).

Referenced by [getFormattedTime\(\)](#).

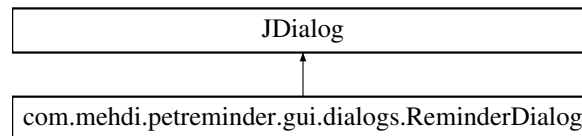
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/model/Reminder.java](#)

7.33 com.mehdi.petreminder.gui.dialogs.ReminderDialog Class Reference

[Reminder](#) ekleme ve düzenleme modal dialog'u.

Inheritance diagram for com.mehdi.petreminder.gui.dialogs.ReminderDialog:



Public Member Functions

- [ReminderDialog](#) (Frame parent, [Reminder](#) editReminder)
ReminderDialog yapıcısı.
- boolean [isConfirmed](#) ()
Onaylandı mı.
- [Reminder](#) [getReminder](#) ()
Oluşturulan reminder'ı döndürür.

7.33.1 Detailed Description

[Reminder](#) ekleme ve düzenleme modal dialog'u.

Tür seçimine göre dinamik form alanları gösterilir.

Author

Muhammed Mehdi Karagülle
İbrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 25 of file [ReminderDialog.java](#).

7.33.2 Constructor & Destructor Documentation

7.33.2.1 ReminderDialog()

```
com.mehdi.petreminder.gui.dialogs.ReminderDialog.ReminderDialog (
    Frame parent,
    Reminder editReminder)
```

[ReminderDialog](#) yapıcısı.

Parameters

<i>parent</i>	Ana pencere
<i>editReminder</i>	Düzenlenecek reminder (yeni için null)

Definition at line 97 of file [ReminderDialog.java](#).

```
00097
00098         super(parent, editReminder == null ? "Yeni Hatırlatıcı" : "Hatırlatıcı Düzenle", true);
00099         this.editReminder = editReminder;
00100         buildUI();
00101         if (editReminder != null) fillForm(editReminder);
00102         pack();
00103         setLocationRelativeTo(parent);
00104         setMinimumSize(new Dimension(440, 480));
00105     }
```

7.33.3 Member Function Documentation

7.33.3.1 getReminder()

[Reminder](#) com.mehdi.petreminder.gui.dialogs.ReminderDialog.getReminder ()

Oluşturulan reminder'ı döndürür.

Returns

[Reminder](#) nesnesi

Definition at line 318 of file [ReminderDialog.java](#).

```
00318 { return resultReminder; }
```

7.33.3.2 isConfirmed()

boolean com.mehdi.petreminder.gui.dialogs.ReminderDialog.isConfirmed ()

Onaylandı mı.

Returns

true ise kaydet tıklandı

Definition at line 312 of file [ReminderDialog.java](#).

```
00312 { return confirmed; }
```

The documentation for this class was generated from the following file:

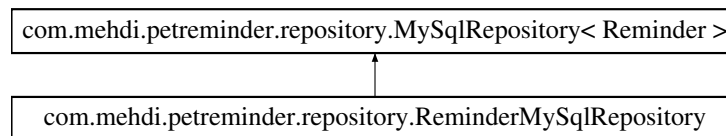
- petreminder-app/src/main/java/com/mehdi/petreminder/gui/dialogs/[ReminderDialog.java](#)

7.34 com.mehdi.petreminder.repository.ReminderMySqlRepository

Class Reference

MySQL üzerinde [Reminder](#) CRUD işlemleri.

Inheritance diagram for com.mehdi.petreminder.repository.ReminderMySqlRepository:



Public Member Functions

- [ReminderMySqlRepository](#) ()
Varsayılan yapıcı.
- [ReminderMySqlRepository](#) (Connection [connection](#))
Test yapıcısı.

Public Member Functions inherited from

[com.mehdi.petreminder.repository.MySqlRepository](#) < [Reminder](#) >

- [MySqlRepository](#) ()
MySQL bağlantısını açar.
- int [save](#) (T entity)
- Optional< T > [findById](#) (int id)
- List< T > [findAll](#) ()
- boolean [update](#) (T entity)
- boolean [delete](#) (int id)
- void [deleteAll](#) ()
- int [count](#) ()
- void [close](#) ()
- Connection [getConnection](#) ()
Bağlantıyı döndürür.

Protected Member Functions

- String [getTableName](#) ()
- String [getCreateTableSql](#) ()
- String [getInsertSql](#) ()
- String [getUpdateSql](#) ()
- void [setInsertParams](#) (PreparedStatement ps, [Reminder](#) r) throws SQLException
- void [setUpdateParams](#) (PreparedStatement ps, [Reminder](#) r) throws SQLException
- [Reminder](#) [mapRow](#) (ResultSet rs) throws SQLException

Protected Member Functions inherited from [com.mehdi.petreminder.repository.MySqlRepository](#) < [Reminder](#) >

- abstract String [getTableName](#) ()
Tablo adı.
- abstract String [getCreateTableSql](#) ()
Tablo oluşturma SQL.
- abstract String [getInsertSql](#) ()
INSERT SQL.
- abstract String [getUpdateSql](#) ()
UPDATE SQL.
- abstract void [setInsertParams](#) (PreparedStatement ps, T entity) throws SQLException
INSERT parametrelerini set eder.
- abstract void [setUpdateParams](#) (PreparedStatement ps, T entity) throws SQLException
UPDATE parametrelerini set eder.
- abstract T [mapRow](#) (ResultSet rs) throws SQLException
ResultSet'ten varlık oluşturur.
- void [createTableIfNotExists](#) ()
Tabloyu oluşturur (yoksa).
- void [setEntityId](#) (T entity, int id)
Reflection ile entity'ye id set eder.

Additional Inherited Members

Protected Attributes inherited from [com.mehdi.petreminder.repository.MySqlRepository](#) < [Reminder](#) >

- Connection [connection](#)
MySQL JDBC bağlantısı.

Static Protected Attributes inherited from [com.mehdi.petreminder.repository.MySqlRepository](#) < [Reminder](#) >

- static final Logger [logger](#)
Logger.

7.34.1 Detailed Description

MySQL üzerinde [Reminder](#) CRUD işlemleri.

Author

Muhammed Mehdi Karagülle
İbrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 21 of file [ReminderMySqlRepository.java](#).

7.34.2 Constructor & Destructor Documentation

7.34.2.1 ReminderMySqlRepository() [1/2]

`com.mehdi.petreminder.repository.ReminderMySqlRepository.ReminderMySqlRepository ()`

Varsayılan yapıcı.

Definition at line 26 of file [ReminderMySqlRepository.java](#).

```
00026                                     {
00027         super();
00028     }
```

7.34.2.2 ReminderMySqlRepository() [2/2]

`com.mehdi.petreminder.repository.ReminderMySqlRepository.ReminderMySqlRepository (Connection connection)`

Test yapıcısı.

Parameters

<i>connection</i>	Test bağlantısı
-------------------	-----------------

Definition at line 34 of file [ReminderMySqlRepository.java](#).

```
00034                                     {
00035         super(connection);
00036     }
```

References [com.mehdi.petreminder.repository.MySqlRepository<Reminder>.connection](#).

7.34.3 Member Function Documentation

7.34.3.1 getCreateTableSql()

`String com.mehdi.petreminder.repository.ReminderMySqlRepository.getCreateTableSql () [protected]`

Member documentation.

Definition at line 52 of file [ReminderMySqlRepository.java](#).

```
00052                                     {
00053         return "CREATE TABLE IF NOT EXISTS reminders ("
00054             + "id INT AUTO_INCREMENT PRIMARY KEY,"
00055             + "pet_id INT NOT NULL,"
00056             + "pet_name VARCHAR(50),"
00057             + "reminder_type VARCHAR(30) NOT NULL,"
00058             + "scheduled_time DATETIME NOT NULL,"
00059             + "completed BOOLEAN DEFAULT FALSE,"
00060             + "description TEXT,"
00061             + "priority VARCHAR(10) DEFAULT 'ORTA',"
00062             + "recurring BOOLEAN DEFAULT FALSE,"
00063             + "recurrence_pattern VARCHAR(30),"
00064             + "food_type VARCHAR(50),"
00065             + "portion_grams DOUBLE,"
00066             + "includes_water_change BOOLEAN DEFAULT FALSE,"
00067             + "medication_name VARCHAR(100),"
00068             + "dosage DOUBLE,"
00069             + "dosage_unit VARCHAR(20),"
00070             + "prescribed BOOLEAN DEFAULT FALSE,"
```

```

00071         + "exercise_type VARCHAR(50),"
00072         + "duration_minutes INT,"
00073         + "distance_km DOUBLE,"
00074         + "grooming_type VARCHAR(50),"
00075         + "professional BOOLEAN DEFAULT FALSE,"
00076         + "groomer_name VARCHAR(100),"
00077         + "veterinarian_name VARCHAR(100),"
00078         + "clinic_name VARCHAR(100),"
00079         + "reason VARCHAR(100),"
00080         + "estimated_cost DOUBLE,"
00081         + "confirmed BOOLEAN DEFAULT FALSE,"
00082         + "clinic_phone VARCHAR(20)"
00083         + ");";
00084     }

```

7.34.3.2 getInsertSql()

String com.mehdi.petreminder.repository.ReminderMySQLRepository.getInsertSql () [protected]

Member documentation.

Definition at line 91 of file [ReminderMySQLRepository.java](#).

```

00091     {
00092         return "INSERT INTO reminders (pet_id,pet_name,reminder_type,scheduled_time,"
00093         + "completed,description,priority,recurring,recurrence_pattern,"
00094         + "food_type,portion_grams,includes_water_change,"
00095         + "medication_name,dosage,dosage_unit,prescribed,"
00096         + "exercise_type,duration_minutes,distance_km,"
00097         + "grooming_type,professional,groomer_name,"
00098         + "veterinarian_name,clinic_name,reason,estimated_cost,confirmed,clinic_phone)"
00099         + "VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";
00100     }

```

7.34.3.3 getTableName()

String com.mehdi.petreminder.repository.ReminderMySQLRepository.getTableName () [protected]

Member documentation.

Definition at line 43 of file [ReminderMySQLRepository.java](#).

```

00043     {
00044         return "reminders";
00045     }

```

7.34.3.4 getUpdateSql()

String com.mehdi.petreminder.repository.ReminderMySQLRepository.getUpdateSql () [protected]

Member documentation.

Definition at line 107 of file [ReminderMySQLRepository.java](#).

```

00107     {
00108         return "UPDATE reminders SET pet_id=?,pet_name=?,reminder_type=?,scheduled_time=,"
00109         + "completed=?,description=?,priority=?,recurring=?,recurrence_pattern=?, "
00110         + "food_type=?,portion_grams=?,includes_water_change=?, "
00111         + "medication_name=?,dosage=?,dosage_unit=?,prescribed=?, "
00112         + "exercise_type=?,duration_minutes=?,distance_km=?, "
00113         + "grooming_type=?,professional=?,groomer_name=?, "
00114         + "veterinarian_name=?,clinic_name=?,reason=?,estimated_cost=?,confirmed=?, "
00115         + "clinic_phone=? WHERE id=?";
00116     }

```

7.34.3.5 mapRow()

`Reminder` com.mehdi.petreminder.repository.ReminderMySQLRepository.mapRow (ResultSet rs) throws SQLException [protected]

Member documentation.

Definition at line 170 of file `ReminderMySQLRepository.java`.

```

00170                                     {
00171         String type = rs.getString("reminder_type");
00172         int id = rs.getInt("id");
00173         int petId = rs.getInt("pet_id");
00174         String petName = rs.getString("pet_name");
00175         Timestamp ts = rs.getTimestamp("scheduled_time");
00176         LocalDateTime scheduledTime = ts != null ? ts.toLocalDateTime() : null;
00177         String description = rs.getString("description");
00178         String priority = rs.getString("priority");
00179         boolean recurring = rs.getBoolean("recurring");
00180         String recurrencePattern = rs.getString("recurrence_pattern");
00181         boolean completed = rs.getBoolean("completed");
00182
00183         Reminder reminder;
00184         switch (type != null ? type : "") {
00185             case "Besleme":
00186                 reminder = new FeedingReminder(id, petId, petName, scheduledTime,
00187                     description, priority, recurring, recurrencePattern,
00188                     rs.getString("food_type"), rs.getDouble("portion_grams"),
00189                     rs.getBoolean("includes_water_change"));
00190                 break;
00191             case "Ilac":
00192                 reminder = new MedicationReminder(id, petId, petName, scheduledTime,
00193                     description, priority, recurring, recurrencePattern,
00194                     rs.getString("medication_name"), rs.getDouble("dosage"),
00195                     rs.getString("dosage_unit"), rs.getBoolean("prescribed"), null);
00196                 break;
00197             case "Egzersiz":
00198                 reminder = new ExerciseReminder(id, petId, petName, scheduledTime,
00199                     description, priority, recurring, recurrencePattern,
00200                     rs.getString("exercise_type"), rs.getInt("duration_minutes"),
00201                     rs.getDouble("distance_km"));
00202                 break;
00203             case "Bakim":
00204                 reminder = new GroomingReminder(id, petId, petName, scheduledTime,
00205                     description, priority, recurring, recurrencePattern,
00206                     rs.getString("grooming_type"), rs.getBoolean("professional"),
00207                     rs.getString("groomer_name"));
00208                 break;
00209             case "Veteriner":
00210                 reminder = new VetAppointment(id, petId, petName, scheduledTime,
00211                     description, priority, recurring, recurrencePattern,
00212                     rs.getString("veterinarian_name"), rs.getString("clinic_name"),
00213                     rs.getString("reason"), rs.getDouble("estimated_cost"),
00214                     rs.getBoolean("confirmed"), rs.getString("clinic_phone"));
00215                 break;
00216             default:
00217                 reminder = new FeedingReminder(id, petId, petName, scheduledTime, "Mama", 100);
00218         }
00219         reminder.setCompleted(completed);
00220         return reminder;
00221     }

```

References `com.mehdi.petreminder.model.Reminder.setCompleted()`.

7.34.3.6 setInsertParams()

`void` com.mehdi.petreminder.repository.ReminderMySQLRepository.setInsertParams (PreparedStatement ps, `Reminder` r) throws SQLException [protected]

Member documentation.

Definition at line 123 of file `ReminderMySQLRepository.java`.

```

00123                                     {
00124         ps.setInt(1, r.getPetId());

```



```

00125         ps.setString(2, r.getPetName());
00126         ps.setString(3, r.getReminderType());
00127         ps.setObject(4, r.getScheduledTime() != null
00128             ? Timestamp.valueOf(r.getScheduledTime()) : null);
00129         ps.setBoolean(5, r.isCompleted());
00130         ps.setString(6, r.getDescription());
00131         ps.setString(7, r.getPriority());
00132         ps.setBoolean(8, r.isRecurring());
00133         ps.setString(9, r.getRecurrencePattern());
00134         ps.setString(10, r instanceof FeedingReminder ? ((FeedingReminder) r).getFoodType() : null);
00135         ps.setObject(11, r instanceof FeedingReminder ? ((FeedingReminder) r).getPortionGrams() :
null);
00136         ps.setBoolean(12, r instanceof FeedingReminder && ((FeedingReminder)
r).isIncludesWaterChange());
00137         ps.setString(13, r instanceof MedicationReminder ? ((MedicationReminder)
r).getMedicationName() : null);
00138         ps.setObject(14, r instanceof MedicationReminder ? ((MedicationReminder) r).getDosage() :
null);
00139         ps.setString(15, r instanceof MedicationReminder ? ((MedicationReminder) r).getDosageUnit() :
null);
00140         ps.setBoolean(16, r instanceof MedicationReminder && ((MedicationReminder) r).isPrescribed());
00141         ps.setString(17, r instanceof ExerciseReminder ? ((ExerciseReminder) r).getExerciseType() :
null);
00142         ps.setObject(18, r instanceof ExerciseReminder ? ((ExerciseReminder) r).getDurationMinutes() :
null);
00143         ps.setObject(19, r instanceof ExerciseReminder ? ((ExerciseReminder) r).getDistanceKm() :
null);
00144         ps.setString(20, r instanceof GroomingReminder ? ((GroomingReminder) r).getGroomingType() :
null);
00145         ps.setBoolean(21, r instanceof GroomingReminder && ((GroomingReminder) r).isProfessional());
00146         ps.setString(22, r instanceof GroomingReminder ? ((GroomingReminder) r).getGroomerName() :
null);
00147         ps.setString(23, r instanceof VetAppointment ? ((VetAppointment) r).getVeterinarianName() :
null);
00148         ps.setString(24, r instanceof VetAppointment ? ((VetAppointment) r).getClinicName() : null);
00149         ps.setString(25, r instanceof VetAppointment ? ((VetAppointment) r).getReason() : null);
00150         ps.setObject(26, r instanceof VetAppointment ? ((VetAppointment) r).getEstimatedCost() :
null);
00151         ps.setBoolean(27, r instanceof VetAppointment && ((VetAppointment) r).isConfirmed());
00152         ps.setString(28, r instanceof VetAppointment ? ((VetAppointment) r).getClinicPhone() : null);
00153     }

```

Referenced by [setUpdateParams\(\)](#).

7.34.3.7 setUpdateParams()

```

void com.mehdi.petreminder.repository.ReminderMySQLRepository.setUpdateParams (
    PreparedStatement ps,
    Reminder r) throws SQLException [protected]

```

Member documentation.

Definition at line 160 of file [ReminderMySQLRepository.java](#).

```

00160
00161         setInsertParams(ps, r);
00162         ps.setInt(29, r.getId());
00163     }

```

References [setInsertParams\(\)](#).

The documentation for this class was generated from the following file:

- petreminder-app/src/main/java/com/mehdi/petreminder/repository/[ReminderMySQLRepository.java](#)

7.35 com.mehdi.petreminder.service.ReminderService Class Reference

[Reminder](#) CRUD ve iş kuralları.

Public Member Functions

- [ReminderService](#) ()
Varsayılan yapıcı.
- [ReminderService](#) ([IRepository](#)< [Reminder](#) > repository)
Test yapıcısı.
- [Reminder](#) [addReminder](#) ([Reminder](#) reminder)
Yeni hatırlatıcı ekler.
- [Reminder](#) [getReminderByld](#) (int id)
ID ile hatırlatıcı getirir.
- List< [Reminder](#) > [getAllReminders](#) ()
Tüm hatırlatıcıları listeler.
- List< [Reminder](#) > [getPendingReminders](#) ()
Bekleyen (tamamlanmamış) hatırlatıcıları getirir.
- List< [Reminder](#) > [getOverdueReminders](#) ()
Gecikmiş hatırlatıcıları getirir.
- List< [Reminder](#) > [getRemindersByPetId](#) (int petId)
Pet'e ait hatırlatıcıları getirir.
- void [markCompleted](#) (int id)
Hatırlatıcıyı tamamlandı olarak işaretler.
- void [updateReminder](#) ([Reminder](#) reminder)
Hatırlatıcı günceller.
- void [deleteReminder](#) (int id)
Hatırlatıcı siler.
- void [deleteAllReminders](#) ()
Tüm hatırlatıcıları siler.
- int [getReminderCount](#) ()
Toplam hatırlatıcı sayısı.
- void [close](#) ()
Repository kapatır.

7.35.1 Detailed Description

[Reminder](#) CRUD ve iş kuralları.

Observer Pattern: [Reminder](#) eklendiğinde, tamamlandığında veya silindiğinde [EventManager](#) üzerinden kayıtlı observer'lara bildirim gönderilir. Böylece Dashboard gibi bileşenler otomatik güncellenir.

Author

Muhammed Mehdi Karagülle
İbrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 32 of file [ReminderService.java](#).

7.35.2 Constructor & Destructor Documentation

7.35.2.1 ReminderService() [1/2]

com.mehdi.petreminder.service.ReminderService.ReminderService ()

Varsayılan yapıcı.

Definition at line 44 of file [ReminderService.java](#).

```
00044         {
00045             this.repository = RepositoryFactory.createReminderRepository();
00046         }
```

References [com.mehdi.petreminder.repository.RepositoryFactory.createReminderRepository\(\)](#).

7.35.2.2 ReminderService() [2/2]

com.mehdi.petreminder.service.ReminderService.ReminderService (
 IRepository< Reminder > repository)

Test yapıcısı.

Parameters

<i>repository</i>	Test repository
-------------------	-----------------

Definition at line 52 of file [ReminderService.java](#).

```
00052         {
00053             this.repository = repository;
00054         }
```

7.35.3 Member Function Documentation

7.35.3.1 addReminder()

[Reminder](#) com.mehdi.petreminder.service.ReminderService.addReminder (
 [Reminder](#) reminder)

Yeni hatırlatıcı ekler.

Parameters

<i>reminder</i>	Eklenecek Reminder
-----------------	------------------------------------

Returns

Oluşturulan [Reminder](#)

Exceptions

ServiceException

Doğrulama başarısız

Definition at line 62 of file [ReminderService.java](#).

```
00062                                     {
00063         validateReminder(reminder);
00064         int id = repository.save(reminder);
00065         logger.info("Reminder eklendi: id={}, type={}", id, reminder.getReminderType());
00066         // Observer Pattern: Reminder eklendi bildirimi
00067         EventManager.getInstance().notify(EventType.REMINDER_ADDED, reminder);
00068         return reminder;
00069     }
```

References [com.mehdi.petreminder.observer.EventManager.getInstance\(\)](#), [com.mehdi.petreminder.model.Reminder.getReminderType\(\)](#), [com.mehdi.petreminder.observer.EventManager.notify\(\)](#), and [com.mehdi.petreminder.observer.EventType.REMINDER_ADDED](#).

7.35.3.2 close()

```
void com.mehdi.petreminder.service.ReminderService.close ()
```

Repository kapatır.

Definition at line 179 of file [ReminderService.java](#).

```
00179         {
00180         repository.close();
00181     }
```

7.35.3.3 deleteAllReminders()

```
void com.mehdi.petreminder.service.ReminderService.deleteAllReminders ()
```

Tüm hatırlatıcıları siler.

Definition at line 164 of file [ReminderService.java](#).

```
00164         {
00165         repository.deleteAll();
00166     }
```

7.35.3.4 deleteReminder()

```
void com.mehdi.petreminder.service.ReminderService.deleteReminder (
    int id)
```

Hatırlatıcı siler.

Parameters

<i>id</i>	Silinecek ID
-----------	--------------

Definition at line 151 of file [ReminderService.java](#).

```
00151         {
00152             boolean deleted = repository.delete(id);
00153             if (!deleted) {
00154                 throw new ServiceException("Reminder silinemedi: id=" + id);
00155             }
00156             logger.info("Reminder silindi: id={}", id);
00157             // Observer Pattern: Reminder silindi bildirimi
00158             EventManager.getInstance().notify(EventType.REMINDER_DELETED, id);
00159         }
```

References [com.mehdi.petreminder.observer.EventManager.getInstance\(\)](#), [com.mehdi.petreminder.observer.EventManager.notify\(\)](#), and [com.mehdi.petreminder.observer.EventType.REMINDER_DELETED](#).

7.35.3.5 getAllReminders()

```
List< Reminder > com.mehdi.petreminder.service.ReminderService.getAllReminders ()
```

Tüm hatırlatıcıları listeler.

Returns

[Reminder](#) listesi

Definition at line 85 of file [ReminderService.java](#).

```
00085         {
00086             return repository.findAll();
00087         }
```

7.35.3.6 getOverdueReminders()

```
List< Reminder > com.mehdi.petreminder.service.ReminderService.getOverdueReminders ()
```

Gecikmiş hatırlatıcıları getirir.

Returns

Overdue reminder listesi

Definition at line 105 of file [ReminderService.java](#).

```
00105         {
00106             return repository.findAll().stream()
00107                 .filter(Reminder::isOverdue)
00108                 .collect(Collectors.toList());
00109         }
```

References [com.mehdi.petreminder.model.Reminder.isOverdue\(\)](#).

7.35.3.7 getPendingReminders()

```
List< Reminder > com.mehdi.petreminder.service.ReminderService.getPendingReminders ()
```

Bekleyen (tamamlanmamış) hatırlatıcıları getirir.

Returns

Pending reminder listesi

Definition at line 93 of file [ReminderService.java](#).

```
00093         {
00094             return repository.findAll().stream()
00095                 .filter(r -> !r.isCompleted())
00096                 .sorted(Comparator.comparing(r -> r.getScheduledTime() != null
00097                     ? r.getScheduledTime() : LocalDateTime.MAX))
00098                 .collect(Collectors.toList());
00099         }
```

7.35.3.8 getReminderById()

```
Reminder com.mehdi.petreminder.service.ReminderService.getReminderById (
    int id)
```

ID ile hatırlatıcı getirir.

Parameters

<i>id</i>	Reminder ID
-----------	-------------

Returns

Reminder nesnesi

Definition at line 76 of file [ReminderService.java](#).

```
00076                                     {
00077         return repository.findById(id)
00078             .orElseThrow(() -> new ServiceException("Reminder bulunamadı: id=" + id));
00079     }
```

Referenced by [markCompleted\(\)](#).

7.35.3.9 getReminderCount()

```
int com.mehdi.petreminder.service.ReminderService.getReminderCount ()
```

Toplam hatırlatıcı sayısı.

Returns

Sayı

Definition at line 172 of file [ReminderService.java](#).

```
00172                                     {
00173         return repository.count();
00174     }
```

7.35.3.10 getRemindersByPetId()

```
List< Reminder > com.mehdi.petreminder.service.ReminderService.getRemindersByPetId (
    int petId)
```

Pet'e ait hatırlatıcıları getirir.

Parameters

<i>pet↔ Id</i>	Pet ID
--------------------	--------

Returns

Reminder listesi

Definition at line 116 of file [ReminderService.java](#).

```
00116                                     {
00117         return repository.findAll().stream()
00118             .filter(r -> r.getPetId() == petId)
00119             .collect(Collectors.toList());
00120     }
```

7.35.3.11 markCompleted()

```
void com.mehdi.petreminder.service.ReminderService.markCompleted (
    int id)
```

Hatırlatıcıyı tamamlandı olarak işaretler.

Parameters

<i>id</i>	Reminder ID
-----------	-------------

Definition at line 126 of file [ReminderService.java](#).

```
00126         {
00127             Reminder r = getReminderById(id);
00128             r.setCompleted(true);
00129             repository.update(r);
00130             logger.info("Reminder tamamlandı id={}", id);
00131             // Observer Pattern: Reminder tamamlandı bildirimi
00132             EventManager.getInstance().notify(EventType.REMINDER_COMPLETED, r);
00133         }
```

References [com.mehdi.petreminder.observer.EventManager.getInstance\(\)](#), [getReminderById\(\)](#), [com.mehdi.petreminder.observer.EventManager.notify\(\)](#), [com.mehdi.petreminder.observer.EventType.REMINDER_COMPLETED](#), and [com.mehdi.petreminder.model.Reminder.setCompleted\(\)](#).

7.35.3.12 updateReminder()

```
void com.mehdi.petreminder.service.ReminderService.updateReminder (
    Reminder reminder)
```

Hatırlatıcı günceller.

Parameters

<i>reminder</i>	Güncellenecek Reminder
-----------------	--

Definition at line 139 of file [ReminderService.java](#).

```
00139         {
00140             validateReminder(reminder);
00141             boolean updated = repository.update(reminder);
00142             if (!updated) {
00143                 throw new ServiceException("Reminder güncellenemedi: id=" + reminder.getId());
00144             }
00145         }
```

References [com.mehdi.petreminder.model.Reminder.getId\(\)](#).

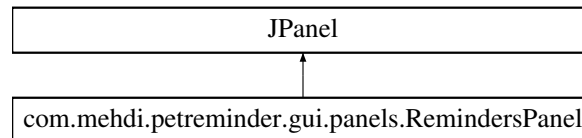
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/service/ReminderService.java](#)

7.36 com.mehdi.petreminder.gui.panels.RemindersPanel Class Reference

Hatırlatıcılar CRUD paneli.

Inheritance diagram for com.mehdi.petreminder.gui.panels.RemindersPanel:



Public Member Functions

- [RemindersPanel](#) ([MainFrame](#) mainFrame)
[RemindersPanel](#) yapıcısı.
- void [refresh](#) ()
Tabloyu yeniler.

7.36.1 Detailed Description

Hatırlatıcılar CRUD paneli.

Author

Muhammed Mehdi Karagülle
İbrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 28 of file [RemindersPanel.java](#).

7.36.2 Constructor & Destructor Documentation

7.36.2.1 RemindersPanel()

```
com.mehdi.petreminder.gui.panels.RemindersPanel.RemindersPanel (  
    MainFrame mainFrame)
```

[RemindersPanel](#) yapıcısı.

Parameters

<i>mainFrame</i>	Ana pencere
------------------	-------------

Definition at line 52 of file [RemindersPanel.java](#).

```

00052         {
00053             this.mainFrame      = mainFrame;
00054             this.reminderService = new ReminderService();
00055             setBackground(GuiConstants.BG_COLOR);
00056             setLayout(new BorderLayout(GuiConstants.GAP, GuiConstants.GAP));
00057             setBorder(new EmptyBorder(GuiConstants.GAP_LARGE, GuiConstants.GAP_LARGE,
00058                                     GuiConstants.GAP_LARGE, GuiConstants.GAP_LARGE));
00059             buildUI();
00060         }

```

References [com.mehdi.petreminder.gui.util.GuiConstants.BG_COLOR](#), [com.mehdi.petreminder.gui.util.GuiConstants.GAP](#), and [com.mehdi.petreminder.gui.util.GuiConstants.GAP_LARGE](#).

7.36.3 Member Function Documentation**7.36.3.1 refresh()**

```
void com.mehdi.petreminder.gui.panels.RemindersPanel.refresh ()
```

Tabloyu yeniler.

Definition at line 204 of file [RemindersPanel.java](#).

```

00204         {
00205             tableModel.setRowCount(0);
00206             try {
00207                 List<Reminder> list = chkOnlyPending != null && chkOnlyPending.isSelected()
00208                     ? reminderService.getPendingReminders()
00209                     : reminderService.getAllReminders();
00210                 for (Reminder r : list) {
00211                     String status = r.isCompleted() ? " Tamamlandı"
00212                         : r.isOverdue() ? " Gecikmiş" : " Bekliyor";
00213                     tableModel.addRow(new Object[]{
00214                         r.getId(), r.getPetName(), r.getReminderType(),
00215                         r.getPriority(), r.getFormattedTime(), status
00216                     });
00217                 }
00218             } catch (Exception e) { /* sessiz */ }
00219         }

```

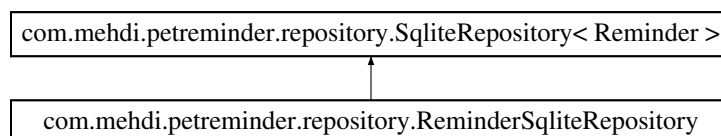
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/RemindersPanel.java](#)

7.37 com.mehdi.petreminder.repository.ReminderSqliteRepository Class Reference

SQLite üzerinde [Reminder](#) (tüm alt türler) CRUD işlemleri.

Inheritance diagram for com.mehdi.petreminder.repository.ReminderSqliteRepository:



Public Member Functions

- [ReminderSqliteRepository](#) ()
Varsayılan yapıcı.
- [ReminderSqliteRepository](#) (Connection [connection](#))
Test yapıcısı.
- List< [Reminder](#) > [findByPetId](#) (int petId)
PetId'ye göre hatırlatıcıları getirir.

Public Member Functions inherited from**[com.mehdi.petreminder.repository.SqliteRepository](#)< [Reminder](#) >**

- [SqliteRepository](#) ()
SQLite bağlantısını açar ve tabloyu oluşturur.
- int [save](#) (T entity)
- Optional< T > [findById](#) (int id)
- List< T > [findAll](#) ()
- boolean [update](#) (T entity)
- boolean [delete](#) (int id)
- void [deleteAll](#) ()
- int [count](#) ()
- void [close](#) ()
- Connection [getConnection](#) ()
Bağlantıyı döndürür (test için).

Protected Member Functions

- String [getTableName](#) ()
- String [getCreateTableSql](#) ()
- String [getInsertSql](#) ()
- String [getUpdateSql](#) ()
- void [setInsertParams](#) (PreparedStatement ps, [Reminder](#) r) throws SQLException
- void [setUpdateParams](#) (PreparedStatement ps, [Reminder](#) r) throws SQLException
- [Reminder](#) [mapRow](#) (ResultSet rs) throws SQLException

Protected Member Functions inherited from**[com.mehdi.petreminder.repository.SqliteRepository](#)< [Reminder](#) >**

- abstract String [getTableName](#) ()
Tablo adını döndürür.
- abstract String [getCreateTableSql](#) ()
Tablo oluşturma SQL'ini döndürür.
- abstract String [getInsertSql](#) ()
INSERT SQL'ini döndürür (ID hariç).
- abstract String [getUpdateSql](#) ()
UPDATE SQL'ini döndürür.
- abstract void [setInsertParams](#) (PreparedStatement ps, T entity) throws SQLException
PreparedStatement'a INSERT parametrelerini set eder.
- abstract void [setUpdateParams](#) (PreparedStatement ps, T entity) throws SQLException
PreparedStatement'a UPDATE parametrelerini set eder.
- abstract T [mapRow](#) (ResultSet rs) throws SQLException
ResultSet'ten varlık oluşturur.
- void [createTableIfNotExists](#) ()
Tabloyu oluşturur (yoksa).
- void [setEntityId](#) (T entity, int id)
Reflection ile entity'ye id set eder.

Additional Inherited Members

Protected Attributes inherited from

[com.mehdi.petreminder.repository.SqliteRepository](#)< [Reminder](#) >

- Connection [connection](#)
SQLite JDBC bağlantısı.

Static Protected Attributes inherited from

[com.mehdi.petreminder.repository.SqliteRepository](#)< [Reminder](#) >

- static final Logger [logger](#)
Logger.

7.37.1 Detailed Description

SQLite üzerinde [Reminder](#) (tüm alt türler) CRUD işlemleri.

Tüm [Reminder](#) alt sınıfları tek tabloda (reminder_type sütunu ile) saklanır.

Author

Muhammed Mehdi Karagülle
İbrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 22 of file [ReminderSqliteRepository.java](#).

7.37.2 Constructor & Destructor Documentation

7.37.2.1 ReminderSqliteRepository() [1/2]

```
com.mehdi.petreminder.repository.ReminderSqliteRepository.ReminderSqliteRepository ()
```

Varsayılan yapıcı.

Definition at line 27 of file [ReminderSqliteRepository.java](#).

```
00027                                     {  
00028     super ();  
00029 }
```

7.37.2.2 ReminderSqliteRepository() [2/2]

```
com.mehdi.petreminder.repository.ReminderSqliteRepository.ReminderSqliteRepository (
    Connection connection)
```

Test yapıcısı.

Parameters

<i>connection</i>	Test bağlantısı
-------------------	-----------------

Definition at line 35 of file [ReminderSqliteRepository.java](#).

```
00035                                     {
00036         super(connection);
00037     }
```

References [com.mehdi.petreminder.repository.SqliteRepository< Reminder >.connection](#).

7.37.3 Member Function Documentation

7.37.3.1 findByPetId()

```
List< Reminder > com.mehdi.petreminder.repository.ReminderSqliteRepository.findByPetId (
    int petId)
```

PetId'ye göre hatırlatıcıları getirir.

Parameters

<i>petId</i>	Pet ID
--------------	--------

Returns

Hatırlatıcı listesi

Definition at line 240 of file [ReminderSqliteRepository.java](#).

```
00240                                     {
00241         List<Reminder> list = new ArrayList<>();
00242         try (PreparedStatement ps = connection.prepareStatement(
00243             "SELECT * FROM reminders WHERE pet_id = ?")) {
00244             ps.setInt(1, petId);
00245             ResultSet rs = ps.executeQuery();
00246             while (rs.next()) list.add(mapRow(rs));
00247         } catch (SQLException e) {
00248             throw new RepositoryException("findByPetId hatası", e);
00249         }
00250         return list;
00251     }
```

References [com.mehdi.petreminder.repository.SqliteRepository< Reminder >.connection](#), and [mapRow\(\)](#).

7.37.3.2 getCreateTableSql()

String com.mehdi.petreminder.repository.ReminderSqliteRepository.getCreateTableSql () [protected]

Member documentation.

Definition at line 53 of file [ReminderSqliteRepository.java](#).

```

00053         {
00054             return "CREATE TABLE IF NOT EXISTS reminders ("
00055                 + "id INTEGER PRIMARY KEY AUTOINCREMENT,"
00056                 + "pet_id INTEGER NOT NULL,"
00057                 + "pet_name TEXT,"
00058                 + "reminder_type TEXT NOT NULL,"
00059                 + "scheduled_time TEXT NOT NULL,"
00060                 + "completed INTEGER DEFAULT 0,"
00061                 + "description TEXT,"
00062                 + "priority TEXT DEFAULT 'ORTA',"
00063                 + "recurring INTEGER DEFAULT 0,"
00064                 + "recurrence_pattern TEXT,"
00065                 + "food_type TEXT,"
00066                 + "portion_grams REAL,"
00067                 + "includes_water_change INTEGER DEFAULT 0,"
00068                 + "medication_name TEXT,"
00069                 + "dosage REAL,"
00070                 + "dosage_unit TEXT,"
00071                 + "prescribed INTEGER DEFAULT 0,"
00072                 + "exercise_type TEXT,"
00073                 + "duration_minutes INTEGER,"
00074                 + "distance_km REAL,"
00075                 + "grooming_type TEXT,"
00076                 + "professional INTEGER DEFAULT 0,"
00077                 + "groomer_name TEXT,"
00078                 + "veterinarian_name TEXT,"
00079                 + "clinic_name TEXT,"
00080                 + "reason TEXT,"
00081                 + "estimated_cost REAL,"
00082                 + "confirmed INTEGER DEFAULT 0,"
00083                 + "clinic_phone TEXT"
00084                 + "));"
00085         }

```

7.37.3.3 getInsertSql()

String com.mehdi.petreminder.repository.ReminderSqliteRepository.getInsertSql () [protected]

Member documentation.

Definition at line 92 of file [ReminderSqliteRepository.java](#).

```

00092         {
00093             return "INSERT INTO reminders (pet_id,pet_name,reminder_type,scheduled_time,"
00094                 + "completed,description,priority,recurring,recurrence_pattern,"
00095                 + "food_type,portion_grams,includes_water_change,"
00096                 + "medication_name,dosage,dosage_unit,prescribed,"
00097                 + "exercise_type,duration_minutes,distance_km,"
00098                 + "grooming_type,professional,groomer_name,"
00099                 + "veterinarian_name,clinic_name,reason,estimated_cost,confirmed,clinic_phone) "
00100                 + "VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";
00101         }

```

7.37.3.4 getTableName()

String com.mehdi.petreminder.repository.ReminderSqliteRepository.getTableName () [protected]

Member documentation.

Definition at line 44 of file [ReminderSqliteRepository.java](#).

```

00044         {
00045             return "reminders";
00046         }

```

7.37.3.5 getUpdateSql()

String com.mehdi.petreminder.repository.ReminderSqliteRepository.getUpdateSql () [protected]

Member documentation.

Definition at line 108 of file [ReminderSqliteRepository.java](#).

```
00108      {
00109          return "UPDATE reminders SET pet_id=?,pet_name=?,reminder_type=?,scheduled_time=,"
00110              + "completed=?,description=?,priority=?,recurring=?,recurrence_pattern=,"
00111              + "food_type=?,portion_grams=?,includes_water_change=,"
00112              + "medication_name=?,dosage=?,dosage_unit=?,prescribed=,"
00113              + "exercise_type=?,duration_minutes=?,distance_km=,"
00114              + "grooming_type=?,professional=?,groomer_name=,"
00115              + "veterinarian_name=?,clinic_name=?,reason=?,estimated_cost=?,confirmed=,"
00116              + "clinic_phone=? WHERE id=?";
00117      }
```

7.37.3.6 mapRow()

[Reminder](#) com.mehdi.petreminder.repository.ReminderSqliteRepository.mapRow (
 ResultSet rs) throws SQLException [protected]

Member documentation.

Definition at line 175 of file [ReminderSqliteRepository.java](#).

```
00175      {
00176          String type = rs.getString("reminder_type");
00177          int id = rs.getInt("id");
00178          int petId = rs.getInt("pet_id");
00179          String petName = rs.getString("pet_name");
00180          String timeStr = rs.getString("scheduled_time");
00181          LocalDateTime scheduledTime = timeStr != null ? LocalDateTime.parse(timeStr) : null;
00182          String description = rs.getString("description");
00183          String priority = rs.getString("priority");
00184          boolean recurring = rs.getInt("recurring") == 1;
00185          String recurrencePattern = rs.getString("recurrence_pattern");
00186          boolean completed = rs.getInt("completed") == 1;
00187
00188          Reminder reminder;
00189          switch (type) {
00190              case "Besleme":
00191                  FeedingReminder fr = new FeedingReminder(id, petId, petName, scheduledTime,
00192                      description, priority, recurring, recurrencePattern,
00193                      rs.getString("food_type"), rs.getDouble("portion_grams"),
00194                      rs.getInt("includes_water_change") == 1);
00195                  reminder = fr;
00196                  break;
00197              case "Ilac":
00198                  MedicationReminder mr = new MedicationReminder(id, petId, petName, scheduledTime,
00199                      description, priority, recurring, recurrencePattern,
00200                      rs.getString("medication_name"), rs.getDouble("dosage"),
00201                      rs.getString("dosage_unit"), rs.getInt("prescribed") == 1, null);
00202                  reminder = mr;
00203                  break;
00204              case "Egzersiz":
00205                  ExerciseReminder er = new ExerciseReminder(id, petId, petName, scheduledTime,
00206                      description, priority, recurring, recurrencePattern,
00207                      rs.getString("exercise_type"), rs.getInt("duration_minutes"),
00208                      rs.getDouble("distance_km"));
00209                  reminder = er;
00210                  break;
00211              case "Bakim":
00212                  GroomingReminder gr = new GroomingReminder(id, petId, petName, scheduledTime,
00213                      description, priority, recurring, recurrencePattern,
00214                      rs.getString("grooming_type"), rs.getInt("professional") == 1,
00215                      rs.getString("groomer_name"));
00216                  reminder = gr;
00217                  break;
00218              case "Veteriner":
00219                  VetAppointment va = new VetAppointment(id, petId, petName, scheduledTime,
00220                      description, priority, recurring, recurrencePattern,
00221                      rs.getString("veterinarian_name"), rs.getString("clinic_name"),
00222                      rs.getString("reason"), rs.getDouble("estimated_cost"),
00223                      rs.getInt("confirmed") == 1, rs.getString("clinic_phone"));
```

```

00224         reminder = va;
00225         break;
00226     default:
00227         FeedingReminder def = new FeedingReminder(id, petId, petName, scheduledTime,
00228             "Mama", 100);
00229         reminder = def;
00230     }
00231     reminder.setCompleted(completed);
00232     return reminder;
00233 }

```

References [com.mehdi.petreminder.model.Reminder.setCompleted\(\)](#).

Referenced by [findByPetId\(\)](#).

7.37.3.7 setInsertParams()

```

void com.mehdi.petreminder.repository.ReminderSqliteRepository.setInsertParams (
    PreparedStatement ps,
    Reminder r) throws SQLException [protected]

```

Member documentation.

Definition at line 124 of file [ReminderSqliteRepository.java](#).

```

00124                                                                 {
00125         ps.setInt(1, r.getPetId());
00126         ps.setString(2, r.getPetName());
00127         ps.setString(3, r.getReminderType());
00128         ps.setString(4, r.getScheduledTime() != null ? r.getScheduledTime().toString() : null);
00129         ps.setInt(5, r.isCompleted() ? 1 : 0);
00130         ps.setString(6, r.getDescription());
00131         ps.setString(7, r.getPriority());
00132         ps.setInt(8, r.isRecurring() ? 1 : 0);
00133         ps.setString(9, r.getRecurrencePattern());
00134         // FeedingReminder
00135         ps.setString(10, r instanceof FeedingReminder ? ((FeedingReminder) r).getFoodType() : null);
00136         ps.setObject(11, r instanceof FeedingReminder ? ((FeedingReminder) r).getPortionGrams() :
null);
00137         ps.setInt(12, r instanceof FeedingReminder && ((FeedingReminder) r).isIncludesWaterChange() ?
1 : 0);
00138         // MedicationReminder
00139         ps.setString(13, r instanceof MedicationReminder ? ((MedicationReminder)
r).getMedicationName() : null);
00140         ps.setObject(14, r instanceof MedicationReminder ? ((MedicationReminder) r).getDosage() :
null);
00141         ps.setString(15, r instanceof MedicationReminder ? ((MedicationReminder) r).getDosageUnit() :
null);
00142         ps.setInt(16, r instanceof MedicationReminder && ((MedicationReminder) r).isPrescribed() ? 1 :
0);
00143         // ExerciseReminder
00144         ps.setString(17, r instanceof ExerciseReminder ? ((ExerciseReminder) r).getExerciseType() :
null);
00145         ps.setObject(18, r instanceof ExerciseReminder ? ((ExerciseReminder) r).getDurationMinutes() :
null);
00146         ps.setObject(19, r instanceof ExerciseReminder ? ((ExerciseReminder) r).getDistanceKm() :
null);
00147         // GroomingReminder
00148         ps.setString(20, r instanceof GroomingReminder ? ((GroomingReminder) r).getGroomingType() :
null);
00149         ps.setInt(21, r instanceof GroomingReminder && ((GroomingReminder) r).isProfessional() ? 1 :
0);
00150         ps.setString(22, r instanceof GroomingReminder ? ((GroomingReminder) r).getGroomerName() :
null);
00151         // VetAppointment
00152         ps.setString(23, r instanceof VetAppointment ? ((VetAppointment) r).getVeterinarianName() :
null);
00153         ps.setString(24, r instanceof VetAppointment ? ((VetAppointment) r).getClinicName() : null);
00154         ps.setString(25, r instanceof VetAppointment ? ((VetAppointment) r).getReason() : null);
00155         ps.setObject(26, r instanceof VetAppointment ? ((VetAppointment) r).getEstimatedCost() :
null);
00156         ps.setInt(27, r instanceof VetAppointment && ((VetAppointment) r).isConfirmed() ? 1 : 0);
00157         ps.setString(28, r instanceof VetAppointment ? ((VetAppointment) r).getClinicPhone() : null);
00158     }

```

Referenced by [setUpUpdateParams\(\)](#).

7.37.3.8 setUpdateParams()

```
void com.mehdi.petreminder.repository.ReminderSqliteRepository.setUpdateParams (
    PreparedStatement ps,
    Reminder r) throws SQLException [protected]
```

Member documentation.

Definition at line 165 of file [ReminderSqliteRepository.java](#).

```
00165                                     {
00166         setInsertParams(ps, r);
00167         ps.setInt(29, r.getId());
00168     }
```

References [setInsertParams\(\)](#).

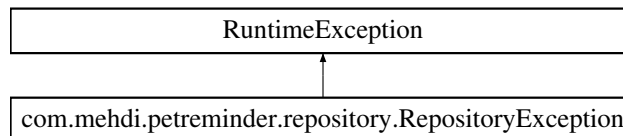
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/repository/ReminderSqliteRepository.java](#)

7.38 com.mehdi.petreminder.repository.RepositoryException Class Reference

Repository işlemlerinde fırlatılan unchecked istisna.

Inheritance diagram for com.mehdi.petreminder.repository.RepositoryException:



Public Member Functions

- [RepositoryException](#) (String message)
Mesajlı yapıcı.
- [RepositoryException](#) (String message, Throwable cause)
Mesaj ve neden'li yapıcı.

7.38.1 Detailed Description

Repository işlemlerinde fırlatılan unchecked istisna.

OOP: RuntimeException türetilmiş özel istisna.

Author

Muhammed Mehdi Karagülle
Ibrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 16 of file [RepositoryException.java](#).

7.38.2 Constructor & Destructor Documentation

7.38.2.1 RepositoryException() [1/2]

```
com.mehdi.petreminder.repository.RepositoryException.RepositoryException (
    String message)
```

Mesajlı yapıcı.

Parameters

<i>message</i>	Hata mesajı
----------------	-------------

Definition at line 25 of file [RepositoryException.java](#).

```
00025                                     {
00026         super (message);
00027     }
```

7.38.2.2 RepositoryException() [2/2]

```
com.mehdi.petreminder.repository.RepositoryException.RepositoryException (
    String message,
    Throwable cause)
```

Mesaj ve neden'li yapıcı.

Parameters

<i>message</i>	Hata mesajı
<i>cause</i>	Altta yatan istisna

Definition at line 34 of file [RepositoryException.java](#).

```
00034                                     {
00035         super (message, cause);
00036     }
```

The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/repository/RepositoryException.java](#)

7.39 com.mehdi.petreminder.repository.RepositoryFactory Class Reference

Aktif storage backend'e göre repository üretir.

Static Public Member Functions

- static [IRepository](#)< [Pet](#) > [createPetRepository](#) ()
Aktif backend için [Pet](#) repository üretir.
- static [IRepository](#)< [Reminder](#) > [createReminderRepository](#) ()
Aktif backend için [Reminder](#) repository üretir.
- static [IRepository](#)< [User](#) > [createUserRepository](#) ()
Aktif backend için [User](#) repository üretir.
- static [IRepository](#)< [MedicalRecord](#) > [createMedicalRecordRepository](#) ()
Aktif backend için [MedicalRecord](#) repository üretir.
- static [StorageType](#) [getActiveType](#) ()
Aktif backend tipini döndürür.

7.39.1 Detailed Description

Aktif storage backend'e göre repository üretir.

PDF zorunluluğu: [RepositoryFactory](#) pattern — runtime switch. Factory Method Pattern: [createPetRepository\(\)](#), [createReminderRepository\(\)](#) vb. [StorageConfig.getActiveBackend\(\)](#) 'a göre doğru implementasyonu döner.

Author

Muhammed Mehdi Karagülle
Ibrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 25 of file [RepositoryFactory.java](#).

7.39.2 Member Function Documentation

7.39.2.1 createMedicalRecordRepository()

```
IRepository< MedicalRecord > com.mehdi.petreminder.repository.RepositoryFactory.createMedicalRecordRepository () [static]
```

Aktif backend için [MedicalRecord](#) repository üretir.

Returns

[IRepository](#)<[MedicalRecord](#)> implementasyonu

Definition at line 87 of file [RepositoryFactory.java](#).

```
00087                                     {
00088         StorageType type = StorageConfig.getActiveBackend();
00089         logger.info("MedicalRecord repository oluşturuluyor: {}", type);
00090         switch (type) {
00091             case SQLITE: return new MedicalRecordSqliteRepository();
00092             case MYSQL:  return new MedicalRecordMySQLRepository();
00093             case BINARY: return new BinaryRepository<>(BINARY_DIR, "medical_records");
00094             default:
00095         }
00096     }
```

References [com.mehdi.petreminder.config.StorageConfig.getActiveBackend\(\)](#).

Referenced by [com.mehdi.petreminder.service.MedicalRecordService.MedicalRecordService\(\)](#).

7.39.2.2 createPetRepository()

```
IRepository< Pet > com.mehdi.petreminder.repository.RepositoryFactory.createPetRepository ()
[static]
```

Aktif backend için [Pet](#) repository üretir.

Returns

[IRepository<Pet>](#) implementasyonu

Exceptions

RepositoryException	Backend başlatılamazsa
-------------------------------------	------------------------

Definition at line 42 of file [RepositoryFactory.java](#).

```

00042                                     {
00043         StorageType type = StorageConfig.getActiveBackend();
00044         logger.info("Pet repository oluşturuluyor: {}", type);
00045         switch (type) {
00046             case SQLITE:      return new PetSqliteRepository();
00047             case MYSQL:       return new PetMySQLRepository();
00048             case BINARY:      return new BinaryRepository<>(BINARY_DIR, "pets");
00049         }
00050     }
00051 }
```

References [com.mehdi.petreminder.config.StorageConfig.getActiveBackend\(\)](#).

Referenced by [com.mehdi.petreminder.service.PetService.PetService\(\)](#).

7.39.2.3 createReminderRepository()

```
IRepository< Reminder > com.mehdi.petreminder.repository.RepositoryFactory.createReminder↵
Repository () [static]
```

Aktif backend için [Reminder](#) repository üretir.

Returns

[IRepository<Reminder>](#) implementasyonu

Definition at line 57 of file [RepositoryFactory.java](#).

```

00057                                     {
00058         StorageType type = StorageConfig.getActiveBackend();
00059         logger.info("Reminder repository oluşturuluyor: {}", type);
00060         switch (type) {
00061             case SQLITE:      return new ReminderSqliteRepository();
00062             case MYSQL:       return new ReminderMySQLRepository();
00063             case BINARY:      return new BinaryRepository<>(BINARY_DIR, "reminders");
00064         }
00065     }
00066 }
```

References [com.mehdi.petreminder.config.StorageConfig.getActiveBackend\(\)](#).

Referenced by [com.mehdi.petreminder.service.ReminderService.ReminderService\(\)](#).

7.39.2.4 createUserRepository()

`IRepository< User > com.mehdi.petreminder.repository.RepositoryFactory.createUserRepository ()`
[static]

Aktif backend için `User` repository üretir.

Returns

`IRepository<User>` implementasyonu

Definition at line 72 of file `RepositoryFactory.java`.

```
00072                                     {
00073         StorageType type = StorageConfig.getActiveBackend();
00074         logger.info("User repository oluşturuluyor: {}", type);
00075         switch (type) {
00076             case SQLITE: return new UserSqliteRepository();
00077             case MYSQL:  return new UserMySqlRepository();
00078             case BINARY:
00079                 default: return new BinaryRepository<>(BINARY_DIR, "users");
00080         }
00081     }
```

References `com.mehdi.petreminder.config.StorageConfig.getActiveBackend()`.

7.39.2.5 getActiveType()

`StorageType com.mehdi.petreminder.repository.RepositoryFactory.getActiveType ()` [static]

Aktif backend tipini döndürür.

Returns

Aktif `StorageType`

Definition at line 102 of file `RepositoryFactory.java`.

```
00102                                     {
00103         return StorageConfig.getActiveBackend();
00104     }
```

References `com.mehdi.petreminder.config.StorageConfig.getActiveBackend()`.

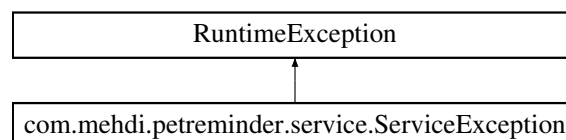
The documentation for this class was generated from the following file:

- `petreminder-app/src/main/java/com/mehdi/petreminder/repository/RepositoryFactory.java`

7.40 com.mehdi.petreminder.service.ServiceException Class Reference

Servis katmanında fırlatılan unchecked istisna.

Inheritance diagram for `com.mehdi.petreminder.service.ServiceException`:



Public Member Functions

- [ServiceException](#) (String message)
Mesajlı yapıcı.
- [ServiceException](#) (String message, Throwable cause)
Mesaj ve neden'li yapıcı.

7.40.1 Detailed Description

Servis katmanında fırlatılan unchecked istisna.

Author

Muhammed Mehdi Karagülle
Ibrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 15 of file [ServiceException.java](#).

7.40.2 Constructor & Destructor Documentation

7.40.2.1 ServiceException() [1/2]

```
com.mehdi.petreminder.service.ServiceException.ServiceException (  
    String message)
```

Mesajlı yapıcı.

Parameters

<i>message</i>	Hata mesajı
----------------	-------------

Definition at line 24 of file [ServiceException.java](#).

```
00024      {  
00025          super (message);  
00026      }
```

7.40.2.2 ServiceException() [2/2]

```
com.mehdi.petreminder.service.ServiceException.ServiceException (
    String message,
    Throwable cause)
```

Mesaj ve neden'li yapıcı.

Parameters

<i>message</i>	Hata mesajı
<i>cause</i>	Neden

Definition at line 33 of file [ServiceException.java](#).

```
00033                                     {
00034         super(message, cause);
00035     }
```

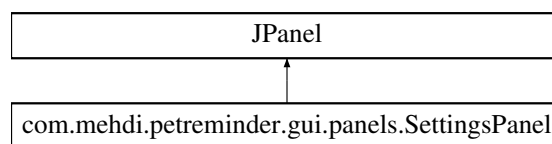
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/service/ServiceException.java](#)

7.41 com.mehdi.petreminder.gui.panels.SettingsPanel Class Reference

Storage backend ve uygulama ayarları.

Inheritance diagram for com.mehdi.petreminder.gui.panels.SettingsPanel:



Public Member Functions

- [SettingsPanel](#) ([MainFrame](#) mainFrame)
SettingsPanel yapıcısı.
- void [refresh](#) ()
Paneli yeniler.

7.41.1 Detailed Description

Storage backend ve uygulama ayarları.

PDF zorunluluğu: Runtime storage switching — no restart required.

Author

Muhammed Mehdi Karagülle
Ibrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 25 of file [SettingsPanel.java](#).

7.41.2 Constructor & Destructor Documentation

7.41.2.1 SettingsPanel()

com.mehdi.petreminder.gui.panels.SettingsPanel.SettingsPanel (
[MainFrame](#) *mainFrame*)

[SettingsPanel](#) yapıcısı.

Parameters

<i>mainFrame</i>	Ana pencere
------------------	-------------

Definition at line 43 of file [SettingsPanel.java](#).

```
00043                                     {
00044         this.mainFrame = mainFrame;
00045         setBackground(GuiConstants.BG\_COLOR);
00046         setLayout(new BorderLayout());
00047         setBorder(new EmptyBorder(GuiConstants.GAP\_LARGE, GuiConstants.GAP\_LARGE,
00048             GuiConstants.GAP\_LARGE, GuiConstants.GAP\_LARGE));
00049         buildUI();
00050     }
```

References [com.mehdi.petreminder.gui.util.GuiConstants.BG_COLOR](#), and [com.mehdi.petreminder.gui.util.GuiConstants.GAP_LARGE](#).

7.41.3 Member Function Documentation

7.41.3.1 refresh()

void com.mehdi.petreminder.gui.panels.SettingsPanel.refresh ()

Paneli yeniler.

Definition at line 178 of file [SettingsPanel.java](#).

```
00178         {
00179         if (lblCurrent != null) {
00180             lblCurrent.setText(StorageConfig.getActiveBackend\(\).getDisplayName\(\));
00181         }
00182         if (cmbStorage != null) {
00183             cmbStorage.setSelectedItem(StorageConfig.getActiveBackend\(\));
00184         }
00185     }
```

References [com.mehdi.petreminder.config.StorageConfig.getActiveBackend\(\)](#), and [com.mehdi.petreminder.config.StorageType.getDisplayName\(\)](#).

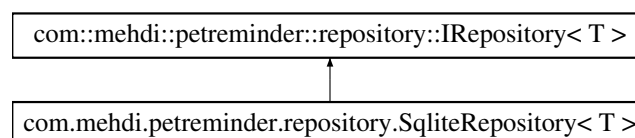
The documentation for this class was generated from the following file:

- petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/[SettingsPanel.java](#)

7.42 com.mehdi.petreminder.repository.SQLiteRepository< T > Class Template Reference

SQLite tabanlı abstract repository temel sınıfı.

Inheritance diagram for com.mehdi.petreminder.repository.SQLiteRepository< T >:



Public Member Functions

- [SQLiteRepository](#) ()
SQLite bağlantısını açar ve tabloyu oluşturur.
- [SQLiteRepository](#) (Connection [connection](#))
Test amaçlı yapıcı — dışarıdan bağlantı verilir.
- int [save](#) (T entity)
- Optional< T > [findById](#) (int id)
- List< T > [findAll](#) ()
- boolean [update](#) (T entity)
- boolean [delete](#) (int id)
- void [deleteAll](#) ()
- int [count](#) ()
- void [close](#) ()
- Connection [getConnection](#) ()
Bağlantıyı döndürür (test için).

Protected Member Functions

- abstract String [getTableName](#) ()
Tablo adını döndürür.
- abstract String [getCreateTableSql](#) ()
Tablo oluşturma SQL'ini döndürür.
- abstract String [getInsertSql](#) ()
INSERT SQL'ini döndürür (ID hariç).
- abstract String [getUpdateSql](#) ()
UPDATE SQL'ini döndürür.
- abstract void [setInsertParams](#) (PreparedStatement ps, T entity) throws SQLException
PreparedStatement'a INSERT parametrelerini set eder.
- abstract void [setUpdateParams](#) (PreparedStatement ps, T entity) throws SQLException
PreparedStatement'a UPDATE parametrelerini set eder.
- abstract T [mapRow](#) (ResultSet rs) throws SQLException
ResultSet'ten varlık oluşturur.
- void [createTableIfNotExists](#) ()
Tabloyu oluşturur (yoksa).
- void [setEntityId](#) (T entity, int id)
Reflection ile entity'ye id set eder.

Protected Attributes

- Connection [connection](#)
SQLite JDBC bağlantısı.

Static Protected Attributes

- static final Logger [logger](#)
Logger.

7.42.1 Detailed Description

SQLite tabanlı abstract repository temel sınıfı.

Alt sınıflar ([PetSqliteRepository](#) vb.) bu sınıfı extend eder ve `mapRow` / `getTableName` / `getInsertSql` / `getUpdateSql` metodlarını implemente eder. Template Method Pattern kullanılmıştır.

Parameters

< T >	Varlık türü
-------	-------------

Author

Muhammed Mehdi Karagülle
Ibrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 27 of file [SqliteRepository.java](#).

7.42.2 Constructor & Destructor Documentation

7.42.2.1 SqliteRepository() [1/2]

```
com.mehdi.petreminder.repository.SqliteRepository< T >.SqliteRepository ()
```

SQLite bağlantısını açar ve tabloyu oluşturur.

Exceptions

RepositoryException	Bağlantı başarısız olursa
-------------------------------------	---------------------------

Definition at line 40 of file [SqliteRepository.java](#).

```
00040         {
00041             try {
00042                 Class.forName("org.sqlite.JDBC");
00043                 String url = "jdbc:sqlite:" + StorageConfig.getSqliteFilePath();
00044                 connection = DriverManager.getConnection(url);
00045                 connection.setAutoCommit(true);
00046                 createTableIfNotExists();
00047                 logger.info("SQLite bağlantısı açıldı: {}", StorageConfig.getSqliteFilePath());
00048             } catch (Exception e) {
00049                 throw new RepositoryException("SQLite bağlantı hatası", e);
00050             }
00051         }
```

References [connection](#), [createTableIfNotExists\(\)](#), [com.mehdi.petreminder.config.StorageConfig.getSqliteFilePath\(\)](#), and [logger](#).

7.42.2.2 SqliteRepository() [2/2]

```
com.mehdi.petreminder.repository.SqliteRepository< T >.SqliteRepository (
    Connection connection)
```

Test amaçlı yapıcı — dışarıdan bağlantı verilir.

Parameters

<i>connection</i>	Test bağlantısı (H2 veya SQLite in-memory)
-------------------	--

Definition at line 57 of file [SqliteRepository.java](#).

```
00057                                     {
00058         this.connection = connection;
00059         try {
00060             createTableIfNotExists();
00061         } catch (Exception e) {
00062             throw new RepositoryException("Tablo oluşturma hatası", e);
00063         }
00064     }
```

References [connection](#), and [createTableIfNotExists\(\)](#).

7.42.3 Member Function Documentation

7.42.3.1 close()

```
void com.mehdi.petreminder.repository.SqliteRepository< T >.close ()
```

Repository bağlantısını/kaynaklarını kapatır.

Implements [com.mehdi.petreminder.repository.IRepository< T >](#).

Definition at line 253 of file [SqliteRepository.java](#).

```
00253                                     {
00254         try {
00255             if (connection != null && !connection.isClosed()) {
00256                 connection.close();
00257                 logger.info("SQLite bağlantısı kapatıldı.");
00258             }
00259         } catch (SQLException e) {
00260             logger.warn("SQLite kapatma hatası: {}", e.getMessage());
00261         }
00262     }
```

References [connection](#), and [logger](#).

7.42.3.2 count()

```
int com.mehdi.petreminder.repository.SqliteRepository< T >.count ()
```

Varlık sayısını döndürür.

Returns

Toplam kayıt sayısı

Implements [com.mehdi.petreminder.repository.IRepository< T >](#).

Definition at line 238 of file [SqliteRepository.java](#).

```
00238                                     {
00239         String sql = "SELECT COUNT(*) FROM " + getTableName();
00240         try (Statement st = connection.createStatement()) {
00241             ResultSet rs = st.executeQuery(sql) {
00242                 if (rs.next()) return rs.getInt(1);
00243             } catch (SQLException e) {
00244                 throw new RepositoryException("SQLite count hatası", e);
00245             }
00246         } return 0;
00247     }
```

References [connection](#), and [getTableName\(\)](#).

7.42.3.3 createTableIfNotExists()

void [com.mehdi.petreminder.repository.SqliteRepository](#)< T >.createTableIfNotExists () [protected]

Tabloyu oluşturur (yoksa).

Definition at line 119 of file [SqliteRepository.java](#).

```

00119         {
00120             try (Statement st = connection.createStatement()) {
00121                 String sql = getCreateTableSql();
00122                 // H2 does not support SQLite's AUTOINCREMENT; use AUTO_INCREMENT instead
00123                 String dbProduct = connection.getMetaData().getDatabaseProductName();
00124                 if ("H2".equalsIgnoreCase(dbProduct)) {
00125                     sql = sql.replace("AUTOINCREMENT", "AUTO_INCREMENT");
00126                 }
00127                 st.execute(sql);
00128             } catch (SQLException e) {
00129                 throw new RepositoryException("Tablo oluşturma hatası", e);
00130             }
00131         }

```

References [connection](#), and [getCreateTableSql\(\)](#).

Referenced by [SqliteRepository\(\)](#), and [SqliteRepository\(\)](#).

7.42.3.4 delete()

boolean [com.mehdi.petreminder.repository.SqliteRepository](#)< T >.delete (int id)

ID'ye göre varlık siler.

Parameters

<i>id</i>	Silinecek varlığın ID'si
-----------	--------------------------

Returns

Silme başarılıysa true

Exceptions

RepositoryException	Silme başarısız olursa
-------------------------------------	------------------------

Implements [com.mehdi.petreminder.repository.IRepository](#)< T >.

Definition at line 211 of file [SqliteRepository.java](#).

```

00211         {
00212             String sql = "DELETE FROM " + getTableName() + " WHERE id = ?";
00213             try (PreparedStatement ps = connection.prepareStatement(sql)) {
00214                 ps.setInt(1, id);
00215                 int rows = ps.executeUpdate();
00216                 return rows > 0;
00217             } catch (SQLException e) {
00218                 throw new RepositoryException("SQLite delete hatası", e);
00219             }
00220         }

```

References [connection](#), and [getTableName\(\)](#).

7.42.3.5 deleteAll()

```
void com.mehdi.petreminder.repository.SQLiteRepository< T >.deleteAll ()
```

Tüm varlıkları siler.

Exceptions

<i>RepositoryException</i>	Silme başarısız olursa
--	------------------------

Implements [com.mehdi.petreminder.repository.IRepository< T >](#).

Definition at line 226 of file [SQLiteRepository.java](#).

```
00226      {
00227          try (Statement st = connection.createStatement()) {
00228              st.execute("DELETE FROM " + getTableName());
00229          } catch (SQLException e) {
00230              throw new RepositoryException("SQLite deleteAll hatası", e);
00231          }
00232      }
```

References [connection](#), and [getTableName\(\)](#).

7.42.3.6 findAll()

```
List< T > com.mehdi.petreminder.repository.SQLiteRepository< T >.findAll ()
```

Tüm varlıkları listeler.

Returns

Varlık listesi (boş olabilir)

Exceptions

<i>RepositoryException</i>	Okuma başarısız olursa
--	------------------------

Implements [com.mehdi.petreminder.repository.IRepository< T >](#).

Definition at line 178 of file [SQLiteRepository.java](#).

```
00178      {
00179          String sql = "SELECT * FROM " + getTableName();
00180          List<T> list = new ArrayList<>();
00181          try (Statement st = connection.createStatement();
00182              ResultSet rs = st.executeQuery(sql)) {
00183              while (rs.next()) {
00184                  list.add(mapRow(rs));
00185              }
00186          } catch (SQLException e) {
00187              throw new RepositoryException("SQLite findAll hatası", e);
00188          }
00189          return list;
00190      }
```

References [connection](#), [getTableName\(\)](#), and [mapRow\(\)](#).

7.42.3.7 findById()

Optional< T > [com.mehdi.petreminder.repository.SqliteRepository](#)< T >.findById (
 int id)

ID'ye göre varlık getirir.

Parameters

<i>id</i>	Varlık ID'si
-----------	--------------

Returns

Varlık (bulunamazsa empty Optional)

Exceptions

RepositoryException	Okuma başarısız olursa
-------------------------------------	------------------------

Implements [com.mehdi.petreminder.repository.IRepository](#)< T >.

Definition at line 160 of file [SqliteRepository.java](#).

```

00160                                     {
00161         String sql = "SELECT * FROM " + getTableName() + " WHERE id = ?";
00162         try (PreparedStatement ps = connection.prepareStatement(sql)) {
00163             ps.setInt(1, id);
00164             ResultSet rs = ps.executeQuery();
00165             if (rs.next()) {
00166                 return Optional.of(mapRow(rs));
00167             }
00168         } catch (SQLException e) {
00169             throw new RepositoryException("SQLite findById hatası", e);
00170         }
00171         return Optional.empty();
00172     }

```

References [connection](#), [getTableName\(\)](#), and [mapRow\(\)](#).

7.42.3.8 getConnection()

Connection [com.mehdi.petreminder.repository.SqliteRepository](#)< T >.getConnection ()

Bağlantıyı döndürür (test için).

Returns

JDBC Connection

Definition at line 268 of file [SqliteRepository.java](#).

```

00268                                     {
00269         return connection;
00270     }

```

References [connection](#).

7.42.3.9 getCreateTableSql()

```
abstract String com.mehdi.petreminder.repository.SqliteRepository< T >.getCreateTableSql ()  
[abstract], [protected]
```

Tablo oluşturma SQL'ini döndürür.

Returns

CREATE TABLE IF NOT EXISTS SQL

Referenced by [createTableIfNotExists\(\)](#).

7.42.3.10 getInsertSql()

```
abstract String com.mehdi.petreminder.repository.SqliteRepository< T >.getInsertSql () [abstract],  
[protected]
```

INSERT SQL'ini döndürür (ID hariç).

Returns

INSERT INTO sql

Referenced by [save\(\)](#).

7.42.3.11 getTableName()

```
abstract String com.mehdi.petreminder.repository.SqliteRepository< T >.getTableName () [abstract],  
[protected]
```

Tablo adını döndürür.

Returns

SQL tablo adı

Referenced by [count\(\)](#), [delete\(\)](#), [deleteAll\(\)](#), [findAll\(\)](#), and [findById\(\)](#).

7.42.3.12 getUpdateSql()

```
abstract String com.mehdi.petreminder.repository.SqliteRepository< T >.getUpdateSql () [abstract],  
[protected]
```

UPDATE SQL'ini döndürür.

Returns

UPDATE sql

Referenced by [update\(\)](#).

7.42.3.13 mapRow()

```
abstract T com.mehdi.petreminder.repository.SqliteRepository< T >.mapRow (
    ResultSet rs) throws SQLException [abstract], [protected]
```

ResultSet'ten varlık oluşturur.

Parameters

<i>rs</i>	ResultSet
-----------	-----------

Returns

Varlık nesnesi

Exceptions

<i>SQLException</i>	SQL hatası
---------------------	------------

Referenced by [findAll\(\)](#), and [findById\(\)](#).

7.42.3.14 save()

```
int com.mehdi.petreminder.repository.SqliteRepository< T >.save (
    T entity)
```

Yeni bir varlık kaydeder.

Parameters

<i>entity</i>	Kaydedilecek varlık
---------------	---------------------

Returns

Oluşturulan varlığın ID'si

Exceptions

<i>RepositoryException</i>	Kayıt başarısız olursa
----------------------------	------------------------

Implements [com.mehdi.petreminder.repository.IRepository< T >](#).

Definition at line 137 of file [SqliteRepository.java](#).

```
00137         {
00138             if (entity == null) throw new RepositoryException("Entity null olamaz");
00139             try (PreparedStatement ps = connection.prepareStatement(
00140                 getInsertSql(), Statement.RETURN_GENERATED_KEYS)) {
00141                 setInsertParams(ps, entity);
00142                 ps.executeUpdate();
00143                 ResultSet keys = ps.getGeneratedKeys();
```

```

00144         if (keys.next()) {
00145             int id = keys.getInt(1);
00146             setEntityId(entity, id);
00147             logger.debug("SQLite kayıt: id={}", id);
00148             return id;
00149         }
00150         throw new RepositoryException("ID üretilemedi");
00151     } catch (SQLException e) {
00152         throw new RepositoryException("SQLite save hatası", e);
00153     }
00154 }

```

References [connection](#), [getInsertSql\(\)](#), [logger](#), [setEntityId\(\)](#), and [setInsertParams\(\)](#).

7.42.3.15 setEntityId()

```

void com.mehdi.petreminder.repository.SQLiteRepository< T >.setEntityId (
    T entity,
    int id) [protected]

```

Reflection ile entity'ye id set eder.

Parameters

<i>entity</i>	Varlık
<i>id</i>	ID

Definition at line 277 of file [SQLiteRepository.java](#).

```

00277         {
00278         try {
00279             var method = entity.getClass().getMethod("setId", int.class);
00280             method.invoke(entity, id);
00281         } catch (Exception e) {
00282             logger.warn("setId çağrılmadı: {}", e.getMessage());
00283         }
00284     }

```

References [logger](#).

Referenced by [save\(\)](#).

7.42.3.16 setInsertParams()

```

abstract void com.mehdi.petreminder.repository.SQLiteRepository< T >.setInsertParams (
    PreparedStatement ps,
    T entity) throws SQLException [abstract], [protected]

```

PreparedStatement'a INSERT parametrelerini set eder.

Parameters

<i>ps</i>	PreparedStatement
<i>entity</i>	Varlık

Exceptions

<i>SQLException</i>	SQL hatası
---------------------	------------

Referenced by [save\(\)](#).

7.42.3.17 setUpdateParams()

```
abstract void com.mehdi.petreminder.repository.SqliteRepository< T >.setUpdateParams (
    PreparedStatement ps,
    T entity) throws SQLException [abstract], [protected]
```

PreparedStatement'a UPDATE parametrelerini set eder.

Parameters

<i>ps</i>	PreparedStatement
<i>entity</i>	Varlık

Exceptions

<i>SQLException</i>	SQL hatası
---------------------	------------

Referenced by [update\(\)](#).

7.42.3.18 update()

```
boolean com.mehdi.petreminder.repository.SqliteRepository< T >.update (
    T entity)
```

Mevcut varlığı günceller.

Parameters

<i>entity</i>	Güncellenecek varlık
---------------	----------------------

Returns

Güncelleme başarılıysa true

Exceptions

<i>RepositoryException</i>	Güncelleme başarısız olursa
----------------------------	-----------------------------

Implements [com.mehdi.petreminder.repository.IRepository< T >](#).

Definition at line 196 of file [SqliteRepository.java](#).

```

00196         {
00197             if (entity == null) return false;
00198             try (PreparedStatement ps = connection.prepareStatement(getUpdateSql())) {
00199                 setUpdateParams(ps, entity);
00200                 int rows = ps.executeUpdate();
00201                 return rows > 0;
00202             } catch (SQLException e) {
00203                 throw new RepositoryException("SQLite update hatası", e);
00204             }
00205         }

```

References [connection](#), [getUpdateSql\(\)](#), and [setUpdateParams\(\)](#).

7.42.4 Member Data Documentation

7.42.4.1 connection

Connection [com.mehdi.petreminder.repository.SqliteRepository](#)< T >.connection [protected]

SQLite JDBC bağlantısı.

Definition at line 34 of file [SqliteRepository.java](#).

Referenced by [close\(\)](#), [count\(\)](#), [createTableIfNotExists\(\)](#), [delete\(\)](#), [deleteAll\(\)](#), [findAll\(\)](#), [findById\(\)](#), [getConnection\(\)](#), [save\(\)](#), [SqliteRepository\(\)](#), [SqliteRepository\(\)](#), and [update\(\)](#).

7.42.4.2 logger

final Logger [com.mehdi.petreminder.repository.SqliteRepository](#)< T >.logger [static], [protected]

Initial value:

```

=
    (Logger) LoggerFactory.getLogger(SqliteRepository.class)

```

Logger.

Definition at line 30 of file [SqliteRepository.java](#).

Referenced by [close\(\)](#), [save\(\)](#), [setEntityId\(\)](#), and [SqliteRepository\(\)](#).

The documentation for this class was generated from the following file:

- petreminder-app/src/main/java/com/mehdi/petreminder/repository/[SqliteRepository.java](#)

7.43 com.mehdi.petreminder.config.StorageConfig Class Reference

Aktif storage backend'i yönetir.

Static Public Member Functions

- static [StorageType](#) [getActiveBackend](#) ()
Aktif backend'i döndürür.
- static void [setActiveBackend](#) ([StorageType](#) type)
Aktif backend'i değiştirir ve kaydeder.
- static String [getMysqlUrl](#) ()
MySQL URL'ini döndürür.
- static void [setMysqlUrl](#) (String url)
MySQL URL'ini set eder.
- static String [getMysqlUsername](#) ()
MySQL kullanıcı adını döndürür.
- static String [getMysqlPassword](#) ()
MySQL şifresini döndürür.
- static String [getSqliteFilePath](#) ()
SQLite dosya yolunu döndürür.
- static String [getBinaryDirectory](#) ()
Binary dizin yolunu döndürür.
- static void [loadFromFile](#) ()
Ayarları properties dosyasından yükler.
- static void [saveToFile](#) ()
Ayarları properties dosyasına kaydeder.
- static void [reset](#) ()
Config'i sıfırlar (test için).

7.43.1 Detailed Description

Aktif storage backend'i yönetir.

PDF zorunluluğu: Runtime storage switching — no restart required. config/storage.properties dosyasına kaydeder/yükler.

Author

Muhammed Mehdi Karagülle
Ibrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 32 of file [StorageConfig.java](#).

7.43.2 Member Function Documentation

7.43.2.1 getActiveBackend()

`StorageType com.mehdi.petreminder.config.StorageConfig.getActiveBackend () [static]`

Aktif backend'i döndürür.

Returns

`StorageType`

Definition at line 71 of file [StorageConfig.java](#).

```
00071                                     {
00072     return activeBackend;
00073 }
```

Referenced by [com.mehdi.petreminder.repository.RepositoryFactory.createMedicalRecordRepository\(\)](#), [com.mehdi.petreminder.repository.RepositoryFactory.createReminderRepository\(\)](#), [com.mehdi.petreminder.repository.RepositoryFactory.getActiveType\(\)](#), [com.mehdi.petreminder.gui.panels.SettingsPanel.refresh\(\)](#), [com.mehdi.petreminder.ConsoleApp.showMainMenu\(\)](#), [com.mehdi.petreminder.ConsoleApp.showSettingsMenu\(\)](#), and [com.mehdi.petreminder.petreminderApp.startConsole\(\)](#).

7.43.2.2 getBinaryDirectory()

`String com.mehdi.petreminder.config.StorageConfig.getBinaryDirectory () [static]`

Binary dizin yolunu döndürür.

Returns

Dizin yolu

Definition at line 121 of file [StorageConfig.java](#).

```
00121 { return binaryDirectory; }
```

7.43.2.3 getMysqlPassword()

`String com.mehdi.petreminder.config.StorageConfig.getMysqlPassword () [static]`

MySQL şifresini döndürür.

Returns

password

Definition at line 109 of file [StorageConfig.java](#).

```
00109 { return mysqlPassword; }
```

Referenced by [com.mehdi.petreminder.repository.MySqlRepository< T >.MySqlRepository\(\)](#).

7.43.2.4 getMysqlUrl()

```
String com.mehdi.petreminder.config.StorageConfig.getMysqlUrl () [static]
```

MySQL URL'ini döndürür.

Returns

JDBC URL

Definition at line 91 of file [StorageConfig.java](#).

```
00091 { return mysqlUrl; }
```

Referenced by [com.mehdi.petreminder.repository.MySqlRepository< T >.MySqlRepository\(\)](#).

7.43.2.5 getMysqlUsername()

```
String com.mehdi.petreminder.config.StorageConfig.getMysqlUsername () [static]
```

MySQL kullanıcı adını döndürür.

Returns

username

Definition at line 103 of file [StorageConfig.java](#).

```
00103 { return mysqlUsername; }
```

Referenced by [com.mehdi.petreminder.repository.MySqlRepository< T >.MySqlRepository\(\)](#).

7.43.2.6 getSqliteFilePath()

```
String com.mehdi.petreminder.config.StorageConfig.getSqliteFilePath () [static]
```

SQLite dosya yolunu döndürür.

Returns

Dosya yolu

Definition at line 115 of file [StorageConfig.java](#).

```
00115 { return sqliteFilePath; }
```

Referenced by [com.mehdi.petreminder.repository.SqliteRepository< T >.SqliteRepository\(\)](#).

7.43.2.7 loadFromFile()

```
void com.mehdi.petreminder.config.StorageConfig.loadFromFile () [static]
```

Ayarları properties dosyasından yükler.

Definition at line 126 of file [StorageConfig.java](#).

```
00126         {
00127             File file = new File(CONFIG_FILE);
00128             if (!file.exists()) {
00129                 logger.debug("Config dosyası bulunamadı, varsayılanlar kullanılıyor.");
00130                 return;
00131             }
00132             Properties props = new Properties();
00133             try (FileInputStream fis = new FileInputStream(file)) {
00134                 props.load(fis);
00135                 String backend = props.getProperty("storage.backend", "BINARY");
00136                 try {
00137                     activeBackend = StorageType.valueOf(backend.toUpperCase());
00138                 } catch (IllegalArgumentException e) {
00139                     activeBackend = StorageType.BINARY;
00140                 }
00141                 mysqlUrl = props.getProperty("mysql.url", mysqlUrl);
00142                 mysqlUsername = props.getProperty("mysql.username", mysqlUsername);
00143                 mysqlPassword = props.getProperty("mysql.password", mysqlPassword);
00144                 sqliteFilePath = props.getProperty("sqlite.path", sqliteFilePath);
00145                 binaryDirectory = props.getProperty("binary.dir", binaryDirectory);
00146                 logger.info("Config yüklendi: backend={}, activeBackend");
00147             } catch (IOException e) {
00148                 logger.warn("Config yüklenemedi: {}", e.getMessage());
00149             }
00150         }
```

References [com.mehdi.petreminder.config.StorageType.BINARY](#).

7.43.2.8 reset()

```
void com.mehdi.petreminder.config.StorageConfig.reset () [static]
```

Config'i sıfırlar (test için).

Definition at line 176 of file [StorageConfig.java](#).

```
00176         {
00177             activeBackend = StorageType.BINARY;
00178             mysqlUrl =
00179                 "jdbc:mysql://localhost:3306/petreminder_db?useSSL=false&serverTimezone=UTC";
00179             mysqlUsername = "petreminder_user";
00180             mysqlPassword = "petreminder_pass";
00181             sqliteFilePath = "data/petreminder.db";
00182             binaryDirectory = "data/binary";
00183         }
```

References [com.mehdi.petreminder.config.StorageType.BINARY](#).

7.43.2.9 saveToFile()

```
void com.mehdi.petreminder.config.StorageConfig.saveToFile () [static]
```

Ayarları properties dosyasına kaydeder.

Definition at line 155 of file [StorageConfig.java](#).

```
00155         {
00156             File dir = new File("config");
00157             if (!dir.exists()) dir.mkdirs();
00158             Properties props = new Properties();
00159             props.setProperty("storage.backend", activeBackend.name());
00160             props.setProperty("mysql.url", mysqlUrl);
```

```

00161         props.setProperty("mysql.username", mysqlUsername);
00162         props.setProperty("mysql.password", mysqlPassword);
00163         props.setProperty("sqlite.path", sqliteFilePath);
00164         props.setProperty("binary.dir", binaryDirectory);
00165         try (FileOutputStream fos = new FileOutputStream(CONFIG_FILE)) {
00166             props.store(fos, "PetReminder Storage Configuration");
00167             logger.debug("Config kaydedildi.");
00168         } catch (IOException e) {
00169             logger.warn("Config kaydedilemedi: {}", e.getMessage());
00170         }
00171     }

```

Referenced by [setActiveBackend\(\)](#).

7.43.2.10 setActiveBackend()

```

void com.mehdi.petreminder.config.StorageConfig.setActiveBackend (
    StorageType type) [static]

```

Aktif backend'i değiştirir ve kaydeder.

Parameters

<i>type</i>	Yeni StorageType
-------------	----------------------------------

Definition at line 79 of file [StorageConfig.java](#).

```

00079                                     {
00080         if (type != null) {
00081             activeBackend = type;
00082             saveToFile();
00083             logger.info("Storage backend değiştirildi: {}", type);
00084         }
00085     }

```

References [saveToFile\(\)](#).

Referenced by [com.mehdi.petreminder.ConsoleApp.handleStorageSwitch\(\)](#), [com.mehdi.petreminder.petreminderApp.main\(\)](#), and [com.mehdi.petreminder.gui.MainFrame.onStorageChanged\(\)](#).

7.43.2.11 setMysqlUrl()

```

void com.mehdi.petreminder.config.StorageConfig.setMysqlUrl (
    String url) [static]

```

MySQL URL'ini set eder.

Parameters

<i>url</i>	JDBC URL
------------	----------

Definition at line 97 of file [StorageConfig.java](#).

```

00097 { mysqlUrl = url; }

```

The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/config/StorageConfig.java](#)

7.44 com.mehdi.petreminder.config.StorageType Enum Reference

Public Member Functions

- [StorageType](#) (String displayName)
Yapıcı.
- String [getDisplayName](#) ()
Gösterim adını döndürür.
- String [toString](#) ()
toString override.

Public Attributes

- [BINARY](#) =("Binary Dosya")
Binary dosya (.bin).
- [SQLITE](#) =("SQLite")
SQLite veritabanı
- [MYSQL](#) =("MySQL (Docker)")
MySQL veritabanı (Docker).

7.44.1 Detailed Description

Enum documentation.

Definition at line 18 of file [StorageType.java](#).

7.44.2 Constructor & Destructor Documentation

7.44.2.1 StorageType()

```
com.mehdi.petreminder.config.StorageType.StorageType (  
    String displayName)
```

Yapıcı.

Parameters

<i>displayName</i>	Gösterim adı
--------------------	--------------

Definition at line 36 of file [StorageType.java](#).

```
00036         {  
00037     this.displayName = displayName;  
00038 }
```


7.44.3 Member Function Documentation

7.44.3.1 getDisplayName()

```
String com.mehdi.petreminder.config.StorageType.getDisplayName ()
```

Gösterim adını döndürür.

Returns

displayName

Definition at line 44 of file [StorageType.java](#).

```
00044 {
00045     return displayName;
00046 }
```

Referenced by [com.mehdi.petreminder.gui.MainFrame.onStorageChanged\(\)](#), [com.mehdi.petreminder.gui.panels.SettingsPanel.refresh\(\)](#), [com.mehdi.petreminder.ConsoleApp.showMainMenu\(\)](#), [com.mehdi.petreminder.ConsoleApp.showSettingsMenu\(\)](#), and [com.mehdi.petreminder.petreminderApp.startConsole\(\)](#).

7.44.3.2 toString()

```
String com.mehdi.petreminder.config.StorageType.toString ()
```

toString override.

Returns

displayName

Definition at line 53 of file [StorageType.java](#).

```
00053 {
00054     return displayName;
00055 }
```

7.44.4 Member Data Documentation

7.44.4.1 BINARY

```
com.mehdi.petreminder.config.StorageType.BINARY = ("Binary Dosya")
```

Binary dosya (.bin).

Definition at line 21 of file [StorageType.java](#).

Referenced by [com.mehdi.petreminder.ConsoleApp.handleStorageSwitch\(\)](#), [com.mehdi.petreminder.config.StorageConfig.loadFromFile\(\)](#), [com.mehdi.petreminder.petreminderApp.parseStorageArg\(\)](#), and [com.mehdi.petreminder.config.StorageConfig.reset\(\)](#).

7.44.4.2 MYSQL

```
com.mehdi.petreminder.config.StorageType.MYSQL = ("MySQL (Docker) ")
```

MySQL veritabanı (Docker).

Definition at line 27 of file [StorageType.java](#).

Referenced by [com.mehdi.petreminder.ConsoleApp.handleStorageSwitch\(\)](#).

7.44.4.3 SQLITE

```
com.mehdi.petreminder.config.StorageType.SQLITE = ("SQLite")
```

SQLite veritabanı

Definition at line 24 of file [StorageType.java](#).

Referenced by [com.mehdi.petreminder.ConsoleApp.handleStorageSwitch\(\)](#).

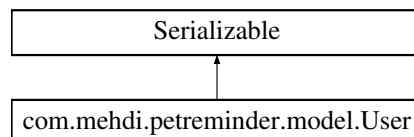
The documentation for this enum was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/config/StorageType.java](#)

7.45 com.mehdi.petreminder.model.User Class Reference

Uygulamayı kullanan evcil hayvan sahibi.

Inheritance diagram for com.mehdi.petreminder.model.User:



Public Member Functions

- [User](#) ()
Varsayılan yapıcı.
- [User](#) (int id, String username, String email, String passwordHash, String fullName)
Temel parametrelili yapıcı.
- int [getId](#) ()
ID getter.
- void [setId](#) (int id)
ID setter.
- String [getUsername](#) ()
Kullanıcı adı getter.
- void [setUsername](#) (String username)

- Kullanıcı adı setter.*
- String [getEmail](#) ()
- E-posta getter.*
- void [setEmail](#) (String email)
- E-posta setter.*
- String [getPasswordHash](#) ()
- Şifre hash getter.*
- void [setPasswordHash](#) (String passwordHash)
- Şifre hash setter.*
- String [getFullName](#) ()
- Tam ad getter.*
- void [setFullName](#) (String fullName)
- Tam ad setter.*
- LocalDateTime [getCreatedAt](#) ()
- Kayıt tarihi getter.*
- void [setCreatedAt](#) (LocalDateTime createdAt)
- Kayıt tarihi setter.*
- LocalDateTime [getLastLogin](#) ()
- Son giriş getter.*
- void [setLastLogin](#) (LocalDateTime lastLogin)
- Son giriş setter.*
- boolean [isActive](#) ()
- Aktif mi getter.*
- void [setActive](#) (boolean active)
- Aktif setter.*
- String [toString](#) ()
- String temsili — şifre hash'i GİZLENİR (KVKK).*
- boolean [equals](#) (Object obj)
- Eşitlik kontrolü — ID bazlı.*
- int [hashCode](#) ()
- Hash kodu.*

7.45.1 Detailed Description

Uygulamayı kullanan evcil hayvan sahibi.

OOP Encapsulation: tüm alanlar private. KVKK: şifre hash'lenir, kişisel veriler korunur. Serializable: Binary↔ Repository için.

Author

Muhammed Mehdi Karagülle, İbrahim Demirci, Zümre Uykun

Version

1.0

Definition at line 20 of file [User.java](#).

7.45.2 Constructor & Destructor Documentation

7.45.2.1 User() [1/2]

```
com.mehdi.petreminder.model.User.User ()
```

Varsayılan yapıcı.

Definition at line 50 of file [User.java](#).

```
00050 {}
```

Referenced by [equals\(\)](#).

7.45.2.2 User() [2/2]

```
com.mehdi.petreminder.model.User.User (
    int id,
    String username,
    String email,
    String passwordHash,
    String fullName)
```

Temel parametrelili yapıcı.

Parameters

<i>id</i>	Kullanıcı ID
<i>username</i>	Kullanıcı adı
<i>email</i>	E-posta
<i>passwordHash</i>	Şifre hash'i
<i>fullName</i>	Tam ad

Definition at line 60 of file [User.java](#).

```
00061                                     {
00062         this.id = id;
00063         this.username = username;
00064         this.email = email;
00065         this.passwordHash = passwordHash;
00066         this.fullName = fullName;
00067         this.createdAt = LocalDateTime.now();
00068         this.active = true;
00069     }
```

7.45.3 Member Function Documentation

7.45.3.1 equals()

```
boolean com.mehdi.petreminder.model.User.equals (
    Object obj)
```

Eşitlik kontrolü — ID bazlı.

Parameters

<i>obj</i>	Karşılaştırılan nesne
------------	-----------------------

Returns

ID'ler eşitse true

Definition at line 135 of file [User.java](#).

```
00135                                     {
00136         if (this == obj) return true;
00137         if (obj == null || !(obj instanceof User)) return false;
00138         User other = (User) obj;
00139         return this.id == other.id;
00140     }
```

References [User\(\)](#).

7.45.3.2 getCreatedAt()

LocalDateTime com.mehdi.petreminder.model.User.getCreatedAt ()

Kayıt tarihi getter.

Returns

Kayıt tarihi

Definition at line 102 of file [User.java](#).

```
00102 { return createdAt; }
```

7.45.3.3 getEmail()

String com.mehdi.petreminder.model.User.getEmail ()

E-posta getter.

Returns

E-posta

Definition at line 84 of file [User.java](#).

```
00084 { return email; }
```

7.45.3.4 getFullName()

String com.mehdi.petreminder.model.User.getFullName ()

Tam ad getter.

Returns

Tam ad

Definition at line 96 of file [User.java](#).

```
00096 { return fullName; }
```

7.45.3.5 getId()

```
int com.mehdi.petreminder.model.User.getId ()
```

ID getter.

Returns

Kullanıcı ID

Definition at line 72 of file [User.java](#).

```
00072 { return id; }
```

7.45.3.6 getLastLogin()

```
LocalDateTime com.mehdi.petreminder.model.User.getLastLogin ()
```

Son giriş getter.

Returns

Son giriş

Definition at line 108 of file [User.java](#).

```
00108 { return lastLogin; }
```

7.45.3.7 getPasswordHash()

```
String com.mehdi.petreminder.model.User.getPasswordHash ()
```

Şifre hash getter.

Returns

Şifre hash'i

Definition at line 90 of file [User.java](#).

```
00090 { return passwordHash; }
```

7.45.3.8 getUsername()

```
String com.mehdi.petreminder.model.User.getUsername ()
```

Kullanıcı adı getter.

Returns

Kullanıcı adı

Definition at line 78 of file [User.java](#).

```
00078 { return username; }
```

7.45.3.9 hashCode()

```
int com.mehdi.petreminder.model.User.hashCode ()
```

Hash kodu.

Returns

ID'ye göre hash

Definition at line 147 of file [User.java](#).

```
00147         {  
00148             return Integer.hashCode(id);  
00149     }
```

7.45.3.10 isActive()

```
boolean com.mehdi.petreminder.model.User.isActive ()
```

Aktif mi getter.

Returns

Aktif mi

Definition at line 114 of file [User.java](#).

```
00114 { return active; }
```

7.45.3.11 setActive()

```
void com.mehdi.petreminder.model.User.setActive (  
        boolean active)
```

Aktif setter.

Parameters

<i>active</i>	Aktif mi
---------------	----------

Definition at line 117 of file [User.java](#).

```
00117 { this.active = active; }
```

Referenced by [com.mehdi.petreminder.repository.UserMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository.UserSqliteRepository.mapRow\(\)](#).

7.45.3.12 setCreatedAt()

```
void com.mehdi.petreminder.model.User.setCreatedAt (
    LocalDateTime createdAt)
```

Kayıt tarihi setter.

Parameters

<i>createdAt</i>	Kayıt tarihi
------------------	--------------

Definition at line 105 of file [User.java](#).

```
00105 { this.createdAt = createdAt; }
```

Referenced by [com.mehdi.petreminder.repository.UserMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository.UserSqliteRepository.mapRow\(\)](#).

7.45.3.13 setEmail()

```
void com.mehdi.petreminder.model.User.setEmail (
    String email)
```

E-posta setter.

Parameters

<i>email</i>	E-posta
--------------	---------

Definition at line 87 of file [User.java](#).

```
00087 { this.email = email; }
```

Referenced by [com.mehdi.petreminder.repository.UserMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository.UserSqliteRepository.mapRow\(\)](#).

7.45.3.14 setFullName()

```
void com.mehdi.petreminder.model.User.setFullName (
    String fullName)
```

Tam ad setter.

Parameters

<i>fullName</i>	Tam ad
-----------------	--------

Definition at line 99 of file [User.java](#).

```
00099 { this.fullName = fullName; }
```

Referenced by [com.mehdi.petreminder.repository.UserMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository.UserSqliteRepository.mapRow\(\)](#).

7.45.3.15 setId()

```
void com.mehdi.petreminder.model.User.setId (
    int id)
```

ID setter.

Parameters

<i>id</i>	Kullanıcı ID
-----------	--------------

Definition at line 75 of file [User.java](#).

```
00075 { this.id = id; }
```

Referenced by [com.mehdi.petreminder.repository.UserMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository.UserSQLiteRepository.mapRow\(\)](#).

7.45.3.16 setLastLogin()

```
void com.mehdi.petreminder.model.User.setLastLogin (
    LocalDateTime lastLogin)
```

Son giriş setter.

Parameters

<i>lastLogin</i>	Son giriş
------------------	-----------

Definition at line 111 of file [User.java](#).

```
00111 { this.lastLogin = lastLogin; }
```

7.45.3.17 setPasswordHash()

```
void com.mehdi.petreminder.model.User.setPasswordHash (
    String passwordHash)
```

Şifre hash setter.

Parameters

<i>passwordHash</i>	Şifre hash'i
---------------------	--------------

Definition at line 93 of file [User.java](#).

```
00093 { this.passwordHash = passwordHash; }
```

Referenced by [com.mehdi.petreminder.repository.UserMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository.UserSQLiteRepository.mapRow\(\)](#).

7.45.3.18 setUsername()

```
void com.mehdi.petreminder.model.User.setUsername (
    String username)
```

Kullanıcı adı setter.

Parameters

<i>username</i>	Kullanıcı adı
-----------------	---------------

Definition at line 81 of file [User.java](#).

```
00081 { this.username = username; }
```

Referenced by [com.mehdi.petreminder.repository.UserMySqlRepository.mapRow\(\)](#), and [com.mehdi.petreminder.repository.UserSqliteRepository.mapRow\(\)](#).

7.45.3.19 toString()

```
String com.mehdi.petreminder.model.User.toString ()
```

String temsili — şifre hash'i GİZLENİR (KVKK).

Returns

Kullanıcı bilgisi, şifre hariç

Definition at line 124 of file [User.java](#).

```
00124 {
00125     return String.format("User{id=%d, username='%s', email='%s', active=%b}",
00126         id, username, email, active);
00127 }
```

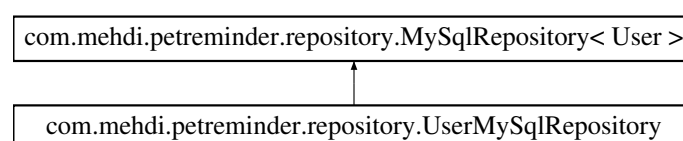
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/model/User.java](#)

7.46 com.mehdi.petreminder.repository.UserMySqlRepository Class Reference

MySQL üzerinde [User](#) CRUD işlemleri.

Inheritance diagram for [com.mehdi.petreminder.repository.UserMySqlRepository](#):



Public Member Functions

- [UserMySqlRepository](#) ()
Varsayılan yapıcı.
- [UserMySqlRepository](#) (Connection [connection](#))
Test yapıcısı.

Public Member Functions inherited from
[com.mehdi.petreminder.repository.MySqlRepository](#) < [User](#) >

- [MySqlRepository](#) ()
MySQL bağlantısını açar.
- int [save](#) (T entity)
- Optional< T > [findById](#) (int id)
- List< T > [findAll](#) ()
- boolean [update](#) (T entity)
- boolean [delete](#) (int id)
- void [deleteAll](#) ()
- int [count](#) ()
- void [close](#) ()
- Connection [getConnection](#) ()
Bağlantıyı döndürür.

Protected Member Functions

- String [getTableName](#) ()
- String [getCreateTableSql](#) ()
- String [getInsertSql](#) ()
- String [getUpdateSql](#) ()
- void [setInsertParams](#) (PreparedStatement ps, [User](#) u) throws SQLException
- void [setUpdateParams](#) (PreparedStatement ps, [User](#) u) throws SQLException
- [User](#) [mapRow](#) (ResultSet rs) throws SQLException

Protected Member Functions inherited from
[com.mehdi.petreminder.repository.MySqlRepository](#) < [User](#) >

- abstract String [getTableName](#) ()
Tablo adı.
- abstract String [getCreateTableSql](#) ()
Tablo oluşturma SQL.
- abstract String [getInsertSql](#) ()
INSERT SQL.
- abstract String [getUpdateSql](#) ()
UPDATE SQL.
- abstract void [setInsertParams](#) (PreparedStatement ps, T entity) throws SQLException
INSERT parametrelerini set eder.
- abstract void [setUpdateParams](#) (PreparedStatement ps, T entity) throws SQLException
UPDATE parametrelerini set eder.
- abstract T [mapRow](#) (ResultSet rs) throws SQLException
ResultSet'ten varlık oluşturur.
- void [createTableIfNotExists](#) ()
Tabloyu oluşturur (yoksa).
- void [setEntityId](#) (T entity, int id)
Reflection ile entity'ye id set eder.

Additional Inherited Members

Protected Attributes inherited from

[com.mehdi.petreminder.repository.MySqlRepository](#) < [User](#) >

- Connection [connection](#)
MySQL JDBC bağlantısı.

Static Protected Attributes inherited from

[com.mehdi.petreminder.repository.MySqlRepository](#) < [User](#) >

- static final Logger [logger](#)
Logger.

7.46.1 Detailed Description

MySQL üzerinde [User](#) CRUD işlemleri.

Author

Muhammed Mehdi Karagülle
İbrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 20 of file [UserMySqlRepository.java](#).

7.46.2 Constructor & Destructor Documentation

7.46.2.1 [UserMySqlRepository\(\)](#) [1/2]

```
com.mehdi.petreminder.repository.UserMySqlRepository.UserMySqlRepository ()
```

Varsayılan yapıcı.

Definition at line 23 of file [UserMySqlRepository.java](#).

```
00023 { super(); }
```

7.46.2.2 UserMySqlRepository() [2/2]

```
com.mehdi.petreminder.repository.UserMySqlRepository.UserMySqlRepository (
    Connection connection)
```

Test yapıcısı.

Parameters

<i>connection</i>	Test bağlantısı
-------------------	-----------------

Definition at line 29 of file [UserMySqlRepository.java](#).

```
00029 { super(connection); }
```

References [com.mehdi.petreminder.repository.MySqlRepository< User >.connection](#).

7.46.3 Member Function Documentation

7.46.3.1 getCreateTableSql()

```
String com.mehdi.petreminder.repository.UserMySqlRepository.getCreateTableSql () [protected]
```

Member documentation.

Definition at line 43 of file [UserMySqlRepository.java](#).

```
00043 {
00044     return "CREATE TABLE IF NOT EXISTS users ("
00045         + "id INT AUTO_INCREMENT PRIMARY KEY,"
00046         + "username VARCHAR(50) NOT NULL UNIQUE,"
00047         + "email VARCHAR(100) NOT NULL UNIQUE,"
00048         + "password_hash VARCHAR(255) NOT NULL,"
00049         + "full_name VARCHAR(100),"
00050         + "created_at DATETIME,"
00051         + "last_login DATETIME,"
00052         + "active BOOLEAN DEFAULT TRUE"
00053         + ")";
00054 }
```

7.46.3.2 getInsertSql()

```
String com.mehdi.petreminder.repository.UserMySqlRepository.getInsertSql () [protected]
```

Member documentation.

Definition at line 61 of file [UserMySqlRepository.java](#).

```
00061 {
00062     return "INSERT INTO users (username,email,password_hash,full_name,created_at,active)"
00063         + " VALUES (?, ?, ?, ?, ?, ?)";
00064 }
```

7.46.3.3 getTableName()

```
String com.mehdi.petreminder.repository.UserMySqlRepository.getTableName () [protected]
```

Member documentation.

Definition at line 36 of file [UserMySqlRepository.java](#).

```
00036 { return "users"; }
```

7.46.3.4 getUpdateSql()

String com.mehdi.petreminder.repository.UserMySqlRepository.getUpdateSql () [protected]

Member documentation.

Definition at line 71 of file [UserMySqlRepository.java](#).

```
00071         {
00072             return "UPDATE users SET username=?,email=?,password_hash=?,full_name=?, "
00073                 + "created_at=?,active=? WHERE id=?";
00074         }
```

7.46.3.5 mapRow()

User com.mehdi.petreminder.repository.UserMySqlRepository.mapRow (
 ResultSet rs) throws SQLException [protected]

Member documentation.

Definition at line 106 of file [UserMySqlRepository.java](#).

```
00106         {
00107             User u = new User();
00108             u.setId(rs.getInt("id"));
00109             u.setUsername(rs.getString("username"));
00110             u.setEmail(rs.getString("email"));
00111             u.setPasswordHash(rs.getString("password_hash"));
00112             u.setFullName(rs.getString("full_name"));
00113             Timestamp ts = rs.getTimestamp("created_at");
00114             if (ts != null) u.setCreatedAt(ts.toLocalDateTime());
00115             u.setActive(rs.getBoolean("active"));
00116             return u;
00117         }
```

References [com.mehdi.petreminder.model.User.setActive\(\)](#), [com.mehdi.petreminder.model.User.setCreatedAt\(\)](#), [com.mehdi.petreminder.model.User.setEmail\(\)](#), [com.mehdi.petreminder.model.User.setFullName\(\)](#), [com.mehdi.petreminder.model.User.setPasswordHash\(\)](#), and [com.mehdi.petreminder.model.User.setUsername\(\)](#).

7.46.3.6 setInsertParams()

void com.mehdi.petreminder.repository.UserMySqlRepository.setInsertParams (
 PreparedStatement ps,
 User u) throws SQLException [protected]

Member documentation.

Definition at line 81 of file [UserMySqlRepository.java](#).

```
00081         {
00082             ps.setString(1, u.getUsername());
00083             ps.setString(2, u.getEmail());
00084             ps.setString(3, u.getPasswordHash());
00085             ps.setString(4, u.getFullName());
00086             ps.setObject(5, u.getCreatedAt() != null
00087                 ? Timestamp.valueOf(u.getCreatedAt()) : null);
00088             ps.setBoolean(6, u.isActive());
00089         }
```

Referenced by [setUpdateParams\(\)](#).

7.46.3.7 setUpdateParams()

```
void com.mehdi.petreminder.repository.UserMySqlRepository.setUpdateParams (
    PreparedStatement ps,
    User u) throws SQLException [protected]
```

Member documentation.

Definition at line 96 of file [UserMySqlRepository.java](#).

```
00096                                     {
00097         setInsertParams(ps, u);
00098         ps.setInt(7, u.getId());
00099     }
```

References [setInsertParams\(\)](#).

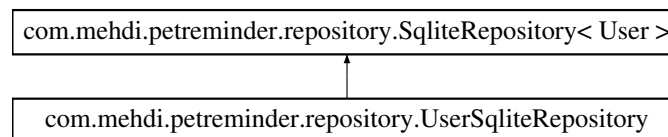
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/repository/UserMySqlRepository.java](#)

7.47 com.mehdi.petreminder.repository.UserSqliteRepository Class Reference

SQLite üzerinde [User](#) CRUD işlemleri.

Inheritance diagram for com.mehdi.petreminder.repository.UserSqliteRepository:



Public Member Functions

- [UserSqliteRepository](#) ()
Varsayılan yapıcı.
- [UserSqliteRepository](#) (Connection [connection](#))
Test yapıcısı.
- [User findByUsername](#) (String username)
Username ile kullanıcı arar.

Public Member Functions inherited from [com.mehdi.petreminder.repository.SqliteRepository< User >](#)

- [SqliteRepository](#) ()
SQLite bağlantısını açar ve tabloyu oluşturur.
- int [save](#) (T entity)
- Optional< T > [findByld](#) (int id)
- List< T > [findAll](#) ()
- boolean [update](#) (T entity)
- boolean [delete](#) (int id)
- void [deleteAll](#) ()
- int [count](#) ()
- void [close](#) ()
- Connection [getConnection](#) ()
Bağlantıyı döndürür (test için).

Protected Member Functions

- String [getTableName](#) ()
- String [getCreateTableSql](#) ()
- String [getInsertSql](#) ()
- String [getUpdateSql](#) ()
- void [setInsertParams](#) (PreparedStatement ps, [User](#) u) throws SQLException
- void [setUpdateParams](#) (PreparedStatement ps, [User](#) u) throws SQLException
- [User](#) [mapRow](#) (ResultSet rs) throws SQLException

Protected Member Functions inherited from [com.mehdi.petreminder.repository.SqliteRepository< User >](#)

- abstract String [getTableName](#) ()
Tablo adını döndürür.
- abstract String [getCreateTableSql](#) ()
Tablo oluşturma SQL'ini döndürür.
- abstract String [getInsertSql](#) ()
INSERT SQL'ini döndürür (ID hariç).
- abstract String [getUpdateSql](#) ()
UPDATE SQL'ini döndürür.
- abstract void [setInsertParams](#) (PreparedStatement ps, T entity) throws SQLException
PreparedStatement'a INSERT parametrelerini set eder.
- abstract void [setUpdateParams](#) (PreparedStatement ps, T entity) throws SQLException
PreparedStatement'a UPDATE parametrelerini set eder.
- abstract T [mapRow](#) (ResultSet rs) throws SQLException
ResultSet'ten varlık oluşturur.
- void [createTableIfNotExists](#) ()
Tabloyu oluşturur (yoksa).
- void [setEntityId](#) (T entity, int id)
Reflection ile entity'ye id set eder.

Additional Inherited Members

Protected Attributes inherited from [com.mehdi.petreminder.repository.SqliteRepository< User >](#)

- Connection [connection](#)
SQLite JDBC bağlantısı.

Static Protected Attributes inherited from [com.mehdi.petreminder.repository.SqliteRepository< User >](#)

- static final Logger [logger](#)
Logger.

7.47.1 Detailed Description

SQLite üzerinde [User](#) CRUD işlemleri.

Author

Muhammed Mehdi Karagülle
Ibrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 20 of file [UserSqliteRepository.java](#).

7.47.2 Constructor & Destructor Documentation

7.47.2.1 UserSqliteRepository() [1/2]

```
com.mehdi.petreminder.repository.UserSqliteRepository.UserSqliteRepository ()
```

Varsayılan yapıcı.

Definition at line 23 of file [UserSqliteRepository.java](#).

```
00023 { super(); }
```

7.47.2.2 UserSqliteRepository() [2/2]

```
com.mehdi.petreminder.repository.UserSqliteRepository.UserSqliteRepository (  
    Connection connection)
```

Test yapıcısı.

Parameters

<i>connection</i>	Test bağlantısı
-------------------	-----------------

Definition at line 29 of file [UserSqliteRepository.java](#).

```
00029 { super(connection); }
```

References [com.mehdi.petreminder.repository.SqliteRepository< User >.connection](#).

7.47.3 Member Function Documentation

7.47.3.1 findByUsername()

`User` `com.mehdi.petreminder.repository.UserSqliteRepository.findByUsername (String username)`

Username ile kullanıcı arar.

Parameters

<code>username</code>	Kullanıcı adı
-----------------------	---------------

Returns

`User` veya null

Definition at line 123 of file `UserSqliteRepository.java`.

```

00123     {
00124         try (PreparedStatement ps = connection.prepareStatement(
00125             "SELECT * FROM users WHERE username = ?")) {
00126             ps.setString(1, username);
00127             ResultSet rs = ps.executeQuery();
00128             if (rs.next()) return mapRow(rs);
00129         } catch (SQLException e) {
00130             throw new RepositoryException("findByUsername hatası", e);
00131         }
00132         return null;
00133     }

```

References `com.mehdi.petreminder.repository.SqliteRepository< User >.connection`, and `mapRow()`.

7.47.3.2 getCreateTableSql()

`String` `com.mehdi.petreminder.repository.UserSqliteRepository.getCreateTableSql ()` [protected]

Member documentation.

Definition at line 43 of file `UserSqliteRepository.java`.

```

00043     {
00044         return "CREATE TABLE IF NOT EXISTS users ("
00045             + "id INTEGER PRIMARY KEY AUTOINCREMENT,"
00046             + "username TEXT NOT NULL UNIQUE,"
00047             + "email TEXT NOT NULL UNIQUE,"
00048             + "password_hash TEXT NOT NULL,"
00049             + "full_name TEXT,"
00050             + "created_at TEXT,"
00051             + "last_login TEXT,"
00052             + "active INTEGER DEFAULT 1"
00053             + ")";
00054     }

```

7.47.3.3 getInsertSql()

`String` `com.mehdi.petreminder.repository.UserSqliteRepository.getInsertSql ()` [protected]

Member documentation.

Definition at line 61 of file `UserSqliteRepository.java`.

```

00061     {
00062         return "INSERT INTO users (username,email,password_hash,full_name,created_at,active)"
00063             + " VALUES (?, ?, ?, ?, ?, ?)";
00064     }

```

7.47.3.4 getTableName()

```
String com.mehdi.petreminder.repository.UserSqliteRepository.getTableName () [protected]
```

Member documentation.

Definition at line 36 of file [UserSqliteRepository.java](#).

```
00036 { return "users"; }
```

7.47.3.5 getUpdateSql()

```
String com.mehdi.petreminder.repository.UserSqliteRepository.getUpdateSql () [protected]
```

Member documentation.

Definition at line 71 of file [UserSqliteRepository.java](#).

```
00071 {
00072     return "UPDATE users SET username=?,email=?,password_hash=?,full_name=?,
00073         + "created_at=?,active=? WHERE id=?";
00074 }
```

7.47.3.6 mapRow()

```
User com.mehdi.petreminder.repository.UserSqliteRepository.mapRow (
    ResultSet rs) throws SQLException [protected]
```

Member documentation.

Definition at line 105 of file [UserSqliteRepository.java](#).

```
00105 {
00106     User u = new User();
00107     u.setId(rs.getInt("id"));
00108     u.setUsername(rs.getString("username"));
00109     u.setEmail(rs.getString("email"));
00110     u.setPasswordHash(rs.getString("password_hash"));
00111     u.setFullName(rs.getString("full_name"));
00112     String createdAt = rs.getString("created_at");
00113     if (createdAt != null) u.setCreatedAt(LocalDate.parse(createdAt));
00114     u.setActive(rs.getInt("active") == 1);
00115     return u;
00116 }
```

References [com.mehdi.petreminder.model.User.setActive\(\)](#), [com.mehdi.petreminder.model.User.setCreatedAt\(\)](#), [com.mehdi.petreminder.model.User.setEmail\(\)](#), [com.mehdi.petreminder.model.User.setFullName\(\)](#), [com.mehdi.petreminder.model.User.setPasswordHash\(\)](#), and [com.mehdi.petreminder.model.User.setUsername\(\)](#).

Referenced by [findByUsername\(\)](#).

7.47.3.7 setInsertParams()

```
void com.mehdi.petreminder.repository.UserSqliteRepository.setInsertParams (
    PreparedStatement ps,
    User u) throws SQLException [protected]
```

Member documentation.

Definition at line 81 of file [UserSqliteRepository.java](#).

```
00081 {
00082     ps.setString(1, u.getUsername());
00083     ps.setString(2, u.getEmail());
00084     ps.setString(3, u.getPasswordHash());
00085     ps.setString(4, u.getFullName());
00086     ps.setString(5, u.getCreatedAt() != null ? u.getCreatedAt().toString() : null);
00087     ps.setInt(6, u.isActive() ? 1 : 0);
00088 }
```

Referenced by [setUpdateParams\(\)](#).

7.47.3.8 setUpdateParams()

```
void com.mehdi.petreminder.repository.UserSqliteRepository.setUpdateParams (
    PreparedStatement ps,
    User u) throws SQLException [protected]
```

Member documentation.

Definition at line 95 of file [UserSqliteRepository.java](#).

```
00095                                     {
00096         setInsertParams(ps, u);
00097         ps.setInt(7, u.getId());
00098     }
```

References [setInsertParams\(\)](#).

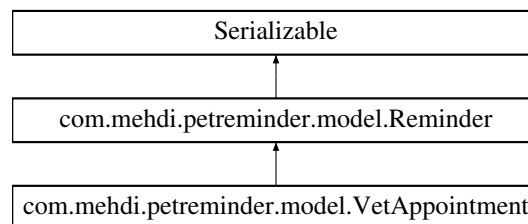
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/repository/UserSqliteRepository.java](#)

7.48 com.mehdi.petreminder.model.VetAppointment Class Reference

Veteriner randevusu — [Reminder](#)'dan türetilmiş somut sınıf.

Inheritance diagram for com.mehdi.petreminder.model.VetAppointment:



Public Member Functions

- [VetAppointment](#) ()
Varsayılan yapıcı.
- [VetAppointment](#) (int id, int petId, String petName, LocalDateTime scheduledTime, String veterinarianName, String clinicName, String reason)
Temel parametrelili yapıcı.
- [VetAppointment](#) (int id, int petId, String petName, LocalDateTime scheduledTime, String description, String priority, boolean recurring, String recurrencePattern, String veterinarianName, String clinicName, String reason, double estimatedCost, boolean confirmed, String clinicPhone)
Tam parametrelili yapıcı.
- String [getVeterinarianName](#) ()
Veteriner adı getter.
- void [setVeterinarianName](#) (String veterinarianName)
Veteriner adı setter.
- String [getClinicName](#) ()
Klinik adı getter.
- void [setClinicName](#) (String clinicName)

- Klinik adı setter.*
- String [getReason](#) ()
- Muayene sebebi getter.*
- void [setReason](#) (String reason)
- Muayene sebebi setter.*
- double [getEstimatedCost](#) ()
- Tahmini maliyet getter.*
- void [setEstimatedCost](#) (double estimatedCost)
- Tahmini maliyet setter.*
- boolean [isConfirmed](#) ()
- Onay durumu getter.*
- void [setConfirmed](#) (boolean confirmed)
- Onay durumu setter.*
- String [getClinicPhone](#) ()
- Klinik telefonu getter.*
- void [setClinicPhone](#) (String clinicPhone)
- Klinik telefonu setter.*
- String [getReminderType](#) ()
- Hatırlatıcı türü.*
- boolean [validate](#) ()
- Veteriner randevusunu doğrular.*
- String [getSummary](#) ()
- Özet bilgi.*
- String [toString](#) ()
- String temsili.*

Public Member Functions inherited from [com.mehdi.petreminder.model.Reminder](#)

- int [getId](#) ()
- ID getter.*
- void [setId](#) (int id)
- ID setter.*
- int [getPetId](#) ()
- Pet ID getter.*
- void [setPetId](#) (int petId)
- Pet ID setter.*
- String [getPetName](#) ()
- Pet adı getter.*
- void [setPetName](#) (String petName)
- Pet adı setter.*
- LocalDateTime [getScheduledTime](#) ()
- Planlanan zaman getter.*
- void [setScheduledTime](#) (LocalDateTime scheduledTime)
- Planlanan zaman setter.*
- boolean [isCompleted](#) ()
- Tamamlandı mı getter.*
- void [setCompleted](#) (boolean completed)
- Tamamlandı setter.*
- String [getDescription](#) ()
- Açıklama getter.*

- void [setDescription](#) (String description)
Açıklama setter.
- String [getPriority](#) ()
Öncelik getter.
- void [setPriority](#) (String priority)
Öncelik setter.
- boolean [isRecurring](#) ()
Tekrarlıyor mu getter.
- void [setRecurring](#) (boolean recurring)
Tekrarlama setter.
- String [getRecurrencePattern](#) ()
Tekrar deseni getter.
- void [setRecurrencePattern](#) (String recurrencePattern)
Tekrar deseni setter.
- String [getFormattedTime](#) ()
Planlanan zamanı biçimli string olarak döndürür.
- boolean [isOverdue](#) ()
Hatırlatıcının geçmiş mi olduğunu kontrol eder.
- boolean [equals](#) (Object obj)
Eşitlik kontrolü — ID bazlı.
- int [hashCode](#) ()
Hash kodu.

Additional Inherited Members

Protected Member Functions inherited from [com.mehdi.petreminder.model.Reminder](#)

- [Reminder](#) ()
Varsayılan yapıcı — Serialization için.
- [Reminder](#) (int id, int petId, String petName, LocalDateTime scheduledTime, String description, String priority, boolean recurring, String recurrencePattern)
Tam parametrelili yapıcı.

Static Protected Attributes inherited from [com.mehdi.petreminder.model.Reminder](#)

- static final DateTimeFormatter [FORMATTER](#)
Tarih saat formatı.

7.48.1 Detailed Description

Veteriner randevusu — [Reminder](#)'dan türetilmiş somut sınıf.

OOP Inheritance: [Reminder](#)'dan miras alır. OOP Polymorphism: [getReminderType\(\)](#), [validate\(\)](#), [getSummary\(\)](#) override.

Author

Muhammed Mehdi Karagülle, İbrahim Demirci, Zümre Uykun

Version

1.0

Definition at line 17 of file [VetAppointment.java](#).

7.48.2 Constructor & Destructor Documentation

7.48.2.1 VetAppointment() [1/3]

com.mehdi.petreminder.model.VetAppointment.VetAppointment ()

Varsayılan yapıcı.

Definition at line 41 of file [VetAppointment.java](#).

```
00041 { super(); }
```

7.48.2.2 VetAppointment() [2/3]

```
com.mehdi.petreminder.model.VetAppointment.VetAppointment (
    int id,
    int petId,
    String petName,
    LocalDateTime scheduledTime,
    String veterinarianName,
    String clinicName,
    String reason)
```

Temel parametrelili yapıcı.

Parameters

<i>id</i>	ID
<i>petId</i>	Pet ID
<i>petName</i>	Pet adı
<i>scheduledTime</i>	Randevu zamanı
<i>veterinarianName</i>	Veteriner adı
<i>clinicName</i>	Klinik adı
<i>reason</i>	Muayene sebebi

Definition at line 53 of file [VetAppointment.java](#).

```
00056 {
00057     super(id, petId, petName, scheduledTime,
00058         "Veteriner: " + reason, "YUKSEK", false, "");
00059     this.veterinarianName = veterinarianName;
00060     this.clinicName = clinicName;
00061     this.reason = reason;
00062     this.estimatedCost = 0.0;
00063     this.confirmed = false;
00064     this.clinicPhone = "";
00065 }
```

7.48.2.3 VetAppointment() [3/3]

```
com.mehdi.petreminder.model.VetAppointment.VetAppointment (
    int id,
    int petId,
    String petName,
    LocalDateTime scheduledTime,
    String description,
    String priority,
    boolean recurring,
    String recurrencePattern,
    String veterinarianName,
    String clinicName,
    String reason,
    double estimatedCost,
    boolean confirmed,
    String clinicPhone)
```

Tam parametrelili yapıcı.

Parameters

<i>id</i>	ID
<i>petId</i>	Pet ID
<i>petName</i>	Pet adı
<i>scheduledTime</i>	Zaman
<i>description</i>	Açıklama
<i>priority</i>	Öncelik
<i>recurring</i>	Tekrarlıyor mu
<i>recurrencePattern</i>	Tekrar deseni
<i>veterinarianName</i>	Veteriner adı
<i>clinicName</i>	Klinik adı
<i>reason</i>	Muayene sebebi
<i>estimatedCost</i>	Tahmini maliyet
<i>confirmed</i>	Onaylandı mı
<i>clinicPhone</i>	Klinik telefonu

Definition at line 84 of file [VetAppointment.java](#).

```
00090     {
00091         super(id, petId, petName, scheduledTime, description,
00092             priority, recurring, recurrencePattern);
00093         this.veterinarianName = veterinarianName;
00094         this.clinicName = clinicName;
00095         this.reason = reason;
00096         this.estimatedCost = estimatedCost;
00097         this.confirmed = confirmed;
00098         this.clinicPhone = clinicPhone;
00099     }
```

7.48.3 Member Function Documentation

7.48.3.1 getClinicName()

```
String com.mehdi.petreminder.model.VetAppointment.getClinicName ()
```


Klinik adı getter.

Returns

Klinik adı

Definition at line 110 of file [VetAppointment.java](#).

```
00110 { return clinicName; }
```

Referenced by [com.mehdi.petreminder.gui.panels.VetPanel.refresh\(\)](#).

7.48.3.2 getClinicPhone()

```
String com.mehdi.petreminder.model.VetAppointment.getClinicPhone ()
```

Klinik telefonu getter.

Returns

Telefon

Definition at line 134 of file [VetAppointment.java](#).

```
00134 { return clinicPhone; }
```

7.48.3.3 getEstimatedCost()

```
double com.mehdi.petreminder.model.VetAppointment.getEstimatedCost ()
```

Tahmini maliyet getter.

Returns

Tahmini maliyet (TL)

Definition at line 122 of file [VetAppointment.java](#).

```
00122 { return estimatedCost; }
```

7.48.3.4 getReason()

```
String com.mehdi.petreminder.model.VetAppointment.getReason ()
```

Muayene sebebi getter.

Returns

Muayene sebebi

Definition at line 116 of file [VetAppointment.java](#).

```
00116 { return reason; }
```

Referenced by [com.mehdi.petreminder.gui.panels.VetPanel.refresh\(\)](#).

7.48.3.5 getReminderType()

```
String com.mehdi.petreminder.model.VetAppointment.getReminderType ()
```

Hatırlatıcı türü.

Returns

"Veteriner"

Reimplemented from [com.mehdi.petreminder.model.Reminder](#).

Definition at line 141 of file [VetAppointment.java](#).

```
00141 { return "Veteriner"; }
```

7.48.3.6 getSummary()

```
String com.mehdi.petreminder.model.VetAppointment.getSummary ()
```

Özet bilgi.

Returns

"Veteriner: Dr.Ali - Rize Vet Kliniği (Rutin Muayene)"

Reimplemented from [com.mehdi.petreminder.model.Reminder](#).

Definition at line 159 of file [VetAppointment.java](#).

```
00159         {  
00160             return String.format("Veteriner: %s - %s (%s)%s",  
00161                 veterinarianName, clinicName, reason,  
00162                 confirmed ? " [Onaylandi]" : " [Bekliyor]");  
00163         }
```

7.48.3.7 getVeterinarianName()

```
String com.mehdi.petreminder.model.VetAppointment.getVeterinarianName ()
```

Veteriner adı getter.

Returns

Veteriner adı

Definition at line 102 of file [VetAppointment.java](#).

```
00102 { return veterinarianName; }
```

Referenced by [com.mehdi.petreminder.gui.panels.VetPanel.refresh\(\)](#).

7.48.3.8 isConfirmed()

```
boolean com.mehdi.petreminder.model.VetAppointment.isConfirmed ()
```

Onay durumu getter.

Returns

Onaylandı mı

Definition at line 128 of file [VetAppointment.java](#).

```
00128 { return confirmed; }
```

Referenced by [com.mehdi.petreminder.gui.panels.VetPanel.refresh\(\)](#).

7.48.3.9 setClinicName()

```
void com.mehdi.petreminder.model.VetAppointment.setClinicName (  
    String clinicName)
```

Klinik adı setter.

Parameters

<i>clinicName</i>	Klinik adı
-------------------	------------

Definition at line 113 of file [VetAppointment.java](#).

```
00113 { this.clinicName = clinicName; }
```

7.48.3.10 setClinicPhone()

```
void com.mehdi.petreminder.model.VetAppointment.setClinicPhone (  
    String clinicPhone)
```

Klinik telefonu setter.

Parameters

<i>clinicPhone</i>	Telefon
--------------------	---------

Definition at line 137 of file [VetAppointment.java](#).

```
00137 { this.clinicPhone = clinicPhone; }
```

7.48.3.11 setConfirmed()

```
void com.mehdi.petreminder.model.VetAppointment.setConfirmed (
    boolean confirmed)
```

Onay durumu setter.

Parameters

<i>confirmed</i>	Onaylandı mı
------------------	--------------

Definition at line 131 of file [VetAppointment.java](#).

```
00131 { this.confirmed = confirmed; }
```

7.48.3.12 setEstimatedCost()

```
void com.mehdi.petreminder.model.VetAppointment.setEstimatedCost (
    double estimatedCost)
```

Tahmini maliyet setter.

Parameters

<i>estimatedCost</i>	Tahmini maliyet
----------------------	-----------------

Definition at line 125 of file [VetAppointment.java](#).

```
00125 { this.estimatedCost = estimatedCost; }
```

7.48.3.13 setReason()

```
void com.mehdi.petreminder.model.VetAppointment.setReason (
    String reason)
```

Muayene sebebi setter.

Parameters

<i>reason</i>	Muayene sebebi
---------------	----------------

Definition at line 119 of file [VetAppointment.java](#).

```
00119 { this.reason = reason; }
```

7.48.3.14 setVeterinarianName()

```
void com.mehdi.petreminder.model.VetAppointment.setVeterinarianName (  
    String veterinarianName)
```

Veteriner adı setter.

Parameters

<i>veterinarianName</i>	Veteriner adı
-------------------------	---------------

Definition at line 105 of file [VetAppointment.java](#).

```
00105                                     {  
00106         this.veterinarianName = veterinarianName;  
00107     }
```

7.48.3.15 toString()

```
String com.mehdi.petreminder.model.VetAppointment.toString ()
```

String temsili.

Returns

[VetAppointment](#) bilgisi

Reimplemented from [com.mehdi.petreminder.model.Reminder](#).

Definition at line 167 of file [VetAppointment.java](#).

```
00167                                     {  
00168         return String.format(  
00169             "VetAppointment{id=%d, pet='%s', vet='%s', clinic='%s', reason='%s'}",  
00170             getId(), getPetName(), veterinarianName, clinicName, reason);  
00171     }
```

References [com.mehdi.petreminder.model.Reminder.getId\(\)](#), and [com.mehdi.petreminder.model.Reminder.getPetName\(\)](#).

7.48.3.16 validate()

```
boolean com.mehdi.petreminder.model.VetAppointment.validate ()
```

Veteriner randevusunu doğrular.

Returns

Veteriner ve klinik bilgileri dolu ise true

Reimplemented from [com.mehdi.petreminder.model.Reminder](#).

Definition at line 148 of file [VetAppointment.java](#).

```
00148                                     {  
00149         return veterinarianName != null && !veterinarianName.trim().isEmpty()  
00150             && clinicName != null && !clinicName.trim().isEmpty()  
00151             && reason != null && !reason.trim().isEmpty();  
00152     }
```

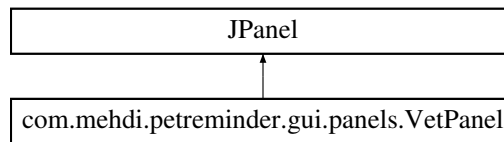
The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/model/VetAppointment.java](#)

7.49 com.mehdi.petreminder.gui.panels.VetPanel Class Reference

Veteriner randevuları CRUD.

Inheritance diagram for com.mehdi.petreminder.gui.panels.VetPanel:



Public Member Functions

- [VetPanel](#) ([MainFrame](#) mainFrame)
[VetPanel](#) yapıcısı.
- void [refresh](#) ()
Tabloyu yeniler.

7.49.1 Detailed Description

Veteriner randevuları CRUD.

Author

Muhammed Mehdi Karagülle
İbrahim Demirci
Zumre Uykun

Version

1.0

Definition at line 29 of file [VetPanel.java](#).

7.49.2 Constructor & Destructor Documentation

7.49.2.1 VetPanel()

```
com.mehdi.petreminder.gui.panels.VetPanel.VetPanel (  
    MainFrame mainFrame)
```

[VetPanel](#) yapıcısı.

Parameters

<i>mainFrame</i>	Ana pencere
------------------	-------------

Definition at line 50 of file [VetPanel.java](#).

```

00050         {
00051             this.mainFrame      = mainFrame;
00052             this.reminderService = new ReminderService();
00053             setBackground(GuiConstants.BG_COLOR);
00054             setLayout(new BorderLayout(GuiConstants.GAP, GuiConstants.GAP));
00055             setBorder(new EmptyBorder(GuiConstants.GAP_LARGE, GuiConstants.GAP_LARGE,
00056                                     GuiConstants.GAP_LARGE, GuiConstants.GAP_LARGE));
00057             buildUI();
00058         }

```

References [com.mehdi.petreminder.gui.util.GuiConstants.BG_COLOR](#), [com.mehdi.petreminder.gui.util.GuiConstants.GAP](#), and [com.mehdi.petreminder.gui.util.GuiConstants.GAP_LARGE](#).

7.49.3 Member Function Documentation

7.49.3.1 refresh()

```
void com.mehdi.petreminder.gui.panels.VetPanel.refresh ()
```

Tabloyu yeniler.

Definition at line 171 of file [VetPanel.java](#).

```

00171         {
00172             tableModel.setRowCount(0);
00173             try {
00174                 List<Reminder> all = reminderService.getAllReminders();
00175                 List<VetAppointment> vets = all.stream()
00176                     .filter(r -> r instanceof VetAppointment)
00177                     .map(r -> (VetAppointment) r)
00178                     .collect(Collectors.toList());
00179                 for (VetAppointment va : vets) {
00180                     tableModel.addRow(new Object[]{
00181                         va.getId(), va.getPetName(), va.getVeterinarianName(),
00182                         va.getClinicName(), va.getReason(), va.getFormattedTime(),
00183                         va.isConfirmed() ? "" : ""
00184                     });
00185                 }
00186             } catch (Exception e) { /* sessiz */ }
00187         }

```

References [com.mehdi.petreminder.model.VetAppointment.getClinicName\(\)](#), [com.mehdi.petreminder.model.Reminder.getFormattedTime\(\)](#), [com.mehdi.petreminder.model.Reminder.getId\(\)](#), [com.mehdi.petreminder.model.Reminder.getPetName\(\)](#), [com.mehdi.petreminder.model.VetAppointment.getReason\(\)](#), [com.mehdi.petreminder.model.VetAppointment.getVeterinarianName\(\)](#), and [com.mehdi.petreminder.model.VetAppointment.isConfirmed\(\)](#).

The documentation for this class was generated from the following file:

- [petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/VetPanel.java](#)

Chapter 8

File Documentation

8.1 petreminder-app/src/main/java/com/mehdi/petreminder/config/↵ StorageConfig.java File Reference

Storage backend konfigürasyonu — runtime switch destekler.

```
import org.slf4j.LoggerFactory;
import ch.qos.logback.classic.Logger;
import java.io.*;
import java.util.Properties;
```

Classes

- class [com.mehdi.petreminder.config.StorageConfig](#)
Aktif storage backend'i yönetir.

Packages

- package [com.mehdi.petreminder.config](#)

8.1.1 Detailed Description

Storage backend konfigürasyonu — runtime switch destekler.

Member documentation. Member documentation.

Definition in file [StorageConfig.java](#).

8.2 StorageConfig.java

[Go to the documentation of this file.](#)

```

00001
00014 package com.mehdi.petreminder.config;
00015
00016 import org.slf4j.LoggerFactory;
00017 import ch.qos.logback.classic.Logger;
00018
00019 import java.io.*;
00020 import java.util.Properties;
00021
00032 public class StorageConfig {
00033
00035     private static final Logger logger =
00036         (Logger) LoggerFactory.getLogger(StorageConfig.class);
00037
00039     private static final String CONFIG_FILE = "config/storage.properties";
00040
00042     private static StorageType activeBackend = StorageType.BINARY;
00043
00045     private static String mysqlUrl =
00046         "jdbc:mysql://localhost:3306/petreminder_db?useSSL=false&serverTimezone=UTC";
00047
00049     private static String mysqlUsername = "petreminder_user";
00050
00052     private static String mysqlPassword = "petreminder_pass";
00053
00055     private static String sqliteFilePath = "data/petreminder.db";
00056
00058     private static String binaryDirectory = "data/binary";
00059
00060     static {
00061         loadFromFile();
00062     }
00063
00065     private StorageConfig() { }
00066
00071     public static StorageType getActiveBackend() {
00072         return activeBackend;
00073     }
00074
00079     public static void setActiveBackend(StorageType type) {
00080         if (type != null) {
00081             activeBackend = type;
00082             saveToFile();
00083             logger.info("Storage backend değiştirildi: {}", type);
00084         }
00085     }
00086
00091     public static String getMySQLUrl() { return mysqlUrl; }
00092
00097     public static void setMySQLUrl(String url) { mysqlUrl = url; }
00098
00103     public static String getMySQLUsername() { return mysqlUsername; }
00104
00109     public static String getMySQLPassword() { return mysqlPassword; }
00110
00115     public static String getSqliteFilePath() { return sqliteFilePath; }
00116
00121     public static String getBinaryDirectory() { return binaryDirectory; }
00122
00126     public static void loadFromFile() {
00127         File file = new File(CONFIG_FILE);
00128         if (!file.exists()) {
00129             logger.debug("Config dosyası bulunamadı, varsayılanlar kullanılıyor.");
00130             return;
00131         }
00132         Properties props = new Properties();
00133         try (FileInputStream fis = new FileInputStream(file)) {
00134             props.load(fis);
00135             String backend = props.getProperty("storage.backend", "BINARY");
00136             try {
00137                 activeBackend = StorageType.valueOf(backend.toUpperCase());
00138             } catch (IllegalArgumentException e) {
00139                 activeBackend = StorageType.BINARY;
00140             }
00141             mysqlUrl = props.getProperty("mysql.url", mysqlUrl);
00142             mysqlUsername = props.getProperty("mysql.username", mysqlUsername);
00143             mysqlPassword = props.getProperty("mysql.password", mysqlPassword);
00144             sqliteFilePath = props.getProperty("sqlite.path", sqliteFilePath);
00145             binaryDirectory = props.getProperty("binary.dir", binaryDirectory);
00146             logger.info("Config yüklendi: backend={}, activeBackend");
00147         } catch (IOException e) {
00148             logger.warn("Config yüklenemedi: {}", e.getMessage());

```

```

00149     }
00150 }
00151
00155 public static void saveToFile() {
00156     File dir = new File("config");
00157     if (!dir.exists()) dir.mkdirs();
00158     Properties props = new Properties();
00159     props.setProperty("storage.backend", activeBackend.name());
00160     props.setProperty("mysql.url", mysqlUrl);
00161     props.setProperty("mysql.username", mysqlUsername);
00162     props.setProperty("mysql.password", mysqlPassword);
00163     props.setProperty("sqlite.path", sqliteFilePath);
00164     props.setProperty("binary.dir", binaryDirectory);
00165     try (FileOutputStream fos = new FileOutputStream(CONFIG_FILE)) {
00166         props.store(fos, "PetReminder Storage Configuration");
00167         logger.debug("Config kaydedildi.");
00168     } catch (IOException e) {
00169         logger.warn("Config kaydedilemedi: {}", e.getMessage());
00170     }
00171 }
00172
00176 public static void reset() {
00177     activeBackend = StorageType.BINARY;
00178     mysqlUrl =
"jdbc:mysql://localhost:3306/petreminder_db?useSSL=false&serverTimezone=UTC";
00179     mysqlUsername = "petreminder_user";
00180     mysqlPassword = "petreminder_pass";
00181     sqliteFilePath = "data/petreminder.db";
00182     binaryDirectory = "data/binary";
00183 }
00184 }

```

8.3 petreminder-app/src/main/java/com/mehdi/petreminder/config/StorageType.java File Reference ↩

Storage backend türleri.

Classes

- enum [com.mehdi.petreminder.config.StorageType](#)

Packages

- package [com.mehdi.petreminder.config](#)

8.3.1 Detailed Description

Storage backend türleri.

Definition in file [StorageType.java](#).

8.4 StorageType.java

[Go to the documentation of this file.](#)

```

00001
00005 package com.mehdi.petreminder.config;
00006
00018 public enum StorageType {
00019
00021     BINARY("Binary Dosya"),
00022
00024     SQLITE("SQLite"),
00025
00027     MYSQL("MySQL (Docker)");
00028
00030     private final String displayName;
00031
00036     StorageType(String displayName) {
00037         this.displayName = displayName;
00038     }
00039
00044     public String getDisplayName() {
00045         return displayName;
00046     }
00047
00052     @Override
00053     public String toString() {
00054         return displayName;
00055     }
00056 }

```

8.5 petreminder-app/src/main/java/com/mehdi/petreminder/ConsoleApp.java File Reference ↩↪

Console application main loop.

```

import com.mehdi.petreminder.config.StorageConfig;
import com.mehdi.petreminder.config.StorageType;
import com.mehdi.petreminder.model.*;
import com.mehdi.petreminder.service.MedicalRecordService;
import com.mehdi.petreminder.service.PetService;
import com.mehdi.petreminder.service.ReminderService;
import com.mehdi.petreminder.service.ServiceException;
import org.jline.terminal.Terminal;
import org.jline.terminal.TerminalBuilder;
import org.slf4j.LoggerFactory;
import ch.qos.logback.classic.Logger;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;
import java.util.List;
import java.util.Scanner;

```

Classes

- class [com.mehdi.petreminder.ConsoleApp](#)
Console application main loop and menu manager.

8.5.1 Detailed Description

Console application main loop.

Definition in file [ConsoleApp.java](#).

8.6 ConsoleApp.java

[Go to the documentation of this file.](#)

```
00001
00005 package com.mehdi.petreminder;
00006
00007 import com.mehdi.petreminder.config.StorageConfig;
00008 import com.mehdi.petreminder.config.StorageType;
00009 import com.mehdi.petreminder.model.*;
00010 import com.mehdi.petreminder.service.MedicalRecordService;
00011 import com.mehdi.petreminder.service.PetService;
00012 import com.mehdi.petreminder.service.ReminderService;
00013 import com.mehdi.petreminder.service.ServiceException;
00014
00015 import org.jline.terminal.Terminal;
00016 import org.jline.terminal.TerminalBuilder;
00017
00018 import org.slf4j.LoggerFactory;
00019 import ch.qos.logback.classic.Logger;
00020
00021 import java.time.LocalDate;
00022 import java.time.LocalDateTime;
00023 import java.time.format.DateTimeFormatter;
00024 import java.time.format.DateTimeParseException;
00025 import java.util.List;
00026 import java.util.Scanner;
00027
00033 public class ConsoleApp {
00034
00036     private static final Logger logger = (Logger) LoggerFactory.getLogger(ConsoleApp.class);
00037
00039     private final Scanner scanner;
00040
00042     private boolean running;
00043
00045     private PetService petService;
00046
00048     private ReminderService reminderService;
00049
00051     private MedicalRecordService medicalRecordService;
00052
00054     private Terminal terminal;
00055
00059     public ConsoleApp() {
00060         this.scanner = new Scanner(System.in);
00061         this.running = false;
00062         initTerminal();
00063         initServices();
00064     }
00065
00069     public ConsoleApp(Scanner scanner) {
00070         this.scanner = scanner;
00071         this.running = false;
00072         initServices();
00073     }
00074
00075     private void initTerminal() {
00076         if (System.console() != null) {
00077             try {
00078                 this.terminal = TerminalBuilder.builder().system(true).build();
00079             } catch (Exception e) {
00080                 this.terminal = null;
00081             }
00082         }
00083     }
00084
00088     private void initServices() {
00089         try {
00090             this.petService = new PetService();
00091             this.reminderService = new ReminderService();
00092             this.medicalRecordService = new MedicalRecordService();
00093         } catch (Exception e) {
```

```

00094         logger.error("Failed to initialize services Component: {}", e.getMessage());
00095     }
00096 }
00097
00101 public void start() {
00102     this.running = true;
00103     logger.info("ConsoleApp started.");
00104     showMainMenu();
00105 }
00106
00110 public void showMainMenu() {
00111     while (running) {
00112         String choice = selectMenuOption(
00113             "\n=====
PET CARE REMINDER SYSTEM - MAIN MENU
\n=====
",
00114             new String[]{"Pets", "Reminders", "Vet Appointments", "Medical Records", "Settings",
(Storage: " + StorageConfig.getActiveBackend().getDisplayName() + ") ", "Exit"},
00115             new String[]{"1", "2", "3", "4", "5", "0"}
00116         );
00117         handleMainMenuChoice(choice);
00118     }
00119 }
00120
00124 public void printMainMenu() {
00125     System.out.println("\n=====
");
00126     System.out.println("    PET CARE REMINDER SYSTEM - MAIN MENU    ");
00127     System.out.println("=====
");
00128     System.out.println("    Pets");
00129     System.out.println("    Reminders");
00130     System.out.println("    Vet Appointments");
00131     System.out.println("    Medical Records");
00132     System.out.println("    Settings");
00133     System.out.println("    Exit");
00134     System.out.println("=====
");
00135     System.out.print("Your choice: ");
00136 }
00137
00141 public void handleMainMenuChoice(String choice) {
00142     if (choice == null) return;
00143     switch (choice.trim()) {
00144         case "1": showPetsMenu(); break;
00145         case "2": showRemindersMenu(); break;
00146         case "3": showVetMenu(); break;
00147         case "4": showMedicalMenu(); break;
00148         case "5": showSettingsMenu(); initServices(); break; // re-init on storage change
00149         case "0": exitApp(); break;
00150         default:
00151             System.out.println("Invalid choice. Please try again.");
00152     }
00153 }
00154
00158 private String selectMenuOption(String title, String[] options, String[] returnValues) {
00159     if (terminal == null) {
00160         System.out.println(title);
00161         for (int i = 0; i < options.length; i++) {
00162             System.out.println("    [" + returnValues[i] + "] " + options[i]);
00163         }
00164         System.out.print("Your choice: ");
00165         return readInput();
00166     }
00167
00168     try {
00169         terminal.enterRawMode();
00170         int selectedIndex = 0;
00171         boolean firstDraw = true;
00172
00173         while (running) {
00174             if (!firstDraw) {
00175                 System.out.print("\033[" + (options.length + 2) + "A");
00176             }
00177             firstDraw = false;
00178
00179             System.out.println(title);
00180             for (int i = 0; i < options.length; i++) {
00181                 if (i == selectedIndex) {
00182                     System.out.println("\033[2K\r > [" + returnValues[i] + "] " + options[i]);
00183                 } else {
00184                     System.out.println("\033[2K\r    [" + returnValues[i] + "] " + options[i]);
00185                 }
00186             }
00187             System.out.print("\033[2K\r (Up/Down/Tab to navigate, Enter to select)");
00188             System.out.flush();
00189
00190             int c = terminal.reader().read();
00191             if (c == 9) { // Tab
00192                 selectedIndex = (selectedIndex + 1) % options.length;
00193             } else if (c == 13 || c == 10) { // Enter

```

```

00194         System.out.println();
00195         terminal.echo(true); // Restore echo before returning
00196         return returnValues[selectedIndex];
00197     } else if (c == 27) { // ESC sequence (Arrow keys)
00198         int next1 = terminal.reader().read(100);
00199         int next2 = terminal.reader().read(100);
00200         if (next1 == 91) { // '['
00201             if (next2 == 65) { // Up
00202                 selectedIndex = (selectedIndex - 1 + options.length) % options.length;
00203             } else if (next2 == 66) { // Down
00204                 selectedIndex = (selectedIndex + 1) % options.length;
00205             }
00206         }
00207     }
00208 }
00209 } catch (Exception e) {
00210     // fallback gracefully
00211 } finally {
00212     try { terminal.echo(true); } catch (Exception ignored) {}
00213 }
00214
00215 System.out.println(title);
00216 for (int i = 0; i < options.length; i++) {
00217     System.out.println(" [" + returnValues[i] + "] " + options[i]);
00218 }
00219 System.out.print("Your choice: ");
00220 return readInput();
00221 }
00222
00223 // =====
00224 // PETS MENU
00225 // =====
00229 public void showPetsMenu() {
00230     while (running) {
00231         String choice = selectMenuOption(
00232             "\n--- PETS ---",
00233             new String[]{"List All Pets", "Add New Pet", "Edit Pet", "Delete Pet", "Back to Main
Menu"},
00234             new String[]{"1", "2", "3", "4", "0"}
00235         );
00236
00237         switch (choice) {
00238             case "1": listAllPets(); break;
00239             case "2": addNewPet(); break;
00240             case "3": editPet(); break;
00241             case "4": deletePet(); break;
00242             case "0": return;
00243             default: System.out.println("Invalid choice.");
00244         }
00245     }
00246 }
00247
00251 private void listAllPets() {
00252     System.out.println("\n--- LIST OF PETS ---");
00253     try {
00254         List<Pet> pets = petService.getAllPets();
00255         if (pets.isEmpty()) {
00256             System.out.println("No pets found. Please add a pet first.");
00257             return;
00258         }
00259         for (Pet p : pets) {
00260             System.out.println(String.format("ID: %d | Name: %s | Species: %s | Age: %s",
00261                 p.getId(), p.getName(), p.getSpecies(), p.getAgeString()));
00262         }
00263     } catch (Exception e) {
00264         System.out.println("Error listing pets: " + e.getMessage());
00265     }
00266 }
00267
00271 private void addNewPet() {
00272     System.out.println("\n--- ADD NEW PET ---");
00273
00274     System.out.print("Species (1: Dog, 2: Cat, 3: Bird): ");
00275     String typeChoice = readInput();
00276
00277     System.out.print("Name: ");
00278     String name = readInput();
00279     if (name.isEmpty()) {
00280         System.out.println("Error: Name cannot be empty.");
00281         return;
00282     }
00283
00284     System.out.print("Birth Date (YYYY-MM-DD): ");
00285     LocalDate birthDate = readDateInput();
00286     if (birthDate == null) return;
00287
00288     System.out.print("Gender (Male/Female): ");

```

```

00289     String gender = readInput();
00290
00291     System.out.print("Weight (kg): ");
00292     double weight = readDoubleInput();
00293     if (weight <= 0) return;
00294
00295     System.out.print("Breed: ");
00296     String breed = readInput();
00297
00298     Pet newPet = null;
00299     try {
00300         if ("1".equals(typeChoice)) {
00301             Dog dog = new Dog(0, name, birthDate, 1);
00302             dog.setGender(gender);
00303             dog.setWeight(weight);
00304             dog.setBreed(breed);
00305             System.out.print("Is trained? (true/false): ");
00306             dog.setTrained(Boolean.parseBoolean(readInput()));
00307             newPet = dog;
00308         } else if ("2".equals(typeChoice)) {
00309             Cat cat = new Cat(0, name, birthDate, 1);
00310             cat.setGender(gender);
00311             cat.setWeight(weight);
00312             cat.setBreed(breed);
00313             System.out.print("Is indoor? (true/false): ");
00314             cat.setIndoor(Boolean.parseBoolean(readInput()));
00315             newPet = cat;
00316         } else if ("3".equals(typeChoice)) {
00317             Bird bird = new Bird(0, name, birthDate, 1);
00318             bird.setGender(gender);
00319             bird.setWeight(weight);
00320             bird.setBirdType(breed);
00321             System.out.print("Can talk? (true/false): ");
00322             bird.setCanTalk(Boolean.parseBoolean(readInput()));
00323             newPet = bird;
00324         } else {
00325             System.out.println("Invalid species choice.");
00326             return;
00327         }
00328
00329         petService.addPet(newPet);
00330         System.out.println("Pet '" + name + "' added successfully.");
00331     } catch (ServiceException e) {
00332         System.out.println("Failed to add pet: " + e.getMessage());
00333     }
00334 }
00335
00336 private void editPet() {
00337     listAllPets();
00338     System.out.print("\nEnter ID of pet to edit (or 0 to cancel): ");
00339     int id = readIntInput();
00340     if (id <= 0) return;
00341
00342     try {
00343         Pet pet = petService.getPetById(id);
00344         System.out.println("Editing Pet: " + pet.getName());
00345
00346         System.out.print("New Weight (" + pet.getWeight() + "): ");
00347         String weightStr = readInput();
00348         if (!weightStr.isEmpty()) {
00349             pet.setWeight(Double.parseDouble(weightStr));
00350         }
00351
00352         System.out.print("New Notes: ");
00353         String notes = readInput();
00354         if (!notes.isEmpty()) {
00355             pet.setNotes(notes);
00356         }
00357
00358         petService.updatePet(pet);
00359         System.out.println("Pet updated successfully.");
00360     } catch (ServiceException | NumberFormatException e) {
00361         System.out.println("Failed to update pet: " + e.getMessage());
00362     }
00363 }
00364
00365 private void deletePet() {
00366     listAllPets();
00367     System.out.print("\nEnter ID of pet to delete (or 0 to cancel): ");
00368     int id = readIntInput();
00369     if (id <= 0) return;
00370
00371     try {
00372         Pet p = petService.getPetById(id);
00373         System.out.print("Are you sure you want to delete '" + p.getName() + "'? (y/n): ");
00374         if ("y".equalsIgnoreCase(readInput())) {
00375             petService.deletePet(id);
00376         }
00377     }
00378 }

```



```

00382         System.out.println("Pet deleted successfully.");
00383     }
00384 } catch (ServiceException e) {
00385     System.out.println("Failed to delete pet: " + e.getMessage());
00386 }
00387 }
00388
00389 // =====
00390 // REMINDERS MENU
00391 // =====
00395 public void showRemindersMenu() {
00396     while (running) {
00397         String choice = selectMenuOption(
00398             "\n--- REMINDERS ---",
00399             new String[]{"List All Pending Reminders", "Add New Reminder", "Mark Reminder as
Completed", "Delete Reminder", "Back to Main Menu"},
00400             new String[]{"1", "2", "3", "4", "0"}
00401         );
00402
00403         switch (choice) {
00404             case "1": listPendingReminders(); break;
00405             case "2": addNewReminder(); break;
00406             case "3": markReminderCompleted(); break;
00407             case "4": deleteReminder(); break;
00408             case "0": return;
00409             default: System.out.println("Invalid choice.");
00410         }
00411     }
00412 }
00413
00417 private void listPendingReminders() {
00418     System.out.println("\n--- PENDING REMINDERS ---");
00419     try {
00420         List<Reminder> reminders = reminderService.getPendingReminders();
00421         if (reminders.isEmpty()) {
00422             System.out.println("No pending reminders found.");
00423             return;
00424         }
00425         for (Reminder r : reminders) {
00426             System.out.println(String.format("ID: %d | Pet ID: %d | Time: %s | Desc: %s",
00427                 r.getId(), r.getPetId(),
00428                 r.getScheduledTime().format(DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm")), r.getDescription()));
00429         } catch (Exception e) {
00430             System.out.println("Error listing reminders: " + e.getMessage());
00431         }
00432     }
00433
00437 private void addNewReminder() {
00438     System.out.println("\n--- ADD NEW REMINDER ---");
00439     System.out.print("Enter Pet ID for this reminder: ");
00440     int petId = readIntInput();
00441     if (petId <= 0) return;
00442
00443     // Verify pet exists
00444     try {
00445         petService.getPetById(petId);
00446     } catch (ServiceException e) {
00447         System.out.println("Error: Pet not found.");
00448         return;
00449     }
00450
00451     System.out.print("Reminder Type (1: Feeding, 2: Medication, 3: Grooming, 4: Exercise): ");
00452     String typeChoice = readInput();
00453
00454     System.out.print("Description: ");
00455     String desc = readInput();
00456     if (desc.isEmpty()) {
00457         System.out.println("Error: Description cannot be empty.");
00458         return;
00459     }
00460
00461     System.out.print("Date and Time (YYYY-MM-DD HH:MM): ");
00462     LocalDateTime time = readDateTimeInput();
00463     if (time == null) return;
00464
00465     Reminder newReminder = null;
00466     try {
00467         switch (typeChoice) {
00468             case "1":
00469                 FeedingReminder fr = new FeedingReminder(0, petId, "Unknown", time, "Unknown", 0);
00470                 fr.setDescription(desc);
00471                 System.out.print("Food Type: ");
00472                 fr.setFoodType(readInput());
00473                 System.out.print("Portion (grams): ");
00474                 fr.setPortionGrams((int)readDoubleInput());
00475                 newReminder = fr;

```

```

00476         break;
00477     case "2":
00478         MedicationReminder mr = new MedicationReminder(0, petId, "Unknown", time,
"Unknown", 0.0, "Unknown");
00479         mr.setDescription(desc);
00480         System.out.print("Medication Name: ");
00481         mr.setMedicationName(readInput());
00482         System.out.print("Dosage: ");
00483         mr.setDosage(readDoubleInput());
00484         newReminder = mr;
00485         break;
00486     case "3":
00487         GroomingReminder gr = new GroomingReminder(0, petId, "Unknown", time, "Unknown",
false);
00488         gr.setDescription(desc);
00489         System.out.print("Professional grooming? (true/false): ");
00490         gr.setProfessional(Boolean.parseBoolean(readInput()));
00491         newReminder = gr;
00492         break;
00493     case "4":
00494         ExerciseReminder er = new ExerciseReminder(0, petId, "Unknown", time, "Unknown",
0);
00495         er.setDescription(desc);
00496         System.out.print("Duration (minutes): ");
00497         er.setDurationMinutes((int)readDoubleInput());
00498         newReminder = er;
00499         break;
00500     default:
00501         System.out.println("Invalid type choice.");
00502         return;
00503     }
00504
00505     reminderService.addReminder(newReminder);
00506     System.out.println("Reminder added successfully.");
00507 } catch (ServiceException e) {
00508     System.out.println("Failed to add reminder: " + e.getMessage());
00509 }
00510 }
00511
00515 private void markReminderCompleted() {
00516     listPendingReminders();
00517     System.out.print("\nEnter ID of reminder to complete (or 0 to cancel): ");
00518     int id = readIntInput();
00519     if (id <= 0) return;
00520
00521     try {
00522         reminderService.markCompleted(id);
00523         System.out.println("Reminder marked as completed.");
00524     } catch (ServiceException e) {
00525         System.out.println("Failed to update reminder: " + e.getMessage());
00526     }
00527 }
00528
00532 private void deleteReminder() {
00533     listPendingReminders();
00534     System.out.print("\nEnter ID of reminder to delete (or 0 to cancel): ");
00535     int id = readIntInput();
00536     if (id <= 0) return;
00537
00538     try {
00539         reminderService.deleteReminder(id);
00540         System.out.println("Reminder deleted successfully.");
00541     } catch (ServiceException e) {
00542         System.out.println("Failed to delete reminder: " + e.getMessage());
00543     }
00544 }
00545
00546 // =====
00547 // VET APPOINTMENTS MENU
00548 // =====
00552 public void showVetMenu() {
00553     while (running) {
00554         String choice = selectMenuOption(
00555             "\n--- VET APPOINTMENTS ---",
00556             new String[]{"List Vet Appointments", "Add New Appointment", "Back to Main Menu"},
00557             new String[]{"1", "2", "0"}
00558         );
00559
00560         switch (choice) {
00561             case "1": listVetAppointments(); break;
00562             case "2": addVetAppointment(); break;
00563             case "0": return;
00564             default: System.out.println("Invalid choice.");
00565         }
00566     }
00567 }
00568

```

```

00572     private void listVetAppointments() {
00573         System.out.println("\n--- VET APPOINTMENTS ---");
00574         try {
00575             List<Reminder> reminders = reminderService.getPendingReminders();
00576             boolean found = false;
00577             for (Reminder r : reminders) {
00578                 if (r instanceof VetAppointment) {
00579                     VetAppointment va = (VetAppointment) r;
00580                     System.out.println(String.format("ID: %d | Pet ID: %d | Time: %s | Clinic: %s |
Reason: %s",
00581                         va.getId(), va.getPetId(),
00582                         va.getScheduledTime().format(DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm")),
00583                         va.getClinicName(), va.getReason()));
00584                     found = true;
00585                 }
00586             }
00587             if (!found) System.out.println("No pending vet appointments found.");
00588         } catch (Exception e) {
00589             System.out.println("Error: " + e.getMessage());
00590         }
00591     }
00592     private void addVetAppointment() {
00593         System.out.println("\n--- ADD VET APPOINTMENT ---");
00594         System.out.print("Enter Pet ID: ");
00595         int petId = readIntInput();
00596         if (petId <= 0) return;
00597
00598         System.out.print("Description: ");
00599         String desc = readInput();
00600
00601         System.out.print("Date and Time (YYYY-MM-DD HH:MM): ");
00602         LocalDateTime time = readDateTimeInput();
00603         if (time == null) return;
00604
00605         System.out.print("Clinic Name: ");
00606         String clinic = readInput();
00607
00608         System.out.print("Reason: ");
00609         String reason = readInput();
00610
00611         try {
00612             VetAppointment va = new VetAppointment(0, petId, "Unknown", time, "Unknown", clinic,
reason);
00613             va.setDescription(desc);
00614             reminderService.addReminder(va);
00615             System.out.println("Vet appointment scheduling successful.");
00616         } catch (ServiceException e) {
00617             System.out.println("Failed: " + e.getMessage());
00618         }
00619     }
00620     // =====
00621     // MEDICAL RECORDS MENU
00622     // =====
00623     public void showMedicalMenu() {
00624         while (running) {
00625             String choice = selectMenuOption(
00626                 "\n--- MEDICAL RECORDS ---",
00627                 new String[]{"List Records", "Add New Record", "Back to Main Menu"},
00628                 new String[]{"1", "2", "0"}
00629             );
00630
00631             switch (choice) {
00632                 case "1": listMedicalRecords(); break;
00633                 case "2": addMedicalRecord(); break;
00634                 case "0": return;
00635                 default: System.out.println("Invalid choice.");
00636             }
00637         }
00638     }
00639     private void listMedicalRecords() {
00640         System.out.println("\n--- MEDICAL RECORDS ---");
00641         System.out.print("Enter Pet ID (or 0 for all): ");
00642         int petId = readIntInput();
00643         try {
00644             List<MedicalRecord> records = (petId > 0) ?
00645                 medicalRecordService.getRecordsByPetId(petId) : medicalRecordService.getAllRecords();
00646
00647             if (records.isEmpty()) {
00648                 System.out.println("No records found.");
00649                 return;
00650             }
00651
00652             for (MedicalRecord m : records) {
00653                 System.out.println(String.format("ID: %d | Pet ID: %d | Date: %s | Diagnosis: %s |

```

```

Treatment: %s",
00665         m.getId(), m.getPetId(), m.getRecordDate(), m.getDiagnosis(), m.getTreatment());
00666     }
00667 } catch (Exception e) {
00668     System.out.println("Error: " + e.getMessage());
00669 }
00670 }
00671
00672 private void addMedicalRecord() {
00673     System.out.println("\n--- ADD MEDICAL RECORD ---");
00674     System.out.print("Enter Pet ID: ");
00675     int petId = readIntInput();
00676     if (petId <= 0) return;
00677
00678     System.out.print("Date (YYYY-MM-DD): ");
00679     LocalDate date = readDateInput();
00680     if (date == null) return;
00681
00682     System.out.print("Diagnosis: ");
00683     String diagnosis = readInput();
00684
00685     System.out.print("Treatment: ");
00686     String treatment = readInput();
00687
00688     try {
00689         MedicalRecord record = new MedicalRecord(0, petId, "Unknown", date, "Checkup", diagnosis,
00690 "Unknown");
00691         record.setTreatment(treatment);
00692         medicalRecordService.addRecord(record);
00693         System.out.println("Medical record added successfully.");
00694     } catch (ServiceException e) {
00695         System.out.println("Failed: " + e.getMessage());
00696     }
00697 }
00698
00699 // =====
00700 // SETTINGS MENU
00701 // =====
00702 public void showSettingsMenu() {
00703     String choice = selectMenuOption(
00704         "\n--- SETTINGS ---\nCurrent Storage: " +
00705         StorageConfig.getActiveBackend().getDisplayName(),
00706         new String[]{"Binary File I/O (.bin)", "SQLite (.db)", "MySQL (Docker required)", "Back"},
00707         new String[]{"1", "2", "3", "0"}
00708     );
00709     handleStorageSwitch(choice);
00710 }
00711
00712 public void handleStorageSwitch(String choice) {
00713     if (choice == null) return;
00714     switch (choice.trim()) {
00715         case "1":
00716             StorageConfig.setActiveBackend(StorageType.BINARY);
00717             System.out.println("Storage switched to: Binary File I/O");
00718             break;
00719         case "2":
00720             StorageConfig.setActiveBackend(StorageType.SQLITE);
00721             System.out.println("Storage switched to: SQLite");
00722             break;
00723         case "3":
00724             StorageConfig.setActiveBackend(StorageType.MYSQL);
00725             System.out.println("Storage switched to: MySQL");
00726             System.out.println("NOTE: MySQL requires Docker Compose to be running.");
00727             break;
00728         case "0":
00729             break;
00730         default:
00731             System.out.println("Invalid choice.");
00732     }
00733 }
00734
00735 // =====
00736 // UTILITIES
00737 // =====
00738 public void exitApp() {
00739     running = false;
00740     System.out.println("\nClosing Pet Care Reminder System...");
00741     System.out.println("Goodbye!");
00742 }
00743
00744 public String readInput() {
00745     try {
00746         if (scanner.hasNextLine()) {
00747             return scanner.nextLine().trim();
00748         }
00749     } catch (Exception e) {
00750         logger.error("Input reading error: {}", e.getMessage());
00751     }
00752 }

```

```

00764     }
00765     return "";
00766 }
00767
00768 // Test helper
00772 protected void setScannerSource(String input) {
00773     // Only used in tests via reflection
00774 }
00775
00779 private int readIntInput() {
00780     try {
00781         String input = readInput();
00782         return input.isEmpty() ? 0 : Integer.parseInt(input);
00783     } catch (NumberFormatException e) {
00784         System.out.println("Error: Please enter a valid number.");
00785         return -1;
00786     }
00787 }
00788
00792 private double readDoubleInput() {
00793     try {
00794         String input = readInput();
00795         return input.isEmpty() ? 0.0 : Double.parseDouble(input);
00796     } catch (NumberFormatException e) {
00797         System.out.println("Error: Please enter a valid decimal number.");
00798         return -1.0;
00799     }
00800 }
00801
00805 private LocalDate readDateInput() {
00806     String input = readInput();
00807     try {
00808         return LocalDate.parse(input);
00809     } catch (DateTimeParseException e) {
00810         System.out.println("Error: Invalid date format. Please use YYYY-MM-DD.");
00811         return null;
00812     }
00813 }
00814
00818 private LocalDateTime readDateTimeInput() {
00819     String input = readInput();
00820     try {
00821         DateTimeFormatter formatter = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm");
00822         return LocalDateTime.parse(input, formatter);
00823     } catch (DateTimeParseException e) {
00824         System.out.println("Error: Invalid format. Please use YYYY-MM-DD HH:MM");
00825         return null;
00826     }
00827 }
00828
00833 public boolean isRunning() { return running; }
00834
00839 public void setRunning(boolean running) { this.running = running; }
00840 }

```

8.7 petreminder-app/src/main/java/com/mehdi/petreminder/gui/dialogs/PetDialog.java File Reference ↩

Pet ekleme / düzenleme diyalogu.

```

import com.mehdi.petreminder.gui.util.GuiConstants;
import com.mehdi.petreminder.model.*;
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import java.awt.*;
import java.time.LocalDate;
import java.time.format.DateTimeParseException;

```

Classes

- class [com.mehdi.petreminder.gui.dialogs.PetDialog](#)
Pet ekleme ve düzenleme modal dialog'u.

Packages

- package [com.mehdi.petreminder.gui.dialogs](#)

8.7.1 Detailed Description

[Pet](#) ekleme / düzenleme diyalogu.

Member documentation. Member documentation.

Definition in file [PetDialog.java](#).

8.8 PetDialog.java

[Go to the documentation of this file.](#)

```

00001
00014 package com.mehdi.petreminder.gui.dialogs;
00015
00016 import com.mehdi.petreminder.gui.util.GuiConstants;
00017 import com.mehdi.petreminder.model.*;
00018
00019 import javax.swing.*;
00020 import javax.swing.border.EmptyBorder;
00021 import java.awt.*;
00022 import java.time.LocalDate;
00023 import java.time.format.DateTimeParseException;
00024
00034 public class PetDialog extends JDialog {
00035
00037     private static final long serialVersionUID = 1L;
00038
00040     private boolean confirmed = false;
00041
00043     private Pet resultPet;
00044
00052     private JTextField txtName, txtBreed, txtBirthDate, txtGender, txtWeight, txtNotes;
00054     private JComboBox<String> cmbSpecies;
00056     private JCheckBox chkTrained;
00058     private JCheckBox chkIndoor;
00060     private JCheckBox chkCanTalk;
00061
00063     private final Pet editPet;
00064
00070     public PetDialog(Frame parent, Pet editPet) {
00071         super(parent, editPet == null ? "Yeni Hayvan Ekle" : "Hayvanı Düzenle", true);
00072         this.editPet = editPet;
00073         buildUI();
00074         if (editPet != null) fillForm(editPet);
00075         pack();
00076         setLocationRelativeTo(parent);
00077         setMinimumSize(new Dimension(420, 500));
00078     }
00079
00083     private void buildUI() {
00084         JPanel main = new JPanel(new BorderLayout(8, 8));
00085         main.setBorder(new EmptyBorder(20, 20, 20, 20));
00086
00087         main.add(buildForm(), BorderLayout.CENTER);
00088         main.add(buildButtons(), BorderLayout.SOUTH);
00089         setContentPane(main);
00090     }
00091
00096     private JPanel buildForm() {
00097         JPanel p = new JPanel(new GridBagLayout());
00098         GridBagConstraints gbc = new GridBagConstraints();
00099         gbc.insets = new Insets(5, 5, 5, 5);
00100         gbc.anchor = GridBagConstraints.WEST;
00101         gbc.fill = GridBagConstraints.HORIZONTAL;
00102
00103         int row = 0;
00104         txtName = addField(p, gbc, row++, "Ad *:", new JTextField(20));
00105         cmbSpecies = new JComboBox<>(new String[]{"Dog", "Cat", "Bird"});
00106         cmbSpecies.addActionListener(e -> updateSpecificFields());

```

```

00107         addLabel(p, gbc, row, "Tür *:");
00108         gbc.gridx = 1; gbc.gridy = row++;
00109         p.add(cmbSpecies, gbc);
00110
00111         txtBreed      = addField(p, gbc, row++, "İrk / Tür:", new JTextField(20));
00112         txtBirthDate  = addField(p, gbc, row++, "Doğum Tarihi\n(yyyy-MM-dd):", new JTextField(20));
00113         txtGender     = addField(p, gbc, row++, "Cinsiyet:", new JTextField(20));
00114         txtWeight     = addField(p, gbc, row++, "Ağırlık (kg):", new JTextField(20));
00115
00116         chkTrained    = new JCheckBox("Eğitimli");
00117         chkIndoor     = new JCheckBox("Ev Kedisi");
00118         chkCanTalk    = new JCheckBox("Konuşuyor");
00119
00120         addLabel(p, gbc, row, "Özellik:");
00121         gbc.gridx = 1; gbc.gridy = row++;
00122         JPanel checkPanel = new JPanel(new FlowLayout(FlowLayout.LEFT, 0, 0));
00123         checkPanel.setOpaque(false);
00124         checkPanel.add(chkTrained);
00125         checkPanel.add(chkIndoor);
00126         checkPanel.add(chkCanTalk);
00127         p.add(checkPanel, gbc);
00128
00129         txtNotes = addField(p, gbc, row, "Notlar:", new JTextField(20));
00130         updateSpecificFields();
00131         return p;
00132     }
00133
00137     private void updateSpecificFields() {
00138         String species = (String) cmbSpecies.getSelectedItem();
00139         chkTrained.setEnabled("Dog".equals(species));
00140         chkIndoor.setEnabled("Cat".equals(species));
00141         chkCanTalk.setEnabled("Bird".equals(species));
00142     }
00143
00153     private JTextField addField(JPanel p, GridBagConstraints gbc,
00154                                int row, String label, JTextField field) {
00155         addLabel(p, gbc, row, label);
00156         gbc.gridx = 1; gbc.gridy = row;
00157         p.add(field, gbc);
00158         return field;
00159     }
00160
00164     private void addLabel(JPanel p, GridBagConstraints gbc, int row, String text) {
00165         gbc.gridx = 0; gbc.gridy = row;
00166         JLabel lbl = new JLabel(text.replace("\n", " "));
00167         lbl.setFont(GuiConstants.BODY_FONT);
00168         p.add(lbl, gbc);
00169     }
00170
00175     private JPanel buildButtons() {
00176         JPanel p = new JPanel(new FlowLayout(FlowLayout.RIGHT));
00177         JButton btnOk = new JButton("Kaydet");
00178         btnOk.setBackground(GuiConstants.PRIMARY);
00179         btnOk.setForeground(Color.WHITE);
00180         btnOk.setFocusPainted(false);
00181         btnOk.addActionListener(e -> onSave());
00182
00183         JButton btnCancel = new JButton("İptal");
00184         btnCancel.addActionListener(e -> dispose());
00185
00186         p.add(btnCancel);
00187         p.add(btnOk);
00188         return p;
00189     }
00190
00195     private void fillForm(Pet pet) {
00196         txtName.setText(pet.getName());
00197         cmbSpecies.setSelectedItem(pet.getSpecies());
00198         txtBreed.setText(getBreed(pet));
00199         if (pet.getBirthDate() != null) {
00200             txtBirthDate.setText(pet.getBirthDate().toString());
00201         }
00202         txtGender.setText(pet.getGender() != null ? pet.getGender() : "");
00203         txtWeight.setText(String.valueOf(pet.getWeight()));
00204         txtNotes.setText(pet.getNotes() != null ? pet.getNotes() : "");
00205         if (pet instanceof Dog) chkTrained.setSelected(((Dog) pet).isTrained());
00206         if (pet instanceof Cat) chkIndoor.setSelected(((Cat) pet).isIndoor());
00207         if (pet instanceof Bird) chkCanTalk.setSelected(((Bird) pet).isCanTalk());
00208         updateSpecificFields();
00209     }
00210
00216     private String getBreed(Pet pet) {
00217         if (pet instanceof Dog) return ((Dog) pet).getBreed() != null ? ((Dog) pet).getBreed() : "";
00218         if (pet instanceof Cat) return ((Cat) pet).getBreed() != null ? ((Cat) pet).getBreed() : "";
00219         if (pet instanceof Bird) return ((Bird) pet).getBirdType() != null ? ((Bird)
pet).getBirdType() : "";
00220         return "";

```

```

00221     }
00222
00226     private void onSave() {
00227         String name = txtName.getText().trim();
00228         if (name.isEmpty()) {
00229             JOptionPane.showMessageDialog(this, "Ad boş olamaz!", "Hata",
00230                 JOptionPane.ERROR_MESSAGE);
00231             return;
00232         }
00233
00234         LocalDate birthDate = null;
00235         String bdStr = txtBirthDate.getText().trim();
00236         if (!bdStr.isEmpty()) {
00237             try {
00238                 birthDate = LocalDate.parse(bdStr);
00239             } catch (DateTimeParseException e) {
00240                 JOptionPane.showMessageDialog(this, "Tarih formatı: yyyy-MM-dd", "Hata",
00241                     JOptionPane.ERROR_MESSAGE);
00242                 return;
00243             }
00244         }
00245
00246         double weight = 0;
00247         try {
00248             if (!txtWeight.getText().trim().isEmpty()) {
00249                 weight = Double.parseDouble(txtWeight.getText().trim());
00250             }
00251         } catch (NumberFormatException e) {
00252             JOptionPane.showMessageDialog(this, "Ağırlık sayısal olmalı", "Hata",
00253                 JOptionPane.ERROR_MESSAGE);
00254             return;
00255         }
00256
00257         String species = (String) cmbSpecies.getSelectedItem();
00258         int ownerId = editPet != null ? editPet.getOwnerId() : 1;
00259
00260         Pet pet;
00261         if ("Dog".equals(species)) {
00262             Dog dog = new Dog(editPet != null ? editPet.getId() : 0, name, birthDate, ownerId);
00263             dog.setBreed(txtBreed.getText().trim());
00264             dog.setTrained(chkTrained.isSelected());
00265             pet = dog;
00266         } else if ("Cat".equals(species)) {
00267             Cat cat = new Cat(editPet != null ? editPet.getId() : 0, name, birthDate, ownerId);
00268             cat.setBreed(txtBreed.getText().trim());
00269             cat.setIndoor(chkIndoor.isSelected());
00270             pet = cat;
00271         } else {
00272             Bird bird = new Bird(editPet != null ? editPet.getId() : 0, name, birthDate, ownerId);
00273             bird.setBirdType(txtBreed.getText().trim());
00274             bird.setCanTalk(chkCanTalk.isSelected());
00275             pet = bird;
00276         }
00277         pet.setGender(txtGender.getText().trim());
00278         pet.setWeight(weight);
00279         pet.setNotes(txtNotes.getText().trim());
00280
00281         resultPet = pet;
00282         confirmed = true;
00283         dispose();
00284     }
00285
00290     public boolean isConfirmed() { return confirmed; }
00291
00296     public Pet getPet() { return resultPet; }
00297 }

```

8.9 petreminder-app/src/main/java/com/mehdi/petreminder/gui/dialogs/ReminderDialog.java File Reference

Hatırlatıcı ekleme / düzenleme diyalogu.

```

import com.mehdi.petreminder.gui.util.GuiConstants;
import com.mehdi.petreminder.model.*;
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import java.awt.*;

```



```
import java.time.LocalDateTime;
import java.time.format.DateTimeParseException;
```

Classes

- class [com.mehdi.petreminder.gui.dialogs.ReminderDialog](#)
Reminder ekleme ve düzenleme modal dialog'u.

Packages

- package [com.mehdi.petreminder.gui.dialogs](#)

8.9.1 Detailed Description

Hatırlatıcı ekleme / düzenleme diyalogu.

Definition in file [ReminderDialog.java](#).

8.10 ReminderDialog.java

[Go to the documentation of this file.](#)

```
00001
00005 package com.mehdi.petreminder.gui.dialogs;
00006
00007 import com.mehdi.petreminder.gui.util.GuiConstants;
00008 import com.mehdi.petreminder.model.*;
00009
00010 import javax.swing.*;
00011 import javax.swing.border.EmptyBorder;
00012 import java.awt.*;
00013 import java.time.LocalDateTime;
00014 import java.time.format.DateTimeParseException;
00015
00025 public class ReminderDialog extends JDialog {
00026
00028     private static final long serialVersionUID = 1L;
00029
00031     private boolean confirmed = false;
00032
00034     private Reminder resultReminder;
00035
00036     // Ortak alanlar
00038     private JTextField txtPetId;
00040     private JTextField txtPetName;
00042     private JTextField txtTime;
00044     private JComboBox<String> cmbPriority;
00046     private JComboBox<String> cmbType;
00048     private JTextField txtDesc;
00049
00050     // Tür-özel alanlar
00058     private JTextField txtFoodType, txtPortionGrams;
00066     private JTextField txtMedName, txtDosage, txtDosageUnit;
00074     private JTextField txtExerciseType, txtDuration;
00076     private JTextField txtGroomingType;
00084     private JTextField txtVetName, txtClinic, txtReason;
00085
00087     private JPanel specificPanel;
00088
00090     private final Reminder editReminder;
00091
00097     public ReminderDialog(Frame parent, Reminder editReminder) {
00098         super(parent, editReminder == null ? "Yeni Hatırlatıcı" : "Hatırlatıcı Düzenle", true);
00099         this.editReminder = editReminder;
00100         buildUI();
```

```

00101         if (editReminder != null) fillForm(editReminder);
00102         pack();
00103         setLocationRelativeTo(parent);
00104         setMinimumSize(new Dimension(440, 480));
00105     }
00106
00110     private void buildUI() {
00111         JPanel main = new JPanel(new BorderLayout(8, 8));
00112         main.setBorder(new EmptyBorder(20, 20, 20, 20));
00113         main.add(buildCommonForm(), BorderLayout.NORTH);
00114         specificPanel = new JPanel(new GridBagLayout());
00115         main.add(new JScrollPane(specificPanel), BorderLayout.CENTER);
00116         main.add(buildButtons(), BorderLayout.SOUTH);
00117         setContentPane(main);
00118     }
00119
00123     private JPanel buildCommonForm() {
00124         JPanel p = new JPanel(new GridBagLayout());
00125         GridBagConstraints gbc = new GridBagConstraints();
00126         gbc.insets = new Insets(4, 4, 4, 4);
00127         gbc.fill = GridBagConstraints.HORIZONTAL;
00128         gbc.anchor = GridBagConstraints.WEST;
00129
00130         int row = 0;
00131         txtPetId = addField(p, gbc, row++, "Pet ID:", new JTextField("1", 10));
00132         txtPetName = addField(p, gbc, row++, "Pet Adı:", new JTextField(10));
00133         txtTime = addField(p, gbc, row++, "Zaman (yyyy-MM-ddTHH:mm):",
00134             new JTextField(LocalDate.now().plusHours(1).toString().substring(0, 16), 10));
00135
00136         cmbPriority = new JComboBox<>(new String[]{"YUKSEK", "ORTA", "DUSUK"});
00137         cmbPriority.setSelectedItem("ORTA");
00138         addLabel(p, gbc, row, "Öncelik:");
00139         gbc.gridx = 1; gbc.gridy = row++;
00140         p.add(cmbPriority, gbc);
00141
00142         cmbType = new JComboBox<>(new String[]{"Besleme", "İlac", "Egzersiz", "Bakim", "Veteriner"});
00143         cmbType.addActionListener(e -> updateSpecificFields());
00144         addLabel(p, gbc, row, "Tür:");
00145         gbc.gridx = 1; gbc.gridy = row++;
00146         p.add(cmbType, gbc);
00147
00148         txtDesc = addField(p, gbc, row, "Açıklama:", new JTextField(10));
00149         return p;
00150     }
00151
00155     private void updateSpecificFields() {
00156         specificPanel.removeAll();
00157         String type = (String) cmbType.getSelectedItem();
00158         GridBagConstraints gbc = new GridBagConstraints();
00159         gbc.insets = new Insets(4, 4, 4, 4);
00160         gbc.fill = GridBagConstraints.HORIZONTAL;
00161         gbc.anchor = GridBagConstraints.WEST;
00162
00163         int row = 0;
00164         switch (type != null ? type : "") {
00165             case "Besleme":
00166                 txtFoodType = addField(specificPanel, gbc, row++, "Besin Türü:", new
JTextField(10));
00167                 txtPortionGrams = addField(specificPanel, gbc, row, "Porsiyon (g):", new
JTextField("100", 10));
00168                 break;
00169             case "İlac":
00170                 txtMedName = addField(specificPanel, gbc, row++, "İlaç Adı:", new JTextField(10));
00171                 txtDosage = addField(specificPanel, gbc, row++, "Doz:", new JTextField("1.0", 10));
00172                 txtDosageUnit = addField(specificPanel, gbc, row, "Birim:", new JTextField("mg", 10));
00173                 break;
00174             case "Egzersiz":
00175                 txtExerciseType = addField(specificPanel, gbc, row++, "Egzersiz Türü:", new
JTextField("Yürüyüş", 10));
00176                 txtDuration = addField(specificPanel, gbc, row, "Süre (dk):", new JTextField("30",
10));
00177                 break;
00178             case "Bakim":
00179                 txtGroomingType = addField(specificPanel, gbc, row, "Bakım Türü:", new
JTextField("Tıraş", 10));
00180                 break;
00181             case "Veteriner":
00182                 txtVetName = addField(specificPanel, gbc, row++, "Veteriner:", new JTextField(10));
00183                 txtClinic = addField(specificPanel, gbc, row++, "Klinik:", new JTextField(10));
00184                 txtReason = addField(specificPanel, gbc, row, "Sebebi:", new JTextField("Kontrol",
10));
00185                 break;
00186             default:
00187                 break;
00188         }
00189         specificPanel.revalidate();
00190         specificPanel.repaint();

```

```

00191     pack();
00192 }
00193
00197 private void fillForm(Reminder r) {
00198     txtPetId.setText(String.valueOf(r.getPetId()));
00199     txtPetName.setText(r.getPetName() != null ? r.getPetName() : "");
00200     if (r.getScheduledTime() != null) {
00201         txtTime.setText(r.getScheduledTime().toString().substring(0, 16));
00202     }
00203     cmbPriority.setSelectedItem(r.getPriority());
00204     cmbType.setSelectedItem(r.getReminderType());
00205     updateSpecificFields();
00206     txtDesc.setText(r.getDescription() != null ? r.getDescription() : "");
00207 }
00208
00212 private void onSave() {
00213     try {
00214         int petId = Integer.parseInt(txtPetId.getText().trim());
00215         String petName = txtPetName.getText().trim();
00216         LocalDateTime time = LocalDateTime.parse(txtTime.getText().trim());
00217         String priority = (String) cmbPriority.getSelectedItem();
00218         String type = (String) cmbType.getSelectedItem();
00219         String desc = txtDesc.getText().trim();
00220         int editId = editReminder != null ? editReminder.getId() : 0;
00221
00222         Reminder r;
00223         switch (type != null ? type : "") {
00224             case "Besleme":
00225                 String food = txtFoodType != null ? txtFoodType.getText().trim() : "Mama";
00226                 double portion = txtPortionGrams != null
00227                     ? Double.parseDouble(txtPortionGrams.getText().trim()) : 100;
00228                 r = new FeedingReminder(editId, petId, petName, time, desc,
00229                     priority, false, "", food, portion, false);
00230                 break;
00231             case "Ilac":
00232                 String med = txtMedName != null ? txtMedName.getText().trim() : "İlaç";
00233                 double dose = txtDosage != null
00234                     ? Double.parseDouble(txtDosage.getText().trim()) : 1.0;
00235                 String unit = txtDosageUnit != null ? txtDosageUnit.getText().trim() : "mg";
00236                 r = new MedicationReminder(editId, petId, petName, time, desc,
00237                     priority, false, "", med, dose, unit, false, null);
00238                 break;
00239             case "Egzersiz":
00240                 String exType = txtExerciseType != null ? txtExerciseType.getText().trim() :
00241                     "Yürüyüş";
00242                 int dur = txtDuration != null
00243                     ? Integer.parseInt(txtDuration.getText().trim()) : 30;
00244                 r = new ExerciseReminder(editId, petId, petName, time, desc,
00245                     priority, false, "", exType, dur, 0);
00246                 break;
00247             case "Bakim":
00248                 String gType = txtGroomingType != null ? txtGroomingType.getText().trim() :
00249                     "Tıraş";
00250                 r = new GroomingReminder(editId, petId, petName, time, desc,
00251                     priority, false, "", gType, false, "");
00252                 break;
00253             case "Veteriner":
00254                 String vet = txtVetName != null ? txtVetName.getText().trim() : "Veteriner";
00255                 String clinic = txtClinic != null ? txtClinic.getText().trim() : "Klinik";
00256                 String reason = txtReason != null ? txtReason.getText().trim() : "Kontrol";
00257                 r = new VetAppointment(editId, petId, petName, time, desc,
00258                     priority, false, "", vet, clinic, reason, 0, false, "");
00259                 break;
00260             default:
00261                 r = new FeedingReminder(editId, petId, petName, time, "Mama", 100);
00262         }
00263         resultReminder = r;
00264         confirmed = true;
00265         dispose();
00266     } catch (NumberFormatException e) {
00267         JOptionPane.showMessageDialog(this, "Sayısal alanları kontrol edin.", "Hata",
00268             JOptionPane.ERROR_MESSAGE);
00269     } catch (DateTimeParseException e) {
00270         JOptionPane.showMessageDialog(this, "Zaman formatı: yyyy-MM-ddTHH:mm", "Hata",
00271             JOptionPane.ERROR_MESSAGE);
00272     }
00273 }
00274
00276 private JPanel buildButtons() {
00277     JPanel p = new JPanel(new FlowLayout(FlowLayout.RIGHT));
00278     JButton ok = new JButton("Kaydet");
00279     ok.setBackground(GuiConstants.PRIMARY);
00280     ok.setForeground(Color.WHITE);
00281     ok.setFocusPainted(false);
00282     ok.addActionListener(e -> onSave());
00283     JButton cancel = new JButton("İptal");
00284     cancel.addActionListener(e -> dispose());

```

```

00285         p.add(cancel);
00286         p.add(ok);
00287         return p;
00288     }
00289
00293     private JTextField addField(JPanel p, GridBagConstraints gbc, int row, String label, JTextField
field) {
00294         addLabel(p, gbc, row, label);
00295         gbc.gridx = 1; gbc.gridy = row;
00296         p.add(field, gbc);
00297         return field;
00298     }
00299
00303     private void addLabel(JPanel p, GridBagConstraints gbc, int row, String text) {
00304         gbc.gridx = 0; gbc.gridy = row;
00305         p.add(new JLabel(text), gbc);
00306     }
00307
00312     public boolean isConfirmed() { return confirmed; }
00313
00318     public Reminder getReminder() { return resultReminder; }
00319 }

```

8.11 petreminder-app/src/main/java/com/mehdi/petreminder/gui/MainFrame.java File Reference ↩

Ana Swing penceresi — tüm panelleri barındırır.

```

import com.formdev.flatlaf.FlatLightLaf;
import com.mehdi.petreminder.config.StorageConfig;
import com.mehdi.petreminder.config.StorageType;
import com.mehdi.petreminder.gui.panels.*;
import com.mehdi.petreminder.gui.util.Constants;
import org.slf4j.LoggerFactory;
import ch.qos.logback.classic.Logger;
import javax.swing.*;
import java.awt.*;

```

Classes

- class [com.mehdi.petreminder.gui.MainFrame](#)
Uygulamanın ana Swing penceresi.

Packages

- package [com.mehdi.petreminder.gui](#)

8.11.1 Detailed Description

Ana Swing penceresi — tüm panelleri barındırır.

FlatLaf light tema, sol navigasyon, sağ içerik paneli.

Definition in file [MainFrame.java](#).

8.12 MainFrame.java

[Go to the documentation of this file.](#)

```

00001
00009 package com.mehdi.petreminder.gui;
00010
00011 import com.formdev.flatlaf.FlatLightLaf;
00012 import com.mehdi.petreminder.config.StorageConfig;
00013 import com.mehdi.petreminder.config.StorageType;
00014 import com.mehdi.petreminder.gui.panels.*;
00015 import com.mehdi.petreminder.gui.util.GuiConstants;
00016 import org.slf4j.LoggerFactory;
00017 import ch.qos.logback.classic.Logger;
00018
00019 import javax.swing.*;
00020 import java.awt.*;
00021
00033 public class MainFrame extends JFrame {
00034
00036     private static final Logger logger =
00037         (Logger) LoggerFactory.getLogger(MainFrame.class);
00038
00040     private static final long serialVersionUID = 1L;
00041
00043     private static final String TITLE = "Pet Care Reminder System v1.0";
00044
00046     public static final String CARD_DASHBOARD = "DASHBOARD";
00048     public static final String CARD_PETS = "PETS";
00050     public static final String CARD_REMINDERS = "REMINDERS";
00052     public static final String CARD_VET = "VET";
00054     public static final String CARD_MEDICAL = "MEDICAL";
00056     public static final String CARD_SETTINGS = "SETTINGS";
00057
00059     private JPanel contentPanel;
00060
00062     private CardLayout cardLayout;
00063
00065     private NavPanel navPanel;
00066
00068     private DashboardPanel dashboardPanel;
00069
00071     private PetsPanel petsPanel;
00072
00074     private RemindersPanel remindersPanel;
00075
00077     private VetPanel vetPanel;
00078
00080     private MedicalPanel medicalPanel;
00081
00083     private SettingsPanel settingsPanel;
00084
00088     public MainFrame() {
00089         super(TITLE);
00090         initLookAndFeel();
00091         initComponents();
00092         setupLayout();
00093         setupFrame();
00094         navigateTo(CARD_DASHBOARD);
00095         logger.info("MainFrame başlatıldı.");
00096     }
00097
00101     private void initLookAndFeel() {
00102         try {
00103             FlatLightLaf.setup();
00104             UIManager.put("Button.arc", 8);
00105             UIManager.put("Component.arc", 6);
00106             UIManager.put("TextComponent.arc", 6);
00107             UIManager.put("TabbedPane.tabHeight", 36);
00108         } catch (Exception e) {
00109             logger.warn("FlatLaf yüklenemedi, varsayılan tema kullanılıyor.");
00110         }
00111     }
00112
00116     private void initComponents() {
00117         cardLayout = new CardLayout();
00118         contentPanel = new JPanel(cardLayout);
00119         contentPanel.setBackground(GuiConstants.BG_COLOR);
00120
00121         dashboardPanel = new DashboardPanel(this);
00122         petsPanel = new PetsPanel(this);
00123         remindersPanel = new RemindersPanel(this);
00124         vetPanel = new VetPanel(this);
00125         medicalPanel = new MedicalPanel(this);
00126         settingsPanel = new SettingsPanel(this);
00127

```

```

00128         contentPanel.add(dashboardPanel, CARD_DASHBOARD);
00129         contentPanel.add(petsPanel, CARD_PETS);
00130         contentPanel.add(remindersPanel, CARD_REMINDERS);
00131         contentPanel.add(vetPanel, CARD_VET);
00132         contentPanel.add(medicalPanel, CARD_MEDICAL);
00133         contentPanel.add(settingsPanel, CARD_SETTINGS);
00134
00135         navPanel = new NavPanel(this);
00136     }
00137
00141     private void setupLayout() {
00142         setLayout(new BorderLayout());
00143         add(navPanel, BorderLayout.WEST);
00144         add(contentPanel, BorderLayout.CENTER);
00145         add(buildStatusBar(), BorderLayout.SOUTH);
00146     }
00147
00151     private void setupFrame() {
00152         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
00153         setSize(1100, 700);
00154         setMinimumSize(new Dimension(900, 600));
00155         setLocationRelativeTo(null);
00156         setIconImage(getAppIcon());
00157     }
00158
00163     private Image getAppIcon() {
00164         // Kaynak yoksa null döner -- sorun çıkarmaz
00165         return null;
00166     }
00167
00172     private JPanel buildStatusBar() {
00173         JPanel bar = new JPanel(new BorderLayout());
00174         bar.setBackground(GuiConstants.NAV_COLOR);
00175         bar.setBorder(BorderFactory.createEmptyBorder(4, 12, 4, 12));
00176         bar.setPreferredSize(new Dimension(0, 28));
00177
00178         JLabel storage = new JLabel("Storage: "
00179             + StorageConfig.getActiveBackend().getDisplayName());
00180         storage.setForeground(Color.WHITE);
00181         storage.setFont(GuiConstants.SMALL_FONT);
00182
00183         JLabel version = new JLabel("CEN206 -- Spring 2026 | RTEU");
00184         version.setForeground(new Color(180, 180, 200));
00185         version.setFont(GuiConstants.SMALL_FONT);
00186
00187         bar.add(storage, BorderLayout.WEST);
00188         bar.add(version, BorderLayout.EAST);
00189         return bar;
00190     }
00191
00196     public void navigateTo(String cardName) {
00197         cardLayout.show(contentPanel, cardName);
00198         navPanel.setActiveButton(cardName);
00199         refreshCurrentPanel(cardName);
00200         logger.debug("Navigasyon: {}", cardName);
00201     }
00202
00207     private void refreshCurrentPanel(String cardName) {
00208         switch (cardName) {
00209             case CARD_DASHBOARD: dashboardPanel.refresh(); break;
00210             case CARD_PETS: petsPanel.refresh(); break;
00211             case CARD_REMINDERS: remindersPanel.refresh(); break;
00212             case CARD_VET: vetPanel.refresh(); break;
00213             case CARD_MEDICAL: medicalPanel.refresh(); break;
00214             case CARD_SETTINGS: settingsPanel.refresh(); break;
00215             default: break;
00216         }
00217     }
00218
00223     public void onStorageChanged(StorageType type) {
00224         StorageConfig.setActiveBackend(type);
00225         // Status bar'ı güncelle
00226         Component[] comps = ((JPanel) getContentPane()).getComponents();
00227         for (Component c : comps) {
00228             if (c instanceof JLabel && ((JLabel) c).getText().startsWith("Storage:")) {
00229                 ((JLabel) c).setText("Storage: " + type.getDisplayName());
00230             }
00231         }
00232         JOptionPane.showMessageDialog(this,
00233             "Storage backend değiştirildi:\n" + type.getDisplayName(),
00234             "Ayarlar", JOptionPane.INFORMATION_MESSAGE);
00235         logger.info("Storage değişti: {}", type);
00236     }
00237
00241     public static void launch() {
00242         SwingUtilities.invokeLater(() -> {
00243             MainFrame frame = new MainFrame();

```

```
00244         frame.setVisible(true);
00245     });
00246 }
00247 }
```

8.13 petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/DashboardPanel.java File Reference

Dashboard — özet istatistikler ve son hatırlatıcılar.

```
import com.mehdi.petreminder.gui.MainFrame;
import com.mehdi.petreminder.gui.util.Constants;
import com.mehdi.petreminder.model.Reminder;
import com.mehdi.petreminder.service.MedicalRecordService;
import com.mehdi.petreminder.service.PetService;
import com.mehdi.petreminder.service.ReminderService;
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import java.awt.*;
import java.util.List;
```

Classes

- class [com.mehdi.petreminder.gui.panels.DashboardPanel](#)
Ana kontrol paneli — özet kartlar ve bekleyen hatırlatıcılar.

Packages

- package [com.mehdi.petreminder.gui.panels](#)

8.13.1 Detailed Description

Dashboard — özet istatistikler ve son hatırlatıcılar.

Definition in file [DashboardPanel.java](#).

8.14 DashboardPanel.java

[Go to the documentation of this file.](#)

```
00001
00008 package com.mehdi.petreminder.gui.panels;
00009
00010 import com.mehdi.petreminder.gui.MainFrame;
00011 import com.mehdi.petreminder.gui.util.Constants;
00012 import com.mehdi.petreminder.model.Reminder;
00013 import com.mehdi.petreminder.service.MedicalRecordService;
00014 import com.mehdi.petreminder.service.PetService;
00015 import com.mehdi.petreminder.service.ReminderService;
00016
00017 import javax.swing.*;
```

```

00018 import javax.swing.border.EmptyBorder;
00019 import java.awt.*;
00020 import java.util.List;
00021
00030 public class DashboardPanel extends JPanel {
00031
00033     private static final long serialVersionUID = 1L;
00034
00036     private final MainFrame mainFrame;
00037
00039     private final PetService petService;
00041     private final ReminderService reminderService;
00043     private final MedicalRecordService medicalService;
00044
00052     private JLabel lblPetCount, lblReminderCount, lblOverdueCount, lblMedicalCount;
00053
00055     private DefaultListModel<String> pendingModel;
00056
00061     public DashboardPanel(MainFrame mainFrame) {
00062         this.mainFrame = mainFrame;
00063         petService = new PetService();
00064         reminderService = new ReminderService();
00065         medicalService = new MedicalRecordService();
00066         setBackground(GuiConstants.BG_COLOR);
00067         setLayout(new BorderLayout(GuiConstants.GAP, GuiConstants.GAP));
00068         setBorder(new EmptyBorder(GuiConstants.GAP_LARGE, GuiConstants.GAP_LARGE,
00069             GuiConstants.GAP_LARGE, GuiConstants.GAP_LARGE));
00070         buildUI();
00071     }
00072
00076     private void buildUI() {
00077         // Başlık
00078         JLabel title = new JLabel("Dashboard");
00079         title.setFont(GuiConstants.TITLE_FONT);
00080         title.setForeground(GuiConstants.TEXT_PRIMARY);
00081         add(title, BorderLayout.NORTH);
00082
00083         // Orta -- Özet kartlar + liste
00084         JPanel center = new JPanel(new BorderLayout(GuiConstants.GAP, GuiConstants.GAP));
00085         center.setOpaque(false);
00086
00087         center.add(buildSummaryCards(), BorderLayout.NORTH);
00088         center.add(buildPendingSection(), BorderLayout.CENTER);
00089
00090         add(center, BorderLayout.CENTER);
00091     }
00092
00097     private JPanel buildSummaryCards() {
00098         JPanel row = new JPanel(new GridLayout(1, 4, GuiConstants.GAP, 0));
00099         row.setOpaque(false);
00100
00101         lblPetCount = new JLabel("0", SwingConstants.CENTER);
00102         lblReminderCount = new JLabel("0", SwingConstants.CENTER);
00103         lblOverdueCount = new JLabel("0", SwingConstants.CENTER);
00104         lblMedicalCount = new JLabel("0", SwingConstants.CENTER);
00105
00106         row.add(summaryCard(" Hayvanlar", lblPetCount, GuiConstants.PRIMARY));
00107         row.add(summaryCard(" Hatırlatıcılar", lblReminderCount, GuiConstants.SUCCESS));
00108         row.add(summaryCard(" Gecikmiş", lblOverdueCount, GuiConstants.DANGER));
00109         row.add(summaryCard(" Sağlık Kayıtları", lblMedicalCount, GuiConstants.WARNING));
00110         return row;
00111     }
00112
00120     private JPanel summaryCard(String label, JLabel value, Color color) {
00121         JPanel card = new JPanel(new BorderLayout());
00122         card.setBackground(GuiConstants.CARD_BG);
00123         card.setBorder(BorderFactory.createCompoundBorder(
00124             BorderFactory.createLineBorder(GuiConstants.BORDER_COLOR),
00125             new EmptyBorder(20, 20, 20, 20)));
00126
00127         JLabel lbl = new JLabel(label, SwingConstants.CENTER);
00128         lbl.setFont(GuiConstants.BODY_FONT);
00129         lbl.setForeground(GuiConstants.TEXT_SECONDARY);
00130
00131         value.setFont(new Font("Segoe UI", Font.BOLD, 36));
00132         value.setForeground(color);
00133
00134         card.add(lbl, BorderLayout.NORTH);
00135         card.add(value, BorderLayout.CENTER);
00136         return card;
00137     }
00138
00143     private JPanel buildPendingSection() {
00144         JPanel panel = new JPanel(new BorderLayout());
00145         panel.setBackground(GuiConstants.CARD_BG);
00146         panel.setBorder(BorderFactory.createCompoundBorder(
00147             BorderFactory.createLineBorder(GuiConstants.BORDER_COLOR),

```



```

00148         new EmptyBorder(16, 16, 16, 16)));
00149
00150         JLabel title = new JLabel("Bekleyen Hatırlatıcılar");
00151         title.setFont(GuiConstants.SECTION_FONT);
00152         title.setForeground(GuiConstants.TEXT_PRIMARY);
00153         panel.add(title, BorderLayout.NORTH);
00154
00155         pendingModel = new DefaultListModel<>();
00156         JList<String> list = new JList<>(pendingModel);
00157         list.setFont(GuiConstants.BODY_FONT);
00158         list.setSelectionBackground(new Color(238, 242, 255));
00159         panel.add(new JScrollPane(list), BorderLayout.CENTER);
00160
00161         JButton btnGoReminders = new JButton("Tümünü Gör →");
00162         btnGoReminders.setBackground(GuiConstants.PRIMARY);
00163         btnGoReminders.setForeground(Color.WHITE);
00164         btnGoReminders.setFocusPainted(false);
00165         btnGoReminders.addActionListener(e -> mainFrame.navigateTo(MainFrame.CARD_REMINDERS));
00166         JPanel south = new JPanel(new FlowLayout(FlowLayout.RIGHT));
00167         south.setOpaque(false);
00168         south.add(btnGoReminders);
00169         panel.add(south, BorderLayout.SOUTH);
00170
00171         return panel;
00172     }
00173
00177     public void refresh() {
00178         try {
00179             lblPetCount.setText(String.valueOf(petService.getPetCount()));
00180             lblReminderCount.setText(String.valueOf(reminderService.getReminderCount()));
00181             List<Reminder> overdue = reminderService.getOverdueReminders();
00182             lblOverdueCount.setText(String.valueOf(overdue.size()));
00183             lblMedicalCount.setText(String.valueOf(medicalService.getRecordCount()));
00184
00185             pendingModel.clear();
00186             List<Reminder> pending = reminderService.getPendingReminders();
00187             for (Reminder r : pending) {
00188                 String item = String.format("[%s] %s -- %s %s",
00189                     r.getPriority(),
00190                     r.getPetName(),
00191                     r.getReminderType(),
00192                     r.getFormattedTime());
00193                 if (r.isOverdue()) item = " " + item;
00194                 pendingModel.addElement(item);
00195             }
00196         } catch (Exception e) {
00197             // Veri yoksa sessizce devam et
00198         }
00199     }
00200 }

```

8.15 petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/MedicalPanel.java File Reference

Sağlık kayıtları yönetim paneli.

```

import com.mehdi.petreminder.gui.MainFrame;
import com.mehdi.petreminder.gui.util.GuiConstants;
import com.mehdi.petreminder.model.MedicalRecord;
import com.mehdi.petreminder.service.MedicalRecordService;
import com.mehdi.petreminder.service.ServiceException;
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.time.LocalDate;
import java.util.List;

```

Classes

- class [com.mehdi.petreminder.gui.panels.MedicalPanel](#)
Sağlık kayıtları CRUD paneli.

Packages

- package [com.mehdi.petreminder.gui.panels](#)

8.15.1 Detailed Description

Sağlık kayıtları yönetim paneli.

Definition in file [MedicalPanel.java](#).

8.16 MedicalPanel.java

[Go to the documentation of this file.](#)

```

00001
00005 package com.mehdi.petreminder.gui.panels;
00006
00007 import com.mehdi.petreminder.gui.MainFrame;
00008 import com.mehdi.petreminder.gui.util.Constants;
00009 import com.mehdi.petreminder.model.MedicalRecord;
00010 import com.mehdi.petreminder.service.MedicalRecordService;
00011 import com.mehdi.petreminder.service.ServiceException;
00012
00013 import javax.swing.*;
00014 import javax.swing.border.EmptyBorder;
00015 import javax.swing.table.DefaultTableModel;
00016 import java.awt.*;
00017 import java.time.LocalDate;
00018 import java.util.List;
00019
00028 public class MedicalPanel extends JPanel {
00029
00031     private static final long serialVersionUID = 1L;
00032
00034     private final MainFrame mainFrame;
00035
00037     private final MedicalRecordService medicalService;
00038
00040     private DefaultTableModel tableModel;
00041
00043     private JTable table;
00044
00049     public MedicalPanel(MainFrame mainFrame) {
00050         this.mainFrame = mainFrame;
00051         this.medicalService = new MedicalRecordService();
00052         setBackground(Constants.BG_COLOR);
00053         setLayout(new BorderLayout(Constants.GAP, Constants.GAP));
00054         setBorder(new EmptyBorder(Constants.GAP_LARGE, Constants.GAP_LARGE,
00055             Constants.GAP_LARGE, Constants.GAP_LARGE));
00056         buildUI();
00057     }
00058
00062     private void buildUI() {
00063         JLabel title = new JLabel(" Sağlık Kayıtları");
00064         title.setFont(Constants.TITLE_FONT);
00065         add(title, BorderLayout.NORTH);
00066         add(buildTable(), BorderLayout.CENTER);
00067         add(buildToolBar(), BorderLayout.SOUTH);
00068     }
00069
00073     private JScrollPane buildTable() {
00074         String[] cols = {"ID", "Pet", "Tür", "Teşhis", "Veteriner", "Tarih", "Maliyet ()"};
00075         tableModel = new DefaultTableModel(cols, 0) {
00076             @Override public boolean isCellEditable(int r, int c) { return false; }
00077         };

```

```

00078         table = new JTable(tableModel);
00079         table.setRowHeight(30);
00080         table.setFont(GuiConstants.BODY_FONT);
00081         table.setGridColor(GuiConstants.BORDER_COLOR);
00082         return new JScrollPane(table);
00083     }
00084
00085     private JPanel buildToolbar() {
00086         JPanel bar = new JPanel(new FlowLayout(FlowLayout.LEFT, 8, 0));
00087         bar.setOpaque(false);
00088         bar.add(btn(" Yeni Kayıt", GuiConstants.PRIMARY, e -> addRecord()));
00089         bar.add(btn(" Sil", GuiConstants.DANGER, e -> deleteSelected()));
00090         return bar;
00091     }
00092
00093     private void addRecord() {
00094         JTextField petId = new JTextField("1");
00095         JTextField petName = new JTextField();
00096         JComboBox<String> recType = new JComboBox<>(new String[]{
00097             "KONTROL", "ASII", "AMELIYAT", "HASTALIK", "DIGER"});
00098         JTextField diagnosis = new JTextField();
00099         JTextField vet = new JTextField();
00100         JTextField cost = new JTextField("0");
00101         JTextField date = new JTextField(LocalDate.now().toString());
00102
00103         Object[] fields = {
00104             "Pet ID:", petId, "Pet Adı:", petName,
00105             "Kayıt Türü:", recType, "Teşhis:", diagnosis,
00106             "Veteriner:", vet, "Maliyet ():", cost,
00107             "Tarih (yyyy-MM-dd):", date
00108         };
00109         int res = JOptionPane.showConfirmDialog(this, fields, "Yeni Sağlık Kaydı",
00110             JOptionPane.OK_CANCEL_OPTION);
00111         if (res == JOptionPane.OK_OPTION) {
00112             try {
00113                 MedicalRecord rec = new MedicalRecord(0,
00114                     Integer.parseInt(petId.getText().trim()),
00115                     petName.getText().trim(),
00116                     LocalDate.parse(date.getText().trim()),
00117                     (String) recType.getSelectedItem(),
00118                     diagnosis.getText().trim(),
00119                     vet.getText().trim());
00120                 rec.setCost(Double.parseDouble(cost.getText().trim()));
00121                 medicalService.addRecord(rec);
00122                 refresh();
00123             } catch (Exception ex) {
00124                 JOptionPane.showMessageDialog(this, "Hata: " + ex.getMessage(), "Hata",
00125                     JOptionPane.ERROR_MESSAGE);
00126             }
00127         }
00128     }
00129
00130     private void deleteSelected() {
00131         int row = table.getSelectedRow();
00132         if (row < 0) return;
00133         int confirm = JOptionPane.showConfirmDialog(this, "Silinsin mi?", "Onayla",
00134             JOptionPane.YES_NO_OPTION);
00135         if (confirm == JOptionPane.YES_OPTION) {
00136             try {
00137                 medicalService.deleteRecord((int) tableModel.getValueAt(row, 0));
00138                 refresh();
00139             } catch (ServiceException ex) {
00140                 JOptionPane.showMessageDialog(this, ex.getMessage());
00141             }
00142         }
00143     }
00144
00145     public void refresh() {
00146         tableModel.setRowCount(0);
00147         try {
00148             List<MedicalRecord> list = medicalService.getAllRecords();
00149             for (MedicalRecord r : list) {
00150                 tableModel.addRow(new Object[]{
00151                     r.getId(), r.getPetName(), r.getRecordType(),
00152                     r.getDiagnosis(), r.getVeterinarianName(),
00153                     r.getFormattedDate(),
00154                     String.format("%.2f", r.getCost())
00155                 });
00156             }
00157         } catch (Exception e) { /* sessiz */ }
00158     }
00159
00160     private JButton btn(String text, Color bg, java.awt.event.ActionListener al) {
00161         JButton b = new JButton(text);
00162         if (bg != null) { b.setBackground(bg); b.setForeground(Color.WHITE); }
00163         b.setFocusPainted(false);
00164         b.addActionListener(al);
00165     }

```

```
00180         return b;
00181     }
00182 }
```

8.17 petreminder- app/src/main/java/com/mehdi/petreminder/gui/panels/NavPanel.java File Reference

Sol navigasyon paneli.

```
import com.mehdi.petreminder.gui.MainFrame;
import com.mehdi.petreminder.gui.util.Constants;
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import java.awt.*;
import java.awt.event.*;
import java.util.HashMap;
import java.util.Map;
```

Classes

- class [com.mehdi.petreminder.gui.panels.NavPanel](#)
Sol navigasyon çubuğu — menü butonları.

Packages

- package [com.mehdi.petreminder.gui.panels](#)

8.17.1 Detailed Description

Sol navigasyon paneli.

Member documentation. Member documentation.

Definition in file [NavPanel.java](#).

8.18 NavPanel.java

[Go to the documentation of this file.](#)

```

00001
00011 package com.mehdi.petreminder.gui.panels;
00012
00013 import com.mehdi.petreminder.gui.MainFrame;
00014 import com.mehdi.petreminder.gui.util.Constants;
00015
00016 import javax.swing.*;
00017 import javax.swing.border.EmptyBorder;
00018 import java.awt.*;
00019 import java.awt.event.*;
00020 import java.util.HashMap;
00021 import java.util.Map;
00022
00033 public class NavPanel extends JPanel {
00034
00036     private static final long serialVersionUID = 1L;
00037
00039     private final MainFrame mainFrame;
00040
00042     private final Map<String, JButton> buttons = new HashMap<>();
00043
00045     private String activeCard = "";
00046
00051     public NavPanel(MainFrame mainFrame) {
00052         this.mainFrame = mainFrame;
00053         setBackground(Constants.NAV_COLOR);
00054         setPreferredSize(new Dimension(Constants.NAV_WIDTH, 0));
00055         setLayout(new BorderLayout(this, BorderLayout.Y_AXIS));
00056         buildNav();
00057     }
00058
00062     private void buildNav() {
00063         add(Box.createVerticalStrut(20));
00064         addLogo();
00065         add(Box.createVerticalStrut(24));
00066         addNavButton(" Dashboard", Constants.CARD_DASHBOARD);
00067         addNavButton(" Hayvanlarım", Constants.CARD_PETS);
00068         addNavButton(" Hatırlatıcılar", Constants.CARD_REMINDERS);
00069         addNavButton(" Vet Randevuları", Constants.CARD_VET);
00070         addNavButton(" Sağlık Kayıtları", Constants.CARD_MEDICAL);
00071         add(Box.createVerticalGlue());
00072         addNavButton(" Ayarlar", Constants.CARD_SETTINGS);
00073         add(Box.createVerticalStrut(16));
00074     }
00075
00079     private void addLogo() {
00080         JLabel logo = new JLabel(" PetCare");
00081         logo.setFont(new Font("Segoe UI", Font.BOLD, 17));
00082         logo.setForeground(Color.WHITE);
00083         logo.setAlignmentX(Component.CENTER_ALIGNMENT);
00084         logo.setBorder(new EmptyBorder(0, 0, 0, 0));
00085         add(logo);
00086
00087         JLabel sub = new JLabel("Reminder System");
00088         sub.setFont(Constants.SMALL_FONT);
00089         sub.setForeground(new Color(160, 170, 190));
00090         sub.setAlignmentX(Component.CENTER_ALIGNMENT);
00091         add(sub);
00092     }
00093
00099     private void addNavButton(String label, String cardName) {
00100         JButton btn = new JButton(label);
00101         btn.setMaximumSize(new Dimension(Constants.NAV_WIDTH - 16, 44));
00102         btn.setPreferredSize(new Dimension(Constants.NAV_WIDTH - 16, 44));
00103         btn.setAlignmentX(Component.CENTER_ALIGNMENT);
00104         btn.setHorizontalAlignment(SwingConstants.LEFT);
00105         btn.setBorderPainted(false);
00106         btn.setFocusPainted(false);
00107         btn.setBackground(Constants.NAV_COLOR);
00108         btn.setForeground(new Color(200, 210, 230));
00109         btn.setFont(Constants.NAV_FONT);
00110         btn.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
00111         btn.setBorder(new EmptyBorder(0, 16, 0, 0));
00112
00113         btn.addMouseListener(new MouseAdapter() {
00114             @Override
00115             public void mouseEntered(MouseEvent e) {
00116                 if (!cardName.equals(activeCard)) {
00117                     btn.setBackground(Constants.NAV_HOVER);
00118                 }
00119             }
00120         });

```

```

00121         @Override
00122         public void mouseExited(MouseEvent e) {
00123             if (!cardName.equals(activeCard)) {
00124                 btn.setBackground(GuiConstants.NAV_COLOR);
00125             }
00126         }
00127     });
00128
00129     btn.addActionListener(e -> mainFrame.navigateTo(cardName));
00130     buttons.put(cardName, btn);
00131     add(Box.createVerticalStrut(4));
00132     add(btn);
00133 }
00134
00139 public void setActiveButton(String cardName) {
00140     if (activeCard != null && buttons.containsKey(activeCard)) {
00141         JButton prev = buttons.get(activeCard);
00142         prev.setBackground(GuiConstants.NAV_COLOR);
00143         prev.setForeground(new Color(200, 210, 230));
00144     }
00145     activeCard = cardName;
00146     if (buttons.containsKey(cardName)) {
00147         JButton active = buttons.get(cardName);
00148         active.setBackground(GuiConstants.NAV_ACTIVE);
00149         active.setForeground(Color.WHITE);
00150     }
00151 }
00152 }

```

8.19 petreminder- app/src/main/java/com/mehdi/petreminder/gui/panels/PetsPanel.java File Reference

Evcil hayvanlar yönetim paneli — tam CRUD.

```

import com.mehdi.petreminder.gui.MainFrame;
import com.mehdi.petreminder.gui.dialogs.PetDialog;
import com.mehdi.petreminder.gui.util.GuiConstants;
import com.mehdi.petreminder.model.*;
import com.mehdi.petreminder.service.PetService;
import com.mehdi.petreminder.service.ServiceException;
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.table.*;
import java.awt.*;
import java.util.List;

```

Classes

- class [com.mehdi.petreminder.gui.panels.PetsPanel](#)
Evcil hayvanlar listesi ve CRUD işlemleri.

Packages

- package [com.mehdi.petreminder.gui.panels](#)

8.19.1 Detailed Description

Evcil hayvanlar yönetim paneli — tam CRUD.

Member documentation. [Member documentation.](#)

Definition in file [PetsPanel.java](#).

8.20 PetsPanel.java

[Go to the documentation of this file.](#)

```

00001
00011 package com.mehdi.petreminder.gui.panels;
00012
00013 import com.mehdi.petreminder.gui.MainFrame;
00014 import com.mehdi.petreminder.gui.dialogs.PetDialog;
00015 import com.mehdi.petreminder.gui.util.GuiConstants;
00016 import com.mehdi.petreminder.model.*;
00017 import com.mehdi.petreminder.service.PetService;
00018 import com.mehdi.petreminder.service.ServiceException;
00019
00020 import javax.swing.*;
00021 import javax.swing.border.EmptyBorder;
00022 import javax.swing.table.*;
00023 import java.awt.*;
00024 import java.util.List;
00025
00035 public class PetsPanel extends JPanel {
00036
00038     private static final long serialVersionUID = 1L;
00039
00041     private final MainFrame mainFrame;
00042
00044     private final PetService petService;
00045
00047     private DefaultTableModel tableModel;
00048
00050     private JTable table;
00051
00053     private JTextField searchField;
00054
00056     private JComboBox<String> speciesFilter;
00057
00062     public PetsPanel(MainFrame mainFrame) {
00063         this.mainFrame = mainFrame;
00064         this.petService = new PetService();
00065         setBackground(GuiConstants.BG_COLOR);
00066         setLayout(new BorderLayout(GuiConstants.GAP, GuiConstants.GAP));
00067         setBorder(new EmptyBorder(GuiConstants.GAP_LARGE, GuiConstants.GAP_LARGE,
00068             GuiConstants.GAP_LARGE, GuiConstants.GAP_LARGE));
00069         buildUI();
00070     }
00071
00075     private void buildUI() {
00076         add(buildHeader(), BorderLayout.NORTH);
00077         add(buildTableArea(), BorderLayout.CENTER);
00078         add(buildToolBar(), BorderLayout.SOUTH);
00079     }
00080
00085     private JPanel buildHeader() {
00086         JPanel p = new JPanel(new BorderLayout(GuiConstants.GAP, 0));
00087         p.setOpaque(false);
00088
00089         JLabel title = new JLabel(" Evcil Hayvanlarım");
00090         title.setFont(GuiConstants.TITLE_FONT);
00091         title.setForeground(GuiConstants.TEXT_PRIMARY);
00092         p.add(title, BorderLayout.WEST);
00093
00094         JPanel controls = new JPanel(new FlowLayout(FlowLayout.RIGHT, 8, 0));
00095         controls.setOpaque(false);
00096
00097         searchField = new JTextField(15);
00098         searchField.putClientProperty("JTextField.placeholderText", "Ad ile ara...");
00099         searchField.addActionListener(e -> refresh());
00100
00101         speciesFilter = new JComboBox<>(new String[]{"Tümü", "Dog", "Cat", "Bird"});
00102         speciesFilter.addActionListener(e -> refresh());
00103
00104         controls.add(new JLabel("Tür:"));
00105         controls.add(speciesFilter);
00106         controls.add(searchField);
00107         controls.add(buildIconButton("", "Ara", e -> refresh()));
00108
00109         p.add(controls, BorderLayout.EAST);
00110         return p;
00111     }
00112
00117     private JScrollPane buildTableArea() {
00118         String[] cols = {"ID", "Ad", "Tür", "İrk", "Doğum Tarihi", "Cinsiyet", "Ağırlık (kg)"};
00119         tableModel = new DefaultTableModel(cols, 0) {
00120             @Override
00121             public boolean isCellEditable(int row, int col) { return false; }
00122         };

```

```

00123         table = new JTable(tableModel);
00124         table.setRowHeight(32);
00125         table.setFont(GuiConstants.BODY_FONT);
00126         table.getTableHeader().setFont(new Font("Segoe UI", Font.BOLD, 12));
00127         table.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
00128         table.setGridColor(GuiConstants.BORDER_COLOR);
00129         table.getColumnModel().getColumn(0).setMaxWidth(50);
00130
00131         // Çift tıklama = düzenle
00132         table.addMouseListener(new java.awt.event.MouseAdapter() {
00133             @Override
00134             public void mouseClicked(java.awt.event.MouseEvent e) {
00135                 if (e.getClickCount() == 2) editSelected();
00136             }
00137         });
00138
00139         return new JScrollPane(table);
00140     }
00141
00142     private JPanel buildToolbar() {
00143         JPanel bar = new JPanel(new FlowLayout(FlowLayout.LEFT, 8, 0));
00144         bar.setOpaque(false);
00145         bar.add(buildPrimaryButton(" Yeni Hayvan", e -> addPet()));
00146         bar.add(buildButton(" Düzenle", e -> editSelected()));
00147         bar.add(buildDangerButton(" Sil", e -> deleteSelected()));
00148         return bar;
00149     }
00150
00151     private void addPet() {
00152         PetDialog dlg = new PetDialog(mainFrame, null);
00153         dlg.setVisible(true);
00154         if (dlg.isConfirmed()) {
00155             try {
00156                 petService.addPet(dlg.getPet());
00157                 refresh();
00158                 JOptionPane.showMessageDialog(this, "Hayvan eklendi!", "Başarılı",
00159                     JOptionPane.INFORMATION_MESSAGE);
00160             } catch (ServiceException ex) {
00161                 JOptionPane.showMessageDialog(this, ex.getMessage(), "Hata",
00162                     JOptionPane.ERROR_MESSAGE);
00163             }
00164         }
00165     }
00166
00167     private void editSelected() {
00168         int row = table.getSelectedRow();
00169         if (row < 0) {
00170             JOptionPane.showMessageDialog(this, "Lütfen bir hayvan seçin.");
00171             return;
00172         }
00173         int id = (int) tableModel.getValueAt(row, 0);
00174         try {
00175             Pet pet = petService.getPetById(id);
00176             PetDialog dlg = new PetDialog(mainFrame, pet);
00177             dlg.setVisible(true);
00178             if (dlg.isConfirmed()) {
00179                 petService.updatePet(dlg.getPet());
00180                 refresh();
00181             }
00182         } catch (ServiceException ex) {
00183             JOptionPane.showMessageDialog(this, ex.getMessage(), "Hata",
00184                 JOptionPane.ERROR_MESSAGE);
00185         }
00186     }
00187
00188     private void deleteSelected() {
00189         int row = table.getSelectedRow();
00190         if (row < 0) {
00191             JOptionPane.showMessageDialog(this, "Lütfen bir hayvan seçin.");
00192             return;
00193         }
00194         String name = (String) tableModel.getValueAt(row, 1);
00195         int confirm = JOptionPane.showConfirmDialog(this,
00196             name + " silinsin mi?", "Onayla", JOptionPane.YES_NO_OPTION);
00197         if (confirm == JOptionPane.YES_OPTION) {
00198             int id = (int) tableModel.getValueAt(row, 0);
00199             petService.deletePet(id);
00200             refresh();
00201         }
00202     }
00203
00204     public void refresh() {
00205         tableModel.setRowCount(0);
00206         try {
00207             String search = searchField != null ? searchField.getText().trim() : "";
00208             String species = speciesFilter != null
00209                 ? (String) speciesFilter.getSelectedItem() : "Tümü";

```



```

00226         List<Pet> pets = search.isEmpty() ? petService.getAllPets()
00227             : petService.searchByName(search);
00228         if (!"Tümü".equals(species) && species != null) {
00229             pets = petService.filterBySpecies(species);
00230         }
00231     }
00232
00233     for (Pet p : pets) {
00234         tableModel.addRow(new Object[]{
00235             p.getId(),
00236             p.getName(),
00237             p.getSpecies(),
00238             getBreed(p),
00239             p.getBirthDate() != null ? p.getBirthDate().toString() : "",
00240             p.getGender() != null ? p.getGender() : "",
00241             String.format("%.1f", p.getWeight())
00242         });
00243     }
00244 } catch (Exception e) {
00245     // Veri yoksa sessiz
00246 }
00247 }
00248
00249 private String getBreed(Pet pet) {
00250     if (pet instanceof Dog) return ((Dog) pet).getBreed() != null ? ((Dog) pet).getBreed() : "";
00251     if (pet instanceof Cat) return ((Cat) pet).getBreed() != null ? ((Cat) pet).getBreed() : "";
00252     if (pet instanceof Bird) return ((Bird) pet).getBirdType() != null ? ((Bird)
00253         pet).getBirdType() : "";
00254     return "";
00255 }
00256
00257 // Yardımcı buton oluşturmalar
00258
00259 private JButton buildPrimaryButton(String text, java.awt.event.ActionListener al) {
00260     JButton b = new JButton(text);
00261     b.setBackground(GuiConstants.PRIMARY);
00262     b.setForeground(Color.WHITE);
00263     b.setFocusPainted(false);
00264     b.setFont(GuiConstants.BODY_FONT);
00265     b.addActionListener(al);
00266     return b;
00267 }
00268
00269 private JButton buildButton(String text, java.awt.event.ActionListener al) {
00270     JButton b = new JButton(text);
00271     b.setFocusPainted(false);
00272     b.setFont(GuiConstants.BODY_FONT);
00273     b.addActionListener(al);
00274     return b;
00275 }
00276
00277 private JButton buildDangerButton(String text, java.awt.event.ActionListener al) {
00278     JButton b = new JButton(text);
00279     b.setBackground(GuiConstants.DANGER);
00280     b.setForeground(Color.WHITE);
00281     b.setFocusPainted(false);
00282     b.setFont(GuiConstants.BODY_FONT);
00283     b.addActionListener(al);
00284     return b;
00285 }
00286
00287 private JButton buildIconButton(String icon, String tooltip,
00288     java.awt.event.ActionListener al) {
00289     JButton b = new JButton(icon);
00290     b.setToolTipText(tooltip);
00291     b.setFocusPainted(false);
00292     b.addActionListener(al);
00293     return b;
00294 }
00295
00296 }
00297
00298 }
00299
00300 }
00301
00302 }
00303
00304 }
00305
00306 }
00307
00308 }
00309
00310 }
00311 }

```

8.21 petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/RemindersPanel.java File Reference

Hatırlatıcılar yönetim paneli.

```

import com.mehdi.petreminder.gui.MainFrame;
import com.mehdi.petreminder.gui.dialogs.ReminderDialog;

```

```

import com.mehdi.petreminder.gui.util.GuiConstants;
import com.mehdi.petreminder.model.Reminder;
import com.mehdi.petreminder.service.ReminderService;
import com.mehdi.petreminder.service.ServiceException;
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.util.List;

```

Classes

- class [com.mehdi.petreminder.gui.panels.RemindersPanel](#)
Hatırlatıcılar CRUD paneli.

Packages

- package [com.mehdi.petreminder.gui.panels](#)

8.21.1 Detailed Description

Hatırlatıcılar yönetim paneli.

Definition in file [RemindersPanel.java](#).

8.22 RemindersPanel.java

[Go to the documentation of this file.](#)

```

00001
00005 package com.mehdi.petreminder.gui.panels;
00006
00007 import com.mehdi.petreminder.gui.MainFrame;
00008 import com.mehdi.petreminder.gui.dialogs.ReminderDialog;
00009 import com.mehdi.petreminder.gui.util.GuiConstants;
00010 import com.mehdi.petreminder.model.Reminder;
00011 import com.mehdi.petreminder.service.ReminderService;
00012 import com.mehdi.petreminder.service.ServiceException;
00013
00014 import javax.swing.*;
00015 import javax.swing.border.EmptyBorder;
00016 import javax.swing.table.DefaultTableModel;
00017 import java.awt.*;
00018 import java.util.List;
00019
00028 public class RemindersPanel extends JPanel {
00029
00031     private static final long serialVersionUID = 1L;
00032
00034     private final MainFrame mainFrame;
00035
00037     private final ReminderService reminderService;
00038
00040     private DefaultTableModel tableModel;
00041
00043     private JTable table;
00044
00046     private JCheckBox chkOnlyPending;
00047
00052     public RemindersPanel(MainFrame mainFrame) {
00053         this.mainFrame = mainFrame;
00054         this.reminderService = new ReminderService();

```

```

00055         setBackground(GuiConstants.BG_COLOR);
00056         setLayout(new BorderLayout(GuiConstants.GAP, GuiConstants.GAP));
00057         setBorder(new EmptyBorder(GuiConstants.GAP_LARGE, GuiConstants.GAP_LARGE,
00058             GuiConstants.GAP_LARGE, GuiConstants.GAP_LARGE));
00059         buildUI();
00060     }
00061
00062     private void buildUI() {
00063         add(buildHeader(), BorderLayout.NORTH);
00064         add(buildTable(), BorderLayout.CENTER);
00065         add(buildToolbar(), BorderLayout.SOUTH);
00066     }
00067
00068     private JPanel buildHeader() {
00069         JPanel p = new JPanel(new BorderLayout());
00070         p.setOpaque(false);
00071         JLabel title = new JLabel(" Hatırlatıcılar");
00072         title.setFont(GuiConstants.TITLE_FONT);
00073         p.add(title, BorderLayout.WEST);
00074         chkOnlyPending = new JCheckBox("Sadece bekleyenler");
00075         chkOnlyPending.addActionListener(e -> refresh());
00076         p.add(chkOnlyPending, BorderLayout.EAST);
00077         return p;
00078     }
00079
00080     private JScrollPane buildTable() {
00081         String[] cols = {"ID", "Pet", "Tür", "Öncelik", "Zaman", "Durum"};
00082         tableModel = new DefaultTableModel(cols, 0) {
00083             @Override public boolean isCellEditable(int r, int c) { return false; }
00084         };
00085         table = new JTable(tableModel);
00086         table.setRowHeight(30);
00087         table.setFont(GuiConstants.BODY_FONT);
00088         table.getTableHeader().setFont(new Font("Segoe UI", Font.BOLD, 12));
00089         table.setGridColor(GuiConstants.BORDER_COLOR);
00090         table.addMouseListener(new java.awt.event.MouseAdapter() {
00091             @Override
00092             public void mouseClicked(java.awt.event.MouseEvent e) {
00093                 if (e.getClickCount() == 2) editSelected();
00094             }
00095         });
00096         return new JScrollPane(table);
00097     }
00098
00099     private JPanel buildToolbar() {
00100         JPanel bar = new JPanel(new FlowLayout(FlowLayout.LEFT, 8, 0));
00101         bar.setOpaque(false);
00102         bar.add(btn(" Yeni", GuiConstants.PRIMARY, e -> addReminder()));
00103         bar.add(btn(" Düzenle", null, e -> editSelected()));
00104         bar.add(btn(" Tamamla", GuiConstants.SUCCESS, e -> completeSelected()));
00105         bar.add(btn(" Sil", GuiConstants.DANGER, e -> deleteSelected()));
00106         return bar;
00107     }
00108
00109     private void addReminder() {
00110         ReminderDialog dlg = new ReminderDialog(
00111             (Frame) SwingUtilities.getWindowAncestor(this), null);
00112         dlg.setVisible(true);
00113         if (dlg.isConfirmed()) {
00114             try {
00115                 reminderService.addReminder(dlg.getReminder());
00116                 refresh();
00117             } catch (ServiceException ex) {
00118                 JOptionPane.showMessageDialog(this, ex.getMessage(), "Hata",
00119                     JOptionPane.ERROR_MESSAGE);
00120             }
00121         }
00122     }
00123
00124     private void editSelected() {
00125         int row = table.getSelectedRow();
00126         if (row < 0) return;
00127         int id = (int) tableModel.getValueAt(row, 0);
00128         try {
00129             Reminder r = reminderService.getReminderById(id);
00130             ReminderDialog dlg = new ReminderDialog(
00131                 (Frame) SwingUtilities.getWindowAncestor(this), r);
00132             dlg.setVisible(true);
00133             if (dlg.isConfirmed()) {
00134                 reminderService.updateReminder(dlg.getReminder());
00135                 refresh();
00136             }
00137         } catch (ServiceException ex) {
00138             JOptionPane.showMessageDialog(this, ex.getMessage(), "Hata",
00139                 JOptionPane.ERROR_MESSAGE);
00140         }
00141     }
00142 }

```

```

00160
00164     private void completeSelected() {
00165         int row = table.getSelectedRow();
00166         if (row < 0) {
00167             JOptionPane.showMessageDialog(this, "Lütfen bir hatırlatıcı seçin.");
00168             return;
00169         }
00170         int id = (int) tableModel.getValueAt(row, 0);
00171         String currentStatus = (String) tableModel.getValueAt(row, 5);
00172         if (currentStatus != null && currentStatus.contains("Tamamlandı")) {
00173             JOptionPane.showMessageDialog(this, "Bu hatırlatıcı zaten tamamlandı.");
00174             return;
00175         }
00176         try {
00177             reminderService.markCompleted(id);
00178             refresh();
00179             JOptionPane.showMessageDialog(this, "Hatırlatıcı tamamlandı olarak işaretlendi!",
00180                 "Başarılı", JOptionPane.INFORMATION_MESSAGE);
00181         } catch (Exception ex) {
00182             JOptionPane.showMessageDialog(this, "Hata: " + ex.getMessage(),
00183                 "Hata", JOptionPane.ERROR_MESSAGE);
00184         }
00185     }
00186
00190     private void deleteSelected() {
00191         int row = table.getSelectedRow();
00192         if (row < 0) return;
00193         int confirm = JOptionPane.showConfirmDialog(this, "Silinsin mi?", "Onayla",
00194             JOptionPane.YES_NO_OPTION);
00195         if (confirm == JOptionPane.YES_OPTION) {
00196             reminderService.deleteReminder((int) tableModel.getValueAt(row, 0));
00197             refresh();
00198         }
00199     }
00200
00204     public void refresh() {
00205         tableModel.setRowCount(0);
00206         try {
00207             List<Reminder> list = chkOnlyPending != null && chkOnlyPending.isSelected()
00208                 ? reminderService.getPendingReminders()
00209                 : reminderService.getAllReminders();
00210             for (Reminder r : list) {
00211                 String status = r.isCompleted() ? " Tamamlandı"
00212                     : r.isOverdue() ? " Gecikmiş" : " Bekliyor";
00213                 tableModel.addRow(new Object[]{
00214                     r.getId(), r.getPetName(), r.getReminderType(),
00215                     r.getPriority(), r.getFormattedTime(), status
00216                 });
00217             }
00218         } catch (Exception e) { /* sessiz */ }
00219     }
00220
00224     private JButton btn(String text, Color bg, java.awt.event.ActionListener al) {
00225         JButton b = new JButton(text);
00226         if (bg != null) { b.setBackground(bg); b.setForeground(Color.WHITE); }
00227         b.setFocusPainted(false);
00228         b.addActionListener(al);
00229         return b;
00230     }
00231 }

```

8.23 petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/SettingsPanel.java File Reference

Ayarlar paneli — runtime storage backend değiştirme.

```

import com.mehdi.petreminder.config.StorageConfig;
import com.mehdi.petreminder.config.StorageType;
import com.mehdi.petreminder.gui.MainFrame;
import com.mehdi.petreminder.gui.util.GuiConstants;
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import java.awt.*;

```

Classes

- class [com.mehdi.petreminder.gui.panels.SettingsPanel](#)
Storage backend ve uygulama ayarları.

Packages

- package [com.mehdi.petreminder.gui.panels](#)

8.23.1 Detailed Description

Ayarlar paneli — runtime storage backend değiştirme.

Definition in file [SettingsPanel.java](#).

8.24 SettingsPanel.java

[Go to the documentation of this file.](#)

```

00001
00005 package com.mehdi.petreminder.gui.panels;
00006
00007 import com.mehdi.petreminder.config.StorageConfig;
00008 import com.mehdi.petreminder.config.StorageType;
00009 import com.mehdi.petreminder.gui.MainFrame;
00010 import com.mehdi.petreminder.gui.util.GuiConstants;
00011
00012 import javax.swing.*;
00013 import javax.swing.border.EmptyBorder;
00014 import java.awt.*;
00015
00025 public class SettingsPanel extends JPanel {
00026
00028     private static final long serialVersionUID = 1L;
00029
00031     private final MainFrame mainFrame;
00032
00034     private JComboBox<StorageType> cmbStorage;
00035
00037     private JLabel lblCurrent;
00038
00043     public SettingsPanel(MainFrame mainFrame) {
00044         this.mainFrame = mainFrame;
00045         setBackground(GuiConstants.BG_COLOR);
00046         setLayout(new BorderLayout());
00047         setBorder(new EmptyBorder(GuiConstants.GAP_LARGE, GuiConstants.GAP_LARGE,
00048             GuiConstants.GAP_LARGE, GuiConstants.GAP_LARGE));
00049         buildUI();
00050     }
00051
00055     private void buildUI() {
00056         JLabel title = new JLabel("Ayarlar");
00057         title.setFont(GuiConstants.TITLE_FONT);
00058         add(title, BorderLayout.NORTH);
00059
00060         JPanel center = new JPanel();
00061         center.setLayout(new BoxLayout(center, BoxLayout.Y_AXIS));
00062         center.setOpaque(false);
00063         center.add(Box.createVerticalStrut(32));
00064         center.add(buildStorageSection());
00065         center.add(Box.createVerticalStrut(24));
00066         center.add(buildInfoSection());
00067         add(center, BorderLayout.CENTER);
00068     }
00069
00073     private JPanel buildStorageSection() {
00074         JPanel card = new JPanel(new GridBagLayout());
00075         card.setBackground(GuiConstants.CARD_BG);
00076         card.setBorder(BorderFactory.createCompoundBorder(
00077             BorderFactory.createLineBorder(GuiConstants.BORDER_COLOR),

```

```

00078         new EmptyBorder(24, 24, 24, 24)));
00079     card.setMaximumSize(new Dimension(500, 200));
00080     card.setAlignmentX(Component.LEFT_ALIGNMENT);
00081
00082     GridBagConstraints gbc = new GridBagConstraints();
00083     gbc.insets = new Insets(8, 8, 8, 8);
00084     gbc.anchor = GridBagConstraints.WEST;
00085
00086     JLabel sectionTitle = new JLabel("Storage Backend");
00087     sectionTitle.setFont(GuiConstants.SECTION_FONT);
00088     gbc.gridx = 0; gbc.gridy = 0; gbc.gridwidth = 2;
00089     card.add(sectionTitle, gbc);
00090     gbc.gridwidth = 1;
00091
00092     gbc.gridx = 0; gbc.gridy = 1;
00093     card.add(new JLabel("Mevcut:"), gbc);
00094     lblCurrent = new JLabel(StorageConfig.getActiveBackend().getDisplayName());
00095     lblCurrent.setFont(new Font("Segoe UI", Font.BOLD, 13));
00096     lblCurrent.setForeground(GuiConstants.PRIMARY);
00097     gbc.gridx = 1;
00098     card.add(lblCurrent, gbc);
00099
00100     gbc.gridx = 0; gbc.gridy = 2;
00101     card.add(new JLabel("Değiştir:"), gbc);
00102     cmbStorage = new JComboBox<>(StorageType.values());
00103     cmbStorage.setSelectedItem(StorageConfig.getActiveBackend());
00104     cmbStorage.setPreferredSize(new Dimension(200, 30));
00105     gbc.gridx = 1;
00106     card.add(cmbStorage, gbc);
00107
00108     JButton btnApply = new JButton("Uygula");
00109     btnApply.setBackground(GuiConstants.PRIMARY);
00110     btnApply.setForeground(Color.WHITE);
00111     btnApply.setFocusPainted(false);
00112     btnApply.addActionListener(e -> applyStorage());
00113     gbc.gridx = 1; gbc.gridy = 3;
00114     card.add(btnApply, gbc);
00115
00116     return card;
00117 }
00118
00122 private JPanel buildInfoSection() {
00123     JPanel card = new JPanel(new GridBagLayout());
00124     card.setBackground(GuiConstants.CARD_BG);
00125     card.setBorder(BorderFactory.createCompoundBorder(
00126         BorderFactory.createLineBorder(GuiConstants.BORDER_COLOR),
00127         new EmptyBorder(24, 24, 24, 24)));
00128     card.setMaximumSize(new Dimension(500, 150));
00129     card.setAlignmentX(Component.LEFT_ALIGNMENT);
00130
00131     GridBagConstraints gbc = new GridBagConstraints();
00132     gbc.insets = new Insets(4, 8, 4, 8);
00133     gbc.anchor = GridBagConstraints.WEST;
00134
00135     JLabel infoTitle = new JLabel("Uygulama Bilgileri");
00136     infoTitle.setFont(GuiConstants.SECTION_FONT);
00137     gbc.gridx = 0; gbc.gridy = 0; gbc.gridwidth = 2;
00138     card.add(infoTitle, gbc);
00139     gbc.gridwidth = 1;
00140
00141     addInfoRow(card, gbc, 1, "Proje:", "Pet Care Reminder System v1.0");
00142     addInfoRow(card, gbc, 2, "Ders:", "CEN206 -- OOP Term Project");
00143     addInfoRow(card, gbc, 3, "Takım:", "Mehdi / Ibrahim / Zümre");
00144     addInfoRow(card, gbc, 4, "Üniversite:", "RTEU -- Spring 2026");
00145     return card;
00146 }
00147
00151 private void addInfoRow(JPanel p, GridBagConstraints gbc, int row,
00152     String key, String value) {
00153     gbc.gridx = 0; gbc.gridy = row;
00154     JLabel k = new JLabel(key);
00155     k.setFont(new Font("Segoe UI", Font.BOLD, 12));
00156     k.setForeground(GuiConstants.TEXT_SECONDARY);
00157     p.add(k, gbc);
00158     gbc.gridx = 1;
00159     JLabel v = new JLabel(value);
00160     v.setFont(GuiConstants.BODY_FONT);
00161     p.add(v, gbc);
00162 }
00163
00167 private void applyStorage() {
00168     StorageType selected = (StorageType) cmbStorage.getSelectedItem();
00169     if (selected != null) {
00170         mainFrame.onStorageChanged(selected);
00171         lblCurrent.setText(selected.getDisplayName());
00172     }
00173 }

```

```

00174
00178     public void refresh() {
00179         if (lblCurrent != null) {
00180             lblCurrent.setText(StorageConfig.getActiveBackend().getDisplayName());
00181         }
00182         if (cmbStorage != null) {
00183             cmbStorage.setSelectedItem(StorageConfig.getActiveBackend());
00184         }
00185     }
00186 }

```

8.25 petreminder-app/src/main/java/com/mehdi/petreminder/gui/panels/VetPanel.java File Reference

Veteriner randevuları paneli.

```

import com.mehdi.petreminder.gui.MainFrame;
import com.mehdi.petreminder.gui.util.GuiConstants;
import com.mehdi.petreminder.model.*;
import com.mehdi.petreminder.service.ReminderService;
import com.mehdi.petreminder.service.ServiceException;
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.time.LocalDateTime;
import java.util.List;
import java.util.stream.Collectors;

```

Classes

- class [com.mehdi.petreminder.gui.panels.VetPanel](#)
Veteriner randevuları CRUD.

Packages

- package [com.mehdi.petreminder.gui.panels](#)

8.25.1 Detailed Description

Veteriner randevuları paneli.

Definition in file [VetPanel.java](#).

8.26 VetPanel.java

[Go to the documentation of this file.](#)

```

00001
00005 package com.mehdi.petreminder.gui.panels;
00006
00007 import com.mehdi.petreminder.gui.MainFrame;
00008 import com.mehdi.petreminder.gui.util.Constants;
00009 import com.mehdi.petreminder.model.*;
00010 import com.mehdi.petreminder.service.ReminderService;
00011 import com.mehdi.petreminder.service.ServiceException;
00012
00013 import javax.swing.*;
00014 import javax.swing.border.EmptyBorder;
00015 import javax.swing.table.DefaultTableModel;
00016 import java.awt.*;
00017 import java.time.LocalDateTime;
00018 import java.util.List;
00019 import java.util.stream.Collectors;
00020
00029 public class VetPanel extends JPanel {
00030
00032     private static final long serialVersionUID = 1L;
00033
00035     private final MainFrame mainFrame;
00036
00038     private final ReminderService reminderService;
00039
00041     private DefaultTableModel tableModel;
00042
00044     private JTable table;
00045
00050     public VetPanel(MainFrame mainFrame) {
00051         this.mainFrame = mainFrame;
00052         this.reminderService = new ReminderService();
00053         setBackground(Constants.BG_COLOR);
00054         setLayout(new BorderLayout(Constants.GAP, Constants.GAP));
00055         setBorder(new EmptyBorder(Constants.GAP_LARGE, Constants.GAP_LARGE,
00056             Constants.GAP_LARGE, Constants.GAP_LARGE));
00057         buildUI();
00058     }
00059
00063     private void buildUI() {
00064         JLabel title = new JLabel(" Veteriner Randevuları");
00065         title.setFont(Constants.TITLE_FONT);
00066         add(title, BorderLayout.NORTH);
00067         add(buildTable(), BorderLayout.CENTER);
00068         add(buildToolBar(), BorderLayout.SOUTH);
00069     }
00070
00074     private JScrollPane buildTable() {
00075         String[] cols = {"ID", "Pet", "Veteriner", "Klinik", "Sebep", "Zaman", "Onay"};
00076         tableModel = new DefaultTableModel(cols, 0) {
00077             @Override public boolean isCellEditable(int r, int c) { return false; }
00078         };
00079         table = new JTable(tableModel);
00080         table.setRowHeight(30);
00081         table.setFont(Constants.BODY_FONT);
00082         table.setGridColor(Constants.BORDER_COLOR);
00083         return new JScrollPane(table);
00084     }
00085
00089     private JPanel buildToolBar() {
00090         JPanel bar = new JPanel(new FlowLayout(FlowLayout.LEFT, 8, 0));
00091         bar.setOpaque(false);
00092         bar.add(btn(" Yeni Randevu", Constants.PRIMARY, e -> addAppointment()));
00093         bar.add(btn(" Onayla", Constants.SUCCESS, e -> confirmSelected()));
00094         bar.add(btn(" Sil", Constants.DANGER, e -> deleteSelected()));
00095         return bar;
00096     }
00097
00101     private void addAppointment() {
00102         JTextField petId = new JTextField("1");
00103         JTextField petName = new JTextField();
00104         JTextField vetName = new JTextField();
00105         JTextField clinic = new JTextField();
00106         JTextField reason = new JTextField();
00107         JTextField time = new JTextField(LocalDateTime.now().plusDays(1)
00108             .toString().substring(0, 16));
00109
00110         Object[] fields = {
00111             "Pet ID:", petId, "Pet Adı:", petName,
00112             "Veteriner:", vetName, "Klinik:", clinic,
00113             "Sebep:", reason, "Zaman (yyyy-MM-ddTHH:mm):", time
00114         };

```



```

00115         int res = JOptionPane.showConfirmDialog(this, fields, "Yeni Randevu",
00116             JOptionPane.OK_CANCEL_OPTION);
00117         if (res == JOptionPane.OK_OPTION) {
00118             try {
00119                 VetAppointment va = new VetAppointment(0,
00120                     Integer.parseInt(petId.getText().trim()),
00121                     petName.getText().trim(),
00122                     LocalDateTime.parse(time.getText().trim()),
00123                     vetName.getText().trim(),
00124                     clinic.getText().trim(),
00125                     reason.getText().trim());
00126                 reminderService.addReminder(va);
00127                 refresh();
00128             } catch (Exception ex) {
00129                 JOptionPane.showMessageDialog(this, "Hata: " + ex.getMessage(), "Hata",
00130                     JOptionPane.ERROR_MESSAGE);
00131             }
00132         }
00133     }
00134
00135     private void confirmSelected() {
00136         int row = table.getSelectedRow();
00137         if (row < 0) return;
00138         int id = (int) tableModel.getValueAt(row, 0);
00139         try {
00140             Reminder r = reminderService.getReminderById(id);
00141             if (r instanceof VetAppointment) {
00142                 ((VetAppointment) r).setConfirmed(true);
00143                 reminderService.updateReminder(r);
00144                 refresh();
00145             }
00146         } catch (ServiceException ex) {
00147             JOptionPane.showMessageDialog(this, ex.getMessage());
00148         }
00149     }
00150
00151     private void deleteSelected() {
00152         int row = table.getSelectedRow();
00153         if (row < 0) return;
00154         int confirm = JOptionPane.showConfirmDialog(this, "Silinsin mi?", "Onayla",
00155             JOptionPane.YES_NO_OPTION);
00156         if (confirm == JOptionPane.YES_OPTION) {
00157             reminderService.deleteReminder((int) tableModel.getValueAt(row, 0));
00158             refresh();
00159         }
00160     }
00161
00162     public void refresh() {
00163         tableModel.setRowCount(0);
00164         try {
00165             List<Reminder> all = reminderService.getAllReminders();
00166             List<VetAppointment> vets = all.stream()
00167                 .filter(r -> r instanceof VetAppointment)
00168                 .map(r -> (VetAppointment) r)
00169                 .collect(Collectors.toList());
00170             for (VetAppointment va : vets) {
00171                 tableModel.addRow(new Object[]{
00172                     va.getId(), va.getPetName(), va.getVeterinarianName(),
00173                     va.getClinicName(), va.getReason(), va.getFormattedTime(),
00174                     va.isConfirmed() ? "" : ""
00175                 });
00176             }
00177         } catch (Exception e) { /* sessiz */ }
00178     }
00179
00180     private JButton btn(String text, Color bg, java.awt.event.ActionListener al) {
00181         JButton b = new JButton(text);
00182         if (bg != null) { b.setBackground(bg); b.setForeground(Color.WHITE); }
00183         b.setFocusPainted(false);
00184         b.addActionListener(al);
00185         return b;
00186     }
00187
00188     private JButton btn(String text, Color bg, java.awt.event.ActionListener al) {
00189         JButton b = new JButton(text);
00190         if (bg != null) { b.setBackground(bg); b.setForeground(Color.WHITE); }
00191         b.setFocusPainted(false);
00192         b.addActionListener(al);
00193         return b;
00194     }
00195
00196     private JButton btn(String text, Color bg, java.awt.event.ActionListener al) {
00197         JButton b = new JButton(text);
00198         if (bg != null) { b.setBackground(bg); b.setForeground(Color.WHITE); }
00199         b.setFocusPainted(false);
00200         b.addActionListener(al);
00201         return b;
00202     }

```

8.27 petreminder-app/src/main/java/com/mehdi/petreminder/gui/util/GuiConstants.java File Reference ↵

GUI için renk, font ve boyut sabitleri.

```
import java.awt.*;
```

Classes

- class [com.mehdi.petreminder.gui.util.GuiConstants](#)
GUI bileşenlerinde kullanılan sabitler.

Packages

- package [com.mehdi.petreminder.gui.util](#)

8.27.1 Detailed Description

GUI için renk, font ve boyut sabitleri.

Definition in file [GuiConstants.java](#).

8.28 GuiConstants.java

[Go to the documentation of this file.](#)

```

00001
00008 package com.mehdi.petreminder.gui.util;
00009
00010 import java.awt.*;
00011
00021 public final class GuiConstants {
00022
00024     private GuiConstants() { }
00025
00026     // Renkler
00027
00029     public static final Color NAV_COLOR      = new Color(45, 55, 72);
00030
00032     public static final Color NAV_HOVER     = new Color(74, 85, 104);
00033
00035     public static final Color NAV_ACTIVE    = new Color(99, 102, 241);
00036
00038     public static final Color BG_COLOR      = new Color(248, 250, 252);
00039
00041     public static final Color CARD_BG       = Color.WHITE;
00042
00044     public static final Color PRIMARY       = new Color(99, 102, 241);
00045
00047     public static final Color DANGER        = new Color(239, 68, 68);
00048
00050     public static final Color SUCCESS       = new Color(34, 197, 94);
00051
00053     public static final Color WARNING       = new Color(245, 158, 11);
00054
00056     public static final Color TEXT_PRIMARY  = new Color(17, 24, 39);
00057
00059     public static final Color TEXT_SECONDARY = new Color(107, 114, 128);
00060
00062     public static final Color BORDER_COLOR  = new Color(229, 231, 235);
00063
00064     // Fontlar
00065
00067     public static final Font TITLE_FONT     = new Font("Segoe UI", Font.BOLD, 22);
00068
00070     public static final Font SECTION_FONT   = new Font("Segoe UI", Font.BOLD, 16);
00071
00073     public static final Font BODY_FONT      = new Font("Segoe UI", Font.PLAIN, 13);
00074
00076     public static final Font SMALL_FONT     = new Font("Segoe UI", Font.PLAIN, 11);
00077
00079     public static final Font NAV_FONT       = new Font("Segoe UI", Font.PLAIN, 13);
00080
00081     // Boyutlar
00082
00084     public static final int NAV_WIDTH       = 200;
00085
00087     public static final int GAP             = 12;
00088
00090     public static final int GAP_LARGE      = 24;
00091 }

```

8.29 petreminder-app/src/main/java/com/mehdi/petreminder/model/Bird.java File Reference ↩

Kus turunu temsil eden somut sınıf.

```
import java.time.LocalDate;
```

Classes

- class [com.mehdi.petreminder.model.Bird](#)
Kus evcil hayvan sınıfı.

Packages

- package [com.mehdi.petreminder.model](#)

8.29.1 Detailed Description

Kus turunu temsil eden somut sınıf.

Definition in file [Bird.java](#).

8.30 Bird.java

[Go to the documentation of this file.](#)

```
00001
00008 package com.mehdi.petreminder.model;
00009
00010 import java.time.LocalDate;
00011
00020 public class Bird extends Pet {
00021
00023     private static final long serialVersionUID = 1L;
00024
00026     private String birdType;
00027
00029     private boolean canTalk;
00030
00032     public Bird() { super(); }
00033
00041     public Bird(int id, String name, LocalDate birthDate, int ownerId) {
00042         super(id, name, "Kus", birthDate, ownerId, "Erkek", 0.0, "");
00043         this.birdType = "Muhabbet Kusu";
00044         this.canTalk = false;
00045     }
00046
00059     public Bird(int id, String name, LocalDate birthDate, int ownerId,
00060                 String gender, double weight, String notes,
00061                 String birdType, boolean canTalk) {
00062         super(id, name, "Kus", birthDate, ownerId, gender, weight, notes);
00063         this.birdType = birdType;
00064         this.canTalk = canTalk;
00065     }
00066
00068     public String getBirdType() { return birdType; }
00069
00071     public void setBirdType(String birdType) { this.birdType = birdType; }
00072
00074     public boolean isCanTalk() { return canTalk; }
00075
```

```

00077     public void setCanTalk(boolean canTalk) { this.canTalk = canTalk; }
00078
00083     @Override
00084     public String makeSound() {
00085         return canTalk ? "Merhaba!" : "Cik Cik!";
00086     }
00087
00092     @Override
00093     public String getIconName() { return "bird"; }
00094
00099     @Override
00100     public String toString() {
00101         return String.format("Bird{id=%d, name='%s', type='%s', age=%s, canTalk=%b}",
00102             getId(), getName(), birdType, getAgeString(), canTalk);
00103     }
00104 }

```

8.31 petreminder-app/src/main/java/com/mehdi/petreminder/model/Cat.java File Reference

Kedi turunu temsil eden somut sınıf.

```
import java.time.LocalDate;
```

Classes

- class [com.mehdi.petreminder.model.Cat](#)
Kedi evcil hayvan sınıfı.

Packages

- package [com.mehdi.petreminder.model](#)

8.31.1 Detailed Description

Kedi turunu temsil eden somut sınıf.

Definition in file [Cat.java](#).

8.32 Cat.java

[Go to the documentation of this file.](#)

```

00001
00005 package com.mehdi.petreminder.model;
00006
00007 import java.time.LocalDate;
00008
00017 public class Cat extends Pet {
00018
00020     private static final long serialVersionUID = 1L;
00021
00023     private String breed;
00024
00026     private boolean indoor;
00027
00029     public Cat() { super(); }
00030

```

```

00038     public Cat(int id, String name, LocalDate birthDate, int ownerId) {
00039         super(id, name, "Kedi", birthDate, ownerId, "Disi", 0.0, "");
00040         this.breed = "Karisik";
00041         this.indoor = true;
00042     }
00043
00056     public Cat(int id, String name, LocalDate birthDate, int ownerId,
00057         String gender, double weight, String notes,
00058         String breed, boolean indoor) {
00059         super(id, name, "Kedi", birthDate, ownerId, gender, weight, notes);
00060         this.breed = breed;
00061         this.indoor = indoor;
00062     }
00063
00065     public String getBreed() { return breed; }
00066
00068     public void setBreed(String breed) { this.breed = breed; }
00069
00071     public boolean isIndoor() { return indoor; }
00072
00074     public void setIndoor(boolean indoor) { this.indoor = indoor; }
00075
00080     @Override
00081     public String makeSound() { return "Miyav!"; }
00082
00087     @Override
00088     public String getIconName() { return "cat"; }
00089
00094     @Override
00095     public String toString() {
00096         return String.format("Cat{id=%d, name='%s', breed='%s', age=%s, indoor=%b}",
00097             getId(), getName(), breed, getAgeString(), indoor);
00098     }
00099 }

```

8.33 petreminder-app/src/main/java/com/mehdi/petreminder/model/Dog.java File Reference ↩

Kopek turunu temsil eden somut sınıf.

```
import java.time.LocalDate;
```

Classes

- class [com.mehdi.petreminder.model.Dog](#)
Kopek evcil hayvan sınıfı.

Packages

- package [com.mehdi.petreminder.model](#)

8.33.1 Detailed Description

Kopek turunu temsil eden somut sınıf.

Definition in file [Dog.java](#).

8.34 Dog.java

[Go to the documentation of this file.](#)

```

00001
00005 package com.mehdi.petreminder.model;
00006
00007 import java.time.LocalDate;
00008
00017 public class Dog extends Pet {
00018
00020     private static final long serialVersionUID = 1L;
00021
00023     private String breed;
00024
00026     private boolean trained;
00027
00029     public Dog() { super(); }
00030
00038     public Dog(int id, String name, LocalDate birthDate, int ownerId) {
00039         super(id, name, "Kopek", birthDate, ownerId, "Erkek", 0.0, "");
00040         this.breed = "Karisik";
00041         this.trained = false;
00042     }
00043
00056     public Dog(int id, String name, LocalDate birthDate, int ownerId,
00057         String gender, double weight, String notes,
00058         String breed, boolean trained) {
00059         super(id, name, "Kopek", birthDate, ownerId, gender, weight, notes);
00060         this.breed = breed;
00061         this.trained = trained;
00062     }
00063
00065     public String getBreed() { return breed; }
00066
00068     public void setBreed(String breed) { this.breed = breed; }
00069
00071     public boolean isTrained() { return trained; }
00072
00074     public void setTrained(boolean trained) { this.trained = trained; }
00075
00080     @Override
00081     public String makeSound() { return "Hav! Hav!"; }
00082
00087     @Override
00088     public String getIconName() { return "dog"; }
00089
00094     @Override
00095     public String toString() {
00096         return String.format("Dog{id=%d, name='%s', breed='%s', age=%s, trained=%b}",
00097             getId(), getName(), breed, getAgeString(), trained);
00098     }
00099 }

```

8.35 petreminder-app/src/main/java/com/mehdi/petreminder/model/ExerciseReminder.java File Reference

Egzersiz hatırlatıcısı sınıfı.

```
import java.time.LocalDateTime;
```

Classes

- class [com.mehdi.petreminder.model.ExerciseReminder](#)
Evcil hayvan egzersiz hatırlatıcısı.

Packages

- package [com.mehdi.petreminder.model](#)

8.35.1 Detailed Description

Egzersiz hatırlatıcısı sınıfı.

Definition in file [ExerciseReminder.java](#).

8.36 ExerciseReminder.java

[Go to the documentation of this file.](#)

```

00001
00005 package com.mehdi.petreminder.model;
00006
00007 import java.time.LocalDateTime;
00008
00016 public class ExerciseReminder extends Reminder {
00017
00019     private static final long serialVersionUID = 1L;
00020
00022     private String exerciseType;
00023
00025     private int durationMinutes;
00026
00028     private double distanceKm;
00029
00031     public ExerciseReminder() { super(); }
00032
00042     public ExerciseReminder(int id, int petId, String petName,
00043                             LocalDateTime scheduledTime,
00044                             String exerciseType, int durationMinutes) {
00045         super(id, petId, petName, scheduledTime,
00046             "Egzersiz: " + exerciseType, "ORTA", false, "");
00047         this.exerciseType = exerciseType;
00048         this.durationMinutes = durationMinutes;
00049         this.distanceKm = 0.0;
00050     }
00051
00066     public ExerciseReminder(int id, int petId, String petName,
00067                             LocalDateTime scheduledTime, String description,
00068                             String priority, boolean recurring,
00069                             String recurrencePattern, String exerciseType,
00070                             int durationMinutes, double distanceKm) {
00071         super(id, petId, petName, scheduledTime, description,
00072             priority, recurring, recurrencePattern);
00073         this.exerciseType = exerciseType;
00074         this.durationMinutes = durationMinutes;
00075         this.distanceKm = distanceKm;
00076     }
00077
00079     public String getExerciseType() { return exerciseType; }
00080
00082     public void setExerciseType(String exerciseType) { this.exerciseType = exerciseType; }
00083
00085     public int getDurationMinutes() { return durationMinutes; }
00086
00088     public void setDurationMinutes(int durationMinutes) {
00089         this.durationMinutes = durationMinutes;
00090     }
00091
00093     public double getDistanceKm() { return distanceKm; }
00094
00096     public void setDistanceKm(double distanceKm) { this.distanceKm = distanceKm; }
00097
00099     @Override
00100     public String getReminderType() { return "Egzersiz"; }
00101
00106     @Override
00107     public boolean validate() {
00108         return exerciseType != null && !exerciseType.trim().isEmpty()
00109             && durationMinutes > 0;
00110     }
00111
00116     @Override
00117     public String getSummary() {
00118         String s = String.format("Egzersiz: %s %d dk", exerciseType, durationMinutes);
00119         if (distanceKm > 0) s += String.format(" (%.1f km)", distanceKm);
00120         return s;
00121     }
00122

```

```

00124     @Override
00125     public String toString() {
00126         return String.format("ExerciseReminder{id=%d, pet='%s', type='%s', %ddk}",
00127             getId(), getPetName(), exerciseType, durationMinutes);
00128     }
00129 }

```

8.37 petreminder-app/src/main/java/com/mehdi/petreminder/model/↵ FeedingReminder.java File Reference

Besleme hatırlatıcısı sınıfı.

```
import java.time.LocalDateTime;
```

Classes

- class [com.mehdi.petreminder.model.FeedingReminder](#)
Evcil hayvan besleme hatırlatıcısı.

Packages

- package [com.mehdi.petreminder.model](#)

8.37.1 Detailed Description

Besleme hatırlatıcısı sınıfı.

Definition in file [FeedingReminder.java](#).

8.38 FeedingReminder.java

[Go to the documentation of this file.](#)

```

00001
00005 package com.mehdi.petreminder.model;
00006
00007 import java.time.LocalDateTime;
00008
00017 public class FeedingReminder extends Reminder {
00018
00020     private static final long serialVersionUID = 1L;
00021
00023     private String foodType;
00024
00026     private double portionGrams;
00027
00029     private boolean includesWaterChange;
00030
00032     public FeedingReminder() { super(); }
00033
00043     public FeedingReminder(int id, int petId, String petName,
00044         LocalDateTime scheduledTime,
00045         String foodType, double portionGrams) {
00046         super(id, petId, petName, scheduledTime,
00047             "Besleme: " + foodType, "ORTA", false, "");
00048         this.foodType = foodType;
00049         this.portionGrams = portionGrams;
00050         this.includesWaterChange = false;

```



```

00051     }
00052
00067     public FeedingReminder(int id, int petId, String petName,
00068                           LocalDateTime scheduledTime, String description,
00069                           String priority, boolean recurring,
00070                           String recurrencePattern, String foodType,
00071                           double portionGrams, boolean includesWaterChange) {
00072         super(id, petId, petName, scheduledTime, description,
00073             priority, recurring, recurrencePattern);
00074         this.foodType = foodType;
00075         this.portionGrams = portionGrams;
00076         this.includesWaterChange = includesWaterChange;
00077     }
00078
00080     public String getFoodType() { return foodType; }
00081
00083     public void setFoodType(String foodType) { this.foodType = foodType; }
00084
00086     public double getPortionGrams() { return portionGrams; }
00087
00089     public void setPortionGrams(double portionGrams) { this.portionGrams = portionGrams; }
00090
00092     public boolean isIncludesWaterChange() { return includesWaterChange; }
00093
00095     public void setIncludesWaterChange(boolean includesWaterChange) {
00096         this.includesWaterChange = includesWaterChange;
00097     }
00098
00103     @Override
00104     public String getReminderType() { return "Besleme"; }
00105
00110     @Override
00111     public boolean validate() {
00112         return foodType != null && !foodType.trim().isEmpty() && portionGrams > 0;
00113     }
00114
00119     @Override
00120     public String getSummary() {
00121         return String.format("Besleme: %s (%.0fg)%s",
00122             foodType, portionGrams,
00123             includesWaterChange ? " + Su değişimi" : "");
00124     }
00125
00130     @Override
00131     public String toString() {
00132         return String.format("FeedingReminder{id=%d, pet='%s', food='%s', portion=%.0fg}",
00133             getId(), getPetName(), foodType, portionGrams);
00134     }
00135 }

```

8.39 petreminder-app/src/main/java/com/mehdi/petreminder/model/GroomingReminder.java File Reference ↩

Tıraş/bakım hatırlatıcısı sınıfı.

```
import java.time.LocalDateTime;
```

Classes

- class [com.mehdi.petreminder.model.GroomingReminder](#)
Evcil hayvan tıraş ve bakım hatırlatıcısı.

Packages

- package [com.mehdi.petreminder.model](#)

8.39.1 Detailed Description

Tıraş/bakım hatırlatıcısı sınıfı.

Definition in file [GroomingReminder.java](#).

8.40 GroomingReminder.java

[Go to the documentation of this file.](#)

```

00001
00005 package com.mehdi.petreminder.model;
00006
00007 import java.time.LocalDateTime;
00008
00016 public class GroomingReminder extends Reminder {
00017
00019     private static final long serialVersionUID = 1L;
00020
00022     private String groomingType;
00023
00025     private boolean professional;
00026
00028     private String groomerName;
00029
00031     public GroomingReminder() { super(); }
00032
00042     public GroomingReminder(int id, int petId, String petName,
00043                             LocalDateTime scheduledTime,
00044                             String groomingType, boolean professional) {
00045         super(id, petId, petName, scheduledTime,
00046               "Bakim: " + groomingType, "DUSUK", false, "");
00047         this.groomingType = groomingType;
00048         this.professional = professional;
00049         this.groomerName = "";
00050     }
00051
00066     public GroomingReminder(int id, int petId, String petName,
00067                             LocalDateTime scheduledTime, String description,
00068                             String priority, boolean recurring,
00069                             String recurrencePattern, String groomingType,
00070                             boolean professional, String groomerName) {
00071         super(id, petId, petName, scheduledTime, description,
00072               priority, recurring, recurrencePattern);
00073         this.groomingType = groomingType;
00074         this.professional = professional;
00075         this.groomerName = groomerName;
00076     }
00077
00079     public String getGroomingType() { return groomingType; }
00080
00082     public void setGroomingType(String groomingType) { this.groomingType = groomingType; }
00083
00085     public boolean isProfessional() { return professional; }
00086
00088     public void setProfessional(boolean professional) { this.professional = professional; }
00089
00091     public String getGroomerName() { return groomerName; }
00092
00094     public void setGroomerName(String groomerName) { this.groomerName = groomerName; }
00095
00097     @Override
00098     public String getReminderType() { return "Bakim"; }
00099
00104     @Override
00105     public boolean validate() {
00106         return groomingType != null && !groomingType.trim().isEmpty();
00107     }
00108
00113     @Override
00114     public String getSummary() {
00115         return String.format("Bakim: %s%s",
00116                               groomingType,
00117                               professional ? " (Profesyonel)" : "",
00118                               groomerName != null && !groomerName.isEmpty() ? " - " + groomerName : "");
00119     }
00120
00122     @Override
00123     public String toString() {
00124         return String.format("GroomingReminder{id=%d, pet='%s', type='%s', prof=%b}",
00125                               getId(), getPetName(), groomingType, professional);
00126     }
00127 }

```

8.41 petreminder-app/src/main/java/com/mehdi/petreminder/model/MedicalRecord.java File Reference ↩

Evciil hayvan sađlık kaydı modeli.

```
import java.io.Serializable;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
```

Classes

- class [com.mehdi.petreminder.model.MedicalRecord](#)
Evciil hayvanın sađlık kaydı.

Packages

- package [com.mehdi.petreminder.model](#)

8.41.1 Detailed Description

Evciil hayvan sađlık kaydı modeli.

Composition ilişkisi: MedicalRecord Pet'e ait.

Definition in file [MedicalRecord.java](#).

8.42 MedicalRecord.java

[Go to the documentation of this file.](#)

```
00001
00006 package com.mehdi.petreminder.model;
00007
00008 import java.io.Serializable;
00009 import java.time.LocalDate;
00010 import java.time.format.DateTimeFormatter;
00011
00021 public class MedicalRecord implements Serializable {
00022
00024     private static final long serialVersionUID = 1L;
00025
00027     private static final DateTimeFormatter DATE_FMT =
00028         DateTimeFormatter.ofPattern("dd/MM/yyyy");
00029
00031     private int id;
00032
00034     private int petId;
00035
00037     private String petName;
00038
00040     private LocalDate recordDate;
00041
00046     private String recordType;
00047
00049     private String diagnosis;
00050
00052     private String treatment;
00053
00055     private String veterinarianName;
00056
```

```

00058     private double cost;
00059
00061     private LocalDate nextCheckDate;
00062
00064     private String notes;
00065
00067     private String vaccineName;
00068
00070     public MedicalRecord() {}
00071
00082     public MedicalRecord(int id, int petId, String petName,
00083                          LocalDate recordDate, String recordType,
00084                          String diagnosis, String veterinarianName) {
00085         this.id = id;
00086         this.petId = petId;
00087         this.petName = petName;
00088         this.recordDate = recordDate;
00089         this.recordType = recordType;
00090         this.diagnosis = diagnosis;
00091         this.veterinarianName = veterinarianName;
00092         this.treatment = "";
00093         this.cost = 0.0;
00094         this.notes = "";
00095         this.vaccineName = "";
00096     }
00097
00099     public int getId() { return id; }
00100
00102     public void setId(int id) { this.id = id; }
00103
00105     public int getPetId() { return petId; }
00106
00108     public void setPetId(int petId) { this.petId = petId; }
00109
00111     public String getPetName() { return petName; }
00112
00114     public void setPetName(String petName) { this.petName = petName; }
00115
00117     public LocalDate getRecordDate() { return recordDate; }
00118
00120     public void setRecordDate(LocalDate recordDate) { this.recordDate = recordDate; }
00121
00123     public String getRecordType() { return recordType; }
00124
00126     public void setRecordType(String recordType) { this.recordType = recordType; }
00127
00129     public String getDiagnosis() { return diagnosis; }
00130
00132     public void setDiagnosis(String diagnosis) { this.diagnosis = diagnosis; }
00133
00135     public String getTreatment() { return treatment; }
00136
00138     public void setTreatment(String treatment) { this.treatment = treatment; }
00139
00141     public String getVeterinarianName() { return veterinarianName; }
00142
00144     public void setVeterinarianName(String veterinarianName) {
00145         this.veterinarianName = veterinarianName;
00146     }
00147
00149     public double getCost() { return cost; }
00150
00152     public void setCost(double cost) { this.cost = cost; }
00153
00155     public LocalDate getNextCheckDate() { return nextCheckDate; }
00156
00158     public void setNextCheckDate(LocalDate nextCheckDate) {
00159         this.nextCheckDate = nextCheckDate;
00160     }
00161
00163     public String getNotes() { return notes; }
00164
00166     public void setNotes(String notes) { this.notes = notes; }
00167
00169     public String getVaccineName() { return vaccineName; }
00170
00172     public void setVaccineName(String vaccineName) { this.vaccineName = vaccineName; }
00173
00178     public String getFormattedDate() {
00179         if (recordDate == null) return "Belirsiz";
00180         return recordDate.format(DATE_FMT);
00181     }
00182
00187     @Override
00188     public String toString() {
00189         return String.format("MedicalRecord{id=%d, pet='%s', type='%s', date='%s'}",
00190                             id, petName, recordType, getFormattedDate());

```

```
00191     }
00192
00193     @Override
00194     public boolean equals(Object obj) {
00200         if (this == obj) return true;
00201         if (obj == null || !(obj instanceof MedicalRecord)) return false;
00202         MedicalRecord other = (MedicalRecord) obj;
00203         return this.id == other.id;
00204     }
00205
00206     @Override
00207     public int hashCode() {
00211         return Integer.hashCode(id);
00212     }
00213 }
00214 }
```

8.43 petreminder-app/src/main/java/com/mehdi/petreminder/model/MedicationReminder.java File Reference ↩

İlaç hatırlatıcısı sınıfı.

```
import java.time.LocalDateTime;
```

Classes

- class [com.mehdi.petreminder.model.MedicationReminder](#)
Evcil hayvan ilaç hatırlatıcısı.

Packages

- package [com.mehdi.petreminder.model](#)

8.43.1 Detailed Description

İlaç hatırlatıcısı sınıfı.

Definition in file [MedicationReminder.java](#).

8.44 MedicationReminder.java

[Go to the documentation of this file.](#)

```
00001
00005 package com.mehdi.petreminder.model;
00006
00007 import java.time.LocalDateTime;
00008
00016 public class MedicationReminder extends Reminder {
00017
00019     private static final long serialVersionUID = 1L;
00020
00022     private String medicationName;
00023
00025     private double dosage;
00026
00028     private String dosageUnit;
00029
00031     private boolean prescribed;
```

```

00032
00034     private String veterinarianName;
00035
00037     public MedicationReminder() { super(); }
00038
00049     public MedicationReminder(int id, int petId, String petName,
00050                               LocalDateTime scheduledTime,
00051                               String medicationName, double dosage,
00052                               String dosageUnit) {
00053         super(id, petId, petName, scheduledTime,
00054               "Ilaç: " + medicationName, "YUKSEK", false, "");
00055         this.medicationName = medicationName;
00056         this.dosage = dosage;
00057         this.dosageUnit = dosageUnit;
00058         this.prescribed = false;
00059         this.veterinarianName = "";
00060     }
00061
00078     public MedicationReminder(int id, int petId, String petName,
00079                               LocalDateTime scheduledTime, String description,
00080                               String priority, boolean recurring,
00081                               String recurrencePattern, String medicationName,
00082                               double dosage, String dosageUnit,
00083                               boolean prescribed, String veterinarianName) {
00084         super(id, petId, petName, scheduledTime, description,
00085               priority, recurring, recurrencePattern);
00086         this.medicationName = medicationName;
00087         this.dosage = dosage;
00088         this.dosageUnit = dosageUnit;
00089         this.prescribed = prescribed;
00090         this.veterinarianName = veterinarianName;
00091     }
00092
00094     public String getMedicationName() { return medicationName; }
00095
00097     public void setMedicationName(String medicationName) {
00098         this.medicationName = medicationName;
00099     }
00100
00102     public double getDosage() { return dosage; }
00103
00105     public void setDosage(double dosage) { this.dosage = dosage; }
00106
00108     public String getDosageUnit() { return dosageUnit; }
00109
00111     public void setDosageUnit(String dosageUnit) { this.dosageUnit = dosageUnit; }
00112
00114     public boolean isPrescribed() { return prescribed; }
00115
00117     public void setPrescribed(boolean prescribed) { this.prescribed = prescribed; }
00118
00120     public String getVeterinarianName() { return veterinarianName; }
00121
00123     public void setVeterinarianName(String veterinarianName) {
00124         this.veterinarianName = veterinarianName;
00125     }
00126
00128     @Override
00129     public String getReminderType() { return "Ilaç"; }
00130
00135     @Override
00136     public boolean validate() {
00137         return medicationName != null && !medicationName.trim().isEmpty()
00138             && dosage > 0
00139             && dosageUnit != null && !dosageUnit.trim().isEmpty();
00140     }
00141
00146     @Override
00147     public String getSummary() {
00148         return String.format("Ilaç: %s %.1f%s%s",
00149                               medicationName, dosage, dosageUnit,
00150                               prescribed ? " (Receteli)" : "");
00151     }
00152
00157     @Override
00158     public String toString() {
00159         return String.format("MedicationReminder{id=%d, pet='%s', med='%s', dose=%.1f%s}",
00160                               getId(), getPetName(), medicationName, dosage, dosageUnit);
00161     }
00162 }

```

8.45 petreminder-app/src/main/java/com/mehdi/petreminder/model/↵ Pet.java File Reference

Tum evcil hayvan turleri icin soyut temel sinif.

```
import java.io.Serializable;
import java.time.LocalDate;
import java.time.Period;
```

Classes

- class [com.mehdi.petreminder.model.Pet](#)
Evcil hayvanin temel soyut sinifi.

Packages

- package [com.mehdi.petreminder.model](#)

8.45.1 Detailed Description

Tum evcil hayvan turleri icin soyut temel sinif.

Encapsulation ve abstraction prensiplerini uygular. Tum pet-specific alt siniflar bu siniften turetilir.

Definition in file [Pet.java](#).

8.46 Pet.java

[Go to the documentation of this file.](#)

```
00001
00007 package com.mehdi.petreminder.model;
00008
00009 import java.io.Serializable;
00010 import java.time.LocalDate;
00011 import java.time.Period;
00012
00022 public abstract class Pet implements Serializable {
00023
00025     private static final long serialVersionUID = 1L;
00026
00028     private int id;
00029
00031     private String name;
00032
00034     private String species;
00035
00037     private LocalDate birthDate;
00038
00040     private int ownerId;
00041
00043     private String gender;
00044
00046     private double weight;
00047
00049     private String notes;
00050
00055     protected Pet() {}
00056
00068     protected Pet(int id, String name, String species, LocalDate birthDate,
```

```

00069         int ownerId, String gender, double weight, String notes) {
00070             this.id = id;
00071             this.name = name;
00072             this.species = species;
00073             this.birthDate = birthDate;
00074             this.ownerId = ownerId;
00075             this.gender = gender;
00076             this.weight = weight;
00077             this.notes = notes;
00078         }
00079
00080         // Getter / Setter
00081
00083         public int getId() { return id; }
00084
00086         public void setId(int id) { this.id = id; }
00087
00089         public String getName() { return name; }
00090
00092         public void setName(String name) { this.name = name; }
00093
00095         public String getSpecies() { return species; }
00096
00098         public void setSpecies(String species) { this.species = species; }
00099
00101         public LocalDate getBirthDate() { return birthDate; }
00102
00104         public void setBirthDate(LocalDate birthDate) { this.birthDate = birthDate; }
00105
00107         public int getOwnerId() { return ownerId; }
00108
00110         public void setOwnerId(int ownerId) { this.ownerId = ownerId; }
00111
00113         public String getGender() { return gender; }
00114
00116         public void setGender(String gender) { this.gender = gender; }
00117
00119         public double getWeight() { return weight; }
00120
00122         public void setWeight(double weight) { this.weight = weight; }
00123
00125         public String getNotes() { return notes; }
00126
00128         public void setNotes(String notes) { this.notes = notes; }
00129
00130         // Abstract Metodlar (Polymorphism)
00131
00137         public abstract String makeSound();
00138
00143         public abstract String getIconName();
00144
00150         public int getAge() {
00151             if (birthDate == null) return -1;
00152             return Period.between(birthDate, LocalDate.now()).getYears();
00153         }
00154
00159         public String getAgeString() {
00160             if (birthDate == null) return "Bilinmiyor";
00161             Period p = Period.between(birthDate, LocalDate.now());
00162             return p.getYears() + " yıl " + p.getMonths() + " ay";
00163         }
00164
00169         @Override
00170         public String toString() {
00171             return String.format("Pet{id=%d, name='%s', species='%s', age=%s}",
00172                 id, name, species, getAgeString());
00173         }
00174
00180         @Override
00181         public boolean equals(Object obj) {
00182             if (this == obj) return true;
00183             if (obj == null || !(obj instanceof Pet)) return false;
00184             Pet other = (Pet) obj;
00185             return this.id == other.id;
00186         }
00187
00192         @Override
00193         public int hashCode() {
00194             return Integer.hashCode(id);
00195         }
00196     }

```


8.47 petreminder-app/src/main/java/com/mehdi/petreminder/model/Reminder.java File Reference ↩

Tüm hatırlatıcı türleri için soyut temel sınıf.

```
import java.io.Serializable;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
```

Classes

- class [com.mehdi.petreminder.model.Reminder](#)
Evcil hayvan bakım hatırlatıcısının soyut temel sınıfı. ↩

Packages

- package [com.mehdi.petreminder.model](#)

8.47.1 Detailed Description

Tüm hatırlatıcı türleri için soyut temel sınıf.

OOP Abstraction ve Polymorphism temel sınıfı. Serializable: BinaryRepository için zorunlu.

Definition in file [Reminder.java](#).

8.48 Reminder.java

[Go to the documentation of this file.](#)

```
00001
00007 package com.mehdi.petreminder.model;
00008
00009 import java.io.Serializable;
00010 import java.time.LocalDateTime;
00011 import java.time.format.DateTimeFormatter;
00012
00022 public abstract class Reminder implements Serializable {
00023
00025     private static final long serialVersionUID = 1L;
00026
00028     protected static final DateTimeFormatter FORMATTER =
00029         DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm");
00030
00032     private int id;
00033
00035     private int petId;
00036
00038     private String petName;
00039
00041     private LocalDateTime scheduledTime;
00042
00044     private boolean completed;
00045
00047     private String description;
00048
00050     private String priority;
00051
00053     private boolean recurring;
00054
```

```

00056     private String recurrencePattern;
00057
00059     protected Reminder() {}
00060
00072     protected Reminder(int id, int petId, String petName,
00073                         LocalDateTime scheduledTime, String description,
00074                         String priority, boolean recurring, String recurrencePattern) {
00075         this.id = id;
00076         this.petId = petId;
00077         this.petName = petName;
00078         this.scheduledTime = scheduledTime;
00079         this.completed = false;
00080         this.description = description;
00081         this.priority = priority;
00082         this.recurring = recurring;
00083         this.recurrencePattern = recurrencePattern;
00084     }
00085
00086     // Getters / Setters
00087
00089     public int getId() { return id; }
00090
00092     public void setId(int id) { this.id = id; }
00093
00095     public int getPetId() { return petId; }
00096
00098     public void setPetId(int petId) { this.petId = petId; }
00099
00101     public String getPetName() { return petName; }
00102
00104     public void setPetName(String petName) { this.petName = petName; }
00105
00107     public LocalDateTime getScheduledTime() { return scheduledTime; }
00108
00110     public void setScheduledTime(LocalDateTime scheduledTime) {
00111         this.scheduledTime = scheduledTime;
00112     }
00113
00115     public boolean isCompleted() { return completed; }
00116
00118     public void setCompleted(boolean completed) { this.completed = completed; }
00119
00121     public String getDescription() { return description; }
00122
00124     public void setDescription(String description) { this.description = description; }
00125
00127     public String getPriority() { return priority; }
00128
00130     public void setPriority(String priority) { this.priority = priority; }
00131
00133     public boolean isRecurring() { return recurring; }
00134
00136     public void setRecurring(boolean recurring) { this.recurring = recurring; }
00137
00139     public String getRecurrencePattern() { return recurrencePattern; }
00140
00142     public void setRecurrencePattern(String recurrencePattern) {
00143         this.recurrencePattern = recurrencePattern;
00144     }
00145
00146     // Abstract Metodlar (Polymorphism)
00147
00153     public abstract String getReminderType();
00154
00160     public abstract boolean validate();
00161
00166     public abstract String getSummary();
00167
00168     // Yardımcı Metodlar
00169
00174     public String getFormattedTime() {
00175         if (scheduledTime == null) return "Belirsiz";
00176         return scheduledTime.format(FORMATTER);
00177     }
00178
00183     public boolean isOverdue() {
00184         if (scheduledTime == null || completed) return false;
00185         return LocalDateTime.now().isAfter(scheduledTime);
00186     }
00187
00192     @Override
00193     public String toString() {
00194         return String.format("Reminder{id=%d, type='%s', pet='%s', time='%s', done=%b}",
00195                               id, getReminderType(), petName, getFormattedTime(), completed);
00196     }
00197
00203     @Override

```

```

00204     public boolean equals(Object obj) {
00205         if (this == obj) return true;
00206         if (obj == null || !(obj instanceof Reminder)) return false;
00207         Reminder other = (Reminder) obj;
00208         return this.id == other.id;
00209     }
00210
00211     @Override
00212     public int hashCode() {
00213         return Integer.hashCode(id);
00214     }
00215 }

```

8.49 petreminder-app/src/main/java/com/mehdi/petreminder/model/↵ User.java File Reference

Kullanıcı modeli sınıfı.

```

import java.io.Serializable;
import java.time.LocalDateTime;

```

Classes

- class [com.mehdi.petreminder.model.User](#)
Uygulamayı kullanan evcil hayvan sahibi.

Packages

- package [com.mehdi.petreminder.model](#)

8.49.1 Detailed Description

Kullanıcı modeli sınıfı.

KVKK uyumu: şifre hash olarak saklanır, plain-text asla.

Definition in file [User.java](#).

8.50 User.java

[Go to the documentation of this file.](#)

```

00001
00002 package com.mehdi.petreminder.model;
00003
00004 import java.io.Serializable;
00005 import java.time.LocalDateTime;
00006
00007 public class User implements Serializable {
00008
00009     private static final long serialVersionUID = 1L;
00010
00011     private int id;
00012
00013     private String username;
00014 }

```

```

00032     private String email;
00033
00035     private String passwordHash;
00036
00038     private String fullName;
00039
00041     private LocalDateTime createdAt;
00042
00044     private LocalDateTime lastLogin;
00045
00047     private boolean active;
00048
00050     public User() {}
00051
00060     public User(int id, String username, String email,
00061                String passwordHash, String fullName) {
00062         this.id = id;
00063         this.username = username;
00064         this.email = email;
00065         this.passwordHash = passwordHash;
00066         this.fullName = fullName;
00067         this.createdAt = LocalDateTime.now();
00068         this.active = true;
00069     }
00070
00072     public int getId() { return id; }
00073
00075     public void setId(int id) { this.id = id; }
00076
00078     public String getUsername() { return username; }
00079
00081     public void setUsername(String username) { this.username = username; }
00082
00084     public String getEmail() { return email; }
00085
00087     public void setEmail(String email) { this.email = email; }
00088
00090     public String getPasswordHash() { return passwordHash; }
00091
00093     public void setPasswordHash(String passwordHash) { this.passwordHash = passwordHash; }
00094
00096     public String getFullName() { return fullName; }
00097
00099     public void setFullName(String fullName) { this.fullName = fullName; }
00100
00102     public LocalDateTime getCreatedAt() { return createdAt; }
00103
00105     public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }
00106
00108     public LocalDateTime getLastLogin() { return lastLogin; }
00109
00111     public void setLastLogin(LocalDateTime lastLogin) { this.lastLogin = lastLogin; }
00112
00114     public boolean isActive() { return active; }
00115
00117     public void setActive(boolean active) { this.active = active; }
00118
00123     @Override
00124     public String toString() {
00125         return String.format("User{id=%d, username='%s', email='%s', active=%b}",
00126                               id, username, email, active);
00127     }
00128
00134     @Override
00135     public boolean equals(Object obj) {
00136         if (this == obj) return true;
00137         if (obj == null || !(obj instanceof User)) return false;
00138         User other = (User) obj;
00139         return this.id == other.id;
00140     }
00141
00146     @Override
00147     public int hashCode() {
00148         return Integer.hashCode(id);
00149     }
00150 }

```

8.51 petreminder-app/src/main/java/com/mehdi/petreminder/model/VetAppointment.java File Reference ↩

Veteriner randevu hatırlatıcısı sınıfı.

```
import java.time.LocalDateTime;
```

Classes

- class [com.mehdi.petreminder.model.VetAppointment](#)
Veteriner randevusu — [Reminder](#)'dan türetilmiş somut sınıf.

Packages

- package [com.mehdi.petreminder.model](#)

8.51.1 Detailed Description

Veteriner randevu hatırlatıcısı sınıfı.

Definition in file [VetAppointment.java](#).

8.52 VetAppointment.java

[Go to the documentation of this file.](#)

```
00001
00005 package com.mehdi.petreminder.model;
00006
00007 import java.time.LocalDateTime;
00008
00017 public class VetAppointment extends Reminder {
00018
00020     private static final long serialVersionUID = 1L;
00021
00023     private String veterinarianName;
00024
00026     private String clinicName;
00027
00029     private String reason;
00030
00032     private double estimatedCost;
00033
00035     private boolean confirmed;
00036
00038     private String clinicPhone;
00039
00041     public VetAppointment() { super(); }
00042
00053     public VetAppointment(int id, int petId, String petName,
00054                           LocalDateTime scheduledTime,
00055                           String veterinarianName, String clinicName,
00056                           String reason) {
00057         super(id, petId, petName, scheduledTime,
00058             "Veteriner: " + reason, "YUKSEK", false, "");
00059         this.veterinarianName = veterinarianName;
00060         this.clinicName = clinicName;
00061         this.reason = reason;
00062         this.estimatedCost = 0.0;
00063         this.confirmed = false;
00064         this.clinicPhone = "";
00065     }
00066
00084     public VetAppointment(int id, int petId, String petName,
00085                           LocalDateTime scheduledTime, String description,
00086                           String priority, boolean recurring,
00087                           String recurrencePattern, String veterinarianName,
00088                           String clinicName, String reason,
00089                           double estimatedCost, boolean confirmed,
00090                           String clinicPhone) {
00091         super(id, petId, petName, scheduledTime, description,
```

```

00092         priority, recurring, recurrencePattern);
00093         this.veterinarianName = veterinarianName;
00094         this.clinicName = clinicName;
00095         this.reason = reason;
00096         this.estimatedCost = estimatedCost;
00097         this.confirmed = confirmed;
00098         this.clinicPhone = clinicPhone;
00099     }
00100
00102     public String getVeterinarianName() { return veterinarianName; }
00103
00105     public void setVeterinarianName(String veterinarianName) {
00106         this.veterinarianName = veterinarianName;
00107     }
00108
00110     public String getClinicName() { return clinicName; }
00111
00113     public void setClinicName(String clinicName) { this.clinicName = clinicName; }
00114
00116     public String getReason() { return reason; }
00117
00119     public void setReason(String reason) { this.reason = reason; }
00120
00122     public double getEstimatedCost() { return estimatedCost; }
00123
00125     public void setEstimatedCost(double estimatedCost) { this.estimatedCost = estimatedCost; }
00126
00128     public boolean isConfirmed() { return confirmed; }
00129
00131     public void setConfirmed(boolean confirmed) { this.confirmed = confirmed; }
00132
00134     public String getClinicPhone() { return clinicPhone; }
00135
00137     public void setClinicPhone(String clinicPhone) { this.clinicPhone = clinicPhone; }
00138
00140     @Override
00141     public String getReminderType() { return "Veteriner"; }
00142
00147     @Override
00148     public boolean validate() {
00149         return veterinarianName != null && !veterinarianName.trim().isEmpty()
00150             && clinicName != null && !clinicName.trim().isEmpty()
00151             && reason != null && !reason.trim().isEmpty();
00152     }
00153
00158     @Override
00159     public String getSummary() {
00160         return String.format("Veteriner: %s - %s (%s)%s",
00161             veterinarianName, clinicName, reason,
00162             confirmed ? " [Onaylandi]" : " [Bekliyor]");
00163     }
00164
00166     @Override
00167     public String toString() {
00168         return String.format(
00169             "VetAppointment{id=%d, pet='%s', vet='%s', clinic='%s', reason='%s'}",
00170             getId(), getPetName(), veterinarianName, clinicName, reason);
00171     }
00172 }

```

8.53 petreminder-app/src/main/java/com/mehdi/petreminder/observer/EventManager.java File Reference

Observer Pattern — Subject (yayinci) sinifi.

```

import org.slf4j.LoggerFactory;
import ch.qos.logback.classic.Logger;
import java.util.*;
import java.util.concurrent.CopyOnWriteArrayList;

```

Classes

- class [com.mehdi.petreminder.observer.EventManager](#)
Observer Pattern Subject — olay yoneticisi.

Packages

- package [com.mehdi.petreminder.observer](#)

8.53.1 Detailed Description

Observer Pattern — Subject (yayinci) sinifi.

Member documentation. [Member documentation.](#)

Observer'leri kaydeder, cikarir ve bildirim gonderir. Singleton Pattern ile tek instance kullanilir.

Definition in file [EventManager.java](#).

8.54 EventManager.java

[Go to the documentation of this file.](#)

```

00001
00016 package com.mehdi.petreminder.observer;
00017
00018 import org.slf4j.LoggerFactory;
00019 import ch.qos.logback.classic.Logger;
00020
00021 import java.util.*;
00022 import java.util.concurrent.CopyOnWriteArrayList;
00023
00037 public class EventManager {
00038
00040     private static final Logger logger =
00041         (Logger) LoggerFactory.getLogger(EventManager.class);
00042
00044     private static EventManager instance;
00045
00051     private final Map<EventType, List<PetReminderObserver> listeners;
00052
00057     private EventManager() {
00058         listeners = new EnumMap<>(EventType.class);
00059         for (EventType type : EventType.values()) {
00060             listeners.put(type, new CopyOnWriteArrayList<>());
00061         }
00062         logger.debug("EventManager olusturuldu.");
00063     }
00064
00069     public static synchronized EventManager getInstance() {
00070         if (instance == null) {
00071             instance = new EventManager();
00072         }
00073         return instance;
00074     }
00075
00081     public void subscribe(EventType eventType, PetReminderObserver observer) {
00082         if (eventType == null || observer == null) {
00083             logger.warn("subscribe: null parametre, atlaniyor.");
00084             return;
00085         }
00086         List<PetReminderObserver> list = listeners.get(eventType);
00087         if (!list.contains(observer)) {
00088             list.add(observer);
00089             logger.debug("Observer kaydedildi: {} -> {}",
00090                 eventType, observer.getClass().getSimpleName());
00091         }
00092     }
00093
00099     public void unsubscribe(EventType eventType, PetReminderObserver observer) {
00100         if (eventType == null || observer == null) return;
00101         List<PetReminderObserver> list = listeners.get(eventType);
00102         list.remove(observer);
00103         logger.debug("Observer cikarildi: {} -> {}",
00104             eventType, observer.getClass().getSimpleName());
00105     }
00106
00111     public void unsubscribeAll(PetReminderObserver observer) {

```

```

00112         if (observer == null) return;
00113         for (EventType type : EventType.values()) {
00114             listeners.get(type).remove(observer);
00115         }
00116         logger.debug("Observer tum olaylardan cikarildi: {}",
00117             observer.getClass().getSimpleName());
00118     }
00119
00125     public void notify(EventType eventType, Object data) {
00126         if (eventType == null) return;
00127         List<PetReminderObserver> list = listeners.get(eventType);
00128         logger.info("Olay tetiklendi: {} ({} observer)",
00129             eventType, list.size());
00130         for (PetReminderObserver observer : list) {
00131             try {
00132                 observer.onEvent(eventType, data);
00133             } catch (Exception e) {
00134                 logger.error("Observer bildirim hatasi: {} - {}",
00135                     observer.getClass().getSimpleName(), e.getMessage());
00136             }
00137         }
00138     }
00139
00145     public int getObserverCount(EventType eventType) {
00146         if (eventType == null) return 0;
00147         return listeners.get(eventType).size();
00148     }
00149
00153     public void clearAll() {
00154         for (EventType type : EventType.values()) {
00155             listeners.get(type).clear();
00156         }
00157         logger.debug("Tum observer'lar temizlendi.");
00158     }
00159
00163     public static synchronized void resetInstance() {
00164         instance = null;
00165     }
00166 }

```

8.55 petreminder-app/src/main/java/com/mehdi/petreminder/observer/EventType.java File Reference

Observer Pattern olay turleri.

Classes

- enum [com.mehdi.petreminder.observer.EventType](#)

Packages

- package [com.mehdi.petreminder.observer](#)

8.55.1 Detailed Description

Observer Pattern olay turleri.

Sistemde gerceklesebilecek olay turlerini tanimlar.

Definition in file [EventType.java](#).

8.56 EventType.java

[Go to the documentation of this file.](#)

```

00001
00006 package com.mehdi.petreminder.observer;
00007
00021 public enum EventType {
00022
00024     STORAGE_CHANGED("Storage Backend Degisti"),
00025
00027     PET_ADDED("Yeni Pet Eklendi"),
00028
00030     PET_DELETED("Pet Silindi"),
00031
00033     PET_UPDATED("Pet Guncellendi"),
00034
00036     REMINDER_ADDED("Yeni Hatirlatici Eklendi"),
00037
00039     REMINDER_COMPLETED("Hatirlatici Tamamlandi"),
00040
00042     REMINDER_DELETED("Hatirlatici Silindi"),
00043
00045     MEDICAL_RECORD_ADDED("Saglik Kaydi Eklendi"),
00046
00048     MEDICAL_RECORD_DELETED("Saglik Kaydi Silindi");
00049
00051     private final String description;
00052
00057     EventType(String description) {
00058         this.description = description;
00059     }
00060
00065     public String getDescription() {
00066         return description;
00067     }
00068
00073     @Override
00074     public String toString() {
00075         return description;
00076     }
00077 }

```

8.57 petreminder-app/src/main/java/com/mehdi/petreminder/observer/↔ PetReminderObserver.java File Reference

Observer Design Pattern — gozlemci arayuzu.

Classes

- interface [com.mehdi.petreminder.observer.PetReminderObserver](#)
Gozlemci arayuzu — durum degisikliklerini dinler.

Packages

- package [com.mehdi.petreminder.observer](#)

8.57.1 Detailed Description

Observer Design Pattern — gozlemci arayuzu.

PDF zorunluluğu: Minimum 3 design pattern. Observer Pattern: Durum degisikliklerinde otomatik bildirim. Factory , Template Method , Observer

Definition in file [PetReminderObserver.java](#).

8.58 PetReminderObserver.java

[Go to the documentation of this file.](#)

```
00001
00008 package com.mehdi.petreminder.observer;
00009
00022 public interface PetReminderObserver {
00023
00031     void onEvent(EventType event, Object data);
00032 }
```

8.59 petreminder-app/src/main/java/com/mehdi/petreminder/petreminder.java File Reference

Pet Care Reminder System kütüphane yardımcı sınıfı.

```
import com.mehdi.petreminder.model.Pet;
import com.mehdi.petreminder.model.Reminder;
import org.slf4j.LoggerFactory;
import ch.qos.logback.classic.Logger;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.List;
```

Classes

- class [com.mehdi.petreminder.petreminder](#)
Pet Care Reminder System yardımcı/facade sınıfı.

8.59.1 Detailed Description

Pet Care Reminder System kütüphane yardımcı sınıfı.

Template'deki [petreminder.java](#)'nın projeye uyarlanmış hali. Bu sınıf utility / facade görevi görür. OOP Encapsulation: tüm yardımcı metodlar bu sınıfta toplanmıştır.

Definition in file [petreminder.java](#).

8.60 petreminder.java

[Go to the documentation of this file.](#)

```
00001
00008 package com.mehdi.petreminder;
00009
00010 import com.mehdi.petreminder.model.Pet;
00011 import com.mehdi.petreminder.model.Reminder;
00012
00013 import org.slf4j.LoggerFactory;
00014 import ch.qos.logback.classic.Logger;
00015
00016 import java.time.LocalDateTime;
```

```

00017 import java.time.format.DateTimeFormatter;
00018 import java.util.List;
00019
00033 public class petreminder {
00034
00038     private static final Logger logger =
00039         (Logger) LoggerFactory.getLogger(petreminder.class);
00040
00044     private static final DateTimeFormatter DISPLAY_FORMAT =
00045         DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm");
00046
00050     public petreminder() {
00051         logger.debug("petreminder yardimci sinifi olusturuldu.");
00052     }
00053
00054     // Temel Aritmetik (template uyumu + test coverage)
00055
00063     public int add(int a, int b) {
00064         logger.debug("add({}, {})", a, b);
00065         return a + b;
00066     }
00067
00074     public int subtract(int a, int b) {
00075         logger.debug("subtract({}, {})", a, b);
00076         return a - b;
00077     }
00078
00085     public int multiply(int a, int b) {
00086         logger.debug("multiply({}, {})", a, b);
00087         return a * b;
00088     }
00089
00099     public double divide(int a, int b) {
00100         if (b == 0) {
00101             logger.error("Sifira bolme denemesi: {} / {}", a, b);
00102             throw new IllegalArgumentException("Sifira bolme yapilamaz.");
00103         }
00104         logger.debug("divide({}, {})", a, b);
00105         return (double) a / b;
00106     }
00107
00108     // Pet Yardimci Metodlari
00109
00115     public void printPets(List<Pet> pets) {
00116         if (pets == null || pets.isEmpty()) {
00117             System.out.println("Hic evcil hayvan bulunamadi.");
00118             return;
00119         }
00120         System.out.println(String.format("%-5s %-15s %-12s %-10s",
00121             "ID", "Ad", "Tur", "Yas"));
00122         System.out.println("-".repeat(45));
00123         for (Pet pet : pets) {
00124             System.out.println(String.format("%-5d %-15s %-12s %-10s",
00125                 pet.getId(), pet.getName(), pet.getSpecies(),
00126                 pet.getAgeString()));
00127         }
00128     }
00129
00135     public void printReminders(List<Reminder> reminders) {
00136         if (reminders == null || reminders.isEmpty()) {
00137             System.out.println("Hic hatirlatici bulunamadi.");
00138             return;
00139         }
00140         System.out.println(String.format("%-5s %-12s %-15s %-20s %-8s",
00141             "ID", "Tur", "Pet", "Zaman", "Durum"));
00142         System.out.println("-".repeat(65));
00143         for (Reminder r : reminders) {
00144             String durum = r.isCompleted() ? "[TAMAM]"
00145                 : r.isOverdue() ? "[GECMIS]" : "[BEKLIYOR]";
00146             System.out.println(String.format("%-5d %-12s %-15s %-20s %-8s",
00147                 r.getId(), r.getReminderType(), r.getPetName(),
00148                 r.getFormattedTime(), durum));
00149         }
00150     }
00151
00156     public String getCurrentTimeFormatted() {
00157         return LocalDateTime.now().format(DISPLAY_FORMAT);
00158     }
00159
00165     public boolean isNullOrEmpty(String value) {
00166         return value == null || value.trim().isEmpty();
00167     }
00168
00173     public void printSeparator(int width) {
00174         System.out.println("-".repeat(width));
00175     }
00176

```

```

00180     public void printHeader() {
00181         printSeparator(50);
00182         System.out.println(" " + petreminderApp.APP_NAME);
00183         System.out.println(" v" + petreminderApp.APP_VERSION);
00184         printSeparator(50);
00185     }
00186 }

```

8.61 petreminder-app/src/main/java/com/mehdi/petreminder/petreminderApp.java File Reference

Pet Care Reminder System ana uygulama giriş noktası.

```

import com.mehdi.petreminder.config.StorageConfig;
import com.mehdi.petreminder.config.StorageType;
import com.mehdi.petreminder.gui.MainFrame;
import org.slf4j.LoggerFactory;
import ch.qos.logback.classic.Logger;
import javax.swing.*;

```

Classes

- class [com.mehdi.petreminder.petreminderApp](#)
Pet Care Reminder System ana sınıfı.

8.61.1 Detailed Description

Pet Care Reminder System ana uygulama giriş noktası.

Console veya GUI modda başlatır. `-gui` flag → Swing GUI `-storage=binary|sqlite|mysql` → backend seçimi

Definition in file [petreminderApp.java](#).

8.62 petreminderApp.java

[Go to the documentation of this file.](#)

```

00001
00008 package com.mehdi.petreminder;
00009
00010 import com.mehdi.petreminder.config.StorageConfig;
00011 import com.mehdi.petreminder.config.StorageType;
00012 import com.mehdi.petreminder.gui.MainFrame;
00013 import org.slf4j.LoggerFactory;
00014 import ch.qos.logback.classic.Logger;
00015
00016 import javax.swing.*;
00017
00030 public class petreminderApp {
00031
00033     private static final Logger logger =
00034         (Logger) LoggerFactory.getLogger(petreminderApp.class);
00035
00037     public static final String APP_NAME = "Pet Care Reminder System";
00038
00040     public static final String APP_VERSION = "1.0.0";

```

```

00041
00050     public static void main(String[] args) {
00051         logger.info("{} v{} baslatiliyor...", APP_NAME, APP_VERSION);
00052
00053         // Storage backend
00054         StorageType storageType = parseStorageArg(args);
00055         StorageConfig.setActiveBackend(storageType);
00056         logger.info("Storage backend: {}", storageType);
00057
00058         boolean guiMode = isGuiRequested(args);
00059
00060         if (guiMode) {
00061             startGui();
00062         } else {
00063             startConsole(args);
00064         }
00065
00066         logger.info("Uygulama kapatildi.");
00067     }
00068
00072     public static void startGui() {
00073         SwingUtilities.invokeLater(() -> {
00074             try {
00075                 MainFrame frame = new MainFrame();
00076                 frame.setVisible(true);
00077                 logger.info("GUI baslatildi.");
00078             } catch (Exception e) {
00079                 logger.error("GUI baslatilamadi: {}", e.getMessage(), e);
00080                 System.err.println("GUI baslatilamadi: " + e.getMessage());
00081                 System.exit(1);
00082             }
00083         });
00084     }
00085
00090     public static void startConsole(String[] args) {
00091         System.out.println("=====");
00092         System.out.println("  " + APP_NAME + " v" + APP_VERSION);
00093         System.out.println("  CEN206 - OOP Term Project - Spring 2026");
00094         System.out.println("=====");
00095         System.out.println("Storage: " + StorageConfig.getActiveBackend().getDisplayName());
00096         System.out.println();
00097         System.out.println("Hint: GUI modda baslatmak için --gui argümanını kullanın.");
00098         System.out.println();
00099
00100         try {
00101             ConsoleApp consoleApp = new ConsoleApp();
00102             consoleApp.start();
00103         } catch (Exception e) {
00104             logger.error("Uygulama hatasi: {}", e.getMessage(), e);
00105             System.err.println("Kritik hata: " + e.getMessage());
00106             System.exit(1);
00107         }
00108     }
00109
00115     public static boolean isGuiRequested(String[] args) {
00116         if (args == null) return false;
00117         for (String arg : args) {
00118             if ("--gui".equalsIgnoreCase(arg)) return true;
00119         }
00120         return false;
00121     }
00122
00128     public static StorageType parseStorageArg(String[] args) {
00129         if (args == null || args.length == 0) return StorageType.BINARY;
00130         for (String arg : args) {
00131             if (arg != null && arg.startsWith("--storage=")) {
00132                 String value = arg.substring("--storage=".length())
00133                     .trim().toUpperCase(java.util.Locale.ROOT);
00134                 try {
00135                     return StorageType.valueOf(value);
00136                 } catch (IllegalArgumentException e) {
00137                     logger.warn("Gecersiz storage: {}. BINARY kullaniliyor.", value);
00138                     return StorageType.BINARY;
00139                 }
00140             }
00141         }
00142         return StorageType.BINARY;
00143     }
00144
00149     public static String getVersion() { return APP_VERSION; }
00150
00155     public static String getAppName() { return APP_NAME; }
00156 }

```

8.63 petreminder-app/src/main/java/com/mehdi/petreminder/repository/BinaryRepository.java File Reference

Binary file I/O repository — ObjectOutputStream / .bin dosyası.

```
import org.slf4j.LoggerFactory;
import ch.qos.logback.classic.Logger;
import java.io.*;
import java.util.*;
```

Classes

- class [com.mehdi.petreminder.repository.BinaryRepository< T extends Serializable >](#)
Generic binary (Object serialization) repository implementasyonu.

Packages

- package [com.mehdi.petreminder.repository](#)

8.63.1 Detailed Description

Binary file I/O repository — ObjectOutputStream / .bin dosyası.

PDF LO.6: Binary storage backend — serialize/deserialize.

Definition in file [BinaryRepository.java](#).

8.64 BinaryRepository.java

[Go to the documentation of this file.](#)

```
00001
00009 package com.mehdi.petreminder.repository;
00010
00011 import org.slf4j.LoggerFactory;
00012 import ch.qos.logback.classic.Logger;
00013
00014 import java.io.*;
00015 import java.util.*;
00016
00029 public class BinaryRepository<T extends Serializable> implements IRepository<T> {
00030
00032     private static final Logger logger =
00033         (Logger) LoggerFactory.getLogger(BinaryRepository.class);
00034
00036     private final String filePath;
00037
00039     private int nextId = 1;
00040
00046     public BinaryRepository(String directoryPath, String entityName) {
00047         File dir = new File(directoryPath);
00048         if (!dir.exists()) {
00049             dir.mkdirs();
00050         }
00051         this.filePath = directoryPath + File.separator + entityName + ".bin";
00052         initNextId();
00053     }
```

```

00054
00058     private void initNextId() {
00059         List<T> all = findAll();
00060         if (all.isEmpty()) {
00061             nextId = 1;
00062             return;
00063         }
00064         // Reflection ile id alanını bulmaya çalış
00065         int maxId = 0;
00066         for (T entity : all) {
00067             try {
00068                 var method = entity.getClass().getMethod("getId");
00069                 Object idObj = method.invoke(entity);
00070                 if (idObj instanceof Integer) {
00071                     int id = (Integer) idObj;
00072                     if (id > maxId) maxId = id;
00073                 }
00074             } catch (Exception e) {
00075                 // getId metodu yoksa pas geç
00076             }
00077         }
00078         nextId = maxId + 1;
00079     }
00080
00086     private void setEntityId(T entity, int id) {
00087         try {
00088             var method = entity.getClass().getMethod("setId", int.class);
00089             method.invoke(entity, id);
00090         } catch (Exception e) {
00091             logger.warn("setId çağrılmadı: {}", e.getMessage());
00092         }
00093     }
00094
00100     private int getEntityId(T entity) {
00101         try {
00102             var method = entity.getClass().getMethod("getId");
00103             Object idObj = method.invoke(entity);
00104             if (idObj instanceof Integer) {
00105                 return (Integer) idObj;
00106             }
00107         } catch (Exception e) {
00108             logger.warn("getId çağrılmadı: {}", e.getMessage());
00109         }
00110         return -1;
00111     }
00112
00116     @Override
00117     public int save(T entity) {
00118         if (entity == null) {
00119             throw new RepositoryException("Entity null olamaz");
00120         }
00121         List<T> all = findAll();
00122         int id = nextId++;
00123         setEntityId(entity, id);
00124         all.add(entity);
00125         writeAll(all);
00126         logger.debug("Kaydedildi: id={}, tip={}", id, entity.getClass().getSimpleName());
00127         return id;
00128     }
00129
00133     @Override
00134     public Optional<T> findById(int id) {
00135         return findAll().stream()
00136             .filter(e -> getEntityId(e) == id)
00137             .findFirst();
00138     }
00139
00143     @Override
00144     @SuppressWarnings("unchecked")
00145     public List<T> findAll() {
00146         File file = new File(filePath);
00147         if (!file.exists() || file.length() == 0) {
00148             return new ArrayList<>();
00149         }
00150         try (FileInputStream fis = new FileInputStream(file);
00151             ObjectInputStream ois = new ObjectInputStream(fis)) {
00152             Object obj = ois.readObject();
00153             if (obj instanceof List) {
00154                 return (List<T>) obj;
00155             }
00156         } catch (EOFException e) {
00157             return new ArrayList<>();
00158         } catch (Exception e) {
00159             logger.error("Binary okuma hatası: {}", e.getMessage());
00160             throw new RepositoryException("Binary okuma hatası", e);
00161         }
00162         return new ArrayList<>();

```

```

00163     }
00164
00165     @Override
00166     public boolean update(T entity) {
00167         if (entity == null) return false;
00168         int id = getEntityId(entity);
00169         List<T> all = findAll();
00170         boolean found = false;
00171         for (int i = 0; i < all.size(); i++) {
00172             if (getEntityId(all.get(i)) == id) {
00173                 all.set(i, entity);
00174                 found = true;
00175                 break;
00176             }
00177         }
00178         if (found) {
00179             writeAll(all);
00180             logger.debug("Güncellendi: id={}", id);
00181         }
00182         return found;
00183     }
00184
00185     @Override
00186     public boolean delete(int id) {
00187         List<T> all = findAll();
00188         boolean removed = all.removeIf(e -> getEntityId(e) == id);
00189         if (removed) {
00190             writeAll(all);
00191             logger.debug("Silindi: id={}", id);
00192         }
00193         return removed;
00194     }
00195
00196     @Override
00197     public void deleteAll() {
00198         writeAll(new ArrayList<>());
00199         nextId = 1;
00200         logger.debug("Tümü silindi: {}", filePath);
00201     }
00202
00203     @Override
00204     public int count() {
00205         return findAll().size();
00206     }
00207
00208     @Override
00209     public void close() {
00210         // Binary repository için kaynak kapatma gerekmez
00211         logger.debug("BinaryRepository kapatıldı.");
00212     }
00213
00214     private void writeAll(List<T> list) {
00215         try (ObjectOutputStream oos = new ObjectOutputStream(
00216             new FileOutputStream(filePath))) {
00217             oos.writeObject(list);
00218         } catch (IOException e) {
00219             logger.error("Binary yazma hatası: {}", e.getMessage());
00220             throw new RepositoryException("Binary yazma hatası", e);
00221         }
00222     }
00223
00224     public String getFilePath() {
00225         return filePath;
00226     }
00227 }
00228
00229 }
00230

```

8.65 petreminder-

app/src/main/java/com/mehdi/petreminder/repository/↵

IRepository.java File Reference

Generic repository interface — PDF LO.6 zorunluluğu.

```

import java.util.List;
import java.util.Optional;

```


Classes

- interface [com.mehdi.petreminder.repository.IRepository< T >](#)
Generic CRUD repository arayüzü.

Packages

- package [com.mehdi.petreminder.repository](#)

8.65.1 Detailed Description

Generic repository interface — PDF LO.6 zorunluluğu.

Tüm storage backend'leri bu interface'i implement eder. OOP Abstraction: implementasyon detayları gizlenir.

Definition in file [IRepository.java](#).

8.66 IRepository.java

[Go to the documentation of this file.](#)

```
00001
00007 package com.mehdi.petreminder.repository;
00008
00009 import java.util.List;
00010 import java.util.Optional;
00011
00023 public interface IRepository<T> {
00024
00031     int save(T entity);
00032
00039     Optional<T> findById(int id);
00040
00046     List<T> findAll();
00047
00054     boolean update(T entity);
00055
00062     boolean delete(int id);
00063
00068     void deleteAll();
00069
00074     int count();
00075
00079     void close();
00080 }
```

8.67 petreminder- app/src/main/java/com/mehdi/petreminder/repository/Medical RecordMySqlRepository.java File Reference

MedicalRecord için MySQL repository.

```
import com.mehdi.petreminder.model.MedicalRecord;
import java.sql.*;
import java.time.LocalDate;
```

Classes

- class [com.mehdi.petreminder.repository.MedicalRecordMySqlRepository](#)
MySQL üzerinde *MedicalRecord* CRUD.

Packages

- package [com.mehdi.petreminder.repository](#)

8.67.1 Detailed Description

MedicalRecord için MySQL repository.

Member documentation. Member documentation.

Definition in file [MedicalRecordMySqlRepository.java](#).

8.68 MedicalRecordMySqlRepository.java

[Go to the documentation of this file.](#)

```

00001
00011 package com.mehdi.petreminder.repository;
00012
00013 import com.mehdi.petreminder.model.MedicalRecord;
00014
00015 import java.sql.*;
00016 import java.time.LocalDate;
00017
00026 public class MedicalRecordMySqlRepository extends MySqlRepository<MedicalRecord> {
00027
00029     public MedicalRecordMySqlRepository() { super(); }
00030
00035     public MedicalRecordMySqlRepository(Connection connection) { super(connection); }
00036
00038     @Override
00042     protected String getTableName() { return "medical_records"; }
00043
00045     @Override
00049     protected String getCreateTableSql() {
00050         return "CREATE TABLE IF NOT EXISTS medical_records ("
00051             + "id INT AUTO_INCREMENT PRIMARY KEY,"
00052             + "pet_id INT NOT NULL,"
00053             + "pet_name VARCHAR(50),"
00054             + "record_date DATE NOT NULL,"
00055             + "record_type VARCHAR(30) NOT NULL,"
00056             + "diagnosis TEXT,"
00057             + "treatment TEXT,"
00058             + "veterinarian_name VARCHAR(100),"
00059             + "cost DOUBLE DEFAULT 0,"
00060             + "next_check_date DATE,"
00061             + "notes TEXT,"
00062             + "vaccine_name VARCHAR(100)"
00063             + ")";
00064     }
00065
00067     @Override
00071     protected String getInsertSql() {
00072         return "INSERT INTO medical_records (pet_id,pet_name,record_date,record_type,"
00073             + "diagnosis,treatment,veterinarian_name,cost,next_check_date,notes,vaccine_name)"
00074             + " VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";
00075     }
00076
00078     @Override
00082     protected String getUpdateSql() {
00083         return "UPDATE medical_records SET pet_id=?,pet_name=?,record_date=?,record_type=,"
00084             + "diagnosis=?,treatment=?,veterinarian_name=?,cost=?,next_check_date=,"
00085             + "notes=?,vaccine_name=? WHERE id=?";
00086     }

```

```

00087
00089     @Override
00093     protected void setInsertParams(PreparedStatement ps, MedicalRecord m) throws SQLException {
00094         ps.setInt(1, m.getPetId());
00095         ps.setString(2, m.getPetName());
00096         ps.setObject(3, m.getRecordDate() != null
00097             ? java.sql.Date.valueOf(m.getRecordDate()) : null);
00098         ps.setString(4, m.getRecordType());
00099         ps.setString(5, m.getDiagnosis());
00100         ps.setString(6, m.getTreatment());
00101         ps.setString(7, m.getVeterinarianName());
00102         ps.setDouble(8, m.getCost());
00103         ps.setObject(9, m.getNextCheckDate() != null
00104             ? java.sql.Date.valueOf(m.getNextCheckDate()) : null);
00105         ps.setString(10, m.getNotes());
00106         ps.setString(11, m.getVaccineName());
00107     }
00108
00110     @Override
00114     protected void setUpdateParams(PreparedStatement ps, MedicalRecord m) throws SQLException {
00115         setInsertParams(ps, m);
00116         ps.setInt(12, m.getId());
00117     }
00118
00120     @Override
00124     protected MedicalRecord mapRow(ResultSet rs) throws SQLException {
00125         MedicalRecord m = new MedicalRecord();
00126         m.setId(rs.getInt("id"));
00127         m.setPetId(rs.getInt("pet_id"));
00128         m.setPetName(rs.getString("pet_name"));
00129         java.sql.Date rd = rs.getDate("record_date");
00130         if (rd != null) m.setRecordDate(rd.toLocalDate());
00131         m.setRecordType(rs.getString("record_type"));
00132         m.setDiagnosis(rs.getString("diagnosis"));
00133         m.setTreatment(rs.getString("treatment"));
00134         m.setVeterinarianName(rs.getString("veterinarian_name"));
00135         m.setCost(rs.getDouble("cost"));
00136         java.sql.Date ncd = rs.getDate("next_check_date");
00137         if (ncd != null) m.setNextCheckDate(ncd.toLocalDate());
00138         m.setNotes(rs.getString("notes"));
00139         m.setVaccineName(rs.getString("vaccine_name"));
00140         return m;
00141     }
00142 }

```

8.69 petreminder-app/src/main/java/com/mehdi/petreminder/repository/MedicalRecordSqliteRepository.java File Reference

MedicalRecord için SQLite repository.

```

import com.mehdi.petreminder.model.MedicalRecord;
import java.sql.*;
import java.time.LocalDate;
import java.util.*;

```

Classes

- class [com.mehdi.petreminder.repository.MedicalRecordSqliteRepository](#)
SQLite üzerinde *MedicalRecord* CRUD.

Packages

- package [com.mehdi.petreminder.repository](#)

8.69.1 Detailed Description

MedicalRecord için SQLite repository.

Definition in file [MedicalRecordSqliteRepository.java](#).

8.70 MedicalRecordSqliteRepository.java

[Go to the documentation of this file.](#)

```

00001
00005 package com.mehdi.petreminder.repository;
00006
00007 import com.mehdi.petreminder.model.MedicalRecord;
00008
00009 import java.sql.*;
00010 import java.time.LocalDate;
00011 import java.util.*;
00012
00021 public class MedicalRecordSqliteRepository extends SqliteRepository<MedicalRecord> {
00022
00024     public MedicalRecordSqliteRepository() { super(); }
00025
00030     public MedicalRecordSqliteRepository(Connection connection) { super(connection); }
00031
00033     @Override
00037     protected String getTableName() { return "medical_records"; }
00038
00040     @Override
00044     protected String getCreateTableSql() {
00045         return "CREATE TABLE IF NOT EXISTS medical_records ("
00046             + "id INTEGER PRIMARY KEY AUTOINCREMENT,"
00047             + "pet_id INTEGER NOT NULL,"
00048             + "pet_name TEXT,"
00049             + "record_date TEXT NOT NULL,"
00050             + "record_type TEXT NOT NULL,"
00051             + "diagnosis TEXT,"
00052             + "treatment TEXT,"
00053             + "veterinarian_name TEXT,"
00054             + "cost REAL DEFAULT 0,"
00055             + "next_check_date TEXT,"
00056             + "notes TEXT,"
00057             + "vaccine_name TEXT"
00058             + ")";
00059     }
00060
00062     @Override
00066     protected String getInsertSql() {
00067         return "INSERT INTO medical_records (pet_id,pet_name,record_date,record_type,"
00068             + "diagnosis,treatment,veterinarian_name,cost,next_check_date,notes,vaccine_name) "
00069             + "VALUES (?,?,?,?,?,?,?,?,?,?,?)";
00070     }
00071
00073     @Override
00077     protected String getUpdateSql() {
00078         return "UPDATE medical_records SET pet_id=?,pet_name=?,record_date=?,record_type=,"
00079             + "diagnosis=?,treatment=?,veterinarian_name=?,cost=?,next_check_date=,"
00080             + "notes=?,vaccine_name=? WHERE id=?";
00081     }
00082
00084     @Override
00088     protected void setInsertParams(PreparedStatement ps, MedicalRecord m) throws SQLException {
00089         ps.setInt(1, m.getPetId());
00090         ps.setString(2, m.getPetName());
00091         ps.setString(3, m.getRecordDate() != null ? m.getRecordDate().toString() : null);
00092         ps.setString(4, m.getRecordType());
00093         ps.setString(5, m.getDiagnosis());
00094         ps.setString(6, m.getTreatment());
00095         ps.setString(7, m.getVeterinarianName());
00096         ps.setDouble(8, m.getCost());
00097         ps.setString(9, m.getNextCheckDate() != null ? m.getNextCheckDate().toString() : null);
00098         ps.setString(10, m.getNotes());
00099         ps.setString(11, m.getVaccineName());
00100     }
00101
00103     @Override
00107     protected void setUpdateParams(PreparedStatement ps, MedicalRecord m) throws SQLException {
00108         setInsertParams(ps, m);
00109         ps.setInt(12, m.getId());

```

```

00110     }
00111
00113     @Override
00117     protected MedicalRecord mapRow(ResultSet rs) throws SQLException {
00118         MedicalRecord m = new MedicalRecord();
00119         m.setId(rs.getInt("id"));
00120         m.setPetId(rs.getInt("pet_id"));
00121         m.setPetName(rs.getString("pet_name"));
00122         String rd = rs.getString("record_date");
00123         if (rd != null) m.setRecordDate(LocalDate.parse(rd));
00124         m.setRecordType(rs.getString("record_type"));
00125         m.setDiagnosis(rs.getString("diagnosis"));
00126         m.setTreatment(rs.getString("treatment"));
00127         m.setVeterinarianName(rs.getString("veterinarian_name"));
00128         m.setCost(rs.getDouble("cost"));
00129         String ncd = rs.getString("next_check_date");
00130         if (ncd != null) m.setNextCheckDate(LocalDate.parse(ncd));
00131         m.setNotes(rs.getString("notes"));
00132         m.setVaccineName(rs.getString("vaccine_name"));
00133         return m;
00134     }
00135
00141     public List<MedicalRecord> findByPetId(int petId) {
00142         List<MedicalRecord> list = new ArrayList<>();
00143         try (PreparedStatement ps = connection.prepareStatement(
00144             "SELECT * FROM medical_records WHERE pet_id = ?")) {
00145             ps.setInt(1, petId);
00146             ResultSet rs = ps.executeQuery();
00147             while (rs.next()) list.add(mapRow(rs));
00148         } catch (SQLException e) {
00149             throw new RepositoryException("findByPetId hatası", e);
00150         }
00151         return list;
00152     }
00153 }

```

8.71 petreminder-app/src/main/java/com/mehdi/petreminder/repository/MySqlRepository.java File Reference

MySQL JDBC repository — mysql-connector-j + Docker Compose.

```

import com.mehdi.petreminder.config.StorageConfig;
import org.slf4j.LoggerFactory;
import ch.qos.logback.classic.Logger;
import java.sql.*;
import java.util.*;

```

Classes

- class [com.mehdi.petreminder.repository.MySqlRepository< T >](#)
MySQL tabanlı abstract repository temel sınıfı.

Packages

- package [com.mehdi.petreminder.repository](#)

8.71.1 Detailed Description

MySQL JDBC repository — mysql-connector-j + Docker Compose.

PDF LO.6: MySQL storage backend.

Definition in file [MySqlRepository.java](#).

8.72 MySqlRepository.java

[Go to the documentation of this file.](#)

```

00001
00006 package com.mehdi.petreminder.repository;
00007
00008 import com.mehdi.petreminder.config.StorageConfig;
00009 import org.slf4j.LoggerFactory;
00010 import ch.qos.logback.classic.Logger;
00011
00012 import java.sql.*;
00013 import java.util.*;
00014
00026 public abstract class MySqlRepository<T> implements IRepository<T> {
00027
00029     protected static final Logger logger =
00030         (Logger) LoggerFactory.getLogger(MySqlRepository.class);
00031
00033     protected Connection connection;
00034
00039     public MySqlRepository() {
00040         try {
00041             Class.forName("com.mysql.cj.jdbc.Driver");
00042             connection = DriverManager.getConnection(
00043                 StorageConfig.getMysqlUrl(),
00044                 StorageConfig.getMysqlUsername(),
00045                 StorageConfig.getMysqlPassword());
00046             connection.setAutoCommit(true);
00047             createTableIfNotExists();
00048             logger.info("MySQL bağlantısı açıldı: {}", StorageConfig.getMysqlUrl());
00049         } catch (Exception e) {
00050             throw new RepositoryException("MySQL bağlantı hatası -- Docker çalışıyor mu?", e);
00051         }
00052     }
00053
00058     public MySqlRepository(Connection connection) {
00059         this.connection = connection;
00060         try {
00061             createTableIfNotExists();
00062         } catch (Exception e) {
00063             throw new RepositoryException("Tablo oluşturma hatası", e);
00064         }
00065     }
00066
00068     protected abstract String getTableName();
00069
00071     protected abstract String getCreateTableSql();
00072
00074     protected abstract String getInsertSql();
00075
00077     protected abstract String getUpdateSql();
00078
00085     protected abstract void setInsertParams(PreparedStatement ps, T entity)
00086         throws SQLException;
00087
00094     protected abstract void setUpdateParams(PreparedStatement ps, T entity)
00095         throws SQLException;
00096
00103     protected abstract T mapRow(ResultSet rs) throws SQLException;
00104
00108     protected void createTableIfNotExists() {
00109         try (Statement st = connection.createStatement()) {
00110             st.execute(getCreateTableSql());
00111         } catch (SQLException e) {
00112             throw new RepositoryException("Tablo oluşturma hatası", e);
00113         }
00114     }
00115
00117     @Override
00118     public int save(T entity) {
00119         if (entity == null) throw new RepositoryException("Entity null olamaz");
00120         try (PreparedStatement ps = connection.prepareStatement(
00121             getInsertSql(), Statement.RETURN_GENERATED_KEYS)) {
00122             setInsertParams(ps, entity);
00123             ps.executeUpdate();
00124             ResultSet keys = ps.getGeneratedKeys();
00125             if (keys.next()) {
00126                 int id = keys.getInt(1);
00127                 setEntityId(entity, id);
00128                 return id;
00129             }
00130             throw new RepositoryException("MySQL ID üretilmedi");
00131         } catch (SQLException e) {
00132             throw new RepositoryException("MySQL save hatası", e);
00133         }
00134     }

```

```

00134     }
00135
00136     @Override
00137     public Optional<T> findById(int id) {
00138         String sql = "SELECT * FROM " + getTableName() + " WHERE id = ?";
00139         try (PreparedStatement ps = connection.prepareStatement(sql)) {
00140             ps.setInt(1, id);
00141             ResultSet rs = ps.executeQuery();
00142             if (rs.next()) return Optional.of(mapRow(rs));
00143         } catch (SQLException e) {
00144             throw new RepositoryException("MySQL findById hatası", e);
00145         }
00146         return Optional.empty();
00147     }
00148
00149     @Override
00150     public List<T> findAll() {
00151         List<T> list = new ArrayList<>();
00152         try (Statement st = connection.createStatement();
00153              ResultSet rs = st.executeQuery("SELECT * FROM " + getTableName())) {
00154             while (rs.next()) list.add(mapRow(rs));
00155         } catch (SQLException e) {
00156             throw new RepositoryException("MySQL findAll hatası", e);
00157         }
00158         return list;
00159     }
00160
00161     @Override
00162     public boolean update(T entity) {
00163         if (entity == null) return false;
00164         try (PreparedStatement ps = connection.prepareStatement(getUpdateSql())) {
00165             setUpdateParams(ps, entity);
00166             return ps.executeUpdate() > 0;
00167         } catch (SQLException e) {
00168             throw new RepositoryException("MySQL update hatası", e);
00169         }
00170     }
00171
00172     @Override
00173     public boolean delete(int id) {
00174         try (PreparedStatement ps = connection.prepareStatement(
00175             "DELETE FROM " + getTableName() + " WHERE id = ?")) {
00176             ps.setInt(1, id);
00177             return ps.executeUpdate() > 0;
00178         } catch (SQLException e) {
00179             throw new RepositoryException("MySQL delete hatası", e);
00180         }
00181     }
00182
00183     @Override
00184     public void deleteAll() {
00185         try (Statement st = connection.createStatement()) {
00186             st.execute("DELETE FROM " + getTableName());
00187         } catch (SQLException e) {
00188             throw new RepositoryException("MySQL deleteAll hatası", e);
00189         }
00190     }
00191
00192     @Override
00193     public int count() {
00194         try (Statement st = connection.createStatement();
00195              ResultSet rs = st.executeQuery("SELECT COUNT(*) FROM " + getTableName())) {
00196             if (rs.next()) return rs.getInt(1);
00197         } catch (SQLException e) {
00198             throw new RepositoryException("MySQL count hatası", e);
00199         }
00200         return 0;
00201     }
00202
00203     @Override
00204     public void close() {
00205         try {
00206             if (connection != null && !connection.isClosed()) {
00207                 connection.close();
00208                 logger.info("MySQL bağlantısı kapatıldı.");
00209             }
00210         } catch (SQLException e) {
00211             logger.warn("MySQL kapatma hatası: {}", e.getMessage());
00212         }
00213     }
00214
00215     public Connection getConnection() {
00216         return connection;
00217     }
00218
00219     protected void setEntityId(T entity, int id) {
00220         try {

```

```

00237         var method = entity.getClass().getMethod("setId", int.class);
00238         method.invoke(entity, id);
00239     } catch (Exception e) {
00240         logger.warn("setId çağrılmadı: {}", e.getMessage());
00241     }
00242 }
00243 }

```

8.73 petreminder-app/src/main/java/com/mehdi/petreminder/repository/PetMySqlRepository.java File Reference

[Pet](#) varlığı için MySQL repository implementasyonu.

```

import com.mehdi.petreminder.model.*;
import java.sql.*;
import java.sql.Date;
import java.time.LocalDate;
import java.util.*;

```

Classes

- class [com.mehdi.petreminder.repository.PetMySqlRepository](#)
MySQL üzerinde [Pet](#) CRUD işlemleri.

Packages

- package [com.mehdi.petreminder.repository](#)

8.73.1 Detailed Description

[Pet](#) varlığı için MySQL repository implementasyonu.

Definition in file [PetMySqlRepository.java](#).

8.74 PetMySqlRepository.java

[Go to the documentation of this file.](#)

```

00001
00005 package com.mehdi.petreminder.repository;
00006
00007 import com.mehdi.petreminder.model.*;
00008
00009 import java.sql.*;
00010 import java.sql.Date;
00011 import java.time.LocalDate;
00012 import java.util.*;
00013
00023 public class PetMySqlRepository extends MySqlRepository<Pet> {
00024
00028     public PetMySqlRepository() {
00029         super();
00030     }

```



```

00031
00036 public PetMySQLRepository(Connection connection) {
00037     super(connection);
00038 }
00039
00041 @Override
00045 protected String getTableName() {
00046     return "pets";
00047 }
00048
00050 @Override
00054 protected String getCreateTableSql() {
00055     return "CREATE TABLE IF NOT EXISTS pets ("
00056         + "id INT AUTO_INCREMENT PRIMARY KEY,"
00057         + "owner_id INT NOT NULL DEFAULT 1,"
00058         + "name VARCHAR(50) NOT NULL,"
00059         + "species VARCHAR(30) NOT NULL,"
00060         + "breed VARCHAR(50),"
00061         + "birth_date DATE,"
00062         + "gender VARCHAR(10),"
00063         + "weight DOUBLE DEFAULT 0,"
00064         + "notes TEXT,"
00065         + "can_talk BOOLEAN DEFAULT FALSE,"
00066         + "bird_type VARCHAR(50),"
00067         + "is_trained BOOLEAN DEFAULT FALSE,"
00068         + "is_indoor BOOLEAN DEFAULT TRUE"
00069         + ")";
00070 }
00071
00073 @Override
00077 protected String getInsertSql() {
00078     return "INSERT INTO pets (owner_id,name,species,breed,birth_date,gender,"
00079         + "weight,notes,can_talk,bird_type,is_trained,is_indoor) "
00080         + "VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";
00081 }
00082
00084 @Override
00088 protected String getUpdateSql() {
00089     return "UPDATE pets SET owner_id=?,name=?,species=?,breed=?,birth_date=,"
00090         + "gender=?,weight=?,notes=?,can_talk=?,bird_type=?,is_trained=?,is_indoor=? "
00091         + "WHERE id=?";
00092 }
00093
00095 @Override
00099 protected void setInsertParams(PreparedStatement ps, Pet pet) throws SQLException {
00100     ps.setInt(1, pet.getOwnerId());
00101     ps.setString(2, pet.getName());
00102     ps.setString(3, pet.getSpecies());
00103     ps.setString(4, pet instanceof Dog ? ((Dog) pet).getBreed()
00104         : pet instanceof Cat ? ((Cat) pet).getBreed() : null);
00105     ps.setObject(5, pet.getBirthDate() != null
00106         ? java.sql.Date.valueOf(pet.getBirthDate()) : null);
00107     ps.setString(6, pet.getGender());
00108     ps.setDouble(7, pet.getWeight());
00109     ps.setString(8, pet.getNotes());
00110     ps.setBoolean(9, pet instanceof Bird && ((Bird) pet).isCanTalk());
00111     ps.setString(10, pet instanceof Bird ? ((Bird) pet).getBirdType() : null);
00112     ps.setBoolean(11, pet instanceof Dog && ((Dog) pet).isTrained());
00113     ps.setBoolean(12, !(pet instanceof Cat) || ((Cat) pet).isIndoor());
00114 }
00115
00117 @Override
00121 protected void setUpdateParams(PreparedStatement ps, Pet pet) throws SQLException {
00122     setInsertParams(ps, pet);
00123     ps.setInt(13, pet.getId());
00124 }
00125
00127 @Override
00131 protected Pet mapRow(ResultSet rs) throws SQLException {
00132     String species = rs.getString("species");
00133     int id = rs.getInt("id");
00134     String name = rs.getString("name");
00135     Date sqlDate = rs.getDate("birth_date");
00136     LocalDate birthDate = sqlDate != null ? sqlDate.toLocalDate() : null;
00137     int ownerId = rs.getInt("owner_id");
00138
00139     Pet pet;
00140     if ("Dog".equalsIgnoreCase(species)) {
00141         Dog dog = new Dog(id, name, birthDate, ownerId);
00142         dog.setBreed(rs.getString("breed"));
00143         dog.setTrained(rs.getBoolean("is_trained"));
00144         pet = dog;
00145     } else if ("Cat".equalsIgnoreCase(species)) {
00146         Cat cat = new Cat(id, name, birthDate, ownerId);
00147         cat.setBreed(rs.getString("breed"));
00148         cat.setIndoor(rs.getBoolean("is_indoor"));
00149         pet = cat;

```

```

00150         } else {
00151             Bird bird = new Bird(id, name, birthDate, ownerId);
00152             bird.setBirdType(rs.getString("bird_type"));
00153             bird.setCanTalk(rs.getBoolean("can_talk"));
00154             pet = bird;
00155         }
00156         pet.setGender(rs.getString("gender"));
00157         pet.setWeight(rs.getDouble("weight"));
00158         pet.setNotes(rs.getString("notes"));
00159         return pet;
00160     }
00161
00162     public List<Pet> findById(int ownerId) {
00163         List<Pet> list = new ArrayList<>();
00164         String sql = "SELECT * FROM pets WHERE owner_id = ?";
00165         try (PreparedStatement ps = connection.prepareStatement(sql)) {
00166             ps.setInt(1, ownerId);
00167             ResultSet rs = ps.executeQuery();
00168             while (rs.next()) list.add(mapRow(rs));
00169         } catch (SQLException e) {
00170             throw new RepositoryException("findById hatası", e);
00171         }
00172         return list;
00173     }
00174 }
00175 }
00176 }

```

8.75 petreminder-app/src/main/java/com/mehdi/petreminder/repository/PetSqliteRepository.java File Reference

[Pet](#) varlığı için SQLite repository implementasyonu.

```

import com.mehdi.petreminder.model.*;
import org.slf4j.LoggerFactory;
import ch.qos.logback.classic.Logger;
import java.sql.*;
import java.time.LocalDate;
import java.util.*;

```

Classes

- class [com.mehdi.petreminder.repository.PetSqliteRepository](#)
SQLite üzerinde [Pet](#) CRUD işlemleri.

Packages

- package [com.mehdi.petreminder.repository](#)

8.75.1 Detailed Description

[Pet](#) varlığı için SQLite repository implementasyonu.

Definition in file [PetSqliteRepository.java](#).

8.76 PetSqliteRepository.java

[Go to the documentation of this file.](#)

```

00001
00005 package com.mehdi.petreminder.repository;
00006
00007 import com.mehdi.petreminder.model.*;
00008 import org.slf4j.LoggerFactory;
00009 import ch.qos.logback.classic.Logger;
00010
00011 import java.sql.*;
00012 import java.time.LocalDate;
00013 import java.util.*;
00014
00025 public class PetSqliteRepository extends SqliteRepository<Pet> {
00026
00028     private static final Logger log =
00029         (Logger) LoggerFactory.getLogger(PetSqliteRepository.class);
00030
00034     public PetSqliteRepository() {
00035         super();
00036     }
00037
00042     public PetSqliteRepository(Connection connection) {
00043         super(connection);
00044     }
00045
00047     @Override
00051     protected String getTableName() {
00052         return "pets";
00053     }
00054
00056     @Override
00060     protected String getCreateTableSql() {
00061         return "CREATE TABLE IF NOT EXISTS pets ("
00062             + "id INTEGER PRIMARY KEY AUTOINCREMENT,"
00063             + "owner_id INTEGER NOT NULL DEFAULT 1,"
00064             + "name TEXT NOT NULL,"
00065             + "species TEXT NOT NULL,"
00066             + "breed TEXT,"
00067             + "birth_date TEXT,"
00068             + "gender TEXT,"
00069             + "weight REAL DEFAULT 0,"
00070             + "notes TEXT,"
00071             + "can_talk INTEGER DEFAULT 0,"
00072             + "bird_type TEXT,"
00073             + "is_trained INTEGER DEFAULT 0,"
00074             + "is_indoor INTEGER DEFAULT 1"
00075             + ")";
00076     }
00077
00079     @Override
00083     protected String getInsertSql() {
00084         return "INSERT INTO pets (owner_id,name,species,breed,birth_date,gender,"
00085             + "weight,notes,can_talk,bird_type,is_trained,is_indoor) "
00086             + "VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";
00087     }
00088
00090     @Override
00094     protected String getUpdateSql() {
00095         return "UPDATE pets SET owner_id=?,name=?,species=?,breed=?,birth_date=,"
00096             + "gender=?,weight=?,notes=?,can_talk=?,bird_type=?,is_trained=?,is_indoor=? "
00097             + "WHERE id=?";
00098     }
00099
00101     @Override
00105     protected void setInsertParams(PreparedStatement ps, Pet pet) throws SQLException {
00106         ps.setInt(1, pet.getOwnerId());
00107         ps.setString(2, pet.getName());
00108         ps.setString(3, pet.getSpecies());
00109         ps.setString(4, getBreed(pet));
00110         ps.setString(5, pet.getBirthDate() != null ? pet.getBirthDate().toString() : null);
00111         ps.setString(6, pet.getGender());
00112         ps.setDouble(7, pet.getWeight());
00113         ps.setString(8, pet.getNotes());
00114         ps.setInt(9, (pet instanceof Bird && ((Bird) pet).isCanTalk()) ? 1 : 0);
00115         ps.setString(10, pet instanceof Bird ? ((Bird) pet).getBirdType() : null);
00116         ps.setInt(11, (pet instanceof Dog && ((Dog) pet).isTrained()) ? 1 : 0);
00117         ps.setInt(12, (pet instanceof Cat && ((Cat) pet).isIndoor()) ? 1 : 0);
00118     }
00119
00121     @Override
00125     protected void setUpdateParams(PreparedStatement ps, Pet pet) throws SQLException {
00126         setInsertParams(ps, pet);
00127         ps.setInt(13, pet.getId());

```

```

00128     }
00129
00131     @Override
00135     protected Pet mapRow(ResultSet rs) throws SQLException {
00136         String species = rs.getString("species");
00137         int id = rs.getInt("id");
00138         String name = rs.getString("name");
00139         String birthDateStr = rs.getString("birth_date");
00140         LocalDate birthDate = birthDateStr != null ? LocalDate.parse(birthDateStr) : null;
00141         int ownerId = rs.getInt("owner_id");
00142
00143         Pet pet;
00144         if ("Dog".equalsIgnoreCase(species) || "Kopek".equalsIgnoreCase(species)) {
00145             Dog dog = new Dog(id, name, birthDate, ownerId);
00146             dog.setBreed(rs.getString("breed"));
00147             dog.setTrained(rs.getInt("is_trained") == 1);
00148             pet = dog;
00149         } else if ("Cat".equalsIgnoreCase(species) || "Kedi".equalsIgnoreCase(species)) {
00150             Cat cat = new Cat(id, name, birthDate, ownerId);
00151             cat.setBreed(rs.getString("breed"));
00152             cat.setIndoor(rs.getInt("is_indoor") == 1);
00153             pet = cat;
00154         } else {
00155             Bird bird = new Bird(id, name, birthDate, ownerId);
00156             bird.setBirdType(rs.getString("bird_type"));
00157             bird.setCanTalk(rs.getInt("can_talk") == 1);
00158             pet = bird;
00159         }
00160         pet.setGender(rs.getString("gender"));
00161         pet.setWeight(rs.getDouble("weight"));
00162         pet.setNotes(rs.getString("notes"));
00163         return pet;
00164     }
00165
00171     private String getBreed(Pet pet) {
00172         if (pet instanceof Dog) return ((Dog) pet).getBreed();
00173         if (pet instanceof Cat) return ((Cat) pet).getBreed();
00174         return null;
00175     }
00176
00182     public List<Pet> findByOwnerId(int ownerId) {
00183         List<Pet> list = new ArrayList<>();
00184         String sql = "SELECT * FROM pets WHERE owner_id = ?";
00185         try (PreparedStatement ps = connection.prepareStatement(sql)) {
00186             ps.setInt(1, ownerId);
00187             ResultSet rs = ps.executeQuery();
00188             while (rs.next()) list.add(mapRow(rs));
00189         } catch (SQLException e) {
00190             throw new RepositoryException("findByOwnerId hatası", e);
00191         }
00192         return list;
00193     }
00194 }

```

8.77 petreminder- app/src/main/java/com/mehdi/petreminder/repository/ReminderMy SqlRepository.java File Reference

[Reminder](#) varlığı için MySQL repository.

```

import com.mehdi.petreminder.model.*;
import java.sql.*;
import java.time.LocalDateTime;
import java.util.*;

```

Classes

- class [com.mehdi.petreminder.repository.ReminderMySqlRepository](#)
MySQL üzerinde [Reminder](#) CRUD işlemleri.

Packages

- package [com.mehdi.petreminder.repository](#)

8.77.1 Detailed Description

[Reminder](#) varlığı için MySQL repository.

Definition in file [ReminderMySqlRepository.java](#).

8.78 ReminderMySqlRepository.java

[Go to the documentation of this file.](#)

```

00001
00005 package com.mehdi.petreminder.repository;
00006
00007 import com.mehdi.petreminder.model.*;
00008
00009 import java.sql.*;
00010 import java.time.LocalDateTime;
00011 import java.util.*;
00012
00021 public class ReminderMySqlRepository extends MySqlRepository<Reminder> {
00022
00026     public ReminderMySqlRepository() {
00027         super();
00028     }
00029
00034     public ReminderMySqlRepository(Connection connection) {
00035         super(connection);
00036     }
00037
00039     @Override
00043     protected String getTableName() {
00044         return "reminders";
00045     }
00046
00048     @Override
00052     protected String getCreateTableSql() {
00053         return "CREATE TABLE IF NOT EXISTS reminders ("
00054             + "id INT AUTO_INCREMENT PRIMARY KEY,"
00055             + "pet_id INT NOT NULL,"
00056             + "pet_name VARCHAR(50),"
00057             + "reminder_type VARCHAR(30) NOT NULL,"
00058             + "scheduled_time DATETIME NOT NULL,"
00059             + "completed BOOLEAN DEFAULT FALSE,"
00060             + "description TEXT,"
00061             + "priority VARCHAR(10) DEFAULT 'ORTA',"
00062             + "recurring BOOLEAN DEFAULT FALSE,"
00063             + "recurrence_pattern VARCHAR(30),"
00064             + "food_type VARCHAR(50),"
00065             + "portion_grams DOUBLE,"
00066             + "includes_water_change BOOLEAN DEFAULT FALSE,"
00067             + "medication_name VARCHAR(100),"
00068             + "dosage DOUBLE,"
00069             + "dosage_unit VARCHAR(20),"
00070             + "prescribed BOOLEAN DEFAULT FALSE,"
00071             + "exercise_type VARCHAR(50),"
00072             + "duration_minutes INT,"
00073             + "distance_km DOUBLE,"
00074             + "grooming_type VARCHAR(50),"
00075             + "professional BOOLEAN DEFAULT FALSE,"
00076             + "groomer_name VARCHAR(100),"
00077             + "veterinarian_name VARCHAR(100),"
00078             + "clinic_name VARCHAR(100),"
00079             + "reason VARCHAR(100),"
00080             + "estimated_cost DOUBLE,"
00081             + "confirmed BOOLEAN DEFAULT FALSE,"
00082             + "clinic_phone VARCHAR(20)"
00083             + ")";
00084     }
00085
00087     @Override
00091     protected String getInsertSql() {

```

```

00092         return "INSERT INTO reminders (pet_id,pet_name,reminder_type,scheduled_time,"
00093             + "completed,description,priority,recurring,recurrence_pattern,"
00094             + "food_type,portion_grams,includes_water_change,"
00095             + "medication_name,dosage,dosage_unit,prescribed,"
00096             + "exercise_type,duration_minutes,distance_km,"
00097             + "grooming_type,professional,groomer_name,"
00098             + "veterinarian_name,clinic_name,reason,estimated_cost,confirmed,clinic_phone) "
00099             + "VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";
00100     }
00101
00102     @Override
00103     protected String getUpdateSql() {
00104         return "UPDATE reminders SET pet_id=?,pet_name=?,reminder_type=?,scheduled_time=?"
00105             + "completed=?,description=?,priority=?,recurring=?,recurrence_pattern=?"
00106             + "food_type=?,portion_grams=?,includes_water_change=?"
00107             + "medication_name=?,dosage=?,dosage_unit=?,prescribed=?"
00108             + "exercise_type=?,duration_minutes=?,distance_km=?"
00109             + "grooming_type=?,professional=?,groomer_name=?"
00110             + "veterinarian_name=?,clinic_name=?,reason=?,estimated_cost=?,confirmed=?"
00111             + "clinic_phone=? WHERE id=?";
00112     }
00113
00114     @Override
00115     protected void setInsertParams(PreparedStatement ps, Reminder r) throws SQLException {
00116         ps.setInt(1, r.getPetId());
00117         ps.setString(2, r.getPetName());
00118         ps.setString(3, r.getReminderType());
00119         ps.setObject(4, r.getScheduledTime() != null
00120             ? Timestamp.valueOf(r.getScheduledTime()) : null);
00121         ps.setBoolean(5, r.isCompleted());
00122         ps.setString(6, r.getDescription());
00123         ps.setString(7, r.getPriority());
00124         ps.setBoolean(8, r.isRecurring());
00125         ps.setString(9, r.getRecurrencePattern());
00126         ps.setString(10, r instanceof FeedingReminder ? ((FeedingReminder) r).getFoodType() : null);
00127         ps.setObject(11, r instanceof FeedingReminder ? ((FeedingReminder) r).getPortionGrams() :
00128             null);
00129         ps.setBoolean(12, r instanceof FeedingReminder && ((FeedingReminder)
00130             r).isIncludesWaterChange());
00131         ps.setString(13, r instanceof MedicationReminder ? ((MedicationReminder)
00132             r).getMedicationName() : null);
00133         ps.setObject(14, r instanceof MedicationReminder ? ((MedicationReminder) r).getDosage() :
00134             null);
00135         ps.setString(15, r instanceof MedicationReminder ? ((MedicationReminder) r).getDosageUnit() :
00136             null);
00137         ps.setBoolean(16, r instanceof MedicationReminder && ((MedicationReminder) r).isPrescribed());
00138         ps.setString(17, r instanceof ExerciseReminder ? ((ExerciseReminder) r).getExerciseType() :
00139             null);
00140         ps.setObject(18, r instanceof ExerciseReminder ? ((ExerciseReminder) r).getDurationMinutes() :
00141             null);
00142         ps.setObject(19, r instanceof ExerciseReminder ? ((ExerciseReminder) r).getDistanceKm() :
00143             null);
00144         ps.setString(20, r instanceof GroomingReminder ? ((GroomingReminder) r).getGroomingType() :
00145             null);
00146         ps.setBoolean(21, r instanceof GroomingReminder && ((GroomingReminder) r).isProfessional());
00147         ps.setString(22, r instanceof GroomingReminder ? ((GroomingReminder) r).getGroomerName() :
00148             null);
00149         ps.setString(23, r instanceof VetAppointment ? ((VetAppointment) r).getVeterinarianName() :
00150             null);
00151         ps.setString(24, r instanceof VetAppointment ? ((VetAppointment) r).getClinicName() : null);
00152         ps.setString(25, r instanceof VetAppointment ? ((VetAppointment) r).getReason() : null);
00153         ps.setObject(26, r instanceof VetAppointment ? ((VetAppointment) r).getEstimatedCost() :
00154             null);
00155         ps.setBoolean(27, r instanceof VetAppointment && ((VetAppointment) r).isConfirmed());
00156         ps.setString(28, r instanceof VetAppointment ? ((VetAppointment) r).getClinicPhone() : null);
00157     }
00158
00159     @Override
00160     protected void setUpdateParams(PreparedStatement ps, Reminder r) throws SQLException {
00161         setInsertParams(ps, r);
00162         ps.setInt(29, r.getId());
00163     }
00164
00165     @Override
00166     protected Reminder mapRow(ResultSet rs) throws SQLException {
00167         String type = rs.getString("reminder_type");
00168         int id = rs.getInt("id");
00169         int petId = rs.getInt("pet_id");
00170         String petName = rs.getString("pet_name");
00171         Timestamp ts = rs.getTimestamp("scheduled_time");
00172         LocalDateTime scheduledTime = ts != null ? ts.toLocalDateTime() : null;
00173         String description = rs.getString("description");
00174         String priority = rs.getString("priority");
00175         boolean recurring = rs.getBoolean("recurring");
00176         String recurrencePattern = rs.getString("recurrence_pattern");
00177         boolean completed = rs.getBoolean("completed");

```

```

00183         Reminder reminder;
00184         switch (type != null ? type : "") {
00185             case "Besleme":
00186                 reminder = new FeedingReminder(id, petId, petName, scheduledTime,
00187                     description, priority, recurring, recurrencePattern,
00188                     rs.getString("food_type"), rs.getDouble("portion_grams"),
00189                     rs.getBoolean("includes_water_change"));
00190                 break;
00191             case "Ilac":
00192                 reminder = new MedicationReminder(id, petId, petName, scheduledTime,
00193                     description, priority, recurring, recurrencePattern,
00194                     rs.getString("medication_name"), rs.getDouble("dosage"),
00195                     rs.getString("dosage_unit"), rs.getBoolean("prescribed"), null);
00196                 break;
00197             case "Egzersiz":
00198                 reminder = new ExerciseReminder(id, petId, petName, scheduledTime,
00199                     description, priority, recurring, recurrencePattern,
00200                     rs.getString("exercise_type"), rs.getInt("duration_minutes"),
00201                     rs.getDouble("distance_km"));
00202                 break;
00203             case "Bakim":
00204                 reminder = new GroomingReminder(id, petId, petName, scheduledTime,
00205                     description, priority, recurring, recurrencePattern,
00206                     rs.getString("grooming_type"), rs.getBoolean("professional"),
00207                     rs.getString("groomer_name"));
00208                 break;
00209             case "Veteriner":
00210                 reminder = new VetAppointment(id, petId, petName, scheduledTime,
00211                     description, priority, recurring, recurrencePattern,
00212                     rs.getString("veterinarian_name"), rs.getString("clinic_name"),
00213                     rs.getString("reason"), rs.getDouble("estimated_cost"),
00214                     rs.getBoolean("confirmed"), rs.getString("clinic_phone"));
00215                 break;
00216             default:
00217                 reminder = new FeedingReminder(id, petId, petName, scheduledTime, "Mama", 100);
00218         }
00219         reminder.setCompleted(completed);
00220         return reminder;
00221     }
00222 }

```

8.79 petreminder-app/src/main/java/com/mehdi/petreminder/repository/ReminderSqliteRepository.java File Reference

[Reminder](#) varlığı için SQLite repository.

```

import com.mehdi.petreminder.model.*;
import java.sql.*;
import java.time.LocalDateTime;
import java.util.*;

```

Classes

- class [com.mehdi.petreminder.repository.ReminderSqliteRepository](#)
SQLite üzerinde [Reminder](#) (tüm alt türler) CRUD işlemleri.

Packages

- package [com.mehdi.petreminder.repository](#)

8.79.1 Detailed Description

[Reminder](#) varlığı için SQLite repository.

Definition in file [ReminderSQLiteRepository.java](#).

8.80 ReminderSQLiteRepository.java

[Go to the documentation of this file.](#)

```

00001
00005 package com.mehdi.petreminder.repository;
00006
00007 import com.mehdi.petreminder.model.*;
00008
00009 import java.sql.*;
00010 import java.time.LocalDateTime;
00011 import java.util.*;
00012
00022 public class ReminderSQLiteRepository extends SQLiteRepository<Reminder> {
00023
00027     public ReminderSQLiteRepository() {
00028         super();
00029     }
00030
00035     public ReminderSQLiteRepository(Connection connection) {
00036         super(connection);
00037     }
00038
00040     @Override
00044     protected String getTableName() {
00045         return "reminders";
00046     }
00047
00049     @Override
00053     protected String getCreateTableSql() {
00054         return "CREATE TABLE IF NOT EXISTS reminders ("
00055             + "id INTEGER PRIMARY KEY AUTOINCREMENT,"
00056             + "pet_id INTEGER NOT NULL,"
00057             + "pet_name TEXT,"
00058             + "reminder_type TEXT NOT NULL,"
00059             + "scheduled_time TEXT NOT NULL,"
00060             + "completed INTEGER DEFAULT 0,"
00061             + "description TEXT,"
00062             + "priority TEXT DEFAULT 'ORTA',"
00063             + "recurring INTEGER DEFAULT 0,"
00064             + "recurrence_pattern TEXT,"
00065             + "food_type TEXT,"
00066             + "portion_grams REAL,"
00067             + "includes_water_change INTEGER DEFAULT 0,"
00068             + "medication_name TEXT,"
00069             + "dosage REAL,"
00070             + "dosage_unit TEXT,"
00071             + "prescribed INTEGER DEFAULT 0,"
00072             + "exercise_type TEXT,"
00073             + "duration_minutes INTEGER,"
00074             + "distance_km REAL,"
00075             + "grooming_type TEXT,"
00076             + "professional INTEGER DEFAULT 0,"
00077             + "groomer_name TEXT,"
00078             + "veterinarian_name TEXT,"
00079             + "clinic_name TEXT,"
00080             + "reason TEXT,"
00081             + "estimated_cost REAL,"
00082             + "confirmed INTEGER DEFAULT 0,"
00083             + "clinic_phone TEXT"
00084             + ")";
00085     }
00086
00088     @Override
00092     protected String getInsertSql() {
00093         return "INSERT INTO reminders (pet_id,pet_name,reminder_type,scheduled_time,"
00094             + "completed,description,priority,recurring,recurrence_pattern,"
00095             + "food_type,portion_grams,includes_water_change,"
00096             + "medication_name,dosage,dosage_unit,prescribed,"
00097             + "exercise_type,duration_minutes,distance_km,"
00098             + "grooming_type,professional,groomer_name,"
00099             + "veterinarian_name,clinic_name,reason,estimated_cost,confirmed,clinic_phone) "
00100             + "VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";

```



```

00101     }
00102
00104     @Override
00108     protected String getUpdateSql() {
00109         return "UPDATE reminders SET pet_id=?,pet_name=?,reminder_type=?,scheduled_time=?,
00110             + "completed=?,description=?,priority=?,recurring=?,recurrence_pattern=?",
00111             + "food_type=?,portion_grams=?,includes_water_change=?",
00112             + "medication_name=?,dosage=?,dosage_unit=?,prescribed=?",
00113             + "exercise_type=?,duration_minutes=?,distance_km=?",
00114             + "grooming_type=?,professional=?,groomer_name=?",
00115             + "veterinarian_name=?,clinic_name=?,reason=?,estimated_cost=?,confirmed=?",
00116             + "clinic_phone=? WHERE id=?";
00117     }
00118
00120     @Override
00124     protected void setInsertParams(PreparedStatement ps, Reminder r) throws SQLException {
00125         ps.setInt(1, r.getPetId());
00126         ps.setString(2, r.getPetName());
00127         ps.setString(3, r.getReminderType());
00128         ps.setString(4, r.getScheduledTime() != null ? r.getScheduledTime().toString() : null);
00129         ps.setInt(5, r.isCompleted() ? 1 : 0);
00130         ps.setString(6, r.getDescription());
00131         ps.setString(7, r.getPriority());
00132         ps.setInt(8, r.isRecurring() ? 1 : 0);
00133         ps.setString(9, r.getRecurrencePattern());
00134         // FeedingReminder
00135         ps.setString(10, r instanceof FeedingReminder ? ((FeedingReminder) r).getFoodType() : null);
00136         ps.setObject(11, r instanceof FeedingReminder ? ((FeedingReminder) r).getPortionGrams() :
00137             null);
00138         ps.setInt(12, r instanceof FeedingReminder && ((FeedingReminder) r).isIncludesWaterChange() ?
00139             1 : 0);
00140         // MedicationReminder
00141         ps.setString(13, r instanceof MedicationReminder ? ((MedicationReminder)
00142             r).getMedicationName() : null);
00143         ps.setObject(14, r instanceof MedicationReminder ? ((MedicationReminder) r).getDosage() :
00144             null);
00145         ps.setString(15, r instanceof MedicationReminder ? ((MedicationReminder) r).getDosageUnit() :
00146             null);
00147         ps.setInt(16, r instanceof MedicationReminder && ((MedicationReminder) r).isPrescribed() ? 1 :
00148             0);
00149         // ExerciseReminder
00150         ps.setString(17, r instanceof ExerciseReminder ? ((ExerciseReminder) r).getExerciseType() :
00151             null);
00152         ps.setObject(18, r instanceof ExerciseReminder ? ((ExerciseReminder) r).getDurationMinutes() :
00153             null);
00154         ps.setObject(19, r instanceof ExerciseReminder ? ((ExerciseReminder) r).getDistanceKm() :
00155             null);
00156         // GroomingReminder
00157         ps.setString(20, r instanceof GroomingReminder ? ((GroomingReminder) r).getGroomingType() :
00158             null);
00159         ps.setInt(21, r instanceof GroomingReminder && ((GroomingReminder) r).isProfessional() ? 1 :
00160             0);
00161         ps.setString(22, r instanceof GroomingReminder ? ((GroomingReminder) r).getGroomerName() :
00162             null);
00163         // VetAppointment
00164         ps.setString(23, r instanceof VetAppointment ? ((VetAppointment) r).getVeterinarianName() :
00165             null);
00166         ps.setString(24, r instanceof VetAppointment ? ((VetAppointment) r).getClinicName() : null);
00167         ps.setString(25, r instanceof VetAppointment ? ((VetAppointment) r).getReason() : null);
00168         ps.setObject(26, r instanceof VetAppointment ? ((VetAppointment) r).getEstimatedCost() :
00169             null);
00170         ps.setInt(27, r instanceof VetAppointment && ((VetAppointment) r).isConfirmed() ? 1 : 0);
00171         ps.setString(28, r instanceof VetAppointment ? ((VetAppointment) r).getClinicPhone() : null);
00172     }
00173
00174     @Override
00175     protected void setUpdateParams(PreparedStatement ps, Reminder r) throws SQLException {
00176         setInsertParams(ps, r);
00177         ps.setInt(29, r.getId());
00178     }
00179
00180     @Override
00181     protected Reminder mapRow(ResultSet rs) throws SQLException {
00182         String type = rs.getString("reminder_type");
00183         int id = rs.getInt("id");
00184         int petId = rs.getInt("pet_id");
00185         String petName = rs.getString("pet_name");
00186         String timeStr = rs.getString("scheduled_time");
00187         LocalDateTime scheduledTime = timeStr != null ? LocalDateTime.parse(timeStr) : null;
00188         String description = rs.getString("description");
00189         String priority = rs.getString("priority");
00190         boolean recurring = rs.getInt("recurring") == 1;
00191         String recurrencePattern = rs.getString("recurrence_pattern");
00192         boolean completed = rs.getInt("completed") == 1;
00193
00194         Reminder reminder;
00195         switch (type) {

```

```

00190         case "Besleme":
00191             FeedingReminder fr = new FeedingReminder(id, petId, petName, scheduledTime,
00192                 description, priority, recurring, recurrencePattern,
00193                 rs.getString("food_type"), rs.getDouble("portion_grams"),
00194                 rs.getInt("includes_water_change") == 1);
00195             reminder = fr;
00196             break;
00197         case "Ilac":
00198             MedicationReminder mr = new MedicationReminder(id, petId, petName, scheduledTime,
00199                 description, priority, recurring, recurrencePattern,
00200                 rs.getString("medication_name"), rs.getDouble("dosage"),
00201                 rs.getString("dosage_unit"), rs.getInt("prescribed") == 1, null);
00202             reminder = mr;
00203             break;
00204         case "Egzersiz":
00205             ExerciseReminder er = new ExerciseReminder(id, petId, petName, scheduledTime,
00206                 description, priority, recurring, recurrencePattern,
00207                 rs.getString("exercise_type"), rs.getInt("duration_minutes"),
00208                 rs.getDouble("distance_km"));
00209             reminder = er;
00210             break;
00211         case "Bakim":
00212             GroomingReminder gr = new GroomingReminder(id, petId, petName, scheduledTime,
00213                 description, priority, recurring, recurrencePattern,
00214                 rs.getString("grooming_type"), rs.getInt("professional") == 1,
00215                 rs.getString("groomer_name"));
00216             reminder = gr;
00217             break;
00218         case "Veteriner":
00219             VetAppointment va = new VetAppointment(id, petId, petName, scheduledTime,
00220                 description, priority, recurring, recurrencePattern,
00221                 rs.getString("veterinarian_name"), rs.getString("clinic_name"),
00222                 rs.getString("reason"), rs.getDouble("estimated_cost"),
00223                 rs.getInt("confirmed") == 1, rs.getString("clinic_phone"));
00224             reminder = va;
00225             break;
00226         default:
00227             FeedingReminder def = new FeedingReminder(id, petId, petName, scheduledTime,
00228                 "Mama", 100);
00229             reminder = def;
00230     }
00231     reminder.setCompleted(completed);
00232     return reminder;
00233 }
00234
00240 public List<Reminder> findByPetId(int petId) {
00241     List<Reminder> list = new ArrayList<>();
00242     try (PreparedStatement ps = connection.prepareStatement(
00243         "SELECT * FROM reminders WHERE pet_id = ?")) {
00244         ps.setInt(1, petId);
00245         ResultSet rs = ps.executeQuery();
00246         while (rs.next()) list.add(mapRow(rs));
00247     } catch (SQLException e) {
00248         throw new RepositoryException("findByPetId hatası", e);
00249     }
00250     return list;
00251 }
00252 }

```

8.81 petreminder-

app/src/main/java/com/mehdi/petreminder/repository/RepositoryException.java File Reference

Repository katmanı için özel istisna sınıfı.

Classes

- class [com.mehdi.petreminder.repository.RepositoryException](#)
Repository işlemlerinde fırlatılan unchecked istisna.

Packages

- package [com.mehdi.petreminder.repository](#)

8.81.1 Detailed Description

Repository katmanı için özel istisna sınıfı.

Definition in file [RepositoryException.java](#).

8.82 RepositoryException.java

[Go to the documentation of this file.](#)

```
00001
00005 package com.mehdi.petreminder.repository;
00006
00016 public class RepositoryException extends RuntimeException {
00017
00019     private static final long serialVersionUID = 1L;
00020
00025     public RepositoryException(String message) {
00026         super(message);
00027     }
00028
00034     public RepositoryException(String message, Throwable cause) {
00035         super(message, cause);
00036     }
00037 }
```

8.83 petreminder-app/src/main/java/com/mehdi/petreminder/repository/RepositoryFactory.java File Reference

Repository nesnesi üretici — Factory Pattern.

```
import com.mehdi.petreminder.config.StorageConfig;
import com.mehdi.petreminder.config.StorageType;
import com.mehdi.petreminder.model.*;
import org.slf4j.LoggerFactory;
import ch.qos.logback.classic.Logger;
```

Classes

- class [com.mehdi.petreminder.repository.RepositoryFactory](#)
Aktif storage backend'e göre repository üretir.

Packages

- package [com.mehdi.petreminder.repository](#)

8.83.1 Detailed Description

Repository nesnesi üretici — Factory Pattern.

PDF zorunluluğu: RepositoryFactory with runtime backend switch.

Definition in file [RepositoryFactory.java](#).

8.84 RepositoryFactory.java

[Go to the documentation of this file.](#)

```

00001
00006 package com.mehdi.petreminder.repository;
00007
00008 import com.mehdi.petreminder.config.StorageConfig;
00009 import com.mehdi.petreminder.config.StorageType;
00010 import com.mehdi.petreminder.model.*;
00011 import org.slf4j.LoggerFactory;
00012 import ch.qos.logback.classic.Logger;
00013
00025 public class RepositoryFactory {
00026
00028     private static final Logger logger =
00029         (Logger) LoggerFactory.getLogger(RepositoryFactory.class);
00030
00032     private static final String BINARY_DIR = "data/binary";
00033
00035     private RepositoryFactory() { }
00036
00042     public static IRepository<Pet> createPetRepository() {
00043         StorageType type = StorageConfig.getActiveBackend();
00044         logger.info("Pet repository oluşturuluyor: {}", type);
00045         switch (type) {
00046             case SQLITE: return new PetSqliteRepository();
00047             case MYSQL:  return new PetMySqlRepository();
00048             case BINARY:
00049                 default: return new BinaryRepository<>(BINARY_DIR, "pets");
00050         }
00051     }
00052
00057     public static IRepository<Reminder> createReminderRepository() {
00058         StorageType type = StorageConfig.getActiveBackend();
00059         logger.info("Reminder repository oluşturuluyor: {}", type);
00060         switch (type) {
00061             case SQLITE: return new ReminderSqliteRepository();
00062             case MYSQL:  return new ReminderMySqlRepository();
00063             case BINARY:
00064                 default: return new BinaryRepository<>(BINARY_DIR, "reminders");
00065         }
00066     }
00067
00072     public static IRepository<User> createUserRepository() {
00073         StorageType type = StorageConfig.getActiveBackend();
00074         logger.info("User repository oluşturuluyor: {}", type);
00075         switch (type) {
00076             case SQLITE: return new UserSqliteRepository();
00077             case MYSQL:  return new UserMySqlRepository();
00078             case BINARY:
00079                 default: return new BinaryRepository<>(BINARY_DIR, "users");
00080         }
00081     }
00082
00087     public static IRepository<MedicalRecord> createMedicalRecordRepository() {
00088         StorageType type = StorageConfig.getActiveBackend();
00089         logger.info("MedicalRecord repository oluşturuluyor: {}", type);
00090         switch (type) {
00091             case SQLITE: return new MedicalRecordSqliteRepository();
00092             case MYSQL:  return new MedicalRecordMySqlRepository();
00093             case BINARY:
00094                 default: return new BinaryRepository<>(BINARY_DIR, "medical_records");
00095         }
00096     }
00097
00102     public static StorageType getActiveType() {
00103         return StorageConfig.getActiveBackend();
00104     }
00105 }

```

8.85 petreminder-app/src/main/java/com/mehdi/petreminder/repository/SqliteRepository.java File Reference

SQLite JDBC repository — org.xerial:sqlite-jdbc.

```
import com.mehdi.petreminder.config.StorageConfig;
import org.slf4j.LoggerFactory;
import ch.qos.logback.classic.Logger;
import java.sql.*;
import java.util.*;
```

Classes

- class [com.mehdi.petreminder.repository.SqliteRepository< T >](#)
SQLite tabanlı abstract repository temel sınıfı.

Packages

- package [com.mehdi.petreminder.repository](#)

8.85.1 Detailed Description

SQLite JDBC repository — org.xerial:sqlite-jdbc.

PDF LO.6: SQLite storage backend.

Definition in file [SqliteRepository.java](#).

8.86 SqliteRepository.java

[Go to the documentation of this file.](#)

```
00001
00006 package com.mehdi.petreminder.repository;
00007
00008 import com.mehdi.petreminder.config.StorageConfig;
00009 import org.slf4j.LoggerFactory;
00010 import ch.qos.logback.classic.Logger;
00011
00012 import java.sql.*;
00013 import java.util.*;
00014
00027 public abstract class SqliteRepository<T> implements IRepository<T> {
00028
00030     protected static final Logger logger =
00031         (Logger) LoggerFactory.getLogger(SqliteRepository.class);
00032
00034     protected Connection connection;
00035
00040     public SqliteRepository() {
00041         try {
00042             Class.forName("org.sqlite.JDBC");
00043             String url = "jdbc:sqlite:" + StorageConfig.getSqliteFilePath();
00044             connection = DriverManager.getConnection(url);
00045             connection.setAutoCommit(true);
00046             createTableIfNotExists();
00047             logger.info("SQLite bağlantısı açıldı: {}", StorageConfig.getSqliteFilePath());
00048         } catch (Exception e) {
00049             throw new RepositoryException("SQLite bağlantı hatası", e);
00050         }
00051     }
00052
00057     public SqliteRepository(Connection connection) {
00058         this.connection = connection;
00059         try {
00060             createTableIfNotExists();
00061         } catch (Exception e) {
00062             throw new RepositoryException("Tablo oluşturma hatası", e);
00062         }
00062     }
00062 }
```

```

00063     }
00064 }
00065
00070 protected abstract String getTableName();
00071
00076 protected abstract String getCreateTableSql();
00077
00082 protected abstract String getInsertSql();
00083
00088 protected abstract String getUpdateSql();
00089
00096 protected abstract void setInsertParams(PreparedStatement ps, T entity)
00097     throws SQLException;
00098
00105 protected abstract void setUpdateParams(PreparedStatement ps, T entity)
00106     throws SQLException;
00107
00114 protected abstract T mapRow(ResultSet rs) throws SQLException;
00115
00119 protected void createTableIfNotExists() {
00120     try (Statement st = connection.createStatement()) {
00121         String sql = getCreateTableSql();
00122         // H2 does not support SQLite's AUTOINCREMENT; use AUTO_INCREMENT instead
00123         String dbProduct = connection.getMetaData().getDatabaseProductName();
00124         if ("H2".equalsIgnoreCase(dbProduct)) {
00125             sql = sql.replace("AUTOINCREMENT", "AUTO_INCREMENT");
00126         }
00127         st.execute(sql);
00128     } catch (SQLException e) {
00129         throw new RepositoryException("Tablo oluşturma hatası", e);
00130     }
00131 }
00132
00136 @Override
00137 public int save(T entity) {
00138     if (entity == null) throw new RepositoryException("Entity null olamaz");
00139     try (PreparedStatement ps = connection.prepareStatement(
00140         getInsertSql(), Statement.RETURN_GENERATED_KEYS)) {
00141         setInsertParams(ps, entity);
00142         ps.executeUpdate();
00143         ResultSet keys = ps.getGeneratedKeys();
00144         if (keys.next()) {
00145             int id = keys.getInt(1);
00146             setEntityId(entity, id);
00147             logger.debug("SQLite kayıt: id={}", id);
00148             return id;
00149         }
00150         throw new RepositoryException("ID üretilemedi");
00151     } catch (SQLException e) {
00152         throw new RepositoryException("SQLite save hatası", e);
00153     }
00154 }
00155
00159 @Override
00160 public Optional<T> findById(int id) {
00161     String sql = "SELECT * FROM " + getTableName() + " WHERE id = ?";
00162     try (PreparedStatement ps = connection.prepareStatement(sql)) {
00163         ps.setInt(1, id);
00164         ResultSet rs = ps.executeQuery();
00165         if (rs.next()) {
00166             return Optional.of(mapRow(rs));
00167         }
00168     } catch (SQLException e) {
00169         throw new RepositoryException("SQLite findById hatası", e);
00170     }
00171     return Optional.empty();
00172 }
00173
00177 @Override
00178 public List<T> findAll() {
00179     String sql = "SELECT * FROM " + getTableName();
00180     List<T> list = new ArrayList<>();
00181     try (Statement st = connection.createStatement();
00182         ResultSet rs = st.executeQuery(sql)) {
00183         while (rs.next()) {
00184             list.add(mapRow(rs));
00185         }
00186     } catch (SQLException e) {
00187         throw new RepositoryException("SQLite findAll hatası", e);
00188     }
00189     return list;
00190 }
00191
00195 @Override
00196 public boolean update(T entity) {
00197     if (entity == null) return false;
00198     try (PreparedStatement ps = connection.prepareStatement(getUpdateSql())) {

```

```

00199         setUpdateParams(ps, entity);
00200         int rows = ps.executeUpdate();
00201         return rows > 0;
00202     } catch (SQLException e) {
00203         throw new RepositoryException("SQLite update hatası", e);
00204     }
00205 }
00206
00210 @Override
00211 public boolean delete(int id) {
00212     String sql = "DELETE FROM " + getTableName() + " WHERE id = ?";
00213     try (PreparedStatement ps = connection.prepareStatement(sql)) {
00214         ps.setInt(1, id);
00215         int rows = ps.executeUpdate();
00216         return rows > 0;
00217     } catch (SQLException e) {
00218         throw new RepositoryException("SQLite delete hatası", e);
00219     }
00220 }
00221
00225 @Override
00226 public void deleteAll() {
00227     try (Statement st = connection.createStatement()) {
00228         st.execute("DELETE FROM " + getTableName());
00229     } catch (SQLException e) {
00230         throw new RepositoryException("SQLite deleteAll hatası", e);
00231     }
00232 }
00233
00237 @Override
00238 public int count() {
00239     String sql = "SELECT COUNT(*) FROM " + getTableName();
00240     try (Statement st = connection.createStatement();
00241          ResultSet rs = st.executeQuery(sql)) {
00242         if (rs.next()) return rs.getInt(1);
00243     } catch (SQLException e) {
00244         throw new RepositoryException("SQLite count hatası", e);
00245     }
00246     return 0;
00247 }
00248
00252 @Override
00253 public void close() {
00254     try {
00255         if (connection != null && !connection.isClosed()) {
00256             connection.close();
00257             logger.info("SQLite bağlantısı kapatıldı.");
00258         }
00259     } catch (SQLException e) {
00260         logger.warn("SQLite kapatma hatası: {}", e.getMessage());
00261     }
00262 }
00263
00268 public Connection getConnection() {
00269     return connection;
00270 }
00271
00277 protected void setEntityId(T entity, int id) {
00278     try {
00279         var method = entity.getClass().getMethod("setId", int.class);
00280         method.invoke(entity, id);
00281     } catch (Exception e) {
00282         logger.warn("setId çağrılmadı: {}", e.getMessage());
00283     }
00284 }
00285 }

```

8.87 petreminder-app/src/main/java/com/mehdi/petreminder/repository/UserMySqlRepository.java File Reference

User varlığı için MySQL repository.

```

import com.mehdi.petreminder.model.User;
import java.sql.*;
import java.time.LocalDateTime;

```

Classes

- class [com.mehdi.petreminder.repository.UserMySqlRepository](#)
MySQL üzerinde [User](#) CRUD işlemleri.

Packages

- package [com.mehdi.petreminder.repository](#)

8.87.1 Detailed Description

User varlığı için MySQL repository.

Definition in file [UserMySqlRepository.java](#).

8.88 UserMySqlRepository.java

[Go to the documentation of this file.](#)

```

00001
00005 package com.mehdi.petreminder.repository;
00006
00007 import com.mehdi.petreminder.model.User;
00008
00009 import java.sql.*;
00010 import java.time.LocalDateTime;
00011
00020 public class UserMySqlRepository extends MySqlRepository<User> {
00021
00023     public UserMySqlRepository() { super(); }
00024
00029     public UserMySqlRepository(Connection connection) { super(connection); }
00030
00032     @Override
00036     protected String getTableName() { return "users"; }
00037
00039     @Override
00043     protected String getCreateTableSql() {
00044         return "CREATE TABLE IF NOT EXISTS users ("
00045             + "id INT AUTO_INCREMENT PRIMARY KEY,"
00046             + "username VARCHAR(50) NOT NULL UNIQUE,"
00047             + "email VARCHAR(100) NOT NULL UNIQUE,"
00048             + "password_hash VARCHAR(255) NOT NULL,"
00049             + "full_name VARCHAR(100),"
00050             + "created_at DATETIME,"
00051             + "last_login DATETIME,"
00052             + "active BOOLEAN DEFAULT TRUE"
00053             + ")";
00054     }
00055
00057     @Override
00061     protected String getInsertSql() {
00062         return "INSERT INTO users (username,email,password_hash,full_name,created_at,active)"
00063             + " VALUES (?, ?, ?, ?, ?, ?)";
00064     }
00065
00067     @Override
00071     protected String getUpdateSql() {
00072         return "UPDATE users SET username=?,email=?,password_hash=?,full_name=,"
00073             + "created_at=?,active=? WHERE id=?";
00074     }
00075
00077     @Override
00081     protected void setInsertParams(PreparedStatement ps, User u) throws SQLException {
00082         ps.setString(1, u.getUsername());
00083         ps.setString(2, u.getEmail());
00084         ps.setString(3, u.getPasswordHash());
00085         ps.setString(4, u.getFullName());
00086         ps.setObject(5, u.getCreatedAt() != null
00087             ? Timestamp.valueOf(u.getCreatedAt()) : null);

```



```
00088         ps.setBoolean(6, u.isActive());
00089     }
00090
00092     @Override
00096     protected void setUpdateParams(PreparedStatement ps, User u) throws SQLException {
00097         setInsertParams(ps, u);
00098         ps.setInt(7, u.getId());
00099     }
00100
00102     @Override
00106     protected User mapRow(ResultSet rs) throws SQLException {
00107         User u = new User();
00108         u.setId(rs.getInt("id"));
00109         u.setUsername(rs.getString("username"));
00110         u.setEmail(rs.getString("email"));
00111         u.setPasswordHash(rs.getString("password_hash"));
00112         u.setFullName(rs.getString("full_name"));
00113         Timestamp ts = rs.getTimestamp("created_at");
00114         if (ts != null) u.setCreatedAt(ts.toLocalDateTime());
00115         u.setActive(rs.getBoolean("active"));
00116         return u;
00117     }
00118 }
```

8.89 petreminder-app/src/main/java/com/mehdi/petreminder/repository/UserSqliteRepository.java File Reference

User varlığı için SQLite repository.

```
import com.mehdi.petreminder.model.User;
import java.sql.*;
import java.time.LocalDateTime;
```

Classes

- class [com.mehdi.petreminder.repository.UserSqliteRepository](#)
SQLite üzerinde [User](#) CRUD işlemleri.

Packages

- package [com.mehdi.petreminder.repository](#)

8.89.1 Detailed Description

User varlığı için SQLite repository.

Definition in file [UserSqliteRepository.java](#).

8.90 UserSqliteRepository.java

[Go to the documentation of this file.](#)

```

00001
00005 package com.mehdi.petreminder.repository;
00006
00007 import com.mehdi.petreminder.model.User;
00008
00009 import java.sql.*;
00010 import java.time.LocalDateTime;
00011
00020 public class UserSqliteRepository extends SqliteRepository<User> {
00021
00023     public UserSqliteRepository() { super(); }
00024
00029     public UserSqliteRepository(Connection connection) { super(connection); }
00030
00032     @Override
00036     protected String getTableName() { return "users"; }
00037
00039     @Override
00043     protected String getCreateTableSql() {
00044         return "CREATE TABLE IF NOT EXISTS users ("
00045             + "id INTEGER PRIMARY KEY AUTOINCREMENT,"
00046             + "username TEXT NOT NULL UNIQUE,"
00047             + "email TEXT NOT NULL UNIQUE,"
00048             + "password_hash TEXT NOT NULL,"
00049             + "full_name TEXT,"
00050             + "created_at TEXT,"
00051             + "last_login TEXT,"
00052             + "active INTEGER DEFAULT 1"
00053             + ")";
00054     }
00055
00057     @Override
00061     protected String getInsertSql() {
00062         return "INSERT INTO users (username,email,password_hash,full_name,created_at,active)"
00063             + " VALUES (?, ?, ?, ?, ?, ?)";
00064     }
00065
00067     @Override
00071     protected String getUpdateSql() {
00072         return "UPDATE users SET username=?,email=?,password_hash=?,full_name=?,
00073             + "created_at=?,active=? WHERE id=?";
00074     }
00075
00077     @Override
00081     protected void setInsertParams(PreparedStatement ps, User u) throws SQLException {
00082         ps.setString(1, u.getUsername());
00083         ps.setString(2, u.getEmail());
00084         ps.setString(3, u.getPasswordHash());
00085         ps.setString(4, u.getFullName());
00086         ps.setString(5, u.getCreatedAt() != null ? u.getCreatedAt().toString() : null);
00087         ps.setInt(6, u.isActive() ? 1 : 0);
00088     }
00089
00091     @Override
00095     protected void setUpdateParams(PreparedStatement ps, User u) throws SQLException {
00096         setInsertParams(ps, u);
00097         ps.setInt(7, u.getId());
00098     }
00099
00101     @Override
00105     protected User mapRow(ResultSet rs) throws SQLException {
00106         User u = new User();
00107         u.setId(rs.getInt("id"));
00108         u.setUsername(rs.getString("username"));
00109         u.setEmail(rs.getString("email"));
00110         u.setPasswordHash(rs.getString("password_hash"));
00111         u.setFullName(rs.getString("full_name"));
00112         String createdAt = rs.getString("created_at");
00113         if (createdAt != null) u.setCreatedAt(LocalDateTime.parse(createdAt));
00114         u.setActive(rs.getInt("active") == 1);
00115         return u;
00116     }
00117
00123     public User findByUsername(String username) {
00124         try (PreparedStatement ps = connection.prepareStatement(
00125             "SELECT * FROM users WHERE username = ?") {
00126             ps.setString(1, username);
00127             ResultSet rs = ps.executeQuery();
00128             if (rs.next()) return mapRow(rs);
00129         } catch (SQLException e) {
00130             throw new RepositoryException("findByUsername hatası", e);
00131         }

```

```
00132         return null;
00133     }
00134 }
```

8.91 petreminder-app/src/main/java/com/mehdi/petreminder/service/MedicalRecordService.java File Reference

Sağlık kayıtları servis katmanı.

```
import com.mehdi.petreminder.model.MedicalRecord;
import com.mehdi.petreminder.observer.EventManager;
import com.mehdi.petreminder.observer.EventType;
import com.mehdi.petreminder.repository.IRepository;
import com.mehdi.petreminder.repository.RepositoryFactory;
import org.slf4j.LoggerFactory;
import ch.qos.logback.classic.Logger;
import java.util.*;
import java.util.stream.Collectors;
```

Classes

- class [com.mehdi.petreminder.service.MedicalRecordService](#)
MedicalRecord CRUD ve iş kuralları.

Packages

- package [com.mehdi.petreminder.service](#)

8.91.1 Detailed Description

Sağlık kayıtları servis katmanı.

Observer Pattern: CRUD işlemlerinde EventManager'a bildirim gönderir.

Definition in file [MedicalRecordService.java](#).

8.92 MedicalRecordService.java

[Go to the documentation of this file.](#)

```
00001
00009 package com.mehdi.petreminder.service;
00010
00011 import com.mehdi.petreminder.model.MedicalRecord;
00012 import com.mehdi.petreminder.observer.EventManager;
00013 import com.mehdi.petreminder.observer.EventType;
00014 import com.mehdi.petreminder.repository.IRepository;
00015 import com.mehdi.petreminder.repository.RepositoryFactory;
00016 import org.slf4j.LoggerFactory;
00017 import ch.qos.logback.classic.Logger;
00018
00019 import java.util.*;
00020 import java.util.stream.Collectors;
```

```

00021
00032 public class MedicalRecordService {
00033
00035     private static final Logger logger =
00036         (Logger) LoggerFactory.getLogger(MedicalRecordService.class);
00037
00039     private final IRepository<MedicalRecord> repository;
00040
00044     public MedicalRecordService() {
00045         this.repository = RepositoryFactory.createMedicalRecordRepository();
00046     }
00047
00052     public MedicalRecordService(IRepository<MedicalRecord> repository) {
00053         this.repository = repository;
00054     }
00055
00061     public MedicalRecord addRecord(MedicalRecord record) {
00062         validateRecord(record);
00063         repository.save(record);
00064         logger.info("MedicalRecord eklendi: petId={}", record.getPetId());
00065         // Observer Pattern: Saglik kaydi eklendi bildirimi
00066         EventManager.getInstance().notify(EventType.MEDICAL_RECORD_ADDED, record);
00067         return record;
00068     }
00069
00075     public MedicalRecord getRecordById(int id) {
00076         return repository.findById(id)
00077             .orElseThrow(() -> new ServiceException("Kayıt bulunamadı: id=" + id));
00078     }
00079
00084     public List<MedicalRecord> getAllRecords() {
00085         return repository.findAll();
00086     }
00087
00093     public List<MedicalRecord> getRecordsByPetId(int petId) {
00094         return repository.findAll().stream()
00095             .filter(r -> r.getPetId() == petId)
00096             .sorted(Comparator.comparing(
00097                 MedicalRecord::getRecordDate,
00098                 Comparator.nullsLast(Comparator.reverseOrder())))
00099             .collect(Collectors.toList());
00100     }
00101
00106     public void updateRecord(MedicalRecord record) {
00107         validateRecord(record);
00108         boolean updated = repository.update(record);
00109         if (!updated) throw new ServiceException("Kayıt güncellenemedi: id=" + record.getId());
00110     }
00111
00116     public void deleteRecord(int id) {
00117         boolean deleted = repository.delete(id);
00118         if (!deleted) throw new ServiceException("Kayıt silinemedi: id=" + id);
00119         logger.info("MedicalRecord silindi: id={}", id);
00120         // Observer Pattern: Saglik kaydi silindi bildirimi
00121         EventManager.getInstance().notify(EventType.MEDICAL_RECORD_DELETED, id);
00122     }
00123
00127     public void deleteAllRecords() {
00128         repository.deleteAll();
00129     }
00130
00135     public int getRecordCount() {
00136         return repository.count();
00137     }
00138
00142     public void close() {
00143         repository.close();
00144     }
00145
00150     private void validateRecord(MedicalRecord r) {
00151         if (r == null) throw new ServiceException("Kayıt null olamaz");
00152         if (r.getPetId() <= 0) throw new ServiceException("Geçerli pet ID gerekli");
00153         if (r.getRecordDate() == null) throw new ServiceException("Kayıt tarihi boş olamaz");
00154         if (r.getRecordType() == null || r.getRecordType().trim().isEmpty()) {
00155             throw new ServiceException("Kayıt türü boş olamaz");
00156         }
00157     }
00158 }

```

8.93 petreminder-app/src/main/java/com/mehdi/petreminder/service/↵ PetService.java File Reference

[Pet](#) iş mantığı servis katmanı.

```
import com.mehdi.petreminder.model.*;
import com.mehdi.petreminder.observer.EventManager;
import com.mehdi.petreminder.observer.EventType;
import com.mehdi.petreminder.repository.IRepository;
import com.mehdi.petreminder.repository.RepositoryFactory;
import org.slf4j.LoggerFactory;
import ch.qos.logback.classic.Logger;
import java.time.LocalDate;
import java.util.*;
import java.util.stream.Collectors;
```

Classes

- class [com.mehdi.petreminder.service.PetService](#)
[Pet](#) CRUD iş mantığı katmanı.

Packages

- package [com.mehdi.petreminder.service](#)

8.93.1 Detailed Description

[Pet](#) iş mantığı servis katmanı.

Repository üzerinde iş kuralları uygular. Observer Pattern: CRUD işlemlerinde EventManager'a bildirim gönderir.

Definition in file [PetService.java](#).

8.94 PetService.java

[Go to the documentation of this file.](#)

```
00001
00007 package com.mehdi.petreminder.service;
00008
00009 import com.mehdi.petreminder.model.*;
00010 import com.mehdi.petreminder.observer.EventManager;
00011 import com.mehdi.petreminder.observer.EventType;
00012 import com.mehdi.petreminder.repository.IRepository;
00013 import com.mehdi.petreminder.repository.RepositoryFactory;
00014 import org.slf4j.LoggerFactory;
00015 import ch.qos.logback.classic.Logger;
00016
00017 import java.time.LocalDate;
00018 import java.util.*;
00019 import java.util.stream.Collectors;
00020
00033 public class PetService {
00034
00036     private static final Logger logger =
00037         (Logger) LoggerFactory.getLogger(PetService.class);
```

```

00038
00040     private final IRepository<Pet> repository;
00041
00045     public PetService() {
00046         this.repository = RepositoryFactory.createPetRepository();
00047     }
00048
00053     public PetService(IRepository<Pet> repository) {
00054         this.repository = repository;
00055     }
00056
00063     public Pet addPet(Pet pet) {
00064         validatePet(pet);
00065         int id = repository.save(pet);
00066         logger.info("Pet eklendi: id={}, name={}", id, pet.getName());
00067         // Observer Pattern: Pet eklendi bildirimi
00068         EventManager.getInstance().notify(EventType.PET_ADDED, pet);
00069         return pet;
00070     }
00071
00078     public Pet getPetById(int id) {
00079         return repository.findById(id)
00080             .orElseThrow(() -> new ServiceException("Pet bulunamadı: id=" + id));
00081     }
00082
00087     public List<Pet> getAllPets() {
00088         return repository.findAll();
00089     }
00090
00096     public void updatePet(Pet pet) {
00097         validatePet(pet);
00098         boolean updated = repository.update(pet);
00099         if (!updated) {
00100             throw new ServiceException("Pet güncellenemedi: id=" + pet.getId());
00101         }
00102         logger.info("Pet güncellendi: id={}", pet.getId());
00103         // Observer Pattern: Pet güncellendi bildirimi
00104         EventManager.getInstance().notify(EventType.PET_UPDATED, pet);
00105     }
00106
00112     public void deletePet(int id) {
00113         boolean deleted = repository.delete(id);
00114         if (!deleted) {
00115             throw new ServiceException("Pet silinemedi: id=" + id);
00116         }
00117         logger.info("Pet silindi: id={}", id);
00118         // Observer Pattern: Pet silindi bildirimi
00119         EventManager.getInstance().notify(EventType.PET_DELETED, id);
00120     }
00121
00125     public void deleteAllPets() {
00126         repository.deleteAll();
00127     }
00128
00133     public int getPetCount() {
00134         return repository.count();
00135     }
00136
00142     public List<Pet> searchByName(String name) {
00143         if (name == null || name.trim().isEmpty()) return getAllPets();
00144         String lower = name.toLowerCase().trim();
00145         return repository.findAll().stream()
00146             .filter(p -> p.getName() != null && p.getName().toLowerCase().contains(lower))
00147             .collect(Collectors.toList());
00148     }
00149
00155     public List<Pet> filterBySpecies(String species) {
00156         if (species == null || species.trim().isEmpty()) return getAllPets();
00157         return repository.findAll().stream()
00158             .filter(p -> p.getSpecies() != null
00159                 && p.getSpecies().equalsIgnoreCase(species.trim()))
00160             .collect(Collectors.toList());
00161     }
00162
00166     public void close() {
00167         repository.close();
00168     }
00169
00175     private void validatePet(Pet pet) {
00176         if (pet == null) throw new ServiceException("Pet null olamaz");
00177         if (pet.getName() == null || pet.getName().trim().isEmpty()) {
00178             throw new ServiceException("Pet adı boş olamaz");
00179         }
00180         if (pet.getBirthDate() != null && pet.getBirthDate().isAfter(LocalDate.now())) {
00181             throw new ServiceException("Doğum tarihi gelecekte olamaz");
00182         }
00183     }

```

```
00184 }
```

8.95 petreminder-app/src/main/java/com/mehdi/petreminder/service/ReminderService.java File Reference

Reminder iş mantığı servis katmanı.

```
import com.mehdi.petreminder.model.Reminder;
import com.mehdi.petreminder.observer.EventManager;
import com.mehdi.petreminder.observer.EventType;
import com.mehdi.petreminder.repository.IRepository;
import com.mehdi.petreminder.repository.RepositoryFactory;
import org.slf4j.LoggerFactory;
import ch.qos.logback.classic.Logger;
import java.time.LocalDateTime;
import java.util.*;
import java.util.stream.Collectors;
```

Classes

- class [com.mehdi.petreminder.service.ReminderService](#)
Reminder CRUD ve iş kuralları.

Packages

- package [com.mehdi.petreminder.service](#)

8.95.1 Detailed Description

Reminder iş mantığı servis katmanı.

Observer Pattern: CRUD işlemlerinde EventManager'a bildirim gönderir.

Definition in file [ReminderService.java](#).

8.96 ReminderService.java

[Go to the documentation of this file.](#)

```
00001
00006 package com.mehdi.petreminder.service;
00007
00008 import com.mehdi.petreminder.model.Reminder;
00009 import com.mehdi.petreminder.observer.EventManager;
00010 import com.mehdi.petreminder.observer.EventType;
00011 import com.mehdi.petreminder.repository.IRepository;
00012 import com.mehdi.petreminder.repository.RepositoryFactory;
00013 import org.slf4j.LoggerFactory;
00014 import ch.qos.logback.classic.Logger;
00015
00016 import java.time.LocalDateTime;
00017 import java.util.*;
```

```

00018 import java.util.stream.Collectors;
00019
00032 public class ReminderService {
00033
00035     private static final Logger logger =
00036         (Logger) LoggerFactory.getLogger(ReminderService.class);
00037
00039     private final IRepository<Reminder> repository;
00040
00044     public ReminderService() {
00045         this.repository = RepositoryFactory.createReminderRepository();
00046     }
00047
00052     public ReminderService(IRepository<Reminder> repository) {
00053         this.repository = repository;
00054     }
00055
00062     public Reminder addReminder(Reminder reminder) {
00063         validateReminder(reminder);
00064         int id = repository.save(reminder);
00065         logger.info("Reminder eklendi: id={}, type={}", id, reminder.getReminderType());
00066         // Observer Pattern: Reminder eklendi bildirimi
00067         EventManager.getInstance().notify(EventType.REMINDER_ADDED, reminder);
00068         return reminder;
00069     }
00070
00076     public Reminder getReminderById(int id) {
00077         return repository.findById(id)
00078             .orElseThrow(() -> new ServiceException("Reminder bulunamadı: id=" + id));
00079     }
00080
00085     public List<Reminder> getAllReminders() {
00086         return repository.findAll();
00087     }
00088
00093     public List<Reminder> getPendingReminders() {
00094         return repository.findAll().stream()
00095             .filter(r -> !r.isCompleted())
00096             .sorted(Comparator.comparing(r -> r.getScheduledTime() != null
00097                 ? r.getScheduledTime() : LocalDateTime.MAX))
00098             .collect(Collectors.toList());
00099     }
00100
00105     public List<Reminder> getOverdueReminders() {
00106         return repository.findAll().stream()
00107             .filter(Reminder::isOverdue)
00108             .collect(Collectors.toList());
00109     }
00110
00116     public List<Reminder> getRemindersByPetId(int petId) {
00117         return repository.findAll().stream()
00118             .filter(r -> r.getPetId() == petId)
00119             .collect(Collectors.toList());
00120     }
00121
00126     public void markCompleted(int id) {
00127         Reminder r = getReminderById(id);
00128         r.setCompleted(true);
00129         repository.update(r);
00130         logger.info("Reminder tamamlandı: id={}", id);
00131         // Observer Pattern: Reminder tamamlandı bildirimi
00132         EventManager.getInstance().notify(EventType.REMINDER_COMPLETED, r);
00133     }
00134
00139     public void updateReminder(Reminder reminder) {
00140         validateReminder(reminder);
00141         boolean updated = repository.update(reminder);
00142         if (!updated) {
00143             throw new ServiceException("Reminder güncellenemedi: id=" + reminder.getId());
00144         }
00145     }
00146
00151     public void deleteReminder(int id) {
00152         boolean deleted = repository.delete(id);
00153         if (!deleted) {
00154             throw new ServiceException("Reminder silinemedi: id=" + id);
00155         }
00156         logger.info("Reminder silindi: id={}", id);
00157         // Observer Pattern: Reminder silindi bildirimi
00158         EventManager.getInstance().notify(EventType.REMINDER_DELETED, id);
00159     }
00160
00164     public void deleteAllReminders() {
00165         repository.deleteAll();
00166     }
00167
00172     public int getReminderCount() {

```



```
00173         return repository.count();
00174     }
00175
00179     public void close() {
00180         repository.close();
00181     }
00182
00187     private void validateReminder(Reminder r) {
00188         if (r == null) throw new ServiceException("Reminder null olamaz");
00189         if (r.getPetId() <= 0) throw new ServiceException("Geçerli bir pet seçin");
00190         if (r.getScheduledTime() == null) throw new ServiceException("Zaman boş olamaz");
00191         if (!r.validate()) throw new ServiceException("Reminder verisi geçersiz");
00192     }
00193 }
```

8.97 petreminder-app/src/main/java/com/mehdi/petreminder/service/ServiceException.java File Reference ↩

Servis katmanı özel istisna sınıfı.

Classes

- class [com.mehdi.petreminder.service.ServiceException](#)
Servis katmanında fırlatılan unchecked istisna.

Packages

- package [com.mehdi.petreminder.service](#)

8.97.1 Detailed Description

Servis katmanı özel istisna sınıfı.

Definition in file [ServiceException.java](#).

8.98 ServiceException.java

[Go to the documentation of this file.](#)

```
00001
00005 package com.mehdi.petreminder.service;
00006
00015 public class ServiceException extends RuntimeException {
00016
00018     private static final long serialVersionUID = 1L;
00019
00024     public ServiceException(String message) {
00025         super(message);
00026     }
00027
00033     public ServiceException(String message, Throwable cause) {
00034         super(message, cause);
00035     }
00036 }
```

